

Malware Data Science

Attack Detection and Attribution



Joshua Saxe with Hillary Sanders

Foreword by Anup Ghosh, PhD



MALWARE DATA SCIENCE

MALWARE DATA SCIENCE

Attack Detection and Attribution

**by Joshua Saxe
with Hillary Sanders**



**no starch
press**

San Francisco

MALWARE DATA SCIENCE. Copyright © 2018 by Joshua Saxe with Hillary Sanders.

All rights reserved. No part of this work may be reproduced or transmitted in any form or by any means, electronic or mechanical, including photocopying, recording, or by any information storage or retrieval system, without the prior written permission of the copyright owner and the publisher.

ISBN-10: 1-59327-859-4

ISBN-13: 978-1-59327-859-5

Publisher: William Pollock

Production Editor: Laurel Chun

Cover Illustration: Jonny Thomas

Interior Design: Octopod Studios

Developmental Editors: Annie Choi and William Pollock

Technical Reviewer: Gabor Szappanos

Copyeditor: Barton Reed

Compositor: Laurel Chun

Proofreader: James Fraleigh

Indexer: BIM Creatives, LLC

For information on distribution, translations, or bulk sales, please contact No Starch Press, Inc. directly:

No Starch Press, Inc.

245 8th Street, San Francisco, CA 94103

phone: 1.415.863.9900; info@nostarch.com

www.nostarch.com

Library of Congress Control Number: 2018949204

No Starch Press and the No Starch Press logo are registered trademarks of No Starch Press, Inc. Other product and company names mentioned herein may be the trademarks of their respective owners. Rather than use a trademark symbol with every occurrence of a trademarked name, we are using the names only in an editorial fashion and to the benefit of the trademark owner, with no intention of infringement of the trademark.

The information in this book is distributed on an “As Is” basis, without warranty. While every precaution has been taken in the preparation of this work, neither the authors nor No Starch Press, Inc. shall have any liability to any person or entity with respect to any loss or damage caused or alleged to be caused directly or indirectly by the information contained in it.

To Alen Capalik,
for bringing me back to computers after a long hiatus

About the Authors

Joshua Saxe is Chief Data Scientist at the major security vendor Sophos, where he leads a security data science research team. He's also a principal inventor of Sophos' neural network-based malware detector, which defends tens of millions of Sophos customers from malware infections. Before joining Sophos, Joshua spent five years leading DARPA-funded security data research projects for the US government.

Hillary Sanders is a senior software engineer and data scientist at Sophos, where she has played a key role in inventing and productizing neural network, machine learning, and malware similarity analysis technologies. Before joining Sophos, Hillary was a data scientist at Premise Data Corporation. She is a regular speaker at security conferences, having given security data science talks at Blackhat USA and BSides Las Vegas. She studied Statistics at UC Berkeley.

About the Technical Reviewer

Gabor Szappanos graduated from the Eotvos Lorand University of Budapest with a degree in physics. His first job was developing diagnostic software and hardware for nuclear power plants at the Computer and Automation Research Institute. Gabor started antivirus work in 1995 and joined VirusBuster in 2001, where he was responsible for taking care of macro virus and script malware; in 2002, he became head of the virus lab. Between 2008 and 2016, he was a member of the board of directors in Anti-Malware Testing Standards Organizations (AMTSO), and, in 2012, he joined Sophos as a Principal Malware Researcher.

BRIEF CONTENTS

Foreword	xvii
Acknowledgments	xix
Introduction	xxi
Chapter 1: Basic Static Malware Analysis.	1
Chapter 2: Beyond Basic Static Analysis: x86 Disassembly	11
Chapter 3: A Brief Introduction to Dynamic Analysis.	25
Chapter 4: Identifying Attack Campaigns Using Malware Networks.	35
Chapter 5: Shared Code Analysis	59
Chapter 6: Understanding Machine Learning–Based Malware Detectors.	89
Chapter 7: Evaluating Malware Detection Systems.	119
Chapter 8: Building Machine Learning Detectors	127
Chapter 9: Visualizing Malware Trends	155
Chapter 10: Deep Learning Basics	175
Chapter 11: Building a Neural Network Malware Detector with Keras	199
Chapter 12: Becoming a Data Scientist	215
Appendix: An Overview of Datasets and Tools	221
Index	233

CONTENTS IN DETAIL

FOREWORD by Anup Ghosh	xvii
-------------------------------	-------------

ACKNOWLEDGMENTS	xix
------------------------	------------

INTRODUCTION	xxi
---------------------	------------

What Is Data Science?	xxii
Why Data Science Matters for Security	xxii
Applying Data Science to Malware	xxiii
Who Should Read This Book?	xxiv
About This Book.	xxiv
How to Use the Sample Code and Data	xxv

1	
BASIC STATIC MALWARE ANALYSIS	1

The Microsoft Windows Portable Executable Format.	2
The PE Header	3
The Optional Header	3
Section Headers	4
Dissecting the PE Format Using pefile	5
Examining Malware Images	7
Examining Malware Strings.	8
Using the strings Program	8
Analyzing Your strings Dump	9
Summary	10

2	
BEYOND BASIC STATIC ANALYSIS: X86 DISASSEMBLY	11

Disassembly Methods	12
Basics of x86 Assembly Language	12
CPU Registers	13
Arithmetic Instructions	15
Data Movement Instructions	15
Disassembling ircbot.exe Using pefile and capstone.	20
Factors That Limit Static Analysis	21
Packing	21
Resource Obfuscation	22
Anti-disassembly Techniques.	22
Dynamically Downloaded Data	22
Summary	23

3	A BRIEF INTRODUCTION TO DYNAMIC ANALYSIS	25
	Why Use Dynamic Analysis?	26
	Dynamic Analysis for Malware Data Science.	26
	Basic Tools for Dynamic Analysis	27
	Typical Malware Behaviors	27
	Loading a File on malwr.com	27
	Analyzing Results on malwr.com.	28
	Limitations of Basic Dynamic Analysis	33
	Summary	34
 4	 IDENTIFYING ATTACK CAMPAIGNS USING MALWARE NETWORKS	 35
	Nodes and Edges	37
	Bipartite Networks	37
	Visualizing Malware Networks	39
	The Distortion Problem	39
	Force-Directed Algorithms	40
	Building Networks with NetworkX	40
	Adding Nodes and Edges.	41
	Adding Attributes	42
	Saving Networks to Disk	42
	Network Visualization with GraphViz.	43
	Using Parameters to Adjust Networks	44
	The GraphViz Command Line Tools	44
	Adding Visual Attributes to Nodes and Edges	48
	Building Malware Networks	51
	Building a Shared Image Relationship Network	54
	Summary	58
 5	 SHARED CODE ANALYSIS	 59
	Preparing Samples for Comparison by Extracting Features	62
	How Bag of Features Models Work.	62
	What are N-Grams?	63
	Using the Jaccard Index to Quantify Similarity	64
	Using Similarity Matrices to Evaluate Malware Shared Code Estimation Methods	66
	Instruction Sequence–Based Similarity	67
	Strings-Based Similarity	70
	Import Address Table–Based Similarity	71
	Dynamic API Call–Based Similarity	72
	Building a Similarity Graph	73
	Scaling Similarity Comparisons	77
	Minhash in a Nutshell	77
	Minhash in Depth	78
	Building a Persistent Malware Similarity Search System	79
	Running the Similarity Search System	85
	Summary	87

6	UNDERSTANDING MACHINE LEARNING-BASED MALWARE DETECTORS	89
Steps for Building a Machine Learning-Based Detector	90	
Gathering Training Examples	91	
Extracting Features	91	
Designing Good Features.	92	
Training Machine Learning Systems.	92	
Testing Machine Learning Systems	93	
Understanding Feature Spaces and Decision Boundaries	93	
What Makes Models Good or Bad: Overfitting and Underfitting	98	
Major Types of Machine Learning Algorithms	101	
Logistic Regression	102	
K-Nearest Neighbors.	105	
Decision Trees	109	
Random Forest	115	
Summary	117	
 7	 EVALUATING MALWARE DETECTION SYSTEMS	 119
Four Possible Detection Outcomes	120	
True and False Positive Rates	120	
Relationship Between True and False Positive Rates	121	
ROC Curves.	123	
Considering Base Rates in Your Evaluation	124	
How Base Rate Affects Precision	124	
Estimating Precision in a Deployment Environment	125	
Summary	126	
 8	 BUILDING MACHINE LEARNING DETECTORS	 127
Terminology and Concepts	128	
Building a Toy Decision Tree-Based Detector.	129	
Training Your Decision Tree Classifier	130	
Visualizing the Decision Tree	131	
Complete Sample Code.	133	
Building Real-World Machine Learning Detectors with sklearn	134	
Real-World Feature Extraction	134	
Why You Can't Use All Possible Features.	137	
Using the Hashing Trick to Compress Features	138	
Building an Industrial-Strength Detector	141	
Extracting Features	141	
Training the Detector	142	
Running the Detector on New Binaries.	144	
What We've Implemented So Far	144	
Evaluating Your Detector's Performance	146	
Using ROC Curves to Evaluate Detector Efficacy	147	
Computing ROC Curves.	147	
Splitting Data into Training and Test Sets	148	

Computing the ROC Curve	149
Cross-Validation	150
Next Steps	153
Summary	154

9 VISUALIZING MALWARE TRENDS 155

Why Visualizing Malware Data Is Important	156
Understanding Our Malware Dataset	158
Loading Data into pandas	158
Working with a pandas DataFrame	159
Filtering Data Using Conditions	161
Using matplotlib to Visualize Data	162
Plotting the Relationship Between Malware Size and Detection	162
Plotting Ransomware Detection Rates	164
Plotting Ransomware and Worm Detection Rates	165
Using seaborn to Visualize Data	168
Plotting the Distribution of Antivirus Detections	169
Creating a Violin Plot	172
Summary	174

10 DEEP LEARNING BASICS 175

What Is Deep Learning?	176
How Neural Networks Work	177
Anatomy of a Neuron	177
A Network of Neurons	180
Universal Approximation Theorem	181
Building Your Own Neural Network	182
Adding Another Neuron to the Network	186
Automatic Feature Generation	188
Training Neural Networks	189
Using Backpropagation to Optimize a Neural Network	190
Path Explosion	192
Vanishing Gradient	192
Types of Neural Networks	193
Feed-Forward Neural Network	193
Convolutional Neural Network	193
Autoencoder Neural Network	194
Generative Adversarial Network	195
Recurrent Neural Network	196
ResNet	196
Summary	197

11 BUILDING A NEURAL NETWORK MALWARE DETECTOR WITH KERAS 199

Defining a Model's Architecture	200
Compiling the Model	202

Training the Model	203
Extracting Features	203
Creating a Data Generator	204
Incorporating Validation Data	207
Saving and Loading the Model	209
Evaluating the Model	209
Enhancing the Model Training Process with Callbacks	211
Using a Built-in Callback	212
Using a Custom Callback	213
Summary	214

12

BECOMING A DATA SCIENTIST 215

Paths to Becoming a Security Data Scientist	216
A Day in the Life of a Security Data Scientist	216
Traits of an Effective Security Data Scientist	218
Open-Mindedness	218
Boundless Curiosity	218
Obsession with Results	219
Skepticism of Results	219
Where to Go from Here	219

APPENDIX

AN OVERVIEW OF DATASETS AND TOOLS 221

Overview of Datasets	222
Chapter 1: Basic Static Malware Analysis	222
Chapter 2: Beyond Basic Static Analysis: x86 Disassembly	222
Chapter 3: A Brief Introduction to Dynamic Analysis	222
Chapter 4: Identifying Attack Campaigns Using Malware Networks	222
Chapter 5: Shared Code Analysis	223
Chapter 6: Understanding Machine Learning–Based Malware Detectors and Chapter 7: Evaluating Malware Detection Systems	223
Chapter 8: Building Machine Learning Detectors	224
Chapter 9: Visualizing Malware Trends	224
Chapter 10: Deep Learning Basics	224
Chapter 11: Building a Neural Network Malware Detector with Keras	224
Chapter 12: Becoming a Data Scientist	224
Tool Implementation Guide	225
Shared Hostname Network Visualization	225
Shared Image Network Visualization	226
Malware Similarity Visualization	227
Malware Similarity Search System	229
Machine Learning Malware Detection System	230

INDEX

233

FOREWORD

Congratulations on picking up *Malware Data Science*. You're on your way to equipping yourself with the skills necessary to become a cybersecurity professional. In this book, you'll find a wonderful introduction to data science as applied to malware analysis, as well as the requisite skills and tools you need to be proficient at it.

There are far more jobs in cybersecurity than there are qualified candidates, so the good news is that cybersecurity is a great field to get into. The bad news is that the skills required to stay current are changing rapidly. As is often the case, necessity is the mother of invention. With far more demand for skilled cybersecurity professionals than there is supply, data science algorithms are filling the gap by providing new insights and predictions about threats against networks. The traditional model of watchmen monitoring network data is rapidly becoming obsolete as data science

is increasingly being used to find threat patterns in terabytes of data. And thank goodness for that, because monitoring a screen of alerts is about as exciting as monitoring a video camera surveillance system of a parking lot.

So what exactly is data science and how does it apply to security? As you'll see in the Introduction, data science applied to security is the art and science of using machine learning, data mining, and visualization to detect threats against networks. While you'll find a lot of hyperbole around machine learning and artificial intelligence driven by marketing, there are, in fact, very good use cases for these technologies that are in production today.

For instance, when it comes to malware detection, both the volume of malware production and the cost to the adversary in changing malware signatures has rendered signature-only based approaches to malware obsolete. Instead, antivirus companies are now training neural networks or other types of machine learning algorithms over very large datasets of malware to learn their characteristics, so that new variants of malware can be detected without having to update the model daily. The combination of signature-based and machine learning-based detection provides coverage for both known and unknown malware. This is a topic both Josh and Hillary are experts in and from which they speak from deep experience.

But malware detection is only one use case for data science. In fact, when it comes to finding threats on the network, today's sophisticated adversaries often will not drop executable programs. Instead, they will exploit existing software for initial access and then leverage system tools to pivot from one machine to the next using the user privileges obtained through exploitation. From an adversarial point of view this approach doesn't leave behind artifacts such as malware that antivirus software will detect. However, a good endpoint logging system or an endpoint detection and response (EDR) system will capture system level activities and send this telemetry to the cloud, from where analysts can attempt to piece together the digital footprints of an intruder. This process of combing through massive streams of data and continuously looking for patterns of intrusion is a problem well-suited for data science, specifically data mining with statistical algorithms and data visualization. You can expect more and more Security Operations Centers (SOCs) to adopt data mining and artificial intelligence technologies. It's really the only way to cull through massive data sets of system events to identify actual attacks.

Cybersecurity is undergoing massive shifts in technology and its operations, and data science is driving the change. We are fortunate to have experts like Josh Saxe and Hillary Sanders not only share their expertise with us, but do it in such an engaging and accessible way. This is your opportunity to learn what they know and apply it to your own work so you can stay ahead of the changes in technology and the adversaries you're charged with defeating.

Anup K. Ghosh, PhD
Founder, Invoicea, Inc
Washington, DC

ACKNOWLEDGMENTS

Thanks to Annie Choi, Laurel Chun, and Bill Pollock at No Starch Press and to my copyeditor, Bart Reed. In all justice, they should be regarded as co-authors of this book. Thanks in advance to the workers responsible for printing, transporting, and selling copies of this book, and the engineers responsible for its digital storage, transmission, and rendering. Thanks to Hillary Sanders for bringing her remarkable talents to the project exactly when they were needed. Gratitude to Gabor Szappanos for his excellent and exacting technical review.

Thanks to my two year old daughter Maya, who, I'm happy to share, slowed this project down dramatically. Thanks to Alen Capalik, Danny Hillis, Chris Greamo, Anup Ghosh, and Joe Levy for their mentorship over the past 10 years. Deep appreciation to the Defense Advanced Research Projects Agency (DARPA) and Timothy Fraser for supporting the research on which much of this book is based. Thanks to Mandiant, and Mila Parkour, for obtaining and curating the APT1 malware samples used for demonstration purposes in this book. Deep appreciation to the authors of Python, NetworkX, matplotlib, numpy, sklearn, Keras, seaborn, pefile, icoutils, *malwr.com*, CuckooBox, capstone, pandas, and sqlite for your contributions to free and open source security and data science software.

Tremendous gratitude to my parents, Maryl Gearhart and Geoff Saxe, for introducing me to computers, for tolerating my teenage hacker phase (and all the illegality that entailed), and for their boundless love and support. Thanks to Gary Glickman for his indispensable love and support. Finally, thanks to Ksenya Gurshtein, my partner in life, for supporting me in this endeavor completely and without hesitation.

Joshua Saxe

Thanks to Josh, for including me in this! Thanks to Ani Adhikari for being an amazing teacher. Thanks to Jacob Michelini, because he really wanted his name in a book.

Hillary Sanders

INTRODUCTION



If you're working in security, chances are you're using data science more than ever before, even if you may not realize it. For example, your antivirus product uses data science algorithms to detect malware. Your firewall vendor may have data science algorithms detecting suspicious network activity. Your security information and event management (SIEM) software probably uses data science to identify suspicious trends in your data. Whether conspicuously or not, the entire security industry is moving toward incorporating more data science into security products.

Advanced IT security professionals are incorporating their own custom machine learning algorithms into their workflows. For example, in recent conference presentations and news articles, security analysts at Target, Mastercard, and Wells Fargo all described developing custom data science

technologies that they use as part of their security workflows.¹ If you're not already on the data science bandwagon, there's no better time to upgrade your skills to include data science into your security practice.

What Is Data Science?

Data science is a growing set of algorithmic tools that allow us to understand and make predictions about data using statistics, mathematics, and artful statistical data visualizations. More specific definitions exist, but generally, data science has three subcomponents: machine learning, data mining, and data visualization.

In the security context, machine learning algorithms learn from training data to detect new threats. These methods have been proven to detect malware that flies under the radar of traditional detection techniques like signatures. Data mining algorithms search security data for interesting patterns (such as relationships between threat actors) that might help us discern attack campaigns targeting our organizations. Finally, data visualization renders sterile, tabular data into graphical format to make it easier for people to spot interesting and suspicious trends. I cover all three areas in depth in this book and show you how to apply them.

Why Data Science Matters for Security

Data science is critically important for the future of cybersecurity for three reasons: first, security is *all about data*. When we seek to detect cyber threats, we're analyzing data in the form of files, logs, network packets, and other artifacts. Traditionally, security professionals didn't use data science techniques to make detections based on these data sources. Instead, they used file hashes, custom-written rules like signatures, and manually defined heuristics. Although these techniques have their merits, they required hand-crafted techniques for each type of attack, necessitating too much manual work to keep up with the changing cyber threat landscape. In recent years, data science techniques have become crucial in bolstering our ability to detect threats.

Second, data science is important to cybersecurity because the number of cyberattacks on the internet has grown dramatically. Take the growth of the malware underworld as an example. In 2008, there were about 1 million unique malware executables known to the security community. By 2012, there were 100 million. As this book goes to press in 2018, there are more than 700 million malicious executables known to the security community (<https://www.av-test.org/en/statistics/malware/>), and this number is likely to grow.

1. Target (<https://www.rsaconference.com/events/us17/agenda/sessions/6662-applied-machine-learning-defeating-modern-malicious>), Mastercard (<https://blogs.wsj.com/cio/2017/11/15/artificial-intelligence-transforms-hacker-arsenal/>), and Wells Fargo (<https://blogs.wsj.com/cio/2017/11/16/the-morning-download-first-ai-powered-cyberattacks-are-detected/>).

Due to the sheer volume of malware, manual detection techniques such as signatures are no longer a reasonable method for detecting all cyberattacks. Because data science techniques automate much of the work that goes into detecting cyberattacks, and vastly decrease the memory usage needed to detect such attacks, they hold tremendous promise in defending networks and users as cyber threats grow.

Finally, data science matters for security because data science is *the* technical trend of the decade, both inside and outside of the security industry, and it will likely remain so through the next decade. Indeed, you've probably seen applications of data science everywhere—in personal voice assistants (Amazon Echo, Siri, and Google Home), self-driving cars, ad recommendation systems, web search engines, medical image analysis systems, and fitness tracking apps.

We can expect data science–driven systems to have major impacts in legal services, education, and other areas. Because data science has become a key enabler across the technical landscape, universities, major companies (Google, Facebook, Microsoft, and IBM), and governments are investing billions of dollars to improve data science tools. Thanks to these investments, data science tools will become even more adept at solving hard attack-detection problems.

Applying Data Science to Malware

This book focuses on data science as it applies to *malware*, which we define as executable programs written with malicious intent, because malware continues to be the primary means by which threat actors gain a foothold on networks and subsequently achieve their goals. For example, in the ransomware scourge that has emerged in recent years, attackers typically send users malicious email attachments that download ransomware executables (malware) to users' machines, which then encrypt users' data and ask them for a ransom to decrypt the data. Although skilled attackers working for governments sometimes avoid using malware altogether to fly under the radar of detection systems, malware continues to be the major enabling technology in cyberattacks today.

By homing in on a specific application of security data science rather than attempting to cover security data science broadly, this book aims to show more thoroughly how data science techniques can be applied to a major security problem. By understanding malware data science, you'll be better equipped to apply data science to other areas of security, like detecting network attacks, phishing emails, or suspicious user behavior. Indeed, almost all the techniques you'll learn in this book apply to building data science detection and intelligence systems in general, not just for malware.

Who Should Read This Book?

This book is aimed toward security professionals who are interested in learning more about how to apply data science to computer security problems. If computer security *and* data science are new to you, you might find yourself having to look up terms to give yourself a little bit of context, but you can still read this book successfully. If you're only interested in data science, but not security, this book is probably not for you.

About This Book

The first part of the book consists of three chapters that cover basic reverse engineering concepts necessary for understanding the malware data science techniques discussed later in the book. If you're new to malware, read the first three chapters first. If you're an old hand at malware reverse engineering, you can skip these chapters.

- **Chapter 1: Basic Static Malware Analysis** covers static analysis techniques for picking apart malware files and discovering how they achieve malicious ends on our computers.
- **Chapter 2: Beyond Basic Static Analysis: x86 Disassembly** gives you a brief overview of x86 assembly language and how to disassemble and reverse engineer malware.
- **Chapter 3: A Brief Introduction to Dynamic Analysis** concludes the reverse engineering section of the book by discussing dynamic analysis, which involves running malware in controlled environments to learn about its behavior.

The next two chapters of the book, Chapters 4 and 5, focus on malware relationship analysis, which involves looking at similarities and differences between collections of malware to identify malware campaigns against your organization, such as a ransomware campaign controlled by a group of cybercriminals, or a concerted, targeted attack on your organization. These stand-alone chapters are for readers who are interested not only in detecting malware, but also in extracting valuable threat intelligence to learn who is attacking their network. If you're less interested in threat intelligence and more interested in data science–driven malware detection, you can safely skip these chapters.

- **Chapter 4: Identifying Attack Campaigns Using Malware Networks** shows you how to analyze and visualize malware based on shared attributes, such as the hostnames that malware programs call out to.
- **Chapter 5: Shared Code Analysis** explains how to identify and visualize shared code relationships between malware samples, which can help you identify whether groups of malware samples came from one or multiple criminal groups.

The next four chapters cover everything you need to know to understand, apply, and implement machine learning–based malware detection systems. These chapters also provide a foundation for applying machine learning to other security contexts.

- **Chapter 6: Understanding Machine Learning–Based Malware Detectors** is an accessible, intuitive, and non-mathematical introduction to basic machine learning concepts. If you have a history with machine learning, this chapter will provide a convenient refresher.
- **Chapter 7: Evaluating Malware Detection Systems** shows you how to evaluate the accuracy of your machine learning systems using basic statistical methods so that you can select the best possible approach.
- **Chapter 8: Building Machine Learning Detectors** introduces open source machine learning tools you can use to build your own machine learning systems and explains how to use them.
- **Chapter 9: Visualizing Malware Trends** covers how to visualize malware threat data to reveal attack campaigns and trends using Python, and how to integrate data visualization into your day-to-day workflow when analyzing security data.

The last three chapters introduce deep learning, an advanced area of machine learning that involves a bit more math. Deep learning is a hot growth area within security data science, and these chapters provide enough to get you started.

- **Chapter 10: Deep Learning Basics** covers the basic concepts that underlie deep learning.
- **Chapter 11: Building a Neural Network Malware Detector with Keras** explains how to implement deep learning–based malware detection systems in Python using open source tools.
- **Chapter 12: Becoming a Data Scientist** concludes the book by sharing different pathways to becoming a data scientist and qualities that can help you succeed in the field.
- **Appendix: An Overview of Datasets and Tools** describes the data and example tool implementations accompanying the book.

How to Use the Sample Code and Data

No good programming book is complete without sample code to play with and extend on your own. Sample code and data accompany each chapter of this book and are described exhaustively in the appendix. All the code targets Python 2.7 in Linux environments. To access the code and data, you can download a VirtualBox Linux virtual machine, which has the code, data, and supporting open source tools all set up and ready to go,

and run it within your own VirtualBox environment. You can download the book's accompanying data at <http://www.malwaredatascience.com/>, and you can download the VirtualBox for free at <https://www.virtualbox.org/wiki/Downloads>. The code has been tested on Linux, but if you prefer to work outside of the Linux VirtualBox, the same code should work almost as well on MacOS, and to a lesser extent on Windows machines.

If you'd rather install the code and data in your own Linux environment, you can download them here: <http://www.malwaredatascience.com/>. You'll find a directory for each chapter in the downloadable archive, and within each chapter's directory there are *code/* and *data/* directories that contain the corresponding code and data. Code files correspond to chapter listings or sections, whichever makes more sense for the application at hand. Some code files are exactly like the listings, whereas others have been changed slightly to make it easier for you to play with parameters and other options. Code directories come with pip *requirements.txt* files, which give the open source libraries that the code in that chapter depends on to run. To install these libraries on your machine, simply type **pip -r requirements.txt** in each chapter's *code/* directory.

Now that you have access to the code and data for this book, let's get started.

1

BASIC STATIC MALWARE ANALYSIS



In this chapter we look at the basics of static malware analysis. Static analysis is performed by analyzing a program file's disassembled code, graphical images, printable strings, and other on-disk resources. It refers to reverse engineering without actually running the program. Although static analysis techniques have their shortcomings, they can help us understand a wide variety of malware. Through careful reverse engineering, you'll be able to better understand the benefits that malware binaries provide attackers after they've taken possession of a target, as well as the ways attackers can hide and continue their attacks on an infected machine. As you'll see, this chapter combines descriptions and examples. Each section introduces a static analysis technique and then illustrates its application in real-world analysis.

I begin this chapter by describing the Portable Executable (PE) file format used by most Windows programs, and then examine how to use the popular Python library `pefile` to dissect a real-world malware binary. I then describe techniques such as imports analysis, graphical image analysis, and strings analysis. In all cases, I show you how to use open source tools to apply the analysis technique to real-world malware. Finally, at the end of the chapter, I introduce ways malware can make life difficult for malware analysts and discuss some ways to mitigate these issues.

You'll find the malware sample used in the examples in this chapter in this book's data under the directory `/ch1`. To demonstrate the techniques discussed in this chapter, we use *ircbot.exe*, an Internet Relay Chat (IRC) bot created for experimental use, as an example of the kinds of malware commonly observed in the wild. As such, the program is designed to stay resident on a target computer while connected to an IRC server. After *ircbot.exe* gets hold of a target, attackers can control the target computer via IRC, allowing them to take actions such as turning on a webcam to capture and surreptitiously extract video feeds of the target's physical location, taking screenshots of the desktop, extracting files from the target machine, and so on. Throughout this chapter, I demonstrate how static analysis techniques can reveal the capabilities of this malware.

The Microsoft Windows Portable Executable Format

To perform static malware analysis, you need to understand the Windows PE format, which describes the structure of modern Windows program files such as `.exe`, `.dll`, and `.sys` files and defines the way they store data. PE files contain x86 instructions, data such as images and text, and metadata that a program needs in order to run.

The PE format was originally designed to do the following:

Tell Windows how to load a program into memory The PE format describes which chunks of a file should be loaded into memory, and where. It also tells you where in the program code Windows should start a program's execution and which dynamically linked code libraries should be loaded into memory.

Supply media (or resources) a running program may use in the course of its execution These resources can include strings of characters like the ones in GUI dialogs or console output, as well as images or videos.

Supply security data such as digital code signatures Windows uses such security data to ensure that code comes from a trusted source.

The PE format accomplishes all of this by leveraging the series of constructs shown in Figure 1-1.

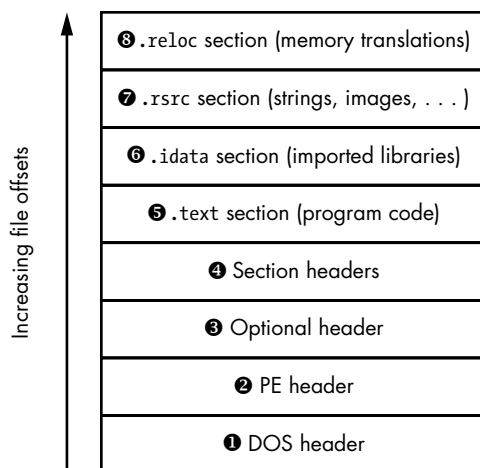


Figure 1-1: The PE file format

As the figure shows, the PE format includes a series of headers telling the operating system how to load the program into memory. It also includes a series of sections that contain the actual program data. Windows loads the sections into memory such that their memory offsets correspond to where they appear on disk. Let's explore this file structure in more detail, starting with the PE header. We'll skip over a discussion of the DOS header, which is a relic of the 1980s-era Microsoft DOS operating system and only present for compatibility reasons.

The PE Header

Shown at the bottom of Figure 1-1, above the DOS header ①, is the PE header ②, which defines a program's general attributes such as binary code, images, compressed data, and other program attributes. It also tells us whether a program is designed for 32- or 64-bit systems. The PE header provides basic but useful contextual information to the malware analyst. For example, the header includes a timestamp field that can give away the time at which the malware author compiled the file. This happens when malware authors forget to replace this field with a bogus value, which they often do.

The Optional Header

The optional header ③ is actually ubiquitous in today's PE executable programs, contrary to what its name suggests. It defines the location of the program's *entry point* in the PE file, which refers to the first instruction the program runs once loaded. It also defines the size of the data that Windows loads into memory as it loads the PE file, the Windows subsystem, the program targets (such as the Windows GUI or the Windows

command line), and other high-level details about the program. The information in this header can prove invaluable to reverse engineers, because a program's entry point tells them where to begin reverse engineering.

Section Headers

Section headers ❹ describe the data sections contained within a PE file. A *section* in a PE file is a chunk of data that either will be mapped into memory when the operating system loads a program or will contain instructions about how the program should be loaded into memory. In other words, a section is a sequence of bytes on disk that will either become a contiguous string of bytes in memory or inform the operating system about some aspect of the loading process.

Section headers also tell Windows what permissions it should grant to sections, such as whether they should be readable, writable, or executable by the program when it's executing. For example, the `.text` section containing x86 code will typically be marked readable and executable but not writable to prevent program code from accidentally modifying itself in the course of execution.

A number of sections, such as `.text` and `.rsrc`, are depicted in Figure 1-1. These get mapped into memory when the PE file is executed. Other special sections, such as the `.reloc` section, aren't mapped into memory. We'll discuss these sections as well. Let's go over the sections shown in Figure 1-1.

The `.text` Section

Each PE program contains at least one section of x86 code marked executable in its section header; these sections are almost always named `.text` ❺. We'll disassemble the data in the `.text` section when performing program disassembly and reverse engineering in Chapter 2.

The `.idata` Section

The `.idata` section ❻, also called *imports*, contains the *Import Address Table (IAT)*, which lists dynamically linked libraries and their functions. The IAT is among the most important PE structures to inspect when initially approaching a PE binary for analysis because it reveals the library calls a program makes, which in turn can betray the malware's high-level functionality.

The Data Sections

The data sections in a PE file can include sections like `.rsrc`, `.data`, and `.rdata`, which store items such as mouse cursor images, button skins, audio, and other media used by a program. For example, the `.rsrc` section ❼ in Figure 1-1 contains printable character strings that a program uses to render text as strings.

The information in the `.rsrc` (resources) section can be vital to malware analysts because by examining the printable character strings, graphical images, and other assets in a PE file, they can gain vital clues about the file's functionality. In “Examining Malware Images” on page 7, you'll learn how to use the `icoutils` toolkit (including `icotool` and `wrestool`) to extract graphical images from malware binaries' resources sections. Then, in “Examining Malware Strings” on page 8, you'll learn how to extract printable strings from malware resources sections.

The `.reloc` Section

A PE binary's code is not *position independent*, which means it will not execute correctly if it's moved from its intended memory location to a new memory location. The `.reloc` section ⑧ gets around this by allowing code to be moved without breaking. It tells the Windows operating system to translate memory addresses in a PE file's code if the code has been moved so that the code still runs correctly. These translations usually involve adding or subtracting an offset from a memory address.

Although a PE file's `.reloc` section may well contain information you'll want to use in your malware analysis, we won't discuss it further in this book because our focus is on applying machine learning and data analysis to malware, not doing the kind of hardcore reverse engineering that involves looking at relocations.

Dissecting the PE Format Using `pefile`

The `pefile` Python module, written and maintained by Ero Carrera, has become an industry-standard malware analysis library for dissecting PE files. In this section, I show you how to use `pefile` to dissect `ircbot.exe`. The `ircbot.exe` file can be found on the virtual machine accompanying this book in the directory `~/malware_data_science/ch1/data`. Listing 1-1 assumes that `ircbot.exe` is in your current working directory.

Enter the following to install the `pefile` library so that we can import it within Python:

```
$ pip install pefile
```

Now, use the commands in Listing 1-1 to start Python, import the `pefile` module, and open and parse the PE file `ircbot.exe` using `pefile`.

```
$ python
>>> import pefile
>>> pe = pefile.PE("ircbot.exe")
```

Listing 1-1: Loading the `pefile` module and parsing a PE file (`ircbot.exe`)

We instantiate `pefile.PE`, which is the core class implemented by the `PE` module. It parses `PE` files so that we can examine their attributes. By calling the `PE` constructor, we load and parse the specified `PE` file, which is *ircbot.exe* in this example. Now that we've loaded and parsed our file, run the code in Listing 1-2 to pull information from *ircbot.exe*'s `PE` fields.

```
# based on Ero Carrera's example code (pefile library author)
for section in pe.sections:
    print (section.Name, hex(section.VirtualAddress),
          hex(section.Misc_VirtualSize), section.SizeOfRawData )
```

Listing 1-2: Iterating through the PE file's sections and printing information about them

Listing 1-3 shows the output.

```
('.text\x00\x00\x00', 0x1000, 0x32830, 207360)
('.rdata\x00\x00', '0x34000', '0x427a', 17408)
('.data\x00\x00\x00', '0x39000', '0x5cff8', 10752)
('.idata\x00\x00', '0x96000', '0xbb0', 3072)
('.reloc\x00\x00', '0x97000', '0x211d', 8704)
```

Listing 1-3: Pulling section data from ircbot.exe using Python's pefile module

As you can see in Listing 1-3, we've pulled data from five different sections of the `PE` file: `.text`, `.rdata`, `.data`, `.idata`, and `.reloc`. The output is given as five tuples, one for each `PE` section pulled. The first entry on each line identifies the `PE` section. (You can ignore the series of `\x00` null bytes, which are simply C-style null string terminators.) The remaining fields tell us what each section's memory utilization will be once it's loaded into memory and where in memory it will be found once loaded.

For example, `0x1000` ❶ is the *base virtual memory address* where these sections will be loaded. Think of this as the section's base memory address. The `0x32830` ❷ in the *virtual size* field specifies the amount of memory required by the section once loaded. The `207360` ❸ in the third field represents the amount of data the section will take up within that chunk of memory.

In addition to using `pefile` to parse a program's sections, we can also use it to list the DLLs a binary will load, as well as the function calls it will request within those DLLs. We can do this by dumping a `PE` file's IAT. Listing 1-4 shows how to use `pefile` to dump the IAT for *ircbot.exe*.

```
$ python
pe = pefile.PE("ircbot.exe")
for entry in pe.DIRECTORY_ENTRY_IMPORT:
    print entry.dll
    for function in entry.imports:
        print '\t',function.name
```

Listing 1-4: Extracting imports from ircbot.exe

Listing 1-4 should produce the output shown in Listing 1-5 (truncated for brevity).

```
KERNEL32.DLL
    GetLocalTime
    ExitThread
    CloseHandle
    ❶ WriteFile
    ❷ CreateFileA
    ExitProcess
    ❸ CreateProcessA
    GetTickCount
    GetModuleFileNameA
--snip--
```

Listing 1-5: Contents of the IAT of ircbot.exe, showing library functions used by this malware

As you can see in Listing 1-5, this output is valuable for malware analysis because it lists a rich array of functions that the malware declares and will reference. For example, the first few lines of the output tell us that the malware will write to files using `WriteFile` ❶, open files using the `CreateFileA` call ❷, and create new processes using `CreateProcessA` ❸. Although this is fairly basic information about the malware, it's a start in understanding the malware's behavior in more detail.

Examining Malware Images

To understand how malware may be designed to game a target, let's look at the icons contained in its `.rsrc` section. For example, malware binaries are often designed to trick users into clicking them by masquerading as Word documents, game installers, PDF files, and so on. You also find images in the malware suggesting programs of interest to the attackers themselves, such as network attack tools and programs run by attackers for the remote control of compromised machines. I have even seen binaries containing desktop icons of jihadists, images of evil-looking cyberpunk cartoon characters, and images of Kalashnikov rifles. For our sample image analysis, let's consider a malware sample the security company Mandiant identified as having been crafted by a Chinese state-sponsored hacking group. You can find this sample malware in this chapter's data directory under the name *fakepdfmalware.exe*. This sample uses an Adobe Acrobat icon to trick users into thinking it is an Adobe Acrobat document, when in fact it's a malicious PE executable.

Before we can extract the images from the *fakepdfmalware.exe* binary using the Linux command line tool `wrestool`, we first need to create a directory to hold the images we'll extract. Listing 1-6 shows how to do all this.

```
$ mkdir images
$ wrestool -x fakepdfmalware.exe -output=images
$ icotool -x -o images images/*.ico
```

Listing 1-6: Shell commands that extract images from a malware sample

We first use `mkdir images` to create a directory to hold the extracted images. Next, we use `wrestool` to extract image resources (-x) from *fakepdfmalware.exe* to */images* and then use `icoutil` to extract (-x) and convert (-o) any resources in the Adobe *.ico* icon format into *.png* graphics so that we can view them using standard image viewer tools. If you don't have `wrestool` installed on your system, you can download it at <http://www.nongnu.org/icoutils/>.

Once you've used `wrestool` to convert the images in the target executable to the PNG format, you should be able to open them in your favorite image viewer and see the Adobe Acrobat icon at various resolutions. As my example here demonstrates, extracting images and icons from PE files is relatively straightforward and can quickly reveal interesting and useful information about malware binaries. Similarly, we can easily extract printable strings from malware for more information, which we'll do next.

Examining Malware Strings

Strings are sequences of printable characters within a program binary. Malware analysts often rely on strings in a malicious sample to get a quick sense of what may be going on inside it. These strings often contain things like HTTP and FTP commands that download web pages and files, IP addresses and hostnames that tell you what addresses the malware connects to, and the like. Sometimes even the language used to write the strings can hint at a malware binary's country of origin, though this can be faked. You may even find text in a string that explains in leetspeak the purpose of a malicious binary.

Strings can also reveal more technical information about a binary. For example, you may find information about the compiler used to create it, the programming language the binary was written in, embedded scripts or HTML, and so on. Although malware authors can obfuscate, encrypt, and compress all of these traces, even advanced malware authors often leave at least some traces exposed, making it particularly important to examine strings dumps when analyzing malware.

Using the strings Program

The standard way to view all strings in a file is to use the command line tool `strings`, which uses the following syntax:

```
$ strings filepath | less
```

This command prints all strings in a file to the terminal, line by line. Adding `| less` at the end prevents the strings from just scrolling across the terminal. By default, the `strings` command finds all printable strings with a minimum length of 4 bytes, but you can set a different minimum length and change various other parameters, as listed in the command's manual page. I recommend simply using the default minimum string length of 4,

but you can change the minimum string length using the `-n` option. For example, `strings -n 10 filepath` would extract only strings with a minimum length of 10 bytes.

Analyzing Your strings Dump

Now that we dumped a malware program's printable strings, the challenge is to understand what the strings mean. For example, let's say we dump the strings to the *ircbotstring.txt* file for *ircbot.exe*, which we explored earlier in this chapter using the *pefile* library, like this:

```
$ strings ircbot.exe > ircbotstring.txt
```

The contents of *ircbotstring.txt* contain thousands of lines of text, but some of these lines should stick out. For example, Listing 1-7 shows a bunch of lines extracted from the string dump that begin with the word `DOWNLOAD`.

```
[DOWNLOAD]: Bad URL, or DNS Error: %s.
[DOWNLOAD]: Update failed: Error executing file: %s.
[DOWNLOAD]: Downloaded %.1fKB to %s @ %.1fKB/sec. Updating.
[DOWNLOAD]: Opened: %s.
--snip--
[DOWNLOAD]: Downloaded %.1f KB to %s @ %.1f KB/sec.
[DOWNLOAD]: CRC Failed (%d != %d).
[DOWNLOAD]: Filesize is incorrect: (%d != %d).
[DOWNLOAD]: Update: %s (%dKB transferred).
[DOWNLOAD]: File download: %s (%dKB transferred).
[DOWNLOAD]: Couldn't open file: %s.
```

Listing 1-7: The strings output showing evidence that the malware can download files specified by the attacker onto a target machine

These lines indicate that *ircbot.exe* will attempt to download files specified by an attacker onto the target machine.

Let's try analyzing another one. The string dump shown in Listing 1-8 indicates that *ircbot.exe* can act as a web server that listens on the target machine for connections from the attacker.

```
❶ GET
❷ HTTP/1.0 200 OK
  Server: myBot
  Cache-Control: no-cache,no-store,max-age=0
  pragma: no-cache
  Content-Type: %s
  Content-Length: %i
  Accept-Ranges: bytes
  Date: %s %s GMT
  Last-Modified: %s %s GMT
  Expires: %s %s GMT
  Connection: close
  HTTP/1.0 200 OK
❸ Server: myBot
```

```
Cache-Control: no-cache,no-store,max-age=0
pragma: no-cache
Content-Type: %s
Accept-Ranges: bytes
Date: %s %s GMT
Last-Modified: %s %s GMT
Expires: %s %s GMT
Connection: close
HH:mm:ss
ddd, dd MMM yyyy
application/octet-stream
text/html
```

Listing 1-8: The strings output showing that the malware has an HTTP server to which the attacker can connect

Listing 1-8 shows a wide variety of HTTP boilerplates used by *ircbot.exe* to implement an HTTP server. It's likely that this HTTP server allows the attacker to connect to a target machine via HTTP to issue commands, such as the command to take a screenshot of the victim's desktop and send it back to the attacker. We see evidence of HTTP functionality throughout the listing. For example, the GET method ❶ requests data from an internet resource. The line HTTP/1.0 200 OK ❷ is an HTTP string that returns the status code 200, indicating that all went well with an HTTP network transaction, and Server: myBot ❸ indicates that the name of the HTTP server is *myBot*, a giveaway that *ircbot.exe* has a built-in HTTP server.

All of this information is useful in understanding and stopping a particular malware sample or malicious campaign. For example, knowing that a malware sample has an HTTP server that outputs certain strings when you connect to it allows you to scan your network to identify infected hosts.

Summary

In this chapter, you got a high-level overview of static malware analysis, which involves inspecting a malware program without actually running it. You learned about the PE file format that defines Windows *.exe* and *.dll* files, and you learned how to use the Python library *pefile* to dissect a real-world malware *ircbot.exe* binary. You also used static analysis techniques such as image analysis and strings analysis to extract more information from malware samples. Chapter 2 continues our discussion of static malware analysis with a focus on analyzing the assembly code that can be recovered from malware.

2

BEYOND BASIC STATIC ANALYSIS: X86 DISASSEMBLY



To thoroughly understand a malicious program, we often need to go beyond basic static analysis of its sections, strings, imports, and images. This involves reverse engineering a program's assembly code. Indeed, disassembly and reverse engineering lie at the heart of deep static analysis of malware samples.

Because reverse engineering is an art, technical craft, and science, a thorough exploration is beyond the scope of this chapter. My goal here is to introduce you to reverse engineering so that you can apply it to malware data science. Understanding this methodology is essential for successfully applying machine learning and data analysis to malware.

In this chapter I start with the concepts you'll need to understand x86 disassembly. Later in the chapter I show how malware authors attempt to bypass disassembly and discuss ways to mitigate these anti-analysis and anti-detection maneuvers. But first, let's review some common disassembly methods as well as the basics of x86 assembly language.

Disassembly Methods

Disassembly is the process by which we translate malware's binary code into valid x86 assembly language. Malware authors generally write malware programs in a high-level language like C or C++ and then use a compiler to compile the source code into x86 binary code. Assembly language is the human-readable representation of this binary code. Therefore, disassembling a malware program into assembly language is necessary to understand how it behaves at its core.

Unfortunately, disassembly is no easy feat because malware authors regularly employ tricks to thwart would-be reverse engineers. In fact, perfect disassembly in the face of deliberate obfuscation is an unsolved problem in computer science. Currently, only approximate, error-prone methods exist for disassembling such programs.

For example, consider the case of *self-modifying code*, or binary code that modifies itself as it executes. The only way to disassemble this code properly is to understand the program logic by which the code modifies itself, but that can be exceedingly complex.

Because perfect disassembly is currently impossible, we must use imperfect methods to accomplish this task. The method we'll use is *linear disassembly*, which involves identifying the contiguous sequence of bytes in the Portable Executable (PE) file that corresponds to its x86 program code and then decoding these bytes. The key limitation of this approach is that it ignores subtleties about how instructions are decoded by the CPU in the course of program execution. Also, it doesn't account for the various obfuscations malware authors sometimes use to make their programs harder to analyze.

The other methods of reverse engineering, which we won't cover here, are the more complex disassembly methods used by industrial-grade disassemblers such as IDA Pro. These more advanced methods actually simulate or reason about program execution to discover which assembly instructions a program might reach as a result of a series of conditional branches.

Although this type of disassembly can be more accurate than linear disassembly, it's far more CPU intensive than linear disassembly methods, making it less suitable for data science purposes where the focus is on disassembling thousands or even millions of programs.

Before you can begin analysis using linear disassembly, however, you'll need to review the basic components of assembly language.

Basics of x86 Assembly Language

Assembly language is the lowest-level human-readable programming language for a given architecture, and it maps closely to the binary instruction format of a particular CPU architecture. A line of assembly language is almost always equivalent to a single CPU instruction. Because assembly is so low level, you can often retrieve it easily from a malware binary by using the right tools.

Gaining basic proficiency in reading disassembled malware x86 code is easier than you might think. This is because most malware assembly code spends most of its time calling into the operating system by way of the Windows operating system's *dynamic-link libraries (DLLs)*, which are loaded into program memory at runtime. Malware programs use DLLs to do most of the real work, such as modifying the system registry, moving and copying files, making network connections and communicating via network protocols, and so on. Therefore, following malware assembly code often involves understanding the ways in which function calls are made from assembly and understanding what various DLL calls do. Of course, things can get much more complicated, but knowing this much can reveal a lot about the malware.

In the following sections I introduce some important assembly language concepts. I also explain some abstract concepts like control flow and control flow graphs. Finally, we disassemble the *ircbot.exe* program and explore how its assembly and control flow can give us insight into its purpose.

There are two major dialects of x86 assembly: Intel and AT&T. In this book I use Intel syntax, which can be obtained from all major disassemblers and is the syntax used in the official Intel documentation of the x86 CPU.

Let's start by taking a look at CPU registers.

CPU Registers

Registers are small data storage units on which x86 CPUs perform computations. Because registers are located on the CPU itself, register access is orders of magnitude faster than memory access. This is why core computational operations, such as arithmetic and condition testing instructions, all target registers. It's also why the CPU uses registers to store information about the status of running programs. Although many registers are available to experienced x86 assembly programmers, we'll just focus on a few important ones here.

General-Purpose Registers

General-purpose registers are like scratch space for assembly programmers. On a 32-bit system, each of these registers contains 32, 16, or 8 bits of space against which we can perform arithmetic operations, bitwise operations, byte order-swapping operations, and more.

In common computational workflows, programs move data into registers from memory or from external hardware devices, perform some operations on this data, and then move the data back out to memory for storage. For example, to sort a long list, a program typically pulls list items in from an array in memory, compares them in the registers, and then writes the comparison results back out to memory.

To understand some of the nuances of the general-purpose register model in the Intel 32-bit architecture, take a look at Figure 2-1.

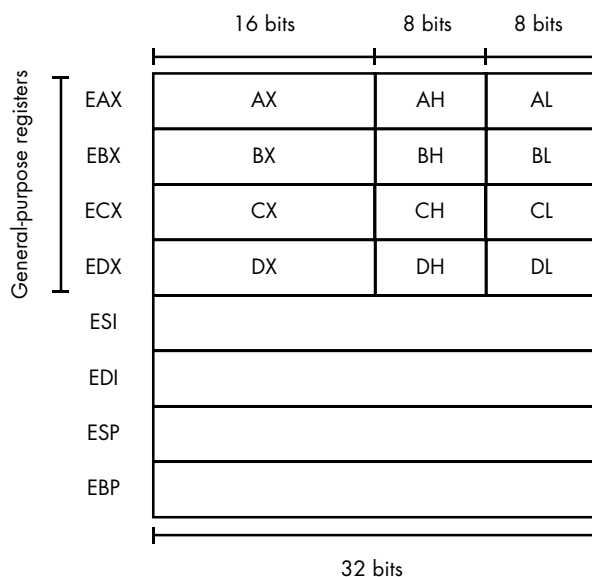


Figure 2-1: Registers in the x86 architecture

The vertical axis shows the layout of the general-purpose registers, and the horizontal axis shows how EAX, EBX, ECX, and EDX are subdivided. EAX, EBX, ECX, and EDX are 32-bit registers that have smaller, 16-bit registers inside them: AX, BX, CX, and DX. As you can see in the figure, these 16-bit registers can be subdivided into upper and lower 8-bit registers: AH, AL, BH, BL, CH, CL, DH, and DL. Although it's sometimes useful to address the subdivisions in EAX, EBX, ECX, and EDX, you'll mostly see direct references to EAX, EBX, ECX, and EDX.

Stack and Control Flow Registers

The stack management registers store critical information about the *program stack*, which is responsible for storing local variables for functions, arguments passed into functions, and control information relating to the program control flow. Let's go over some of these registers.

In simple terms, the ESP register points to the top of the stack for the currently executing function, whereas the EBP register points to the bottom of the stack for the currently executing function. This is crucial information for modern programs, because it means that by referencing data relative to the stack rather than using its absolute address, procedural and object-oriented code can access local variables more gracefully and efficiently.

Although you won't see direct references to the EIP register in x86 assembly code, it's important in security analysis, particularly in the context of vulnerability research and buffer-overflow exploit development. This is because EIP contains the memory address of the currently executing instruction. Attackers can use buffer-overflow exploits to corrupt the value of the EIP register indirectly and take control of program execution.

In addition to its role in exploitation, EIP is also important in the analysis of malicious code deployed by malware. Using a debugger we can inspect EIP's value at any moment, which helps us understand what code malware is executing at any particular time.

EFLAGS is a status register that contains CPU *flags*, which are bits that store status information about the state of the currently executing program. The EFLAGS register is central to the process of making *conditional branches*, or changes in execution flow resulting from the outcome of if/then-style program logic, within x86 programs. Specifically, whenever an x86 assembly program checks whether some value is greater or less than zero and then jumps to a function based on the outcome of this test, the EFLAGS register plays an enabling role, as described in more detail in “Basic Blocks and Control Flow Graphs” on page 19.

Arithmetic Instructions

Instructions operate on general-purpose registers. You can perform simple computations with the general-purpose registers using arithmetic instructions. For example, `add`, `sub`, `inc`, `dec`, and `mul` are examples of arithmetic instructions you'll encounter frequently in malware reverse engineering. Table 2-1 lists some examples of basic instructions and their syntax.

Table 2-1: Arithmetic Instructions

Instructions	Description
<code>add ebx, 100</code>	Adds 100 to the value in EBX and then stores the result in EBX
<code>sub ebx, 100</code>	Subtracts 100 from the value in EBX and then stores the result in EBX
<code>inc ah</code>	Increments the value in AH by 1
<code>dec al</code>	Decrements the value in AL by 1

The `add` instruction adds two integers and stores the result in the first operand specified, whether this is a memory location or a register according to the following syntax. Keep in mind only one argument can be a memory location. The `sub` instruction is similar to `add`, except it subtracts integers. The `inc` instruction increments a register or memory location's integer value, whereas `dec` decrements a register or memory location's integer value.

Data Movement Instructions

The x86 processor provides a robust set of instructions for moving data between registers and memory. These instructions provide the underlying mechanisms that allow us to manipulate data. The staple memory movement instruction is the `mov` instruction. Table 2-2 shows how you can use the `mov` instruction to move data around.

Table 2-2: Data Movement Instructions

Instructions	Description
<code>mov ebx, eax</code>	Moves the value in register EAX into register EBX
<code>mov eax, [0x12345678]</code>	Moves the data at memory address 0x12345678 into the EAX register
<code>mov edx, 1</code>	Moves the value 1 into the register EDX
<code>mov [0x12345678], eax</code>	Moves the value in EAX into the memory location 0x12345678

Related to the `mov` instruction, the `lea` instruction loads the absolute memory address specified into the register used for getting a pointer to a memory location. For example, `lea edx, [esp-4]` subtracts 4 from the value in ESP and loads the resulting value into EDX.

Stack Instructions

The *stack* in x86 assembly is a data structure that allows you to push and pop values onto and off of it. This is similar to how you would add and remove plates on and off the top of a stack of plates.

Because control flow is often expressed through C-style function calls in x86 assembly and because these function calls use the stack to pass arguments, allocate local variables, and remember what part of the program to return to after a function finishes executing, the stack and control flow need to be understood together.

The `push` instruction pushes values onto the program stack when the programmer wants to save a register value onto the stack, and the `pop` instruction deletes values from the stack and places them into a designated register.

The `push` instruction uses the following syntax to perform its operations:

```
push 1
```

In this example, the program points the stack pointer (the register ESP) to a new memory address, thereby making room for the value (1), which is now stored at the top location on the stack. Then it copies the value from the argument to the memory location the CPU has just made room for on the top of the stack.

Let's contrast this with `pop`:

```
pop eax
```

The program uses `pop` to pop the top value off the stack and move it into a specified register. In this example, `pop eax` pops the top value off the stack and moves it into `eax`.

An unintuitive but important detail to understand about the x86 program stack is that it grows downward in memory, so that the highest value on the stack is actually stored at the lowest address in stack memory. This

becomes very important to remember when you analyze assembly code that references data stored on the stack, as it can quickly get confusing unless you know the stack's memory layout.

Because the x86 stack grows downward in memory, when the push instruction allocates space on the program stack for a new value, it decrements the value of ESP so that it points to a lower location in memory and then copies a value from the target register into that memory location, starting at the top address of the stack and growing up. Conversely, the pop instruction actually copies the top value off of the stack and then increments the value of ESP so it points to a higher memory location.

Control Flow Instructions

An x86 program's *control flow* defines the network of possible instruction execution sequences a program may execute, depending on the data, device interactions, and other inputs the program might receive. Control flow instructions define a program's control flow. They are more complicated than stack instructions but still quite intuitive. Because control flow is often expressed through C-style function calls in x86 assembly, the stack and control flow are closely related. They're also related because these function calls use the stack to pass arguments, allocate local variables, and remember what part of the program to return to after a function finishes executing.

The call and ret control flow instructions are the most important in terms of how programs call functions in x86 assembly and how programs return from functions after these functions are done executing.

The call instruction calls a function. Think of this as a function you might write in a higher-level language like C to allow the program to return to the instruction after the call instruction is invoked and the function has finished executing. You can invoke the call instruction using the following syntax, where *address* denotes the memory location where the function's code begins:

```
call address
```

The call instruction does two things. First, it pushes the address of the instruction that will execute after the function call returns onto the top of the stack so that the program knows what address to return to after the called function finishes executing. Second, call replaces the current value of EIP with the value specified by the *address* operand. Then, the CPU begins execution at the new memory location pointed to by EIP.

Just as call initiates a function call, the ret instruction completes it. You can use the ret instruction on its own and without any parameter, as shown here:

```
ret
```

When invoked, `ret` pops the top value off the stack, which we expect to be the saved program counter value (EIP) that the `call` instruction pushed onto the stack when the `call` instruction was invoked. Then it places the popped program counter value back into EIP and resumes execution.

The `jmp` instruction is another important control flow construction, which operates more simply than `call`. Instead of worrying about saving EIP, `jmp` simply tells the CPU to move to the memory address specified as its parameter and begin execution there. For example, `jmp 0x12345678` tells the CPU to start executing the program code stored at memory location `0x12345678` on the next instruction.

You may be wondering how you can make `jmp` and `call` instructions execute in a conditional way, such as “if the program has received a network packet, execute the following function.” The answer is that x86 assembly doesn’t have high-level constructs like `if`, `then`, `else`, `else if`, and so on. Instead, branching to an address within a program’s code typically requires two instructions: a `cmp` instruction, which checks the value in some register against some test value and stores the result of that test in the EFLAGS register, and a conditional branch instruction.

Most conditional branch instructions start with a *j*, which allows the program to jump to a memory address, and are post-fixed with letters that stand for the condition being tested. For example, `jge` tells the program to jump if greater than or equal to. This means that the value in the register being tested must be greater than or equal to the test value.

The `cmp` instruction uses the following syntax:

cmp register, memory location, or literal, register, memory location, or literal

As stated earlier, `cmp` compares the value in the specified general-purpose register with *value* and then stores the result of that comparison in the EFLAGS register.

The various conditional `jmp` instructions are then invoked as follows:

j address*

As you can see, we can prefix *j* to any number of conditional test instructions. For example, to jump only if the value tested is greater than or equal to the value in the register, use the following instruction:

jge address

Note that unlike the case of the `call` and `ret` instructions, the `jmp` family of instructions never touches the program stack. In fact, in the case of the `jmp` family of instructions, the x86 program is responsible for tracking its own execution flow and potentially saving or deleting information about what addresses it has visited and where it should return to after a particular sequence of instructions has executed.

Basic Blocks and Control Flow Graphs

Although x86 programs look sequential when we scroll through their code in a text editor, they actually have loops, conditional branches, and unconditional branches (control flow). All of these give each x86 program a *network* structure. Let's use the simple toy assembly program in Listing 2-1 to see how this works.

```
setup: # symbol standing in for address of instruction on the next line
❶ mov eax, 10
loopstart: # symbol standing in for address of the instruction on the next
line
❷ sub eax, 1
❸ cmp 0, eax
jne $loopstart
loopend: # symbol standing in for address of the instruction on the next line
mov eax, 1
# more code would go here
```

Listing 2-1: Assembly program for understanding control flow graph

As you can see, this program initializes a counter to the value 10, stored in register EAX ❶. Next, it does a loop in which the value in EAX is decremented by 1 ❷ on each iteration. Finally, once EAX has reached a value of 0 ❸, the program breaks out of the loop.

In the language of control flow graph analysis, we can think of these instructions as comprising three basic blocks. A *basic block* is a sequence of instructions that we know will always execute contiguously. In other words, a basic block always ends with either a branching instruction or an instruction that is the target of a branch, and it always begins with either the first instruction of the program, called the program's *entry point*, or a branch target.

In Listing 2-1, you can see where the basic blocks of our simple program begin and end. The first basic block is composed of the instruction `mov eax, 10` under `setup:`. The second basic block is composed of lines beginning with `sub eax, 1` through `jne $loopstart` under `loopstart:`, and the third starts at `mov eax, 1` under `loopend:`. We can visualize the relationships between the basic blocks using the graph in Figure 2-2. (We use the term *graph* synonymously with the term *network*; in computer science, these terms are interchangeable.)

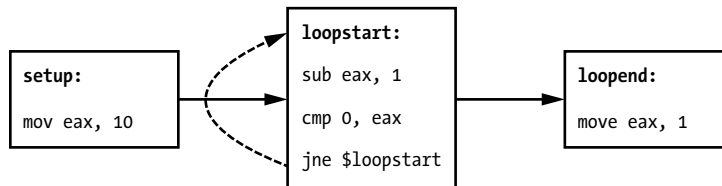


Figure 2-2: A visualization of the control flow graph of our simple assembly program

If one basic block can ever flow into another basic block, we connect it, as shown in Figure 2-2. The figure shows that the setup basic block leads to the loopstart basic block, which repeats 10 times before it transitions to the loopend basic block. Real-world programs have control flow graphs such as these, but they're much more complicated, with thousands of basic blocks and thousands of interconnections.

Disassembling ircbot.exe Using pefile and capstone

Now that you have a good understanding of the basics of assembly language, let's disassemble the first 100 bytes of *ircbot.exe*'s assembly code using linear disassembly. To do this, we'll use the open source Python libraries *pefile* (introduced in Chapter 1) and *capstone*, which is an open source disassembly library that can disassemble 32-bit x86 binary code. You can install both of these libraries with *pip* using the following commands:

```
pip install pefile
pip install capstone
```

Once these two libraries are installed, we can leverage them to disassemble *ircbot.exe* using the code in Listing 2-2.

```
#!/usr/bin/python
import pefile
from capstone import *

# load the target PE file
pe = pefile.PE("ircbot.exe")

# get the address of the program entry point from the program header
entrypoint = pe.OPTIONAL_HEADER.AddressOfEntryPoint

# compute memory address where the entry code will be loaded into memory
entrypoint_address = entrypoint+pe.OPTIONAL_HEADER.ImageBase

# get the binary code from the PE file object
binary_code = pe.get_memory_mapped_image()[entrypoint:entrypoint+100]

# initialize disassembler to disassemble 32 bit x86 binary code
disassembler = Cs(CS_ARCH_X86, CS_MODE_32)

# disassemble the code
for instruction in disassembler.disasm(binary_code, entrypoint_address):
    print "%s\t%s" %(instruction.mnemonic, instruction.op_str)
```

Listing 2-2: Disassembling ircbot.exe

This should produce the following output:

```
❶ push    ebp
   mov     ebp, esp
```

```

push    -1
push    0x437588
push    0x41982c
❷ mov    eax, dword ptr fs:[0]
push    eax
mov     dword ptr fs:[0], esp
❸ add    esp, -0x5c
push    ebx
push    esi
push    edi
mov     dword ptr [ebp - 0x18], esp
❹ call   dword ptr [0x496308]
--snip--

```

Don't worry about understanding all of the instructions in the disassembly output: that would involve an understanding of assembly that goes beyond the scope of this book. However, you should feel comfortable with many of the instructions in the output and have some sense of what they do. For example, the malware pushes the value in register EBP onto the stack ❶, saving its value. Then it proceeds to move the value in ESP into EBP and pushes some numerical values onto the stack. The program moves some data in memory into the EAX register ❷, and it adds the value -0x5c to the value in the ESP register ❸. Finally, the program uses the call instruction to call a function stored at the memory address 0x496308 ❹.

Because this is not a book on reverse engineering, I won't go into any more depth here about what the code means. What I've presented is a start to understanding how assembly language works. For more information on assembly language, I recommend the Intel programmer's manual at <http://www.intel.com/content/www/us/en/processors/architectures-software-developer-manuals.html>.

Factors That Limit Static Analysis

In this chapter and Chapter 1, you learned about a variety of ways in which static analysis techniques can be used to elucidate the purpose and methods of a newly discovered malicious binary. Unfortunately, static analysis has limitations that render it less useful in some circumstances. For example, malware authors can employ certain offensive tactics that are far easier to implement than to defend against. Let's take a look at some of these offensive tactics and see how to defend against them.

Packing

Malware *packing* is the process by which malware authors compress, encrypt, or otherwise mangle the bulk of their malicious program so that it appears inscrutable to malware analysts. When the malware is run, it unpacks itself and then begins execution. The obvious way around malware packing is to actually run the malware in a safe environment, a dynamic analysis technique I'll cover in Chapter 3.

NOTE

Software packing is also used by benign software installers for legitimate reasons. Benign software authors use packing to deliver their code because it allows them to compress program resources to reduce software installer download sizes. It also helps them thwart reverse engineering attempts by business competitors, and it provides a convenient way to bundle many program resources within a single installer file.

Resource Obfuscation

Another anti-detection, anti-analysis technique malware authors use is *resource obfuscation*. They obfuscate the way program resources, such as strings and graphical images, are stored on disk, and then deobfuscate them at runtime so they can be used by the malicious program. For example, a simple obfuscation would be to add a value of 1 to all bytes in images and strings stored in the PE resources section and then subtract 1 from all of this data at runtime. Of course, any number of obfuscations are possible here, all of which make life difficult for malware analysts attempting to make sense of a malware binary using static analysis.

As with packing, one way around resource obfuscation is to just run the malware in a safe environment. When this is not an option, the only mitigation for resource obfuscation is to actually figure out the ways in which malware has obfuscated its resources and to manually deobfuscate them, which is what professional malware analysts often do.

Anti-disassembly Techniques

A third group of anti-detection, anti-analysis techniques used by malware authors are *anti-disassembly* techniques. These techniques are designed to exploit the inherent limitations of state-of-the-art disassembly techniques to hide code from malware analysts or make malware analysts think that a block of code stored on disk contains different instructions than it actually does.

An example of an anti-disassembly technique involves branching to a memory location that the malware author's disassemblers will interpret as a different instruction, essentially hiding the malware's true instructions from reverse engineers. Anti-disassembly techniques have huge potential and there's no perfect way to defend against them. In practice, the two main defenses against these techniques are to run malware samples in a dynamic environment and to manually figure out where anti-disassembly strategies manifest within a malware sample and how to bypass them.

Dynamically Downloaded Data

A final class of anti-analysis techniques malware authors use involves externally sourcing data and code. For example, a malware sample may load code dynamically from an external server at malware startup time. If this is the case, static analysis will be useless against such code. Similarly, malware may source decryption keys from external servers at startup time and then use these keys to decrypt data or code that will be used in the malware's execution.

Obviously, if the malware is using an industrial-strength encryption algorithm, static analysis will not be sufficient to recover the encrypted data and code. Such anti-analysis and anti-detection techniques are quite powerful, and the only way around them is to acquire the code, data, or private keys on the external servers by some means and then use them in one's analysis of the malware in question.

Summary

This chapter introduced x86 assembly code analysis and demonstrated how we can perform disassembly-based static analysis on *ircbot.exe* using open source Python tools. Although this is not meant to be a complete primer on x86 assembly, you should now feel comfortable enough that you have a starting place for figuring out what's going on in a given malware assembly dump. Finally, you learned ways in which malware authors can defend against disassembly and other static analysis techniques, and how you can mitigate these anti-analysis and anti-detection strategies. In Chapter 3, you'll learn to conduct dynamic malware analysis that makes up for many of the weaknesses of static malware analysis.

3

A BRIEF INTRODUCTION TO DYNAMIC ANALYSIS



In Chapter 2, you learned advanced static analysis techniques to disassemble the assembly code recovered from malware.

Although static analysis can be an efficient way to gain useful information about malware by studying its different components on disk, it doesn't allow us to observe malware behavior.

In this chapter, you'll learn about the basics of dynamic malware analysis. Unlike static analysis, which focuses on what malware looks like in file form, dynamic analysis consists of running malware in a safe, contained environment to see how it behaves. This is like introducing a dangerous bacterial strain into a sealed environment to see its effects on other cells.

Using dynamic analysis, we can get around common static analysis hurdles, such as packing and obfuscation, as well as gain more direct insight into the purpose of a given malware sample. We begin by exploring basic dynamic analysis techniques, their relevance to malware data science, and their applications. We use open source tools like *malwr.com* to study examples of dynamic analysis in action. Note that this is a condensed

survey of the topic and is not intended to be comprehensive. For a more complete introduction, check out *Practical Malware Analysis* (No Starch Press, 2012).

Why Use Dynamic Analysis?

To understand why dynamic analysis matters, let's consider the problem of packed malware. Recall that packing malware refers to compressing or obfuscating a malware's x86 assembly code to hide the malicious nature of the program. A packed malware sample unpacks itself when it infects a target machine so that the code can execute.

We could try to disassemble a packed or obfuscated malware sample using the static analysis tools discussed in Chapter 2, but this is a laborious process. For example, with static analysis we'd first have to find the location of the obfuscated code in the malware file. Then we'd have to find the location of the deobfuscation subroutines that deobfuscate this code so that it can run. After locating the subroutines, we'd have to figure out how this deobfuscation procedure works in order to perform it on the code. Only then could we begin the actual process of reverse engineering the malicious code.

A simple yet clever alternative to this process is to execute the malware in a safe, contained environment called a *sandbox*. Running malware in a sandbox allows it to unpack itself as it would when infecting a real target. By simply running malware, we can find out what servers a particular malware binary connects to, what system configuration parameters it changes, and what device I/O (input/output) it attempts to perform.

Dynamic Analysis for Malware Data Science

Dynamic analysis is useful not only for malware reverse engineering but also for malware data science. Because dynamic analysis reveals what a malware sample *does*, we can compare its actions to those of other malware samples. For example, because dynamic analysis shows what files malware samples write to disk, we can use this data to connect those malware samples that write similar filenames to disk. These kinds of clues help us categorize malware samples based on common traits. They can even help us identify malware samples that were authored by the same groups or are part of the same campaigns.

Most importantly, dynamic analysis is useful for building machine learning-based malware detectors. We can train a detector to distinguish between malicious and benign binaries by observing their behaviors during dynamic analysis. For example, after observing thousands of dynamic analysis logs from both malware and benign files, a machine learning system can learn that when *msword.exe* launches a process named *powershell.exe*, this action is malicious, but that when *msword.exe* launches Internet Explorer, this is probably harmless. Chapter 8 will go into more detail about how we

can build malware detectors using data based on both static and dynamic analysis. But before we create sophisticated malware detectors, let's look at some basic tools for dynamic analysis.

Basic Tools for Dynamic Analysis

You can find a number of free, open source tools for dynamic analysis online. This section focuses on *malwr.com* and CuckooBox. The *malwr.com* site has a web interface that allows you to submit binaries for dynamic analysis for free. CuckooBox is a software platform that lets you set up your own dynamic analysis environment so that you can analyze binaries locally. The creators of the CuckooBox platform also operate *malwr.com*, and *malwr.com* runs CuckooBox behind the scenes. Therefore, learning how to analyze results on *malwr.com* will allow you to understand CuckooBox results.

NOTE

At print time, malwr.com's CuckooBox interface was down for maintenance. Hopefully by the time you read this section the site will be back up. If not, the information provided in this chapter can be applied to output from your own CuckooBox instance, which you can set up by following the instructions at <https://cuckoosandbox.org/>.

Typical Malware Behaviors

The following are the major categories of actions a malware sample may take upon execution:

Modifying the file system For example, writing a device driver to disk, changing system configuration files, adding new programs to the file system, and modifying registry keys to ensure the program auto-starts

Modifying the Windows registry to change the system configuration For example, changing firewall settings

Loading device drivers For example, loading a device driver that records user keystrokes

Network actions For example, resolving domain names and making HTTP requests

We'll examine these behaviors in more detail using a malware sample and analyzing its report on *malwr.com*.

Loading a File on malwr.com

To run a malware sample through *malwr.com*, navigate to <https://malwr.com/> and then click the **Submit** button to upload and submit a binary for analysis. We'll use a binary whose SHA256 hash starts with the characters *d676d95*, which you can find in the data directory accompanying this chapter. I encourage you to submit this binary to *malwr.com* and inspect the results yourself as we go. The submit page is shown in Figure 3-1.



By submitting the file, you automatically accept our Terms of Service.

☒ Analyze the sample
☒ Share the sample
☐ Private

6 + 5 =

Figure 3-1: The malware sample submission page

After you submit your sample through this form, the site should prompt you to wait for analysis to complete, which typically takes about five minutes. When the results load, you can inspect them to understand what the executable did when it was run in the dynamic analysis environment.

Analyzing Results on malwr.com

The results page for our sample should look something like Figure 3-2.



Quick Overview

Static Analysis

Behavioral Analysis

Network Analysis

Dropped Files

Comment Board (0)

Tags: None

Analysis

CATEGORY	STARTED	COMPLETED
FILE	2016-12-30 11:56:05	2016-12-30 11:58:22

File Details

FILE NAME	wordplugin.exe
FILE SIZE	410112 bytes
FILE TYPE	PE32 executable (GUI) Intel 80386, for MS Windows, UPX compressed
MD5	9559789bd252e90dcd957de683ce1743
SHA1	c8618b0abe866f3c018a950910f006a1bea2a920
SHA256	d676d9dfab6a4242258362b8ff579cfe6e5e6db3f0cdd3e0069ace50f80af1c5
SHA512	bdaca27f3a5b76baa5928940f4dfc81d641540050b26f02cf1ea7d94bdbfcf3630c54f2d0a54

Figure 3-2: The top of the results page for a malware sample on malwr.com

The results for this file illustrate some key aspects of dynamic analysis, which we'll explore next.

Signatures Panel

The first two panels you'll see on the results page are Analysis and File Details. These contain the time the file was run and other static details about the file. The panel I will focus on here is the Signatures panel, shown in Figure 3-3. This panel contains high-level information derived from the file itself and its behavior when it was run in the dynamic analysis environment. Let's discuss what each of these signatures means.

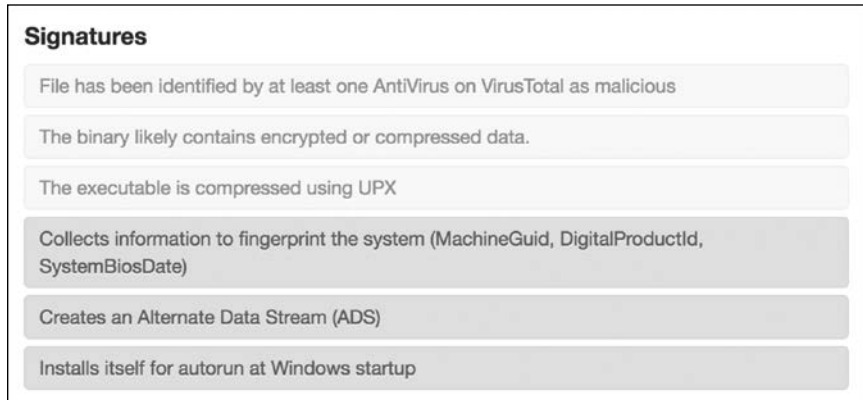


Figure 3-3: The malwr.com signatures that match the behavior of our malware sample

The first three signatures shown in the figure result from static analysis (that is, these are results from the properties of the malware file itself, not its actions). The first signature simply tells us that a number of antivirus engines on the popular antivirus aggregator *VirusTotal.com* marked this file as malware. The second indicates that the binary contains compressed or encrypted data, a common sign of obfuscation. The third tells us that this binary was compressed with the popular UPX packer. Although these static indicators on their own don't tell us what this file does, they do tell us that it's likely malicious. (Note that the color doesn't correspond to static versus dynamic categories; instead, it represents the severity of each rule, with red—the darker gray here—being more suspicious than yellow.)

The next three signatures result from dynamic analysis of the file. The first signature indicates that the program attempts to identify the system's hardware and operating system. The second indicates that the program uses a pernicious feature of Windows known as *Alternate Data Streams (ADS)*, which allows malware to hide data on disk such that it's invisible when using standard file system browsing tools. The third signature indicates that the file changes the Windows registry so that when the system reboots, a program that it specified will automatically execute. This would restart the malware whenever the user reboots their system.

As you can see, even at the level of these automatically triggered signatures, dynamic analysis adds significantly to our knowledge of the file's intended behavior.

Screenshots Panel

Beneath the Signatures panel is the Screenshots panel. This panel shows a screenshot of the dynamic analysis environment desktop as the malware is running. Figure 3-4 shows an example of what this looks like.



Figure 3-4: A screen capture of our malware sample's dynamic behavior

You can see that the malware we're dealing with is *ransomware*, which is a type of malware that encrypts a target's files and forces them to pay up if they want to get their data back. By simply running our malware, we were able to uncover its purpose without resorting to reverse engineering.

Modified System Objects Panel

A row of headings under Screenshots shows the malware sample's network activity. Our binary did not engage in any network communications, but if it had, we would see the hosts it contacted here. Figure 3-5 shows the Summary panel.



Figure 3-5: The Files tab of the Summary pane, showing which files our malware sample modified

This shows which system objects, like files, registry keys, and mutexes, the malware has modified.

Looking at the Files tab in Figure 3-6, it's clear that this ransomware malware has indeed encrypted the user files on disk.



Figure 3-6: File paths in the Files tab of the Summary pane, suggesting that our sample is ransomware

After each file path is a file with a *.locked* extension, which we can infer is the encrypted version of the file it has replaced.

Next, we'll look at the Registry Keys tab, shown in Figure 3-7.

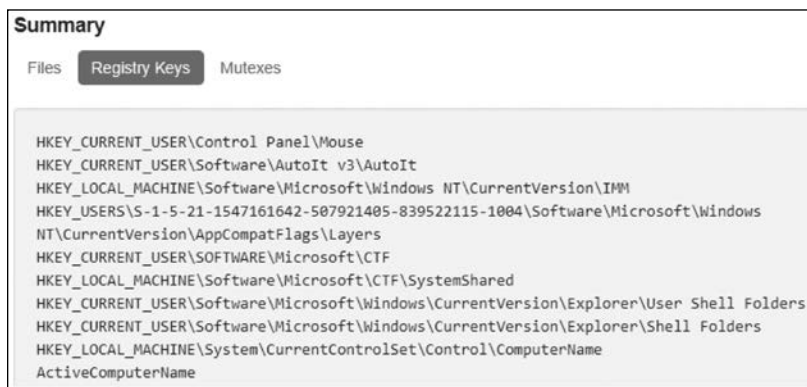


Figure 3-7: The Registry Keys tab of the Summary pane, showing which registry keys our malware sample modified

The registry is a database that Windows uses to store configuration information. Configuration parameters are stored as registry keys, and these keys have associated values. Similar to file paths on the Windows file system, registry keys are backslash delimited. *Malwr.com* shows us what registry keys our malware modified. Although this isn't shown in Figure 3-7, if you view the complete report on *malwr.com*, you should see that one notable registry key our malware changed is HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\Windows\CurrentVersion\Run, which is a registry key that tells Windows to run programs each time a user logs on. It's very likely that our malware modifies this registry to tell Windows to restart the malware every time the system boots up, which ensures that the malware infection persists from reboot to reboot.

The Mutexes tab in the *malwr.com* report contains the names of the mutexes the malware created, as shown in Figure 3-8.



Figure 3-8: The Mutexes tab of the Summary pane, showing which mutexes our malware sample created

Mutexes are lock files that signal that a program has taken possession of some resource. Malware often uses mutexes to prevent itself from infecting a system twice. It turns out that at least one mutex created (*CTF.TimListCache.FMPDefaultS-1-5-21-1547161642-507921405-839522115-1004MUTEX.DefaultS-1-5-21-1547161642-507921405-839522115-1004ShimCacheMutex*) is known by the security community to be associated with malware and may be serving this purpose here.

API Call Analysis

Clicking the Behavioral Analysis tab on the left panel of the *malwr.com* UI, as shown in Figure 3-9, should bring up detailed information about our malware binary's behavior.

This shows what API calls were made by each process launched by the malware, along with their arguments and return values. Perusing this information is time consuming and requires expert knowledge of Windows APIs. Although a detailed discussion of malware API call analysis is beyond the scope of this book, if you're interested in learning more, you can look up individual API calls to discover their effects.

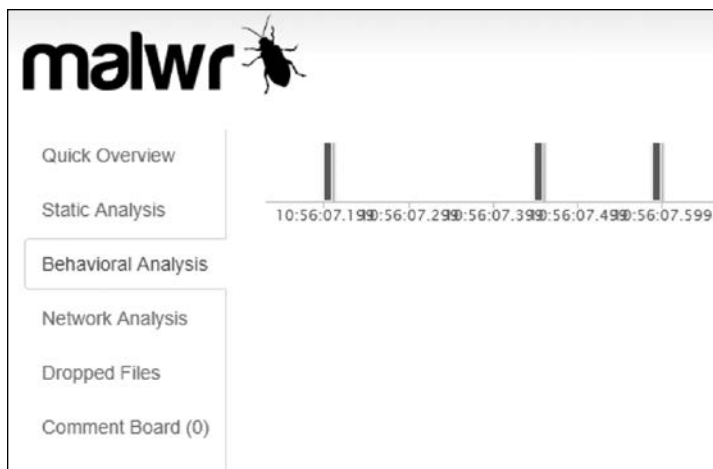


Figure 3-9: The Behavioral Analysis pane of the malwr.com report for our malware sample, showing when API calls were made during the dynamic execution

Although *malwr.com* is a great resource for dynamically analyzing individual malware samples, it isn't great for performing dynamic analysis on large numbers of samples. Executing large numbers of samples in a dynamic environment is important for machine learning and data analysis because it identifies relationships between malware samples' dynamic execution patterns. Creating machine learning systems that can detect instances of malware based on their dynamic execution patterns requires running thousands of malware samples.

In addition to this limitation, *malwr.com* doesn't provide malware analysis results in machine-parseable formats like XML or JSON. To address these issues you must set up and run your own CuckooBox. Fortunately, CuckooBox is free and open source. It also comes with step-by-step instructions for setting up your very own dynamic analysis environment. I encourage you to do so by going to <http://cuckoosandbox.org/>. Now that you understand how to interpret dynamic malware results from *malwr.com*, which uses CuckooBox behind the scenes, you'll also know how to analyze CuckooBox results once you have CuckooBox up and running.

Limitations of Basic Dynamic Analysis

Dynamic analysis is a powerful tool, but it is no malware analysis panacea. In fact, it has serious limitations. One limitation is that malware authors are aware of CuckooBox and other dynamic analysis frameworks and attempt to circumvent them by making their malware fail to execute when it detects that it's running in CuckooBox. The CuckooBox maintainers are aware that malware authors try to do this, so they try to get around attempts by malware to circumvent CuckooBox. This cat-and-mouse game

plays out continuously such that some malware samples will inevitably detect that they are running in dynamic analysis environments and fail to execute when we try to run them.

Another limitation is that even without any circumvention attempts, dynamic analysis might not reveal important malware behaviors. Consider the case of a malware binary that connects back to a remote server upon execution and waits for commands to be issued. These commands may, for example, tell the malware sample to look for certain kinds of files on the victim host, to log keystrokes, or turn on the webcam. In this case, if the remote server sends no commands, or is no longer up, none of these malicious behaviors will be revealed. Because of these limitations, dynamic analysis is not a fix-all for malware analysis. In fact, professional malware analysts combine dynamic and static analysis to achieve the best possible results.

Summary

In this chapter you ran dynamic analysis on a ransomware malware sample with *malwr.com* to analyze the results. You also learned about the advantages and shortcomings of dynamic analysis. Now that you've learned how to conduct basic dynamic analysis, you're ready to dive into malware data science.

The remainder of this book focuses on performing malware data science on static analysis–based malware data. I'll focus on static analysis because it's simpler and easier to get good results with compared to dynamic analysis, making it a good starting place for getting your hands dirty with malware data science. However, in each subsequent chapter I'll also explain how you can apply data science methods to dynamic analysis–based data.

4

IDENTIFYING ATTACK CAMPAIGNS USING MALWARE NETWORKS



Malware network analysis can turn malware datasets into valuable threat intelligence, revealing adversarial attack campaigns, common malware tactics, and sources of malware samples. This approach consists of analyzing the ways in which groups of malware samples are connected by their shared attributes, whether those are embedded IP addresses, hostnames, strings of printable characters, graphics, or similar.

For example, Figure 4-1 shows an example of the power of malware network analysis in a chart that took only seconds to generate with the techniques you'll learn in this chapter.

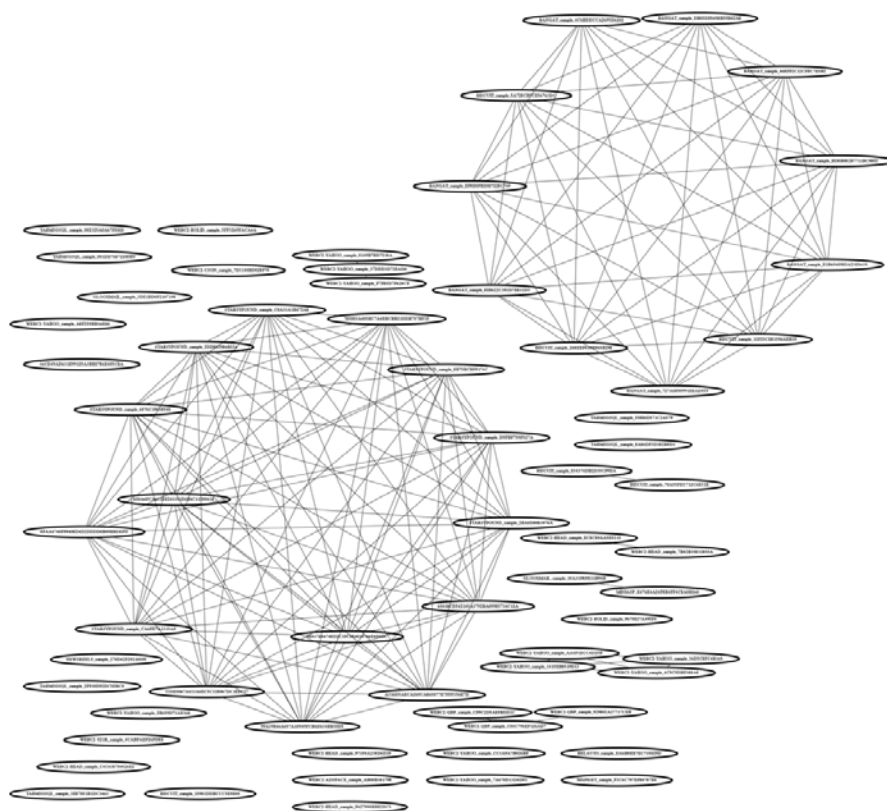


Figure 4-1: Nation-state malware's social network connections revealed via shared attribute analysis

The figure displays a group of nation state–grade malware samples (represented as oval-shaped nodes) and their “social” interconnections (the lines connecting the nodes). The connections are based on the fact that these samples “call back” to the same hostnames and IP addresses, indicating they were deployed by the same attackers. As you’ll learn in this chapter, you can use these connections to help differentiate between a coordinated attack on your organization and a disparate array of criminally motivated attackers.

By the end of the chapter you will have learned:

- The fundamentals of network analysis theory as it relates to extracting threat intelligence from malware
- Ways to use visualizations to identify relationships between malware samples
- How to create, visualize, and extract intelligence from malware networks using Python and various open source toolkits for data analysis and visualization
- How to tie all this knowledge together to reveal and analyze attack campaigns within real-world malware datasets

Nodes and Edges

Before you can perform shared attribute analysis on malware, you need to understand some basics about networks. *Networks* are collections of connected objects (called *nodes*). The connections between these nodes are referred to as *edges*. As abstract mathematical objects, the nodes in a network can represent pretty much anything, as can their edges. What we care about for our purposes is the structure of the interconnections between these nodes and edges, as this can reveal telling details about malware.

When using networks to analyze malware, we can treat each individual malware file as the definition of a node, and we can treat relationships of interest (such as shared code or network behavior) as the definition of an edge. Similar malware files share edges and thus cluster together when we apply force-directed networks (you will see exactly how this works later). Alternatively, we can treat both malware samples and attributes as nodes unto themselves. For example, callback IP addresses have nodes, and so do malware samples. Whenever malware samples call back to a particular IP address, they are connected to that IP address node.

Networks of malware can be more complex than simply a set of nodes and edges. Specifically, they can have *attributes* attached to either nodes or edges, such as the percentage of code that two connected samples share. One common edge attribute is a *weight*, with greater weights indicating stronger connections between samples. Nodes may have their own attributes, such as the file size of the malware samples they represent, but these are typically referred to only as attributes.

Bipartite Networks

A *bipartite network* is one whose nodes can be divided into two partitions (groups), where neither partition contains internal connections. Networks of this type can be used to show shared attributes between malware samples.

Figure 4-2 shows an example of a bipartite network in which malware sample nodes go in the bottom partition, and domain names the samples “call back” to (in order to communicate with the attacker) go in the other partition. Note that callbacks never connect directly to other callbacks, and malware samples never connect directly to other malware samples, as is characteristic of a bipartite network.

As you can see, even such a simple visualization reveals an important piece of intelligence: based on the malware samples’ shared callback servers, we can guess that *sample_014* was probably deployed by the same attacker as *sample_37D*. We can also guess that *sample_37D* and *sample_F7F* probably share the same attacker, and that *sample_014* and *sample_F7F* probably share the same attacker, because they’re connected by sample *sample_37D* (and indeed, the samples shown in Figure 4-2 all come from the same “APT1” Chinese attacker group).

NOTE

We’d like to thank Mandiant and Mila Parkour for curating the APT1 samples and making them available to the research community.

Callback domain names

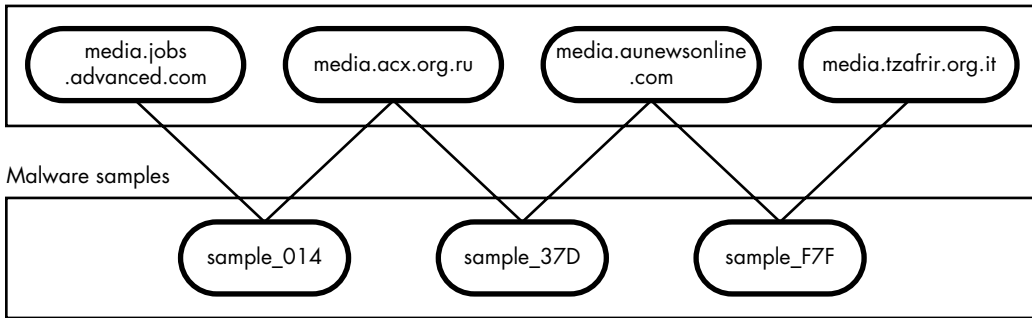


Figure 4-2: A bipartite network. The nodes on top (the attributed partition) are callback domain names. The nodes on the bottom (malware partition) are malware samples.

As the number of nodes and connections in our network grow very large, we might want to see just how the malware samples are related, without having to closely inspect all the attribute connections. We can examine malware sample similarity by creating a bipartite network *projection*, which is a simpler version of a bipartite network in which we link nodes in one partition of the network if they have nodes in the other partition (the *attribute* partition) in common. For example, in the case of the malware samples shown in Figure 4-1, we'd be creating a network in which malware samples are linked if they share callback domain names.

Figure 4-3 shows the projected network of the shared-callback servers of the entire Chinese APT1 dataset referred to previously.

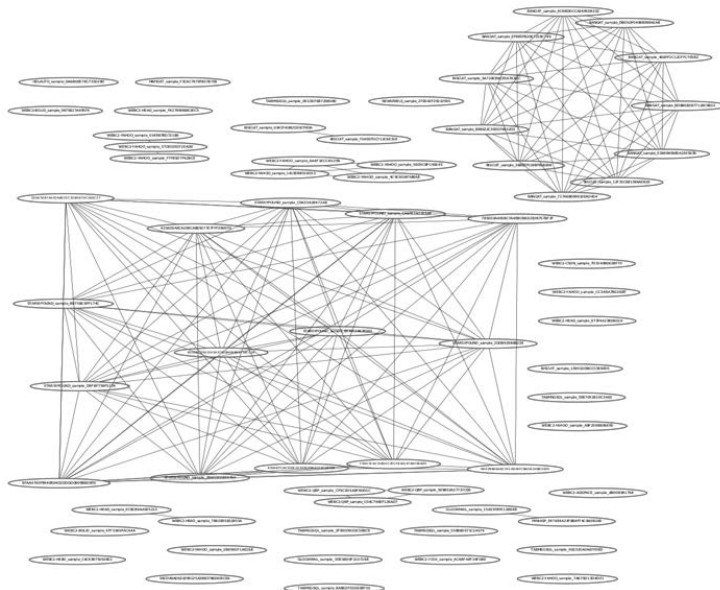


Figure 4-3: A projection of malware samples from the APT1 dataset, showing connections between malware samples only if they share at least one server. The two big clusters were used in two different attack campaigns.

The nodes here are malware samples, and they are linked if they share at least one callback server. By showing connections between malware samples only if they share callback servers, we can begin to see the overall “social network” of these malware samples. As you can see in Figure 4-3, two large groupings exist (the large square cluster in the left-center area and the circular cluster in the top-right area), which upon further inspection turn out to correspond to two different campaigns carried out over the APT1 group’s 10-year history.

Visualizing Malware Networks

As you perform shared attribute analysis of malware using networks, you’ll find that you rely heavily on network visualization software to create the networks like the ones shown thus far. This section introduces how these network visualizations can be created from an algorithmic perspective.

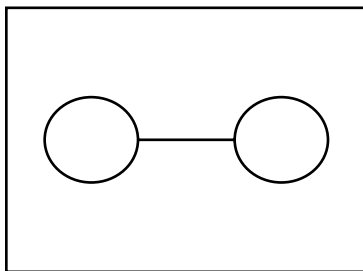
Crucially, the major challenge in doing network visualization is *network layout*, which is the process of deciding where to render each node in a network within a two- or three-dimensional coordinate space, depending on whether you want your visualization to be two- or three-dimensional. When you’re placing nodes on a network, the ideal way is to place them in the coordinate space such that their visual distance from one another is proportional to the shortest-path distance between them in the network. In other words, nodes that are two hops away from one another might be about two inches away from one another, and nodes that are three hops away might be about three inches apart. Doing this allows us to visualize clusters of similar nodes accurately to their actual relationship. As you’ll see in the next section, however, this is often difficult to achieve, especially when you’re working with more than three nodes.

The Distortion Problem

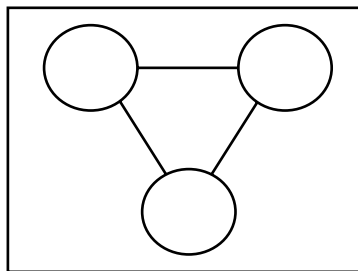
As it turns out, it’s often impossible to solve this network layout problem perfectly. Figure 4-4 illustrates this difficulty.

As you can see in these simple networks, all nodes are connected to all other nodes by edges of equal weights of 1. The ideal layout for these connections would place all nodes equidistant from one another on the page. But as you can see, as we create networks of four and then five nodes, as in (c) and (d), we start to introduce progressively more distortion due to edges of unequal length. Unfortunately, we can only minimize, not eliminate this distortion, and that minimization becomes one of the major goals of network visualization algorithms.

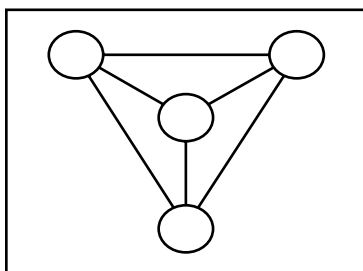
a) Two connected nodes, no distortion, all nodes equal length apart



b) Three connected nodes, no distortion, all nodes equal length apart



c) Four connected nodes, some distortion, some nodes closer than others



d) Five connected nodes, more distortion, heterogeneous node distances

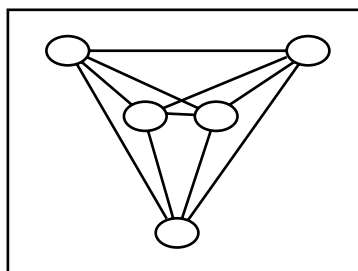


Figure 4-4: Perfect network layout is usually impossible on real-world malware networks. Simple cases like (a) and (b) allow us to lay out all nodes equidistantly. However, (c) adds distortion (the edges are no longer all equal length), and (d) shows even more distortion.

Force-Directed Algorithms

To best minimize layout distortion, computer scientists often use *force-directed* layout algorithms. Force-directed algorithms are based on physical simulations of spring-like forces as well as magnetism. Simulating network edges as physical springs often leads to good node positioning, because the simulated springs push and pull to try to achieve uniform length between nodes and edges. To better visualize this concept, consider how a spring works: when you compress or stretch the spring, it “tries” to get back to its length at equilibrium. These properties correlate well with our desire to have all the edges of our network be equal length. Force-directed algorithms are what we focus on in this chapter.

Building Networks with NetworkX

Now that you have a basic understanding of malware networks, you’re ready to learn how to create networks of malware relationships using the open source NetworkX Python network analysis library and the GraphViz open

source network visualization toolkit. I show you how to programmatically extract malware-related data and then use this data to build, visualize, and analyze networks to represent malware datasets.

Let's begin with NetworkX, which is an open source project maintained by a team centered at Los Alamos National Laboratory and Python's de facto network-processing library (recall that you can install the library dependencies in this chapter, including NetworkX, by entering this chapter's code and data directory and the command `pip install -r requirements.txt`). If you know Python, you should find NetworkX to be surprisingly easy. Use the code in Listing 4-1 to import NetworkX and instantiate a network.

```
#!/usr/bin/python
import networkx

# instantiate a network with no nodes and no edges.
network = networkx.Graph()
```

Listing 4-1: Instantiating a network

This code uses just one function call to the NetworkX Graph constructor to create a network in NetworkX.

NOTE

The NetworkX library uses the term `graph` in place of `network` sometimes, as the two terms are synonymous in computer science—they both indicate a set of nodes connected by edges.

Adding Nodes and Edges

Now that we've instantiated a network, let's add some nodes. A node in a NetworkX network can be any Python object. Here I show you how to add nodes of various types to our network:

```
nodes = ["hello", "world", 1, 2, 3]
for node in nodes:
    network.add_node(node)
```

As shown, we've added five nodes to our network: "hello", "world", 1, 2, and 3.

Then, to add edges, we call `add_edge()`, as shown next:

```
❶ network.add_edge("hello", "world")
network.add_edge(1, 2)
network.add_edge(1, 3)
```

Here, we're connecting some of these five nodes via edges. For example, the first line of code ❶ connects the "hello" and "world" nodes together by creating an edge between them.

Adding Attributes

NetworkX allows us to easily attach attributes to both nodes and edges. To attach an attribute to a node (and to access that attribute later), you can add the attribute as a keyword argument when you add the node to the network, like this:

```
network.add_node(1,myattribute="foo")
```

To add an attribute later, access the network's node dictionary using the following syntax:

```
network.node[1]["myattribute"] = "foo"
```

Then, to access the node, access the node dictionary:

```
print network.node[1]["myattribute"] # prints "foo"
```

As with nodes, you can add attributes to edges using keyword arguments when you add the edges initially, as shown here:

```
network.add_edge("node1", "node2", myattribute="attribute of an edge")
```

Similarly, you can add attributes to edges once they've been added to a network by using the edge dictionary, as shown here:

```
network.edge["node1"]["node2"]["myattribute"] = "attribute of an edge"
```

The edge dictionary is magical in that it allows you to access node attributes the other way around, without having to worry about which node you refer to first, as shown in Listing 4-2.

```
❶ network.edge["node1"]["node2"]["myattribute"] = 321
❷ print network.edge["node2"]["node1"]["myattribute"] # prints 321
```

Listing 4-2: Using the edge dictionary to access node attributes regardless of order

As you can see, the first line sets myattribute on an edge connecting node1 and node2 ❶, and the second line accesses myattribute despite the node1 and node2 references being flipped ❷.

Saving Networks to Disk

To visualize our networks, we need to save them to disk from NetworkX in .dot format—a format commonly used in the network analysis world that can be imported into many network visualization toolkits. To save a network in .dot format, simply call the NetworkX write_dot() function, as shown in Listing 4-3.

```
#!/usr/bin/python
import networkx
from networkx.drawing.nx_agraph import write_dot

# instantiate a network, add some nodes, and connect them
nodes = ["hello", "world", 1, 2, 3]
network = networkx.Graph()
for node in nodes:
    network.add_node(node)
network.add_edge("hello", "world")
write_dot(❶network,❷"network.dot")
```

Listing 4-3: Using write_dot() to save networks to disk

As you can see, at the end of the code, we use the write_dot() function to specify the network we want to save ❶ as well as the path or filename we want to save it to ❷.

Network Visualization with GraphViz

Once we have written a network to disk using the write_dot() NetworkX function, we can visualize the resulting file using GraphViz. GraphViz is the best available command line package for visualizing your networks. It's supported by researchers at AT&T and has become a standard part of the network analysis toolbox used by data analysts. It contains a host of command line network layout tools that can be used to both lay out and render networks. GraphViz comes pre-installed on the virtual machine provided with this book and can also be downloaded at <https://graphviz.gillab.io/download/>. Each GraphViz command line tool ingests networks expressed in .dot format and can be invoked using the following syntax to render a network as a .png file:

```
$ <toolname> <dotfile> -T png -o <outputfile.png>
```

The fdp force-directed graph renderer is one GraphViz network visualization tool. It uses the same basic command line interface as every other GraphViz tool, as shown here:

```
$ fdp apt1callback.dot -T png -o apt1callback.png
```

Here, we specify that we want to use the fdp tool and name the network .dot file we want to lay out, which is apt1callback.dot, found in the ~/ch3/ directory of the data accompanying this book. We specify -T png to indicate the format (PNG) we wish to use. Finally, we specify where we want the output file to be saved using -o apt1callback.png.

Using Parameters to Adjust Networks

The GraphViz tools include many parameters you can use to adjust the way your networks are drawn. Many of these parameters are set using the `-G` command-line flag in the following format:

```
G<parametername>=<parametervalue>
```

Two particularly useful parameters are `overlap` and `splines`. Set `overlap` to `false` to tell GraphViz not to allow any nodes to overlap one another. Use the `splines` parameter to tell GraphViz to draw curved rather than straight lines to make it easier to follow the edges on your networks. The following are some ways to set the `overlap` and `splines` parameters in GraphViz.

Use the following to prevent nodes from overlapping:

```
$ <toolname> <dotfile> -Goverlap=false -T png -o <outputfile.png>
```

Draw edges as curved lines (`splines`) to improve network readability:

```
$ <toolname> <dotfile> -Gsplines=true -T png -o <outputfile.png>
```

Draw edges as curved lines (`splines`) to improve network readability, and don't allow nodes to visually overlap:

```
$ <toolname> <dotfile> -Gsplines=true -Goverlap=false -T png -o <outputfile.png>
```

Note that we simply list one parameter after the other: `-Gsplines=true -Goverlap=false` (the ordering doesn't matter), followed by `-T png -o <outputfile.png>`.

In the next section, I go over some of the most useful GraphViz tools (in addition to `fdp`).

The GraphViz Command Line Tools

Here are some of the available GraphViz tools I have found most useful, as well as some sense of when it is appropriate to use each tool.

fdp

We used the `fdp` layout tool in the previous example, which we used to create a force-directed layout, as described in “Force-Directed Algorithms” on page 40. When you're creating malware networks with fewer than 500 nodes, `fdp` does a good job of revealing network structure in a reasonable amount of time. But when you're working with more than 500 nodes, and especially when connectivity between nodes is complex, you'll find that `fdp` slows down fairly rapidly.

To try out `fdp` on the APT1 shared callback server network shown in Figure 4-3, enter the following from the `ch4` directory of the data accompanying this book (you must have GraphViz installed):

```
$ fdp callback_servers_malware_projection.dot -T png -o fdp_servers.png -
Goverlap=false
```

This command will create a `.png` file (`fdp_servers.png`) that shows a network like the one displayed in Figure 4-5.

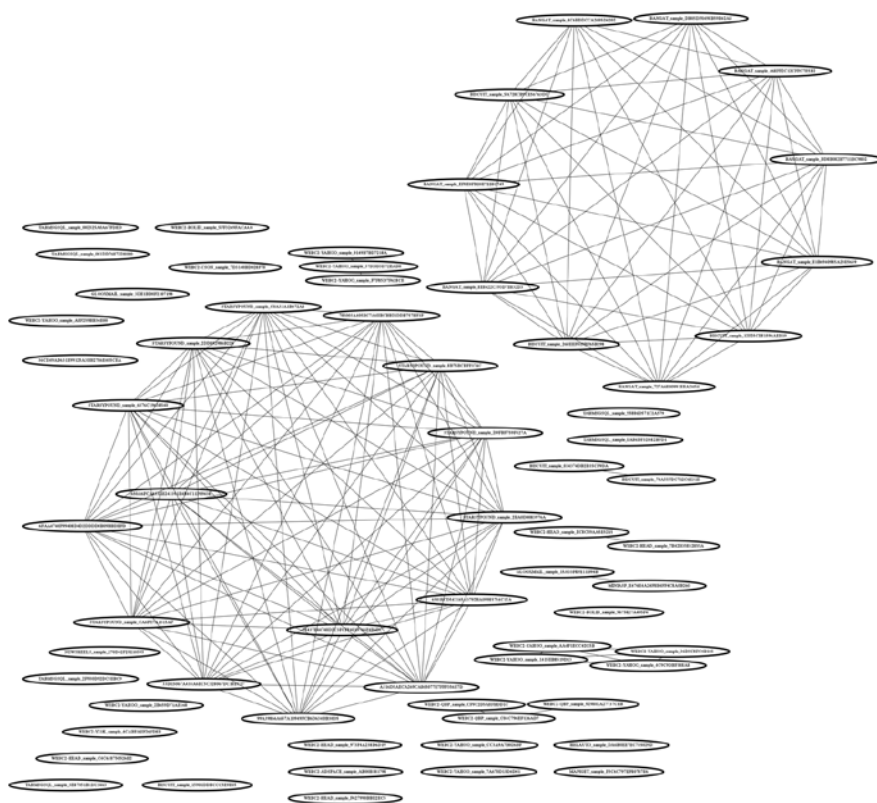


Figure 4-5: Layout of the APT1 samples using the `fdp` tool

The `fdp` layout makes a number of themes apparent in the figure. First, two big clusters of samples are highly interrelated, as clearly seen in the upper-right and lower-left areas of the figure. Second, a number of pairs of samples are related, which can be seen in the lower right. Finally, many samples show no apparent relationship with one another and aren't connected to any other nodes. It's important to recall that this visualization is based on shared callback server relationships between nodes. It's possible that the unconnected samples are related to other samples in the figure by way of other kinds of relationships, such as shared code relationships—relationships we'll explore in Chapter 5.

sfdp

The `sfdp` tool uses roughly the same approach to layout as `fdp`, but it scales better because it creates a hierarchy of simplifications, known as *coarsenings*, where nodes are merged into *supernodes* based on their proximity. After it completes its coarsenings, the `sfdp` tool lays out the merged versions of the graph that have far fewer nodes and edges, which dramatically speeds up the layout process. In this way, `sfdp` is able to perform fewer computations to find the best positions in the network. As a result, `sfdp` can lay out tens of thousands of nodes on a typical laptop, making it by far the best algorithm for laying out very large networks of malware.

This scalability comes at a cost, however: `sfdp` produces layouts that are sometimes less clear than layouts of the same-sized networks in `fdp`. For example, compare Figure 4-6, which I created using `sfdp`, to the network created with `fdp`, shown in Figure 4-5.

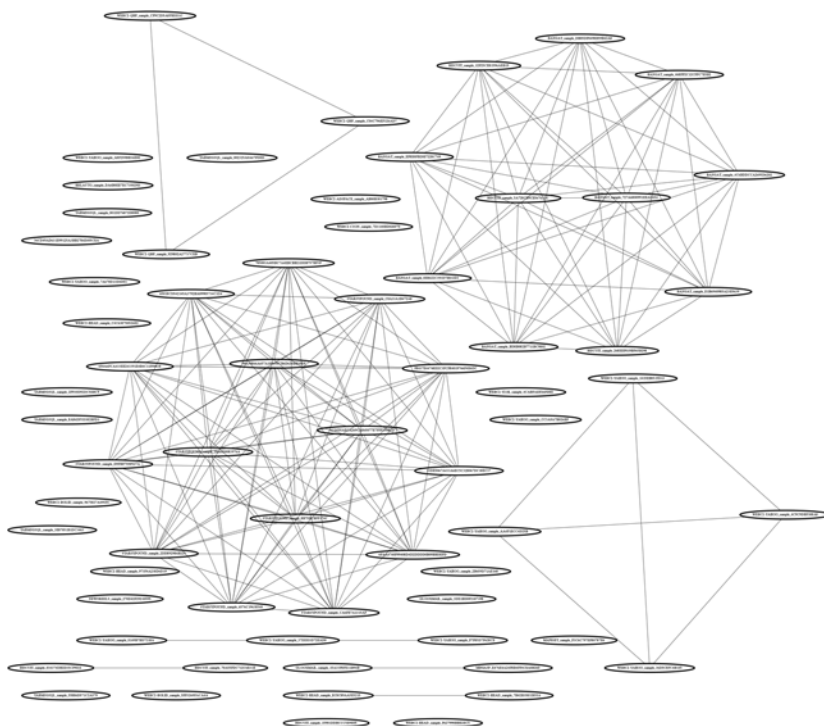


Figure 4-6: Layout of the APT1 samples' shared callback server network using the `sfdp` command

As you can see, there's slightly more noise over each cluster in Figure 4-6, making it slightly harder to see what's going on.

To create this network, enter the `ch4` directory of the data accompanying this book and then enter the following code to produce the `sfdp_servers.png` image file shown in Figure 4-6:

```
$ sfdp callback_servers_malware_projection.dot -T png -o sfdp_servers.png -Goverlap=false
```

Note how the first item in this code specifies that we're using the tool `sfdp`, as opposed to `fdp` from before. Everything else is the same, save the output filename.

neato

The `neato` tool is the GraphViz implementation of a different force-directed network layout algorithm that creates simulated springs between all nodes (including unconnected nodes) to help push things to ideal positions, but at the cost of additional computation. It's hard to know when `neato` will produce the best layout for a given network: my recommendation is that you try it, in conjunction with `fdp`, and see which layout you like more. Figure 4-7 shows what the `neato` layout looks like on the APT1 shared callback server network.

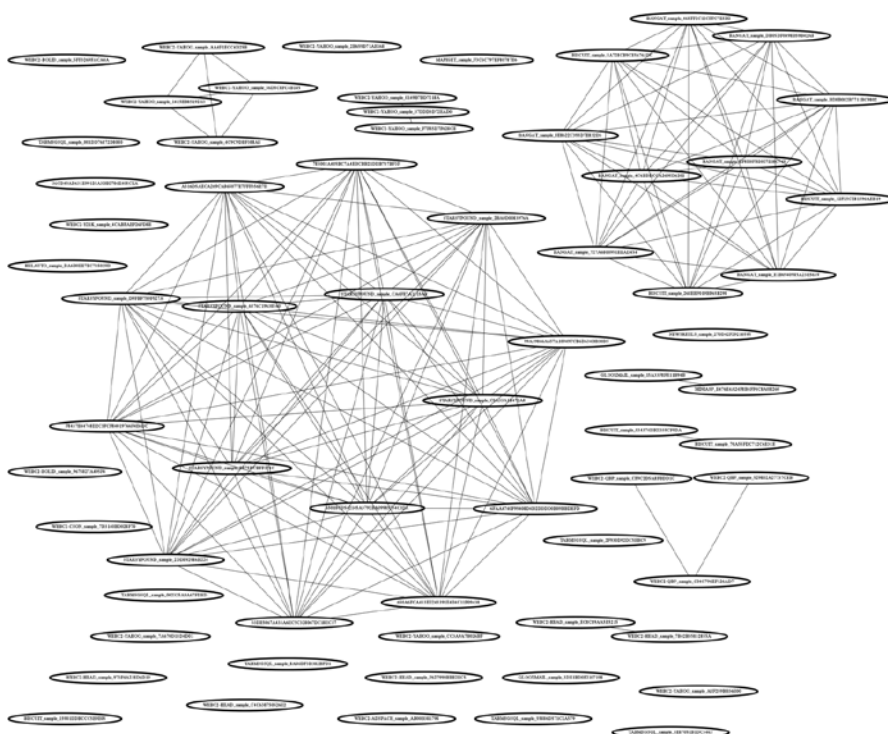


Figure 4-7: Layout of the APT1 shared callback server network using the `neato` layout

As you can see, in this case `neato` produces a similar network layout to those produced by `fdp` and `sfdp`. For some datasets, however, you'll find that `neato` produces a better or worse layout—you just have to try it with your dataset and see. To try out `neato`, enter the following from the `ch4` directory of the data accompanying this book; this should produce the `neato_servers.png` network image file shown in Figure 4-7:

```
$ neato callback_servers_malware_projection.dot -T png -o neato_servers.png -Goverlap=false
```

To create this network, we simply revise the code we used to create Figure 4-6 to specify that we want to use the tool `neato` and then save the `.png` to `neato_servers.png`. Now that you know how to create these network visualizations, let's look at ways to improve them.

Adding Visual Attributes to Nodes and Edges

Beyond deciding on your general network layout, it can be useful to be able to specify how individual nodes and edges are to be rendered. For example, you might want to set edge thickness based on the strength of the connection between two nodes, or set node color based on what compromise each malware sample node is associated with, which would allow you to better visualize clusters of malware. NetworkX and GraphViz make it easy to do this by allowing you to specify visual attributes of nodes and edges simply by assigning values to a set of attributes. I discuss only a few such attributes in the sections that follow, but this topic is deep enough to fill an entire book.

Edge Width

To set the width of the border that GraphViz draws around nodes, or the line that it draws for edges, you can set the `penwidth` attribute of nodes and edges to a number of your choice, as shown in Listing 4-4.

```
#!/usr/bin/python
import networkx
from networkx.drawing.nx_agraph import writedot

❶ g = networkx.Graph()
g.add_node(1)
g.add_node(2)
g.add_edge(1,2,❷penwidth=10) # make the edge extra wide
writedot(g, 'network.dot')
```

Listing 4-4: Setting the `penwidth` attribute

Here, I create a simple network ❶ with two nodes connected by an edge, and I set the `penwidth` attribute of the edge to 10 ❷ (the default value is 1).

Run this code, and you should see an image that looks like Figure 4-8.

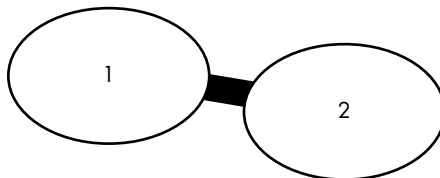


Figure 4-8: A simple network with an edge that has a `penwidth` of 10

As you can see in Figure 4-8, a `penwidth` of 10 results in a very thick edge. The width of the edge (or, the thickness of the node's border if you set the `penwidth` of a node) scales proportionally with the value of

the `penwidth` attribute, so choose accordingly. For example, if your edge strength values vary from 1 to 1000, but you want to be able to see all the edges, you might want to consider assigning `penwidth` attributes based on log scaling of your edge strength values.

Node and Edge Color

To set the color of a node's border or an edge, use the `color` attribute. Listing 4-5 shows how to do this.

```
#!/usr/bin/python

import networkx
from networkx.drawing.nx_agraph import write_dot

g = networkx.Graph()
g.add_node(1, ❶color="blue") # make the node outline blue
g.add_node(2, ❷color="pink") # make the node outline pink
g.add_edge(1,2, ❸color="red") # make the edge red
write_dot(g, 'network.dot')
```

Listing 4-5: Setting node and edge colors

Here, I create the same simple network I created in Listing 4-4, with two nodes and an edge connecting them. For each node that I create, I set its color value (❶ and ❷). I also set the color value for the edge ❸ when I create it.

Figure 4-9 shows the result of Listing 4-5. As expected, you should see that the first node (the edge) and the second node each have a unique color. For a complete list of colors you can use, refer to <http://www.graphviz.org/doc/info/colors.html>.

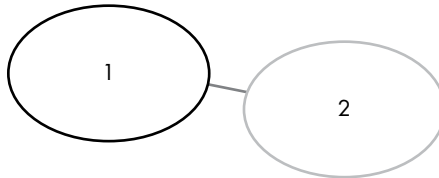


Figure 4-9: A simple network that demonstrates how to set node and edge colors

Colors can be used to show different classes of nodes and edges.

Node Shape

To set the shape of a node, use the `shape` attribute with a string specifying a shape, as defined at <http://www.GraphViz.org/doc/info/shapes.html>. Commonly used values are `box`, `ellipse`, `circle`, `egg`, `diamond`, `triangle`, `pentagon`, and `hexagon`. Listing 4-6 shows how to set the shape attribute of a node.

```
#!/usr/bin/python

import networkx
from networkx.drawing.nx_agraph import write_dot

g = networkx.Graph()
g.add_node(1, ❶shape='diamond')
g.add_node(2, ❷shape='egg')
g.add_edge(1,2)

write_dot(g, 'network.dot')
```

Listing 4-6: Setting node shapes

Similar to the way we set a node's color, we simply use the `shape` keyword argument in the `add_node()` function to specify the shape we want each node to take. Here, we set the first node to a diamond shape ❶ and the second node to an egg shape ❷. The result of this code is shown in Figure 4-10.

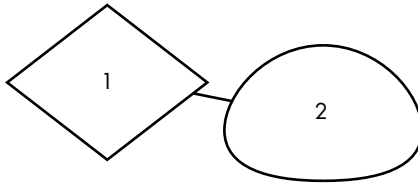


Figure 4-10: A simple network that shows how we can set node shape

The results, showing a diamond-shaped node and an egg-shaped node, reflect the shapes that we specified in Listing 4-6.

Text Labels

Finally, GraphViz also allows you to add labels to nodes and edges with the `label` attribute. Although nodes are automatically labeled based on their assigned ID (for example, the label for a node added as `123` would be `123`), you can specify labels using `label=<my label attribute>`. Unlike nodes, edges aren't labeled by default, but you can assign them labels using the `label` attribute. Listing 4-7 shows how to create our now familiar two-node network but with `label` attributes attached to both nodes and the connecting edge.

```
#!/usr/bin/python

import networkx
from networkx.drawing.nx_agraph import write_dot

g = networkx.Graph()
g.add_node(1, ❶label="first node")
g.add_node(2, ❷label="second node")
```

```
g.add_edge(1,2,📍label="link between first and second node")

write_dot(g,'network.dot')
```

Listing 4-7: Labeling nodes and edges

We label the nodes first node ❶ and second node ❷, respectively. We also label the edge connecting them as the link between first and second node ❸. Figure 4-11 shows the graphical output we expect.

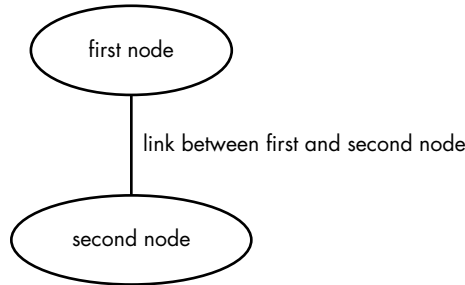


Figure 4-11: A simple network that shows how we can label nodes and edges

Now that you know how to manipulate basic attributes of nodes and edges, you're ready to start building networks from the ground up.

Building Malware Networks

We'll begin our discussion of building malware networks by reproducing and expanding on the shared callback server example shown in Figure 4-1, and then examine shared image analysis of malware.

The following program extracts callback domain names from malware files and then builds a bipartite network of malware samples. Next, it performs one projection of the network to show which malware samples share common callback servers, and it performs another projection to show which callback servers are called by common malware samples. Finally, the program saves the three networks—the original bipartite network, the malware sample projection, and the callback server projection—as files so that they can be visualized with GraphViz.

I walk you through the program, piece by piece. The complete code can be found in the data accompanying this book at the file path *ch4/callback_server_network.py*.

Listing 4-8 shows how to get started by importing the requisite modules.

```
#!/usr/bin/python

import pefile❶
import sys
```

```
import argparse
import os
import pprint
import networkx❷
import re
from networkx.drawing.nx_agraph import write_dot
import collections
from networkx.algorithms import bipartite
```

Listing 4-8: Importing modules

Of the requisite modules we imported, the most notable are the pefile PE parsing module ❶, which we use to parse the target PE binaries, and the networkx library ❷, which we use to create the malware attribute network.

Next, we parse the command line arguments by adding the code in Listing 4-9.

```
args = argparse.ArgumentParser("Visualize shared DLL import relationships
between a directory of malware samples")
args.add_argument(❶"target_path",help="directory with malware samples")
args.add_argument(❷"output_file",help="file to write DOT file to")
args.add_argument(❸"malware_projection",help="file to write DOT file to")
args.add_argument(❹"resource_projection",help="file to write DOT file to")
args = args.parse_args()
```

Listing 4-9: Parsing command line arguments

These arguments include target_path ❶ (the path to the directory where the malware we're analyzing is), output_file ❷ (the path where we write the complete network), malware_projection ❸ (the path where we write a reduced version of the graph and show which malware samples share attributes), and resource_projection ❹ (the path where we write a reduced version of the graph and show which attributes are seen together within the malware samples).

Now we're ready to get into the core of the program. Listing 4-10 shows the code for creating a network for the program.

```
#!/usr/bin/python

import pefile
❶ import sys
import argparse
import os
import pprint
import networkx
import re
from networkx.drawing.nx_agraph import write_dot
import collections
from networkx.algorithms import bipartite

args = argparse.ArgumentParser(
    "Visualize shared hostnames between a directory of malware samples"
)
```

```

args.add_argument("target_path",help="directory with malware samples")
args.add_argument("output_file",help="file to write DOT file to")
args.add_argument("malware_projection",help="file to write DOT file to")
args.add_argument("hostname_projection",help="file to write DOT file to")
args = args.parse_args()
network = networkx.Graph()

valid_hostname_suffixes = map(
    lambda string: string.strip(), open("domain_suffixes.txt")
)
valid_hostname_suffixes = set(valid_hostname_suffixes)
❷ def find_hostnames(string):
    possible_hostnames = re.findall(
        r'(?:[a-zA-Z0-9](?:[a-zA-Z0-9\-\{,61][a-zA-Z0-9])?\.\.)+[a-zA-Z]{2,6}',
        string)
    valid_hostnames = filter(
        lambda hostname: hostname.split(".")[1].lower() \
            in valid_hostname_suffixes,
        possible_hostnames
    )
    return valid_hostnames

# search the target directory for valid Windows PE executable files
for root,dirs,files in os.walk(args.target_path):
    for path in files:
        # try opening the file with pefile to see if it's really a PE file
        try:
            pe = pefile.PE(os.path.join(root,path))
        except pefile.PEFormatError:
            continue
        fullpath = os.path.join(root,path)
        # extract printable strings from the target sample
        ❸ strings = os.popen("strings '{0}'".format(fullpath)).read()

        # use the search_doc function in the included reg module
        # to find hostnames
        ❹ hostnames = find_hostnames(strings)
        if len(hostnames):
            # add the nodes and edges for the bipartite network
            network.add_node(path,label=path[:32],color='black',penwidth=5,
                bipartite=0)
            for hostname in hostnames:
                ❺ network.add_node(hostname,label=hostname,color='blue',
                    penwidth=10,bipartite=1)
                network.add_edge(hostname,path,penwidth=2)
            if hostnames:
                print "Extracted hostnames from:",path
                pprint.pprint(hostnames)

```

Listing 4-10: Creating the network

We first create a fresh network by calling the `networkx.Graph()` constructor ❶. Then we define the function `find_hostnames()`, which extracts

hostnames from strings ❷. Don't worry too much about the mechanics of this function: it's essentially a regular expression and some string-filtering code that tries its best to identify domains.

Next, we iterate through all the files in the target directory, checking whether they are PE files by seeing if the `pefile.PE` class is able to load them (if not, we do not analyze the files). Finally, we extract hostname attributes from the current file by first extracting all printable strings from the file ❸ and then searching the strings for embedded hostname resources ❹. If we find any, we add them as nodes in our network and then add edges from the node for the current malware sample to the hostname resource nodes ❺.

Now we're ready to wrap up the program, as shown in Listing 4-11.

```
# write the dot file to disk
❶ write_dot(network, args.output_file)
❷ malware = set(n for n,d in network.nodes(data=True) if d['bipartite']==0)
❸ hostname = set(network)-malware

# use NetworkX's bipartite network projection function to produce the malware
# and hostname projections
❹ malware_network = bipartite.projected_graph(network, malware)
  hostname_network = bipartite.projected_graph(network, hostname)

# write the projected networks to disk as specified by the user
❺ write_dot(malware_network,args.malware_projection)
  write_dot(hostname_network,args.hostname_projection)
```

Listing 4-11: Writing networks to files

We start by writing our network to disk at the location specified in the command line arguments ❶. Then we create the two reduced networks (the “projections” introduced earlier in this chapter) that show the malware relationships and the hostname resource relationships. We do this by first creating a Python set for containing the IDs of the malware nodes ❷ and another Python set for the IDs of the resource nodes ❸. We then use the NetworkX-specific `projected_graph()` function ❹ to get projections for the malware and resource sets, and we write these networks to disk at the specified locations ❺.

And that's it! You can use this program on any of the malware datasets in this book to see malware relationships between the shared hostname resources embedded in the files. You can even use it on your own datasets to see what threat intelligence you can glean through this mode of analysis.

Building a Shared Image Relationship Network

In addition to analyzing malware based on their shared callback servers, we can also analyze them based on their use of shared icons and other graphical assets. For example, Figure 4-12 shows a portion of the shared image analysis results for the Trojans found in `ch4/data/Trojans`.

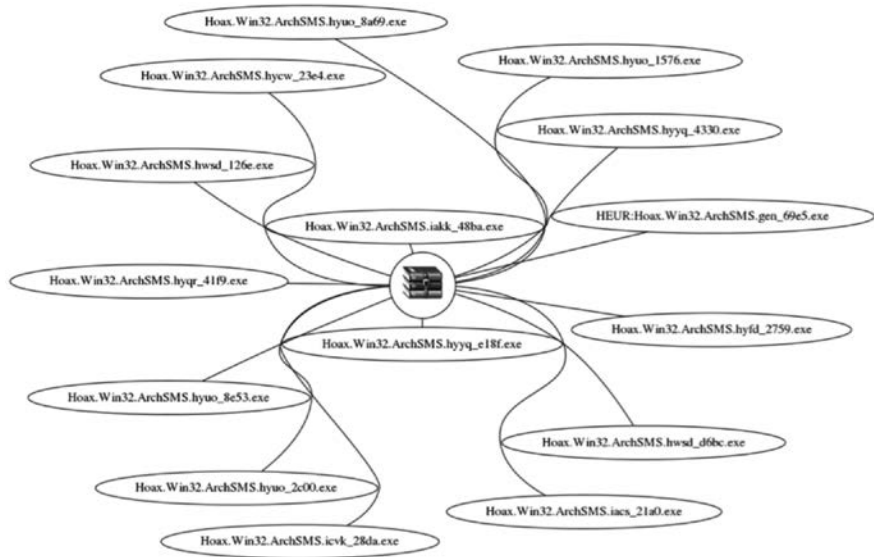


Figure 4-12: A visualization of the shared image asset network for a number of Trojans

You can see that all these Trojan horses pose as archive files and use the same archive file icon (shown in the center of the figure), even though they're executables. The fact that they use exactly the same image as part of their effort to game the user indicates that they probably come from the same attacker. I confirmed this by running the malware samples through the Kaspersky antivirus engine, which assigns them all the same family name (ArchSMS).

Next, I show you how to produce the kind of visualization shown in Figure 4-12, in order to see shared-image relationships between malware samples. To extract the images from the malware, we use the helper library `images`, which in turn relies on `wrestool` (discussed in Chapter 1) to create the `image_network.py` program. Recall that `wrestool` extracts images from Windows executables.

Let's walk through the process of creating a shared image network, starting with the first part of the code, shown in Listing 4-12.

```
#!/usr/bin/python
```

```
import pefile
import sys
import argparse
import os
import pprint
import logging
import networkx
import collections
import tempfile
```



```

from networkx.drawing.nx_agraph import write_dot
from networkx.algorithms import bipartite

# Use argparse to parse any command line arguments

args = argparse.ArgumentParser(
    "Visualize shared image relationships between a directory of malware samples"
)
args.add_argument("target_path", help="directory with malware samples")
args.add_argument("output_file", help="file to write DOT file to")
args.add_argument("malware_projection", help="file to write DOT file to")
args.add_argument("resource_projection", help="file to write DOT file to")
args = args.parse_args()
network = networkx.Graph()

```

```

❶ class ExtractImages():
    def __init__(self, target_binary):
        self.target_binary = target_binary
        self.image_basedir = None
        self.images = []

    def work(self):
        self.image_basedir = tempfile.mkdtemp()
        icondir = os.path.join(self.image_basedir, "icons")
        bitmapdir = os.path.join(self.image_basedir, "bitmaps")
        raw_resources = os.path.join(self.image_basedir, "raw")
        for directory in [icondir, bitmapdir, raw_resources]:
            os.mkdir(directory)
        rawcmd = "wrestool -x {0} -o {1} 2> \
            /dev/null".format(
                self.target_binary, raw_resources
            )
        bmpcmd = "mv {0}/*.bmp {1} 2> /dev/null".format(
            raw_resources, bitmapdir
        )
        icocmd = "icotool -x {0}/*.ico -o {1} \
            2> /dev/null".format(
                raw_resources, icondir
            )
        for cmd in [rawcmd, bmpcmd, icocmd]:
            try:
                os.system(cmd)
            except Exception, msg:
                pass
        for dirname in [icondir, bitmapdir]:
            for path in os.listdir(dirname):
                logging.info(path)
                path = os.path.join(dirname, path)
                imagehash = hash(open(path).read())
                if path.endswith(".png"):
                    self.images.append((path, imagehash))
                if path.endswith(".bmp"):
                    self.images.append((path, imagehash))

```

```

def cleanup(self):
    os.system("rm -rf {}".format(self.image_basedir))

# search the target directory for PE files to extract images from
image_objects = []
for root,dirs,files in os.walk(args.target_path):❷
    for path in files:
        # try to parse the path to see if it's a valid PE file
        try:
            pe = pefile.PE(os.path.join(root,path))
        except pefile.PEFormatError:
            continue

```

Listing 4-12: Parsing the initial argument and file-loading code in our shared image network program

The program starts out much like the hostname graph program (starting at Listing 4-8) we just discussed. It first imports a number of modules, including pefile and networkx. Here, however, we also define the ExtractImages helper class ❶, which we use to extract graphical assets from target malware samples. Then the program enters a loop in which we iterate over all the target malware binaries ❷.

Now that we are in our loop, it's time to extract graphical assets from the target malware binaries using the ExtractImages class (which under the hood is a wrapper around the icoutils programs discussed in Chapter 1). Listing 4-13 shows the part of the code that does this.

```

    fullpath = os.path.join(root,path)
    ❶ images = ExtractImages(fullpath)
    ❷ images.work()
    image_objects.append(images)

    # create the network by linking malware samples to their images
    ❸ for path, image_hash in images.images:
        # set the image attribute on the image nodes to tell GraphViz to
        # render images within these nodes
        if not image_hash in network:
            ❹ network.add_node(image_hash,image=path,label='',type='image')
            node_name = path.split("/")[-1]
            network.add_node(node_name,type="malware")
            ❺ network.add_edge(node_name,image_hash)

```

Listing 4-13: Extracting graphical assets from target malware

First, we pass in a path to a target malware binary to the ExtractImages class ❶, and then we call the resulting instance's work() method ❷. This results in the ExtractImages class creating a temporary directory in which it stores the malware images, and then storing a dictionary containing data about each image in the images class attribute.

Now that we have the list of extracted images from ExtractImages, we iterate over it ❸, creating a new network node for an image if we haven't seen its hash before ❹, and linking the currently processed malware sample to the image in the network ❺.

Now that we have created our network of malware samples linked to the images that they contain, we are ready to write the graph to disk, as shown in Listing 4-14.

```
# write the bipartite network, then do the two projections and write them
❶ write_dot(network, args.output_file)
malware = set(n for n,d in network.nodes(data=True) if d['type']=='malware')
resource = set(network) - malware
malware_network = bipartite.projected_graph(network, malware)
resource_network = bipartite.projected_graph(network, resource)

❷ write_dot(malware_network,args.malware_projection)
write_dot(resource_network,args.resource_projection)
```

Listing 4-14: Writing the graph to disk

We do this in exactly the same way that we did in Listing 4-11. First, we write the complete network to disk ❶, and then we write the two projections (the projection for the malware and the projection for the images, which we refer to as *resources* here) to disk ❷.

You can use *image_network.py* to analyze graphical assets in any of the malware datasets in this book, or to extract intelligence from malware datasets of your choice.

Summary

In this chapter, you learned about the tools and methods necessary to perform shared attribute analysis on your own malware datasets. Specifically, you learned how networks, bipartite networks, and bipartite network projections can help identify the social connections between malware samples, why network layout is central to network visualization, and how force-directed networks work. You also learned how to create and visualize malware networks using Python and open source tools like NetworkX. In Chapter 5, you'll learn how to build malware networks based on shared code relationships between samples.

5

SHARED CODE ANALYSIS



Suppose you discovered a new malware sample on your network. How would you begin to analyze it? You could submit it to a multi-engine antivirus scanner such as VirusTotal to learn what malware family it belongs to. However, such results are often unclear and ambiguous, because engines often label the malware in generic terms like “agent” that mean nothing. You could also run the sample through CuckooBox or some other malware sandbox to get a limited report on the malware sample’s callback servers and behaviors.

When these approaches don’t provide enough information, you may need to reverse-engineer the sample. At this stage, shared code analysis can dramatically improve your workflow. By revealing which previously analyzed samples the new malware sample is similar to, and thus revealing the code they share, shared code analysis allows you to reuse your previous analyses on new malware so that you’re not starting from scratch. Understanding where this previously seen malware came from can also help you figure out who may have deployed the malware.

Shared code analysis, also called *similarity analysis*, is the process by which we compare two malware samples by estimating the percentage of precompilation source code they share. It differs from shared attribute analysis, which compares malware samples based on their external attributes (the desktop icons they use, for example, or the servers they call out to).

In reverse engineering, shared code analysis helps identify samples that can be analyzed *together* (because they were generated from the same malware toolkit or are different versions of the same malware family), which can determine whether the same developers could have been responsible for a group of malware samples.

Consider the output shown in Listing 5-1, which comes from a program you'll build later in this chapter to illustrate the value of malware shared code analysis. It shows previously seen samples that may share code with the new sample as well as comments made on those older samples.

```
Showing samples similar to WEBC2-GREENCAT_sample_E54CE5F0112C9FDFE86DB17E85A5E2C5
Sample name                                     Shared code
[*] WEBC2-GREENCAT_sample_55FB1409170C91740359D1D96364F17B      0.9921875
[*] GREENCAT_sample_55FB1409170C91740359D1D96364F17B            0.9921875
[*] WEBC2-GREENCAT_sample_E83F60FB0E0396EA309FAFOAED64E53F      0.984375
    [comment] This sample was determined to definitely have come from the advanced persistent
threat group observed last July on our West Coast network
[*] GREENCAT_sample_E83F60FB0E0396EA309FAFOAED64E53F              0.984375
```

Listing 5-1: The results of basic shared code analysis

Given a new sample, shared code estimation allows us to see, within seconds, which samples it likely shares code with and what we know about those samples. In this example, it reveals that a very similar sample is from a known APT, or *advanced persistent threat*, thus providing immediate context for this new malware.

We can also visualize sample shared code relationships using network visualization, which you learned about in Chapter 4. For example, Figure 5-1 shows a network of shared code relationships between samples in an advanced persistent threat dataset.

As you can see from the visualization, automated shared code analysis techniques can quickly uncover the existence of malware families that would have taken days or weeks to discover through manual analysis. In this chapter, you'll learn to use these techniques to do the following:

- Identify new malware families that come from the same malware toolkits or were written by the same attackers.
- Determine code similarity between a new sample and previously seen samples.
- Visualize malware relationships to better understand code-sharing patterns between malware samples and to communicate your results to others.
- Use two proof-of-concept tools I built for this book that implement these ideas and allow you to see malware shared code relationships.

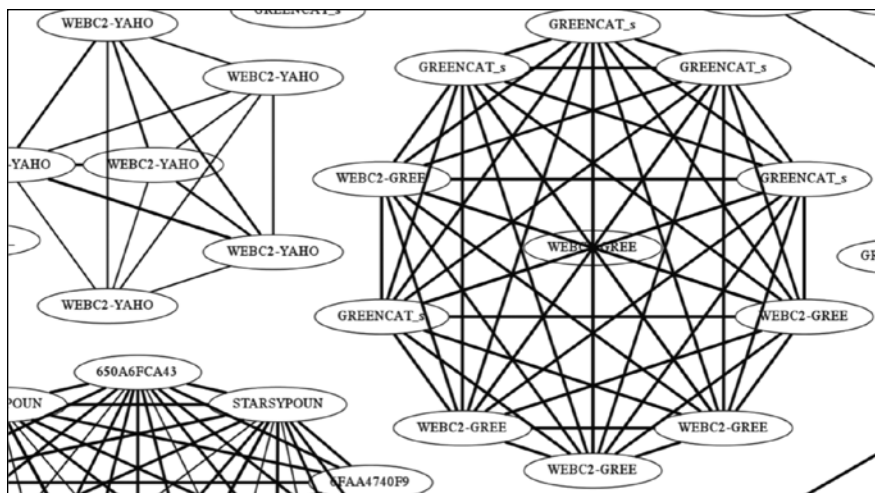


Figure 5-1: An example of the kind of visualization you will learn to create in this chapter, showing shared code relationships between some of the APT1 samples

First, I introduce the test malware samples you'll be using in this chapter, which are the PLA APT1 samples from Chapter 4 and an assortment of crimeware samples. Then, you learn about mathematical similarity comparison and the concept of the *Jaccard index*, a set-theoretic method for comparing malware samples in terms of their shared features. Next, I introduce the concept of features to show how you can use them in conjunction with the Jaccard index to approximate the amount of code two malware samples share. You also learn how to evaluate malware features in terms of their usefulness. Finally, we create visualizations of malware code sharing at multiple scales, as shown in Figure 5-1, by leveraging your knowledge of network visualization from Chapter 4.

MALWARE SAMPLES USED IN THIS CHAPTER

In this chapter, we use real-world malware families that share significant amounts of code with one another to do our experiments. These datasets are available thanks to Mandiant and Mila Parkour, who curated these samples and made them available to the research community. In reality, however, you might not know what family a malware sample belongs to, or to what degree your new malware samples are similar to previously seen samples. But going through examples where we *do* know will be good practice, because it allows us to verify that our automated inferences of sample similarity line up with our knowledge of which samples actually belong in the same group.

The first samples come from the APT1 dataset we used in Chapter 4 to demonstrate shared resource analysis. The other samples consist of thousands

(continued)

of crimeware malware samples developed by criminals to steal people's credit cards, turn their computers into zombie hosts hooked into botnets, and so on. These are real-world samples sourced from a commercial malware feed provided as a paid service for threat intelligence researchers.

To identify their family names, I have input each sample into the Kaspersky antivirus engine. Kaspersky was able to classify 30,104 of these samples with robust hierarchical classifications (such as *trojan.win32.jorik.skor.akr*, indicating the *jorik.skor* family), assigned a class of "unknown" to 41,830 samples, and assigned generic labels (such as, generically, "win32 Trojan") to the remaining 28,481 samples.

Because of the inconsistency of the Kaspersky labels (some Kaspersky label groupings, such as the *jorik* family, represent a very diffuse range of malware, whereas others, such as *webprefix*, represent a very specific set of variants) and the fact that Kaspersky often misses or mislabels malware, I selected seven malware classes that Kaspersky detects with high confidence. Specifically, these include the *dapato*, *pasta*, *skor*, *vbna*, *webprefix*, *xtoober*, and *zango* families.

Preparing Samples for Comparison by Extracting Features

How do we even begin to think about estimating the amount of code two malicious binaries may have shared before they were compiled by attackers? There are many ways one might consider approaching this problem, but in the hundreds of computer science research papers that have been published on the topic, a common theme has emerged: to estimate the amount of shared code between binaries, we group malware samples into "bags of features" before comparing.

By *features* I mean any malware attribute we might possibly want to consider in estimating the code similarity between samples. For example, the features we use could be the printable strings we can extract from the binaries. Instead of thinking of the samples as an interconnected system of functions, dynamic library imports, and so on, we think of malware as a bag of independent features for mathematical convenience (for example, a set of strings that have been extracted from the malware).

How Bag of Features Models Work

To understand how a bag of features works, consider a Venn diagram between two malware samples, as shown in Figure 5-2.

Here, sample A and sample B are shown as bags of features (features are represented as ellipses inside the Venn diagram). We can compare them by examining which features are shared between the two samples. Computing the overlap between two sets of features is fast, and can be used to compare malware samples' similarity based on arbitrary features that we come up with.

For example, when dealing with packed malware, we may want to use features based on malware dynamic run logs since running malware in a sandbox is a way to get malware to unpack itself. In other cases, we may use strings extracted from the static malware binary to perform the comparison.

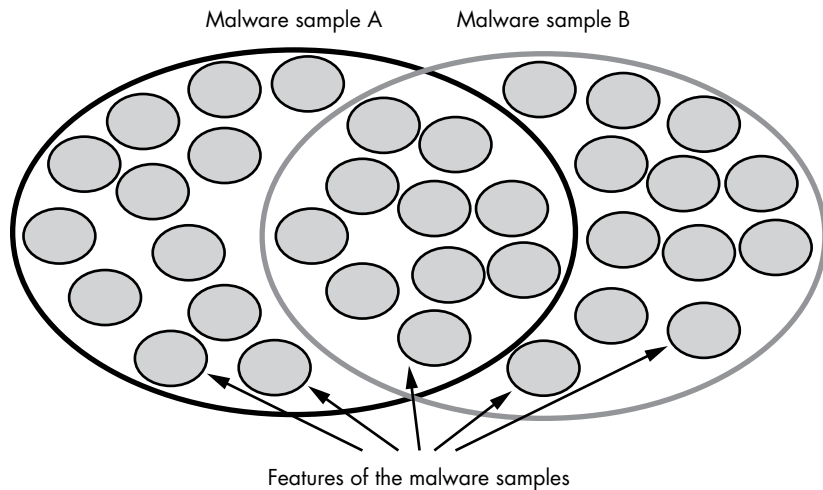


Figure 5-2: An illustration of the “bag of features” model for malware code sharing analysis

In the case of dynamic malware analysis, we may want to compare samples based not just on *what* behaviors they share but also on the order in which they express behaviors, or what we call their *sequences* of behaviors. A common way to incorporate sequence information into malware sample comparisons is to extend the bag of features model to accommodate sequential data using N-grams.

What are N-Grams?

An *N-gram* is a subsequence of events that has a certain length, N , of some larger sequence of events. We extract this subsequence from a larger sequence by sliding a window over the sequential data. In other words, we get N-grams by iterating over a sequence and, at each step, recording the subsequence from the event at index i to the event at index $i + N - 1$, as shown in Figure 5-3.

In Figure 5-3, the sequence of integers (1,2,3,4,5,6,7) is translated into five different subsequences of length 3: (1,2,3), (2,3,4), (3,4,5), (4,5,6), (5,6,7).

Of course, we can do this with any sequential data. For example, using an N-gram word length of 2, the sentence “how now brown cow” yields the following subsequences: “how now”, “now brown”, and “brown cow.” In malware analysis, we would extract N-grams of sequential API calls that a malware sample made. Then we would represent the malware as a bag of features and

use N-gram features to compare the malware sample to some other malware sample's N-grams, thereby incorporating sequence information into the bag of features comparison model.

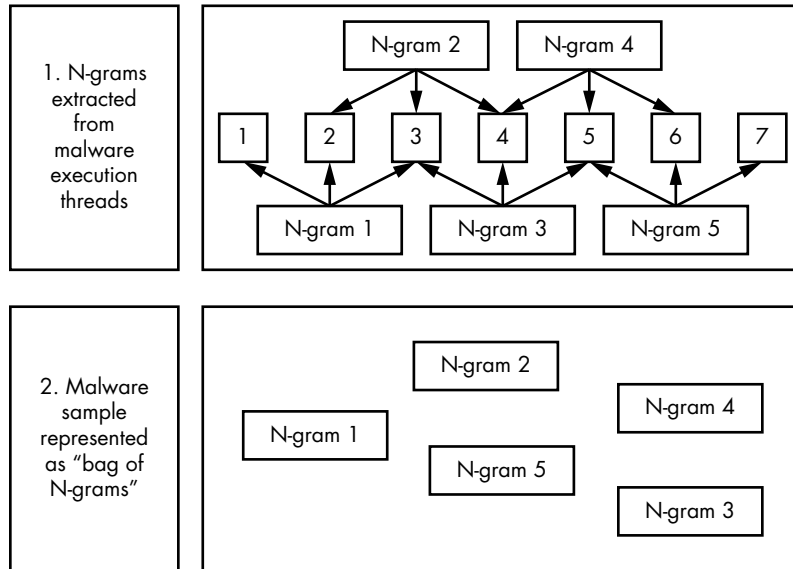


Figure 5-3: A visual explanation of how we can extract N-grams from malware's assembly instructions and dynamic API call sequences, where $N = 3$

Including sequence information in our comparison of malware samples has advantages and disadvantages. The advantage is that when order matters in the comparison (for example, when we care that API call A was observed before API call B, which was observed before API call C), it allows us to capture order, but when order is superfluous (for example, malware randomizing the order of API calls A, B, and C on every run), it can actually make our shared code estimation much worse. Deciding whether to include order information in our malware shared code estimation work depends on what kind of malware we're working with, and requires that we experiment.

Using the Jaccard Index to Quantify Similarity

Once you've represented a malware sample as a bag of features, you'll need to measure the degree of similarity between that sample's bag of features and some other sample's bag of features. To estimate the extent of code sharing between two malware samples, we use a *similarity function*, which should have the following properties:

- It yields a normalized value such that all similarity comparisons between pairs of malware samples can be placed on a common scale. By convention, the function should yield a value ranging from 0 (no code sharing) to 1 (samples share 100 percent of their code).

- The function should help us make accurate estimates of code sharing between two samples (we can determine this empirically through experimentation).
- We should be able to easily understand why the function models code similarity well (it should not be a complicated mathematical black box that takes a lot of effort to understand or explain).

The *Jaccard index* is a simple function that has these properties. In fact, even though other mathematical approaches to code similarity estimation have been tried in the security research community (for example, cosine distance, L1 distance, Euclidean [L2] distance, and so on), the Jaccard index has emerged as the most widely adopted—and for good reason. It simply and intuitively expresses the degree of overlap between two sets of malware features, giving us the percentage of unique features common to both of the two sets normalized by the percentage of unique features that exist in either set.

Figure 5-4 illustrates examples of Jaccard index values.

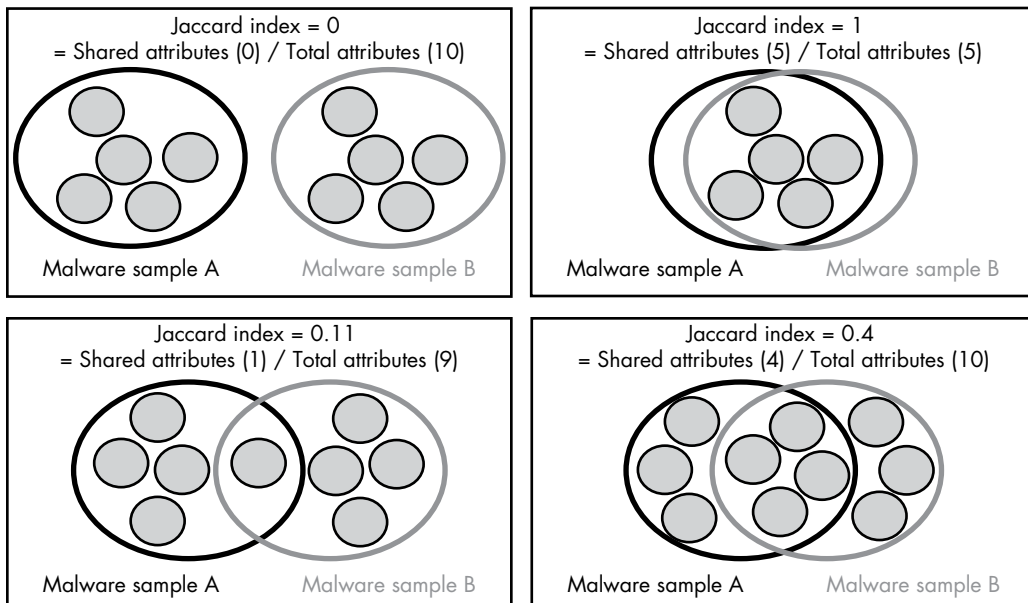


Figure 5-4: A visual illustration of the idea behind the Jaccard index

This illustrates four pairs of malware features extracted from four pairs of malware samples. Each image shows the features shared between the two sets, the features not shared between the two sets, and the resulting Jaccard index for the given pair of malware samples and associated features. You can see that the Jaccard index between the samples is simply the number of features shared between the samples divided by the total number of features drawn in the Venn diagram.

Using Similarity Matrices to Evaluate Malware Shared Code Estimation Methods

Let's discuss four methods for determining whether two malware samples come from the same family: instruction sequence-based similarity, strings-based similarity, Import Address Table-based similarity, and dynamic API call-based similarity. To compare these four methods, we'll use a *similarity matrix* visualization technique. Our goal here will be to compare the relative strengths and weaknesses of each method in terms of its ability to illuminate shared code relationships between samples.

To get started, let's go over the concept of a similarity matrix. Figure 5-5 compares an imaginary set of four malware samples using a similarity matrix.

	Sample 1	Sample 2	Sample 3	Sample 4
Sample 1	Similarity between 1 and 1	Similarity between 1 and 2	Similarity between 1 and 3	Similarity between 1 and 4
Sample 2	Similarity between 2 and 1	Similarity between 2 and 2	Similarity between 2 and 3	Similarity between 2 and 4
Sample 3	Similarity between 3 and 1	Similarity between 3 and 2	Similarity between 3 and 3	Similarity between 3 and 4
Sample 4	Similarity between 4 and 1	Similarity between 4 and 2	Similarity between 4 and 3	Similarity between 4 and 4

Figure 5-5: An illustration of a notional similarity matrix

This matrix allows you to see the similarity relationship between all samples. You can see that some space is wasted in this matrix. For example, we don't care about the similarities represented in shaded boxes, as these entries just contain comparisons between a given sample and itself. You can also see that the information on either side of the shaded boxes is repeated, so you only need to look at one or the other.

Figure 5-6 gives a real-world example of a malware similarity matrix. Note that due to the large number of malware samples shown in the figure, each similarity value is represented by a shaded pixel. Instead of rendering the names of each sample, we render the family names for each sample along the horizontal and vertical axes. A perfect similarity matrix would look like a chain of white squares running diagonally from the top left to

the bottom right, since the rows and columns representing each family are grouped together, and we expect all members of a given family to be similar to one another, but not samples from other families.

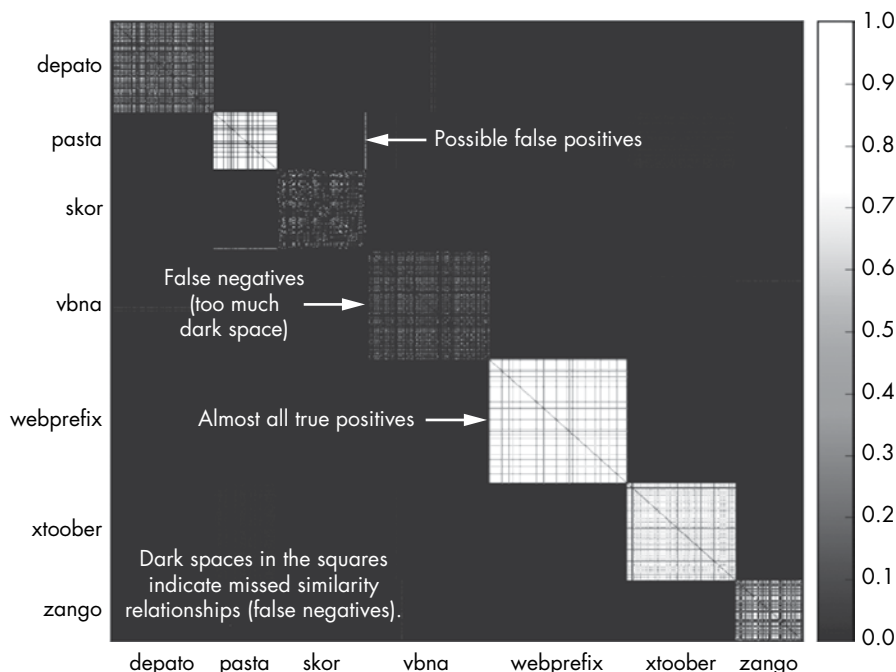


Figure 5-6: A real-world malware similarity matrix computed over the seven malware families

In the results given in Figure 5-6, you can see that some of the family squares are completely white—these are good results, because white pixels within a family square indicate an inferred similarity relationship between samples of the same family. Some are much darker, which means we did not detect strong similarity relationships. Finally, sometimes there are lines of pixels outside the family squares, which are either evidence of related malware families or false positives, meaning that we detected code-sharing between families despite their being inherently different.

Next, we'll use similarity matrix visualizations like Figure 5-6 to compare the results of four different code-sharing estimation methods, starting with a description of instruction sequence-based similarity analysis.

Instruction Sequence-Based Similarity

The most intuitive way to compare two malware binaries in terms of the amount of code they share is by comparing their sequences of x86 assembly instructions, since samples that share sequences of instructions are likely to have shared, before compilation, actual source code. This requires disassembling malware samples using, for example, the *linear disassembly* technique introduced in Chapter 2. Then we can use the N-gram extraction

approach I discussed previously to extract sequences of instructions in the order they appear in the .text section of the malware file. Finally, we can use the instruction N-grams to compute Jaccard indices between samples to estimate how much code we think they share.

The value we use for N during N-gram extraction depends on our analysis goals. The larger N is, the larger our extracted instruction subsequences will be, and thus the harder it will be for malware samples' sequences to match. Setting N to a large number helps identify only samples that are highly likely to share code with one another. On the other hand, you can make N smaller to look for subtle similarities between samples, or if you suspect that the samples employ instruction reordering to obscure similarity analysis.

In Figure 5-7, N is set to 5, which is an aggressive setting that makes it harder for samples to match.

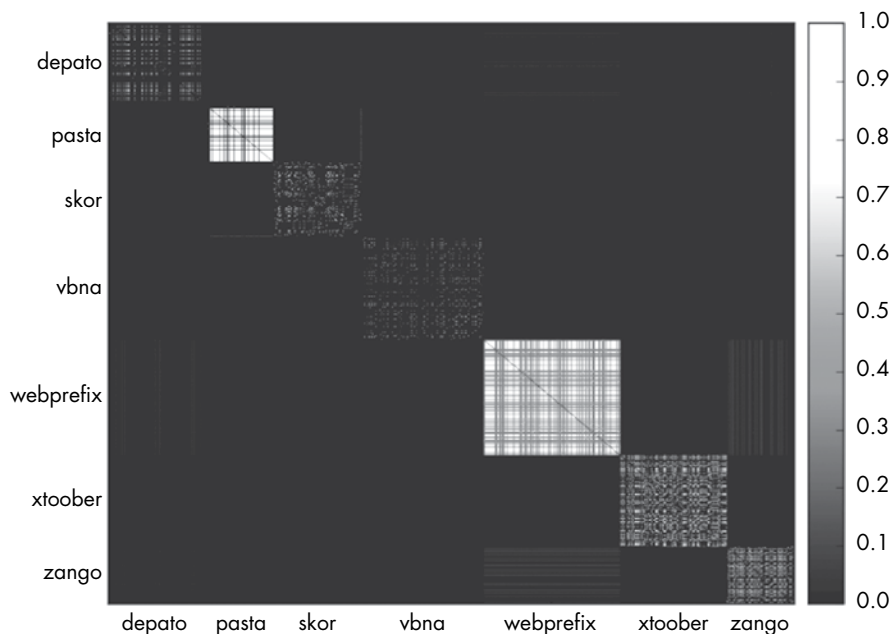


Figure 5-7: The similarity matrix generated using instruction N-gram features. Using $N = 5$, we completely miss many families' similarity relationships but do well on webprefix and pasta.

The results in Figure 5-7 are not very compelling. While the instruction-based similarity analysis correctly identifies similarities between some families, it doesn't within other families (for example, it detects few similarity relationships in depato, skor, and vbna). It's important to note, however, that there are few false positives in this analysis (false inferences of similarity between samples from different families, versus true inferences of similarities within samples of the same family).

As you can see, a limitation of instruction subsequence shared code analysis is that it can miss many code-sharing relationships between samples.

This is because malware samples may be packed such that most of their instructions only become visible once we execute the malware samples and let them unpack themselves. Without unpacking our malware samples, the instruction sequence shared code estimation method will likely not work very well.

Even when we unpack our malware samples, the approach can be problematic, because of the noise introduced by the source code compilation process. Indeed, compilers can compile the same source code into radically different sequences of assembly instructions. Take, for example, the following simple function written in C:

```
int f(void) {  
    int a = 1;  
    int b = 2;  
    ❶ return (a*b)+3;  
}
```

You might think that regardless of compiler, the function would reduce to the same sequence of assembly instructions. But in fact, compilation depends heavily not just on what compiler you use, but also on the compiler settings. For example, compiling this function using the clang compiler under its default settings yields the following instructions corresponding to the line at ❶ in the source code:

```
movl    $1, -4(%rbp)  
movl    $2, -8(%rbp)  
movl    -4(%rbp), %eax  
imull   -8(%rbp), %eax  
addl    $3, %eax
```

In contrast, compiling the same function with the `-O3` flag set, which tells the compiler to optimize the code for speed, yields the following assembly for the same line of the source code:

```
movl    $5, %eax
```

The difference results from the fact that in the second example, the compiler pre-computed the result of the function instead of explicitly computing it, as in the first compilation example. This means that if we compared these functions based on instruction sequences, they wouldn't appear at all similar, even though in reality they were compiled from exactly the same source code.

Beyond the problem of identical C and C++ code appearing to be very different when we're looking at its assembly instructions, there's an additional problem that arises when we compare binaries based on their assembly code: many malware binaries are now authored in high-level languages like C#. These binaries contain standard boilerplate assembly code that simply interprets these higher-level languages' bytecode. So, although

binaries written in the same high-level language may share very similar x86 instructions, their actual bytecode may reflect the fact that they come from very different source code.

Strings-Based Similarity

We can compute strings-based malware similarity by extracting all contiguous printable sequences of characters in the samples and then computing the Jaccard index between all pairs of malware samples based on their shared string relationships.

This approach gets around the compiler problem because the strings extracted from a binary tend to be *format strings* defined by the programmer, which compilers as a general rule do not transform, regardless of which compilers the malware authors are using or what parameters they give the compilers. For example, a typical string extracted from a malware binary might read, “Started key logger at %s on %s and time %s.” Regardless of compiler settings, this string will tend to look identical among multiple binaries and is related to whether or not they’re based on the same source code base.

Figure 5-8 shows how well the string-based code-sharing metric identifies the correct code-sharing relationships in the crimeware dataset.

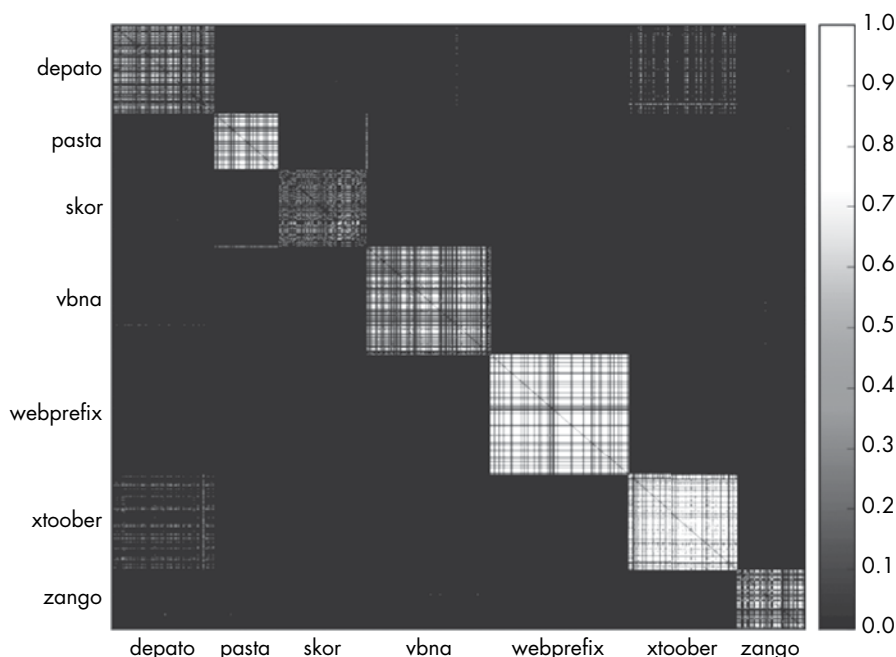


Figure 5-8: The similarity matrix generated using string features

At first glance, this method does far better at identifying the malware families than the instruction-based method, accurately recovering much of the similarity relationships for all seven families. However, unlike the

instruction similarity method, there are a few false positives, since it incorrectly predicts that xtoober and dapato share some level of code. It's also worth noting that this method didn't detect similarities between samples in some families, performing particularly poorly on the zango, skor, and dapato families.

Import Address Table–Based Similarity

We can compute what I call “Import Address Table–based similarity” by comparing the DLL imports made by malware binaries. The idea behind this approach is that even if the malware author has reordered instructions, obfuscated the initialized data section of the malware binary, and implemented anti-debugger and anti-VM anti-analysis techniques, they may have left the exact same import declarations in place. The results for the Import Address Table method are shown in Figure 5-9.

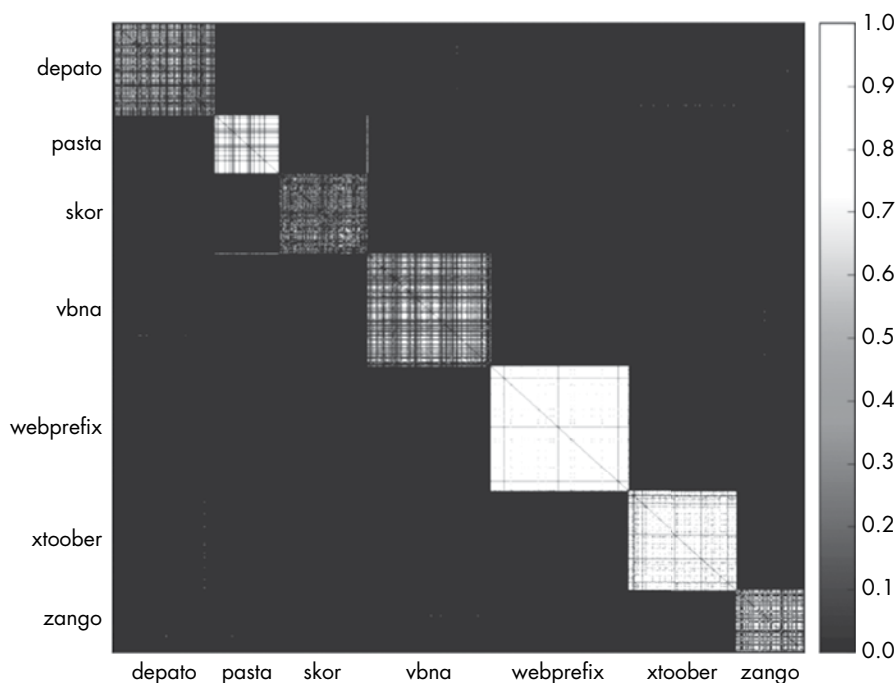


Figure 5-9: The similarity matrix generated using Import Address Table features

The figure shows that the Import Address Table method does better than any of the preceding methods at estimating the similarity relationships between the webprefix and xtoober samples and does very well overall, even though it misses many of the skor, dapato, and vbna relationships. It's also notable that this method gives few false positives on our experimental dataset.

Dynamic API Call–Based Similarity

The final comparison method I introduce in this chapter is dynamic malware similarity. The advantage of comparing dynamic sequences is that even if malware samples are extremely obfuscated or packed, they will tend to perform similar sequences of actions within a sandboxed virtual machine as long as they're derived from the same code or borrow code from one another. To implement this approach, you'll need to run malware samples in a sandbox and record the API calls they make, extract N-grams of API calls from the dynamic logs, and finally compare the samples by taking the Jaccard index between their bags of N-grams.

Figure 5-10 shows that the dynamic N-gram similarity approach does about as well as the import and string methods in most cases.

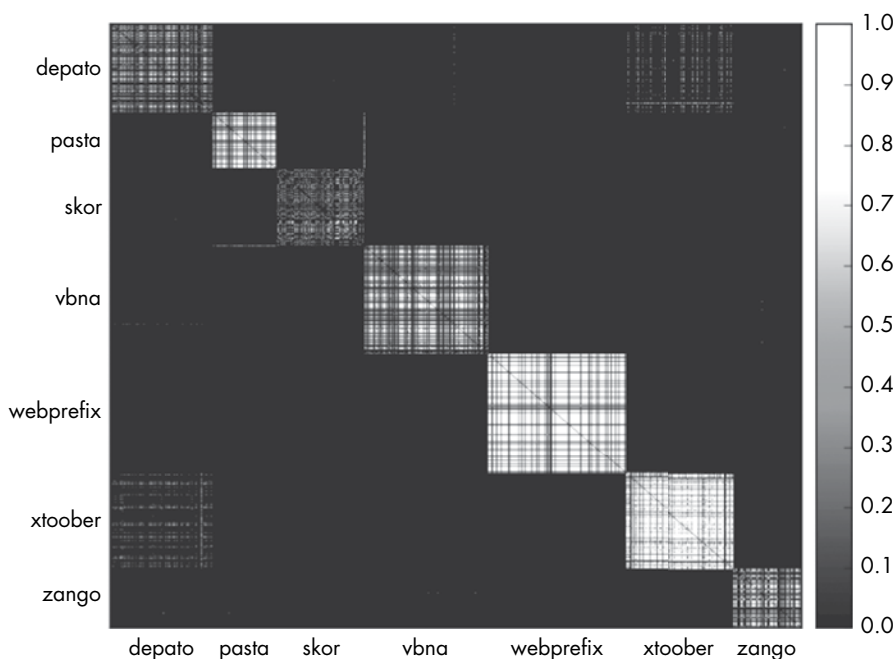


Figure 5-10: The similarity matrix generated using dynamic API call N-gram features

The imperfect results here show that this method is no panacea. Simply running malware in a sandbox is not sufficient to trigger many of its behaviors. Variations of a command line malware tool, for example, may or may not enable an important code module, and therefore execute different sequences of behavior, even though they may share most of their code.

Another problem is that some samples detect that they're running in a sandbox and then promptly exit execution, leaving us with little information to make comparisons. In summary, like the other similarity approaches I've outlined, dynamic API call sequence similarity isn't perfect, but it can provide impressive insight into similarities between samples.

Building a Similarity Graph

Now that you understand the concepts behind methods for identifying malware code sharing, let's build a simple system that performs this analysis over a malware dataset.

First, we need to estimate the amount of code that samples share by extracting the features we would like to use. These could be any of the features described previously, such as Import Address Table-based functions, strings, N-grams of instructions, or N-grams of dynamic behavior. Here, we'll use printable string features because they perform well and are simple to extract and understand.

Once we've extracted the string features, we need to iterate over every pair of malware samples, comparing their features using the Jaccard index. Then, we need to build a code-sharing graph. To do this, we first need to decide on a threshold that defines how much code the two samples share—a standard value I use in my research is 0.8. If the Jaccard index for a given pair of malware samples is above that value, we create a link between them for visualization. The final step is to study the graph to see which samples are connected by shared code relationships.

Listings 5-2 through 5-6 contain our sample program. Because the listing is long, I break it into pieces and explain each piece as I go. Listing 5-2 imports the libraries we'll use, and declares the `jaccard()` function, which computes the Jaccard index between two samples' sets of features.

```
#!/usr/bin/python

import argparse
import os
import networkx
from networkx.drawing.nx_pydot import write_dot
import itertools

def jaccard(set1, set2):
    """
    Compute the Jaccard distance between two sets by taking
    their intersection, union and then dividing the number
    of elements in the intersection by the number of elements
    in their union.
    """
    intersection = set1.intersection(set2)
    intersection_length = float(len(intersection))
    union = set1.union(set2)
    union_length = float(len(union))
    return intersection_length / union_length
```

Listing 5-2: The imports and a helper function to compute the Jaccard index between two samples

Next, in Listing 5-3, we declare two additional utility functions: `getstrings()`, which finds the set of printable string sequences within the malware files we'll be analyzing, and `pecheck()`, which ensures that target

files are indeed Windows PE files. We'll use these functions later when we're performing feature extraction on the target malware binaries.

```
def getstrings(fullpath):
    """
    Extract strings from the binary indicated by the 'fullpath'
    parameter, and then return the set of unique strings in
    the binary.
    """
    strings = os.popen("strings '{0}'".format(fullpath)).read()
    strings = set(strings.split("\n"))
    return strings

def pecheck(fullpath):
    """
    Do a cursory sanity check to make sure 'fullpath' is
    a Windows PE executable (PE executables start with the
    two bytes 'MZ')
    """
    return open(fullpath).read(2) == "MZ"
```

Listing 5-3: Declaring the functions we'll use in feature extraction

Next, in Listing 5-4, we parse our user's command line arguments. These arguments include the target directory in which the malware we'll be analyzing exists, the output *.dot* file to which we'll write the shared code network we build, and the Jaccard index threshold, which determines how high the Jaccard index must be between two samples for the program to decide that they share a common code base with one another.

```
If __name__ == "__main__":
    parser = argparse.ArgumentParser(
        description="Identify similarities between malware samples and build similarity graph"
    )

    parser.add_argument(
        "target_directory",
        help="Directory containing malware"
    )

    parser.add_argument(
        "output_dot_file",
        help="Where to save the output graph DOT file"
    )

    parser.add_argument(
        "--jaccard_index_threshold", "-j", dest="threshold", type=float,
        default=0.8, help="Threshold above which to create an 'edge' between samples"
    )

    args = parser.parse_args()
```

Listing 5-4: Parsing the user's command line arguments

Next, in Listing 5-5, we use the helper functions we declared earlier to do the main work of the program: finding PE binaries in the target directory, extracting features from them, and initializing a network that we'll use to express similarity relationships between the binaries.

```
malware_paths = [] # where we'll store the malware file paths
malware_features = dict() # where we'll store the malware strings
graph = networkx.Graph() # the similarity graph

for root, dirs, paths in os.walk(args.target_directory):
    # walk the target directory tree and store all of the file paths
    for path in paths:
        full_path = os.path.join(root, path)
        malware_paths.append(full_path)

# filter out any paths that aren't PE files
malware_paths = filter(pecheck, malware_paths)

# get and store the strings for all of the malware PE files
for path in malware_paths:
    features = getstrings(path)
    print "Extracted {0} features from {1} ...".format(len(features), path)
    malware_features[path] = features

# add each malware file to the graph
graph.add_node(path, label=os.path.split(path)[-1][:10])
```

Listing 5-5: Extracting features from PE files in the target directory and initializing the shared code network

After extracting features from our target samples, we need to iterate over every pair of malware samples, comparing their features using the Jaccard index. We do this in Listing 5-6. We also build a code-sharing graph where samples are linked together if their Jaccard index is above some user-defined threshold. The threshold I've found works best in my research is 0.8.

```
# iterate through all pairs of malware
for malware1, malware2 in itertools.combinations(malware_paths, 2):

    # compute the jaccard distance for the current pair
    jaccard_index = jaccard(malware_features[malware1], malware_features[malware2])

    # if the jaccard distance is above the threshold, add an edge
    if jaccard_index > args.threshold:
        print malware1, malware2, jaccard_index
        graph.add_edge(malware1, malware2, penwidth=1+(jaccard_index-args.threshold)*10)

# write the graph to disk so we can visualize it
write_dot(graph, args.output_dot_file)
```

Listing 5-6: Creating a code-sharing graph in Python

Scaling Similarity Comparisons

Although the code in Listings 5-2 through 5-6 works well for small malware datasets, it doesn't work well for a large number of malware samples. This is because comparing all pairs of malware samples in a dataset grows quadratically with the number of samples. Specifically, the following equation gives the number of Jaccard index computations necessary to compute a similarity matrix over a dataset of size n :

$$\frac{n^2 - n}{2}$$

For example, let's return to the similarity matrix in Figure 5-5 to see how many Jaccard indices we would need to compute the four samples. At first glance, you might say 16 (4^2), because that's how many cells are in the matrix. However, because the bottom triangle of the matrix contains duplicates of the top triangle of the matrix, we don't need to compute these twice. This means that we can subtract 6 from our total number of computations. Furthermore, we don't need to compare malware samples to themselves, so we can eliminate the diagonal from the matrix, allowing us to subtract four more computations.

The number of computations necessary is as follows:

$$\frac{4^2 - 4}{2} = \frac{16 - 4}{2} = 6$$

This seems manageable, until our dataset grows to 10,000 malware samples, for example, which would require 49,995,000 computations. A dataset that has 50,000 samples would require 1,249,975,000 Jaccard index computations!

To scale malware similarity comparisons, we need to use randomized comparison approximation algorithms. The basic idea is to allow for some error in our computation of comparisons in exchange for a reduction in computation time. For our purposes, an approximate comparison approach known as *minhash* serves this purpose beautifully. The minhash method allows us to compute the Jaccard index using approximation to avoid computing similarities between nonsimilar malware samples below some predefined similarity threshold so that we can analyze shared code relationships between millions of samples.

Before you read about why minhash works, note that this is a tricky algorithm that can take some time to understand. If you decide to skip the "Minhash in Depth" section, just read the "Minhash in a Nutshell" section and use the code provided, and you should have no problems scaling your code sharing analysis.

Minhash in a Nutshell

Minhash takes a malware sample's features and hashes them with k hash functions. For each hash function, we retain only the minimum value of the

hashes computed over all the features, so that the set of malware features is reduced to a fixed size array of k integers, which we call the minhashes. To compute the approximate Jaccard index between two samples based on their minhash arrays, you now just need to check how many of the k minhashes match, and divide that by k .

Magically, the number that falls out of these computations is a close approximation of the true Jaccard index between any two samples. The benefit of using minhash instead of a literal computation of the Jaccard index is that it's much faster to compute.

In fact, we can even use minhash to cleverly index malware in a database such that we only need to compute comparisons between malware samples that are likely to be similar because at least one of their hashes matched, thereby dramatically speeding up computation of similarities within malware datasets.

Minhash in Depth

Let's now discuss the math behind minhash in depth. Figure 5-12 shows the sets of features (represented by the shaded circles) for two malware samples, how they are hashed and then sorted based on their hashes, and how they're finally compared based on the value of the first element of each list.

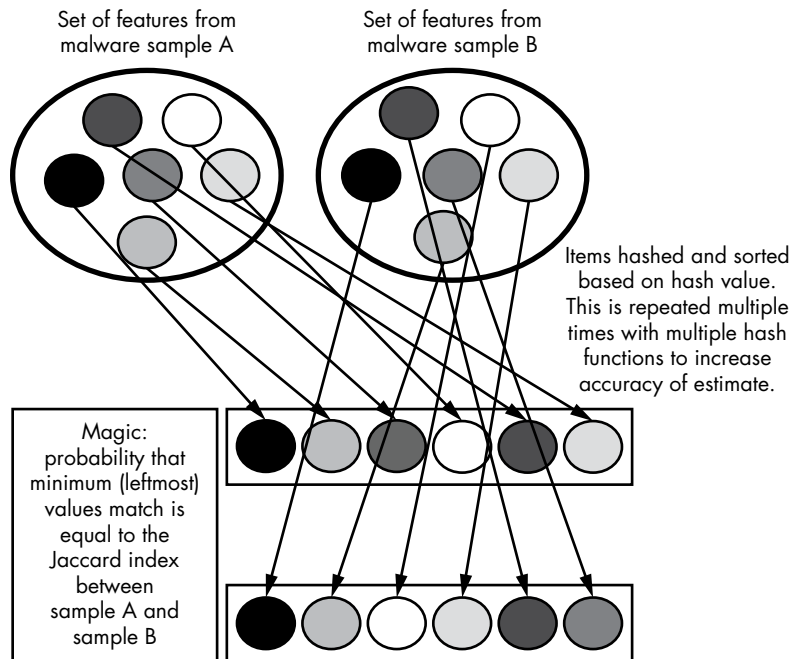


Figure 5-12: An illustration of the idea behind minhash

The probability that the first elements will match is equal to the Jaccard index between the samples. How this works is beyond the scope of this book, but this serendipitous fact is what lets us approximate the Jaccard index using hashes.

Of course, just performing this hashing, sorting, and first-element-checking operation doesn't tell us that much if we only do it once—the hashes either match or they don't, and we can't guess the underlying Jaccard index very accurately based on that one match. To get a better estimate of this underlying value, we have to use k hash functions and repeat this operation k times, and then estimate the Jaccard index by dividing the number of times these first elements matched by k . Our expected *error* in estimating the Jaccard index is defined as the following:

$$\frac{1.0}{\sqrt{k}}$$

So the more times we perform this procedure, the more certain we'll be (I tend to set k to 256 so that the estimate is off by 6 percent, on average).

Suppose we compute a minhash array for every malware sample in a malware dataset containing one million samples. How do we then use the minhashes to speed up the search for malware families in the dataset? We could iterate over every pair of malware samples in the dataset and compare their minhash arrays, which would lead to 499,999,500,000 comparisons. Even though it's faster to compare minhash arrays than to compute the Jaccard index, this is still way too many comparisons to make on modern hardware. We need some way of exploiting the minhashes to optimize the comparison process even more.

The standard approach to this problem is to use sketching combined with database indexing, which creates a system in which we compare only samples that we already know are highly likely to be similar. We make a sketch by hashing multiple minhashes together.

When we get a new sample, we check whether the database contains any sketches that match the new sample's sketches. If so, the new sample is compared with the matching samples using their minhash arrays to approximate the Jaccard index between the new sample and the older, similar samples. This avoids having to compare the new sample to all samples in the database, and instead comparing it to only those samples that are highly likely to have high Jaccard indices with this new sample.

Building a Persistent Malware Similarity Search System

Now that you've learned the pros and cons of using a variety of malware feature types to estimate shared code relationships between malware samples. You've also learned about the Jaccard index, similarity matrices, and the way in which minhash can make computing similarities between malware samples in even very large datasets tractable. With all this knowledge in hand, you understand all of the fundamental concepts necessary to build a scalable malware shared code search system.

Listings 5-7 through 5-12 show an example of a simple system in which I index malware samples based on their string features. In your own work, you should feel confident in modifying this system to use other malware features, or extending it to support more visualization features. Because the listing is long, I've broken it up and we'll cover each subsection in turn.

To begin, Listing 5-7 imports the Python packages required for our program.

```
#!/usr/bin/python

import argparse
import os
import murmur
import shelve
import numpy as np
from listings_5_2_to_5_6 import *

NUM_MINHASHES = 256
SKETCH_RATIO = 8
```

Listing 5-7: Importing Python modules and declaring minhash-related constants

Here, I import packages like `murmur`, `shelve`, and `sim_graph`. For example, `murmur` is a hashing library that we use to compute the minhash algorithm I just discussed. We use `shelve`, a simple database module included in the Python standard library, to store information about samples and their minhashes, which we use to compute similarities. We use `listings_5_2_to_5_6.py` to get functions for computing sample similarity.

We also declare two constants in Listing 5-7: `NUM_MINHASHES` and `SKETCH_RATIO`. These correspond to the number of minhashes and the ratio of minhashes to sketches we compute for each sample. Recall that the more minhashes and sketches we use, the more accurate our similarity computations. For example, 256 minhashes and a ratio of 8:1 (32 sketches) is enough to yield acceptable accuracy at a low computational cost.

Listing 5-8 implements database functionality that we use to initialize, access, and delete the `shelve` database we use to store malware sample information.

```
❶ def wipe_database():
    """
    This problem uses the python standard library 'shelve' database to persist
    information, storing the database in the file 'samples.db' in the same
    directory as the actual Python script. 'wipe_database' deletes this file
    effectively resetting the system.
    """
    dbpath = "".join(__file__.split('/')[:-1] + ['samples.db'])
    os.system("rm -f {}".format(dbpath))

❷ def get_database():
    """
    Helper function to retrieve the 'shelve' database, which is a simple
    key value store.
```

```

"""
dbpath = "/".join(__file__.split('/')[:-1] + ['samples.db'])
return shelve.open(dbpath,protocol=2,writeback=True)

```

Listing 5-8: Database helper functions

We define `wipe_database()` ❶ to delete our program's database in case we want to wipe out sample information we've stored and start over. Then we define `get_database()` ❷ to open our database, creating it if it doesn't yet exist, and then return a database object, allowing us to store and retrieve data about our malware samples.

Listing 5-9 implements a core piece of the code for our shared code analysis: `minhash`.

```

def minhash(features):
    """
    This is where the minhash magic happens, computing both the minhashes of
    a sample's features and the sketches of those minhashes. The number of
    minhashes and sketches computed is controlled by the NUM_MINHASHES and
    NUM_SKETCHES global variables declared at the top of the script.
    """
    minhashes = []
    sketches = []
    ❶ for i in range(NUM_MINHASHES):
        minhashes.append(
            ❷ min([murmur.string_hash(`feature`,i) for feature in features])
        )
    ❸ for i in xrange(0,NUM_MINHASHES,SKETCH_RATIO):
        ❹ sketch = murmur.string_hash(`minhashes[i:i+SKETCH_RATIO]`)
        sketches.append(sketch)
    return np.array(minhashes),sketches

```

Listing 5-9: Obtaining minhashes and sketches for a sample

We loop `NUM_MINHASHES` times ❶ and append one minhash value. Each minhash value is computed by hashing all the features and then taking the minimum hash value. To perform this computation, we use the `murmur` package's `string_hash()` function to hash the features, and then we take the minimum value of the list of hashes by calling Python's `min()` function ❷.

The second argument of `string_hash` is a seed value, which causes the hash function to map to different hashes depending on the seed's value. Because each minhash value requires a unique hash function such that all of our 256 min hash values aren't identical, on each iteration we seed the `string_hash` function with our counter value `i`, which causes the features to map to different hashes on each iteration.

Then, we loop over the minhashes we've computed and use the minhashes to compute sketches ❸. Recall that sketches are hashes of multiple minhashes, which we use for database indexing of our malware samples so that we can quickly retrieve samples that are likely to be similar to one another by querying the database. In the next code listing, we

loop over all of our sample's minhashes with step size `SKETCH_RATIO`, hashing each chunk of hashes as we go to obtain our sketches. Finally, we use `murmur` package's `string_hash` function to hash the minhashes together ❹.

Listing 5-10 uses `get_database()` from Listing 5-8, the `getstrings()` function from the `sim_graph` module we imported, and the `minhash()` function from Listing 5-9 to create a function that indexes samples into our system's database.

```
def store_sample(path):
    """
    Function that stores a sample and its minhashes and sketches in the
    'shelve' database
    """
    ❶ db = get_database()
    ❷ features = getstrings(path)
    ❸ minhashes, sketches = minhash(features)

    ❹ for sketch in sketches:
        sketch = str(sketch)
        ❺ if not sketch in db:
            db[sketch] = set([path])
        else:
            obj = db[sketch]
            ❻ obj.add(path)
            db[sketch] = obj
        db[path] = {'minhashes':minhashes,'comments':[]}
        db.sync()

    print "Extracted {0} features from {1} ...".format(len(features),path)
```

Listing 5-10: Storing a sample's minhashes in the shelve database by using its sketches as keys

We call `get_database()` ❶, `getstrings()` ❷, and `minhash()` ❸ and then iterate over our sample's sketches starting at ❹. Next, to index our samples in the database, we use a technique known as *inverted indexing*, which allows us to store samples based on their *sketch values* instead of an ID. More specifically, for each of a sample's 32 sketch values, we look up that sketch's record in the database and append our sample's ID to the list of samples associated with that sketch. Here, we use a sample's filesystem path as its ID.

You can see how this is implemented in the code: we loop over the sketches we've computed for a sample ❹, we create a record for the sketch if it doesn't already exist (associating our sample with the sketch while we're at it) ❺, and finally, we add the sample path to the sketch's set of associated sample paths if the sketch's record does exist ❻.

Listing 5-11 shows the declaration of two important functions: `comment_sample()` and `search_sample()`.

```
❶ def comment_sample(path):
    """
```

Function that allows a user to comment on a sample. The comment the user provides shows up whenever this sample is seen in a list of similar samples to some new samples, allowing the user to reuse their knowledge about their malware database.

```
"""
db = get_database()
comment = raw_input("Enter your comment:")
if not path in db:
    store_sample(path)
comments = db[path]['comments']
comments.append(comment)
db[path]['comments'] = comments
db.sync()
print "Stored comment:", comment
```

❷ def search_sample(path):

Function searches for samples similar to the sample provided by the 'path' argument, listing their comments, filenames, and similarity values

```
"""
db = get_database()
features = getstrings(path)
minhashes, sketches = minhash(features)
neighbors = []

❸ for sketch in sketches:
    sketch = str(sketch)

    if not sketch in db:
        continue

    ❹ for neighbor_path in db[sketch]:
        neighbor_minhashes = db[neighbor_path]['minhashes']
        similarity = (neighbor_minhashes == minhashes).sum()
        / float(NUM_MINHASHES)
        neighbors.append((neighbor_path, similarity))

neighbors = list(set(neighbors))
❺ neighbors.sort(key=lambda entry:entry[1], reverse=True)
print ""
print "Sample name".ljust(64), "Shared code estimate"
for neighbor, similarity in neighbors:
    short_neighbor = neighbor.split("/")[1]
    comments = db[neighbor]['comments']
    print str("[*] "+short_neighbor).ljust(64), similarity
    for comment in comments:
        print "\t[comment]",comment
```

Listing 5-11: Declaring functions that allow users to comment on samples and search for samples similar to a query sample

As expected, `comment_sample()` ❶ adds a user-defined comment record to a sample's database record. This functionality is useful, because it allows users of the program to include insights gained from reverse-engineering a sample in the database such that when they see a new sample similar to samples they have comments for, they can leverage those comments to more rapidly understand the origins and purpose of the new sample.

Next, `search_sample()` ❷ leverages minhash to find samples similar to a query sample. To do this, first we extract string features, minhashes, and sketches from the query sample. Then, we iterate over the sample's sketches, looking up samples stored in the database that also have that sketch ❸. For each sample that shares a sketch with the query sample, we compute its approximate Jaccard index using its minhashes ❹. Finally, we report the most similar samples to the query sample to the user, along with any comments associated with these samples that have been stored in the database ❺.

Listing 5-12 concludes our program's code by implementing the argument-parsing part of our program.

```

if __name__ == '__main__':
    parser = argparse.ArgumentParser(
        description="""
Simple code-sharing search system which allows you to build up
a database of malware samples (indexed by file paths) and
then search for similar samples given some new sample
"""
    )

    parser.add_argument(
        "-l", "--load", dest="load", default=None,
        help="Path to malware directory or file to store in database"
    )

    parser.add_argument(
        "-s", "--search", dest="search", default=None,
        help="Individual malware file to perform similarity search on"
    )

    parser.add_argument(
        "-c", "--comment", dest="comment", default=None,
        help="Comment on a malware sample path"
    )

    parser.add_argument(
        "-w", "--wipe", action="store_true", default=False,
        help="Wipe sample database"
    )

    args = parser.parse_args()
    ❶ if args.load:
        malware_paths = [] # where we'll store the malware file paths
        malware_features = dict() # where we'll store the malware strings

```

```

for root, dirs, paths in os.walk(args.load):
    # walk the target directory tree and store all of the file paths
    for path in paths:
        full_path = os.path.join(root, path)
        malware_paths.append(full_path)

# filter out any paths that aren't PE files
malware_paths = filter(pecheck, malware_paths)

# get and store the strings for all of the malware PE files
for path in malware_paths:
    store_sample(path)

❷ if args.search:
    search_sample(args.search)

❸ if args.comment:
    comment_sample(args.comment)

❹ if args.wipe:
    wipe_database()

```

Listing 5-12: Performing similarity database updates and queries based on user command line arguments

Here, we allow users to load malware samples into the database so that these samples will be compared with new malware samples when users search similar samples in the database ❶. Next, we allow users to search for samples similar to the sample the user has passed in ❷, printing the results to the terminal. We also allow the user to comment on samples already in the database ❸. Finally, we allow the user to wipe the existing database ❹.

Running the Similarity Search System

Once you've implemented this code, you can run the similarity search system, which consists of four simple operations:

Load Loading the samples into the system stores them in the system database for future code-sharing searches. You can load samples individually or specify a directory, which the system will search recursively for PE files, loading them into the database. You can load samples into the database with the following command run in this chapter's code directory:

```
python listings_5_7_to_5_12.py -l <path to directory or individual malware sample>
```

Comment Commenting on a sample is useful because it allows you to store knowledge about that sample. Also, when you see new samples similar to this sample, a similarity search over those samples will reveal the comments you made on the older, similar sample, thus speeding up your workflow. You can comment on a malware sample with the following command:

```
python listings_5_7_to_5_12.py -c <path to malware sample>
```

Search Given a single malware sample, searching identifies all similar samples in the database and prints them in descending order of similarity. Also, any comments you might have made on those samples are printed. You can search for malware samples similar to a given sample using the following command:

```
python listings_5_7_to_5_12.py -s <path to malware sample>
```

Wipe Wiping the database simply clears all records from the system database, which you can do with the following command:

```
python listings_5_7_to_5_12.py -w
```

Listing 5-13 shows what it looks like when we load the APT1 samples into the system.

```
mds@mds:~/malware_data_science/ch5/code$ python listings_5_7_to_5_12.py -l ../data
Extracted 240 attributes from ../data/APT1_MALWARE_FAMILIES/WEBC2-YAHOO/WEBC2-YAHOO_sample/WEBC2-YAHOO_sample_A8F259BB36E00D124963CFA9B86F502E ...
Extracted 272 attributes from ../data/APT1_MALWARE_FAMILIES/WEBC2-YAHOO/WEBC2-YAHOO_sample/WEBC2-YAHOO_sample_0149B7BD7218AAB4E257D28469FDDB0D ...
Extracted 236 attributes from ../data/APT1_MALWARE_FAMILIES/WEBC2-YAHOO/WEBC2-YAHOO_sample/WEBC2-YAHOO_sample_CC3A9A7B026BFE0E55FF219FD6AA7D94 ...
Extracted 272 attributes from ../data/APT1_MALWARE_FAMILIES/WEBC2-YAHOO/WEBC2-YAHOO_sample/WEBC2-YAHOO_sample_1415EB8519D13328091CC5C76A624E3D ...
Extracted 236 attributes from ../data/APT1_MALWARE_FAMILIES/WEBC2-YAHOO/WEBC2-YAHOO_sample/WEBC2-YAHOO_sample_7A670D13D4D014169C4080328B8FEB86 ...
Extracted 243 attributes from ../data/APT1_MALWARE_FAMILIES/WEBC2-YAHOO/WEBC2-YAHOO_sample/WEBC2-YAHOO_sample_37DDD3D72EAD03C7518F5D47650C8572 ...
--snip--
```

Listing 5-13: Sample output from the loading data into the similarity search system implemented in this chapter

And Listing 5-14 shows what it looks like when we perform a similarity search.

```
mds@mds:~/malware_data_science/ch5/code$ python listings_5_7_to_5_12.py -s \
../data/APT1_MALWARE_FAMILIES/GREENCAT/GREENCAT_sample/GREENCAT_sample_AB20\
8F0B517BA9850F1551C9555B5313
```

Sample name estimate	Shared code
[*] GREENCAT_sample_5AEAA53340A281074FCB539967438E3F	1.0
[*] GREENCAT_sample_1F92FF8711716CA795FBD81C477E45F5	1.0
[*] GREENCAT_sample_3E69945E5865CCC861F69B24BC1166B6	1.0
[*] GREENCAT_sample_AB208F0B517BA9850F1551C9555B5313	1.0
[*] GREENCAT_sample_3E6ED3EE47BCE9946E2541332CB34C69	0.99609375
[*] GREENCAT_sample_C044715C2626AB515F6C85A21C47C7DD	0.6796875
[*] GREENCAT_sample_871CC547FEB9DBEC0285321068E392B8	0.62109375
[*] GREENCAT_sample_57E79F7DF13C0CB01910D0C688FCD296	0.62109375

Listing 5-14: Sample output from the similarity search system implemented in this chapter

Note that our system correctly determines that the query sample (a “greencat” sample) shares code with other greencat samples. If we didn’t have the luxury of already knowing these samples were members of the greencat family, our system would have just saved us a ton of reverse engineering work.

This similarity search system is only a small example of what would be implemented in a production similarity search system. But you should have no problem using what you learned so far to add visualization capabilities to the system and extend it to support multiple similarity search methods.

Summary

In this chapter, you learned how to identify shared code relationships between malware samples, compute code sharing similarity over thousands of malware samples to identify new malware families, determine a new malware sample’s code similarity to thousands of previously seen malware samples, and visualize malware relationships to understand patterns of code sharing.

You should now feel comfortable adding shared code analysis to your malware analysis toolbox, which will allow you to gain fast intelligence over large volumes of malware and accelerate your malware analysis workflow.

In Chapters 6, 7, and 8, you’ll learn to build machine learning systems for detecting malware. Combining these detection techniques with what you’ve already learned will help you catch advanced malware that other tools miss, as well as analyze its relationships to other known malware to gain clues about who deployed the malware and what their goals are.

6

UNDERSTANDING MACHINE LEARNING-BASED MALWARE DETECTORS



With the open source machine learning tools available today, you can build custom, machine learning-based malware detection tools, whether as your primary detection tool or to supplement commercial solutions, with relatively little effort.

But why build your own machine learning tools when commercial anti-virus solutions are already available? When you have access to examples of particular threats, such as malware used by a certain group of attackers targeting your network, building your own machine learning-based detection technologies can allow you to catch new examples of these threats.

In contrast, commercial antivirus engines might miss these threats unless they already include signatures for them. Commercial tools are also “closed books”—that is, we don’t necessarily know how they work and we have limited ability to tune them. When we build our own detection methods, we know how they work and can tune them to our liking to reduce false positives or false negatives. This is helpful because in some applications you might be willing to tolerate more false positives in exchange for fewer false negatives

(for example, when you're searching your network for suspicious files so that you can hand-inspect them to determine if they are malicious), and in other applications you might be willing to tolerate more false negatives in exchange for fewer false positives (for example, if your application blocks programs from executing if it determines they are malicious, meaning that false positives are disruptive to users).

In this chapter, you learn the process of developing your own detection tools at a high level. I start by explaining the big ideas behind machine learning, including feature spaces, decision boundaries, training data, underfitting, and overfitting. Then I focus on four foundational approaches—logistic regression, k-nearest neighbors, decision trees, and random forest—and how these can be applied to perform detection.

You'll then use what you learned in this chapter to learn how to evaluate the accuracy of machine learning systems in Chapter 7 and implement machine learning systems in Python in Chapter 8. Let's get started.

Steps for Building a Machine Learning–Based Detector

There is a fundamental difference between machine learning and other kinds of computer algorithms. Whereas traditional algorithms tell the computer what to do, machine-learning systems learn how to solve a problem by example. For instance, rather than simply pulling from a set of preconfigured rules, machine learning security detection systems can be trained to determine whether a file is bad or good by learning from examples of good and bad files.

The promise of machine learning systems for computer security is that they automate the work of creating signatures, and they have the potential to perform more accurately than signature-based approaches to malware detection, especially on new, previously unseen malware.

Essentially, the workflow we follow to build any machine learning–based detector, including a decision tree, boils down to these steps:

1. **Collect** examples of malware and benignware. We will use these examples (called *training examples*) to train the machine learning system to recognize malware.
2. **Extract** features from each training example to represent the example as an array of numbers. This step also includes research to design good features that will help your machine learning system make accurate inferences.
3. **Train** the machine learning system to recognize malware using the features we have extracted.
4. **Test** the approach on some data not included in our training examples to see how well our detection system works.

Let's discuss each of these steps in more detail in the following sections.

Gathering Training Examples

Machine learning detectors live or die by the training data provided to them. Your malware detector's ability to recognize suspicious binaries depends heavily on the quantity and quality of training examples you provide. Be prepared to spend much of your time gathering training examples when building machine learning-based detectors, because the more examples you feed your system, the more accurate it's likely to be.

The quality of your training examples is also important. The malware and benignware you collect should mirror the kind of malware and benignware you expect your detector to see when you ask it to decide whether new files are malicious or benign.

For example, if you want to detect malware from a specific threat actor group, you must collect as much malware as possible from that group for use in training your system. If your goal is to detect a broad class of malware (such as ransomware), it's essential to collect as many representative samples of this class as possible.

By the same token, the benign training examples you feed your system should mirror the kinds of benign files you will ask your detector to analyze once you deploy it. For example, if you are working on detecting malware on a university network, you should train your system with a broad sampling of the benignware that students and university employees use, in order to avoid false positives. These benign examples would include computer games, document editors, custom software written by the university IT department, and other types of nonmalicious programs.

To give a real-world example, at my current day job, we built a detector that detects malicious Office documents. We spent about half the time on this project gathering training data, and this included collecting benign documents generated by more than a thousand of my company's employees. Using these examples to train our system significantly reduced our false positive rate.

Extracting Features

To classify files as good or bad, we train machine learning systems by showing them features of software binaries; these are file attributes that will help the system distinguish between good and bad files. For example, here are some features we might use to determine whether a file is good or bad:

- Whether it's digitally signed
- The presence of malformed headers
- The presence of encrypted data
- Whether it has been seen on more than 100 network workstations

To obtain these features, we need to extract them from files. For example, we might write code to determine whether a file is digitally signed, has malformed headers, contains encrypted data, and so on.

Often, in security data science, we use a huge number of features in our machine learning detectors. For example, we might create a feature for every library call in the Win32 API, such that a binary would have that feature if it had the corresponding API call. We'll revisit feature extraction in Chapter 8, where we discuss more advanced feature extraction concepts as well as how to use them to implement machine learning systems in Python.

Designing Good Features

Our goal should be to select features that yield the most accurate results. This section provides some general rules to follow.

First, when selecting features, choose ones that represent your best guess as to what might help a machine learning system distinguish bad files from good files. For example, the feature “contains encrypted data” might be a good marker for malware because we know that malware often contains encrypted data, and we're guessing that benignware will contain encrypted data more rarely. The beauty of machine learning is that if this hypothesis is wrong, and benignware contains encrypted data just as often as malware does, the system will more or less ignore this feature. If our hypothesis is right, the system will learn to use the “contains encrypted data” feature to detect malware.

Second, don't use so many features that your set of features becomes too large relative to the number of training examples for your detection system. This is what the machine learning experts call “the curse of dimensionality.” For example, if you have a thousand features and only a thousand training examples, chances are you don't have enough training examples to teach your machine learning system what each feature actually says about a given binary. Statistics tells us that it's better to give your system a few features relative to the number of training examples you have available and let it form well-founded beliefs about which features truly indicate malware.

Finally, make sure your features represent a range of hypotheses about what constitutes malware or benignware. For example, you may choose to build features related to encryption, such as whether a file uses encryption-related API calls or a public key infrastructure (PKI), but make sure to also use features unrelated to encryption to hedge your bets. That way, if your system fails to detect malware based on one type of feature, it might still detect it using other features.

Training Machine Learning Systems

After you've extracted features from your training binaries, it's time to train your machine learning system. What this looks like algorithmically depends completely on the machine learning approach you're using. For example, training a decision tree approach (which we discuss shortly) involves a different learning algorithm than training a logistic regression approach (which we also discuss).

Fortunately, all machine learning detectors provide the same basic interface. You provide them with training data that contains features from sample binaries, as well as corresponding labels that tell the algorithm

which binaries are malware and which are benignware. Then the algorithms learn to determine whether or not new, previously unseen binaries are malicious or benign. We cover training in more detail later in this chapter.

NOTE

In this book, we focus on a class of machine learning algorithms known as supervised machine learning algorithms. To train models using these algorithms, we tell them which examples are malicious and which are benign. Another class of machine learning algorithms, unsupervised algorithms, does not require us to know which examples are malicious or benign in our training set. These algorithms are much less effective at detecting malicious software and malicious behavior, and we will not cover them in this book.

Testing Machine Learning Systems

Once you've trained your machine learning system, you need to check how accurate it is. You do this by running the trained system on data that you didn't train it on and seeing how well it determines whether or not the binaries are malicious or benign. In security, we typically train our systems on binaries that we gathered up to some point in time, and then we test on binaries that we saw *after* that point in time, to measure how well our systems will detect new malware, and to measure how well our systems will avoid producing false positives on new benignware. Most machine learning research involves thousands of iterations that go something like this: we create a machine learning system, test it, and then tweak it, train it again, and test it again, until we're satisfied with the results. I'll cover testing machine learning systems in detail in Chapter 8.

Let's now discuss how a variety of machine learning algorithms work. This is the hard part of the chapter, but also the most rewarding if you take the time to understand it. In this discussion, I talk about the unifying ideas that underlie these algorithms and then move on to each algorithm in detail.

Understanding Feature Spaces and Decision Boundaries

Two simple geometric ideas can help you understand all machine learning-based detection algorithms: the idea of a geometrical feature space and the idea of a decision boundary. A *feature space* is the geometrical space defined by the features you've selected, and a *decision boundary* is a geometrical structure running through this space such that binaries on one side of this boundary are defined as malware, and binaries on the other side of the boundary are defined as benignware. When we use a machine learning algorithm to classify files as malicious or benign, we extract features so that we can place the samples in the feature space, and then we check which side of the decision boundary the samples are on to determine whether the files are malware or benignware.

This geometrical way of understanding feature spaces and decision boundaries is accurate for systems that operate on feature spaces of one, two, or three dimensions (features), but it also holds for feature spaces with

millions of dimensions, even though it's impossible to visualize or conceive of million-dimensional spaces. We'll stick to examples with two dimensions in this chapter to make them easy to visualize, but just remember that real-world security machine learning systems pretty much always use hundreds, thousands, or millions of dimensions, and the basic concepts we discuss in a two-dimensional context hold for real-world systems that have more than two dimensions.

Let's create a toy malware detection problem to clarify the idea of a decision boundary in a feature space. Suppose we have a training dataset consisting of malware and benignware samples. Now suppose we extract the following two features from each binary: the percentage of the file that appears to be compressed, and the number of suspicious functions each binary imports. We can visualize our training dataset as shown in Figure 6-1 (bear in mind I created the data in the plot artificially, for example purposes).

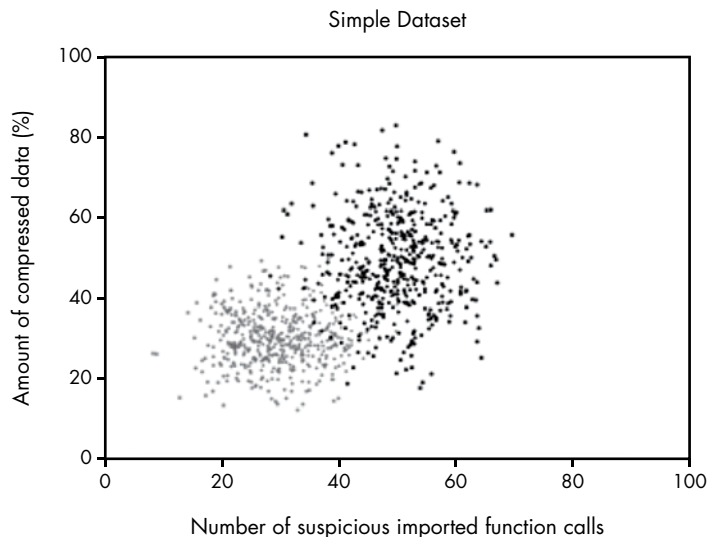


Figure 6-1: A plot of a sample dataset we'll use in this chapter, where gray dots are benignware and black dots are malware

The two-dimensional space shown in Figure 6-1, which is defined by our two features, is the feature space for our sample dataset. You can see a clear pattern in which the black dots (the malware) are generally in the upper-right part of the space. In general, these have more suspicious imported function calls and more compressed data than the benignware, which mostly inhabits the lower-left part of the plot. Suppose, after viewing this plot, you were asked to create a malware detection system based solely on the two features we're using here. It seems clear that, based on the data, you can formulate the following rule: if a binary has both a lot of compressed data and a lot of suspicious imported function calls, it's malware, and if it has neither a lot of suspicious imported calls nor much compressed data, it's benignware.

In geometrical terms, we can visualize this rule as a diagonal line that separates the malware samples from the benignware samples in the feature space, such that binaries with sufficient compressed data and imported function calls (defined as malware) are above the line, and the rest of the binaries (defined as benignware) are below the line. Figure 6-2 shows such a line, which we call a decision boundary.

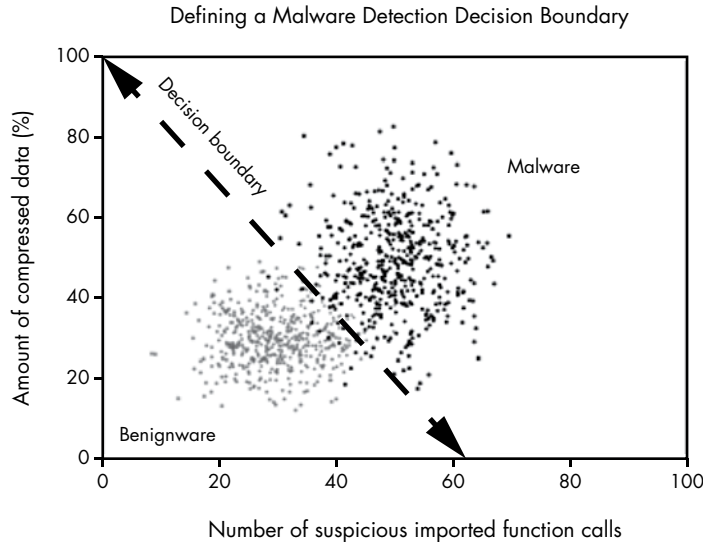


Figure 6-2: A decision boundary drawn through our sample dataset, which defines a rule for detecting malware

As you can see from the line, *most* of the black (malware) dots are on one side of the boundary, and *most* of the gray (benignware) samples are on the other side of the decision boundary. Note that it's impossible to draw a line that separates *all* of the samples from one another, because the black and gray clouds in this dataset overlap one another. But from looking at this example, it appears we've drawn a line that will correctly classify new malware samples and benignware samples in most cases, assuming they follow the pattern seen in the data in this image.

In Figure 6-2, we manually drew a decision boundary through our data. But what if we want a more exact decision boundary and want to do it in an automated way? This is exactly what machine learning does. In other words, all machine learning detection algorithms look at data and use an automated process to determine the ideal decision boundary, such that there's the greatest chance of correctly performing detection on new, previously unseen data.

Let's look at the way a real-world, commonly used machine learning algorithm identifies a decision boundary within the sample data shown in Figure 6-3. This example uses an algorithm called logistic regression.

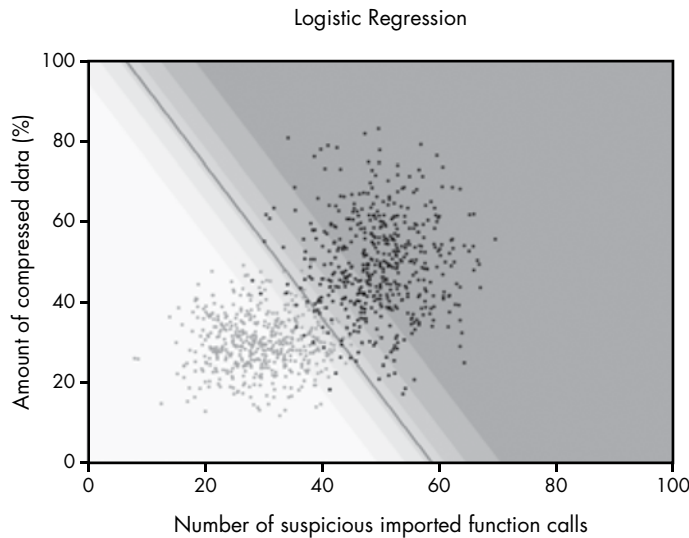


Figure 6-3: The decision boundary automatically created by training a logistic regression model

Notice that we're using the same sample data we used in the previous plots, where gray dots are benignware and black dots are malware. The line running through the center of the plot is the decision boundary that the logistic regression algorithm *learns* by looking at the data. On the right side of the line, the logistic regression algorithm assigns a greater than 50 percent probability that binaries are malware, and on the left side of the line, it assigns a less than 50 percent probability that a binary is malware.

Now note the shaded regions of the plot. The dark gray shaded region is the region where the logistic regression model is highly confident that files are malware. Any new file the logistic regression model sees that has features that land in this region should have a high probability of being malware. As we get closer and closer to the decision boundary, the model has less and less confidence about whether or not binaries are malware or benignware. Logistic regression allows us to easily move the line up into the darker region or down into the lighter region, depending on how aggressive we want to be about detecting malware. For example, if we move it down, we'll catch more malware, but get more false positives. If we move it up, we'll catch less malware, but get fewer false positives.

I want to emphasize that logistic regression, and all other machine learning algorithms, can operate in arbitrarily high dimensional feature spaces. Figure 6-4 illustrates how logistic regression works in a slightly higher dimensional feature space.

In this higher-dimensional space, the decision boundary is not a line, but a *plane* separating the points in the 3D volume. If we were to move to four or more dimensions, logistic regression would create a *hyperplane*, which is an n -dimensional plane-like structure that separates the malware from benignware points in this high dimensional space.

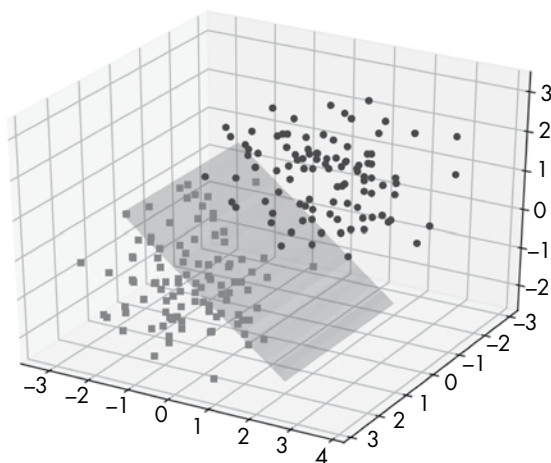


Figure 6-4: A planar decision boundary through a hypothetical three dimensional feature space created by logistic regression

Because logistic regression is a relatively simple machine learning algorithm, it can only create simple geometrical decision boundaries such as lines, planes, and higher dimensional planes. Other machine learning algorithms can create decision boundaries that are more complex. Consider, for example, the decision boundary shown in Figure 6-5, given by the k-nearest neighbors algorithm (which I discuss in detail shortly).

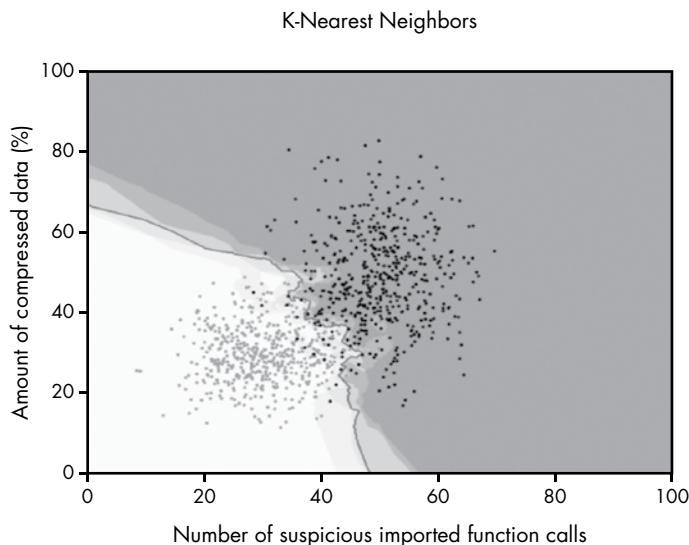


Figure 6-5: A decision boundary created by the k-nearest neighbors algorithm

As you can see, this decision boundary isn't a plane: it's a highly irregular structure. Also note that some machine learning algorithms can generate

disjointed decision boundaries, which define some regions of the feature space as malicious and some regions as benign, even if those regions are not contiguous. Figure 6-6 shows a decision boundary with this irregular structure, using a different sample dataset with a more complex pattern of malware and benignware in our sample feature space.

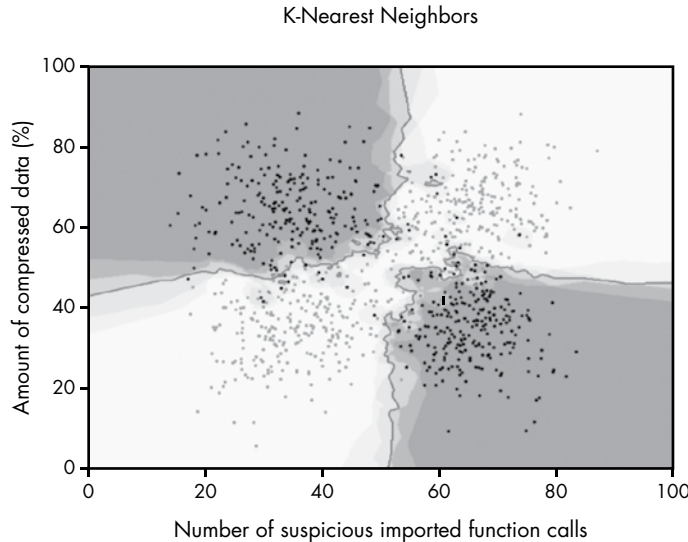


Figure 6-6: A disjoint decision boundary created by the k-nearest neighbors algorithm

Even though the decision boundary is noncontiguous, it's still common machine learning parlance to call these disjoint decision boundaries simply “decision boundaries.” You can use different machine learning algorithms to express different types of decision boundaries, and this difference in expressivity is why we might pick one machine learning algorithm over another for a given project.

Now that we've explored core machine learning concepts like feature spaces and decision boundaries, let's discuss what machine learning practitioners call overfitting and underfitting next.

What Makes Models Good or Bad: Overfitting and Underfitting

I can't overemphasize the importance of overfitting and underfitting in machine learning. Avoiding both cases is what defines a good machine learning algorithm. Good, accurate detection models in machine learning capture the general trend in what the training data says about what distinguishes malware from benignware, without getting distracted by the outliers or the exceptions that prove the rule.

Underfit models ignore outliers but fail to capture the general trend, resulting in poor accuracy on new, previously unseen binaries. Overfit models get distracted by outliers in ways that don't reflect the general trend, and they yield poor accuracy on previously unseen binaries. Building machine learning malware detection models is all about capturing the general trend that distinguishes the malicious from the benign.

Let's use the examples of underfit, well fit, and overfit models in Figures 6-7, 6-8, and 6-9 to illustrate these terms. Figure 6-7 shows an underfit model.

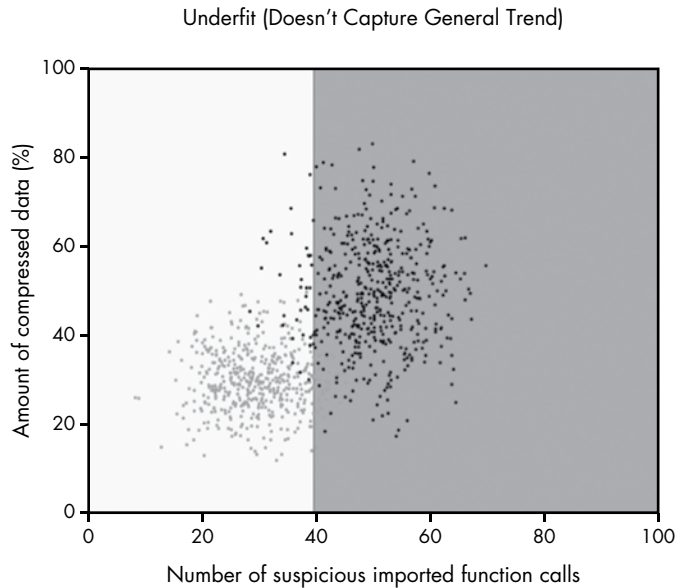


Figure 6-7: An underfit machine learning model

Here, you can see the black dots (malware) cluster in the upper-right region of the plot, and the gray dots (benignware) cluster in the lower-left. However, our machine learning model simply slices the dots down the middle, crudely separating the data without capturing the diagonal trend. Because the model doesn't capture the general trend, we say that it is underfit.

Also note that there are only two shades of certainty that the model gives in all of the regions of the plot: either the shade is dark gray or it's white. In other words, the model is either absolutely certain that points in the feature space are malicious or absolutely certain they're benign. This inability to express certainty correctly is also a reason this model is underfit.

Let's contrast the underfit model in Figure 6-7 with the well-fit model in Figure 6-8.

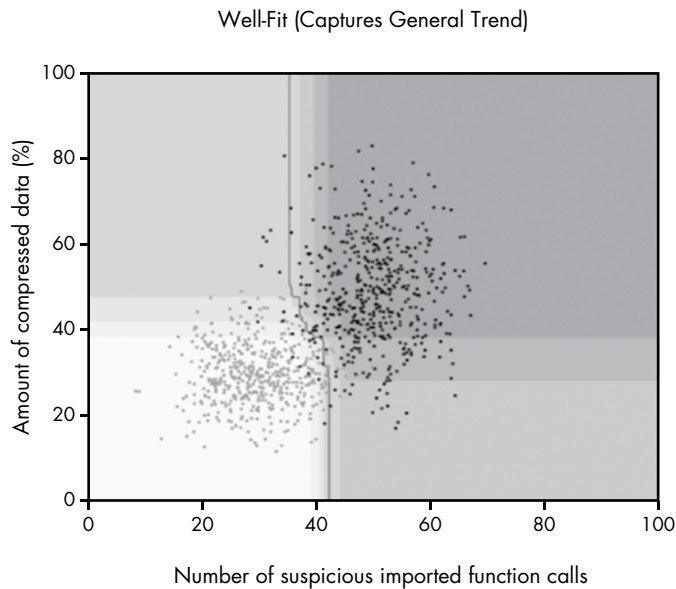


Figure 6-8: A well-fit machine learning model

In this case, the model not only captures the general trend in the data but also creates a reasonable model of certainty with respect to its estimate of which regions of the feature space are definitely malicious, definitely benign, or are in a gray area.

Note the decision line running from the top to the bottom of this plot. The model has a simple theory about what divides the malware from the benignware: a vertical line with a diagonal notch in the middle of the plot. Also note the shaded regions in the plot, which tells us that the model is only certain that data in the upper-right part of the plot are malware, and only certain that binaries in the lower-left corner of the plot are benignware.

Finally, let's contrast the overfit model shown next in Figure 6-9 to the underfit model you saw in Figure 6-7 as well as the well-fit model in Figure 6-8.

The overfit model in Figure 6-9 fails to capture the general trend in the data. Instead, it obsesses over the exceptions in the data, including the handful of black dots (malware training examples) that occur in the cluster of gray dots (benign training examples) and draws decision boundaries around them. Similarly, it focuses on the handful of benignware examples that occur in the malware cluster, drawing boundaries around those as well.

This means that when we see new, previously unseen binaries that happen to have features that place them close to these outliers, the machine learning model will think they are malware when they are almost definitely benignware, and vice versa. In practice, this means that this model won't be as accurate as it could be.

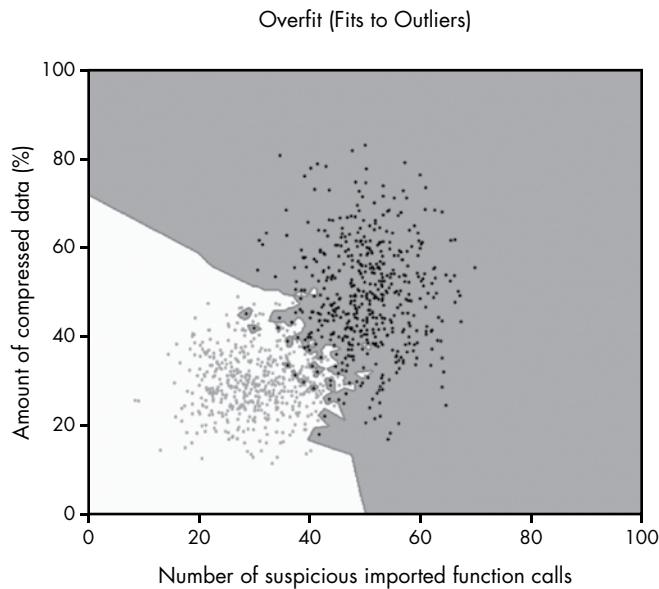


Figure 6-9: An overfit machine learning model

Major Types of Machine Learning Algorithms

So far I've discussed machine learning in very general terms, touching on two machine learning methods: logistic regression and k-nearest neighbors. In the remainder of this chapter, we delve deeper and discuss logistic regression, k-nearest neighbors, decision trees, and random forest algorithms in more detail. We use these algorithms quite often in the security data science community. These algorithms are complex, but the ideas behind them are intuitive and straightforward.

First, let's look at the sample datasets we use to explore the strengths and weaknesses of each algorithm, shown in Figure 6-10.

I created these datasets for example purposes. On the left, we have our simple dataset, which I've already used in Figures 6-7, 6-8, and 6-9. In this case, we can separate the black training examples (malware) from the gray training examples (benignware) using a simple geometric structure such as a line.

The dataset on the right, which I've already shown in Figure 6-6, is complex because we can't separate the malware from the benignware using a simple line. But there is still a clear pattern to the data: we just have to use more complex methods to create a decision boundary. Let's see how different algorithms perform with these two sample datasets.

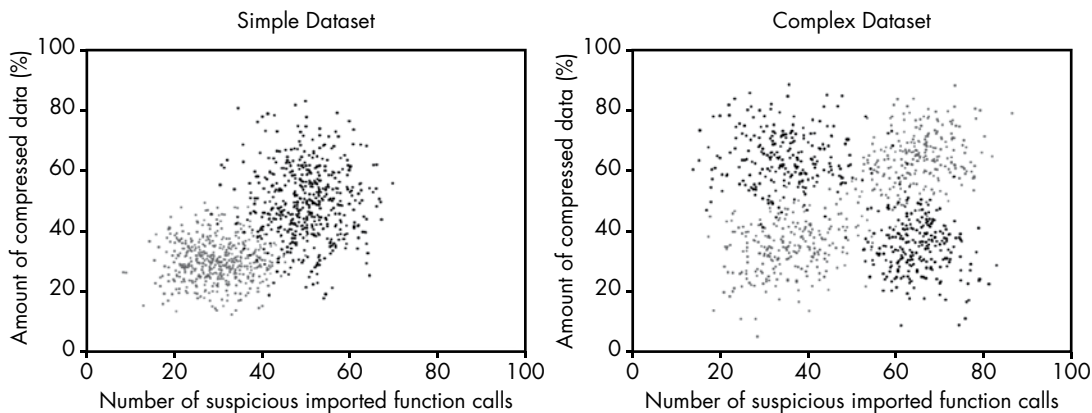


Figure 6-10: The two sample datasets we use in this chapter, with black dots representing malware and gray dots representing benignware

Logistic Regression

As you learned previously, logistic regression is a machine learning algorithm that creates a line, plane, or hyperplane (depending on how many features you provide) that geometrically separates your training malware from your training benignware. When you use the trained model to detect new malware, logistic regression checks whether a previously unseen binary is on the malware side or the benignware side of the boundary to determine whether it's malicious or benign.

A limitation of logistic regression is that if your data can't be separated simply using a line or hyperplane, logistic regression is not the right solution. Whether or not you can use logistic regression for your problem depends on your data and your features. For example, if your problem has lots of individual features that on their own are strong indicators of maliciousness (or "benignness"), then logistic regression might be a winning approach. If your data is such that you need to use complex relationships between features to decide that a file is malware, then another approach, like k-nearest neighbors, decision trees, or random forest, might make more sense.

To illustrate the strengths and weaknesses of logistic regression, let's look at the performance of logistic regression on our two sample datasets, as shown in Figure 6-11. We see that logistic regression yields a very effective separation of the malware and benignware in our simple dataset (on the left). In contrast, the performance of logistic regression on our complex dataset (on the right) is not effective. In this case, the logistic regression algorithm gets confused, because it can only express a linear decision boundary. You can see both binary types on both sides of the line, and the shaded gray confidence bands don't really make any sense relative to the data. For this more complex dataset, we'd need to use an algorithm capable of expressing more geometric structures.

Logistic Regression

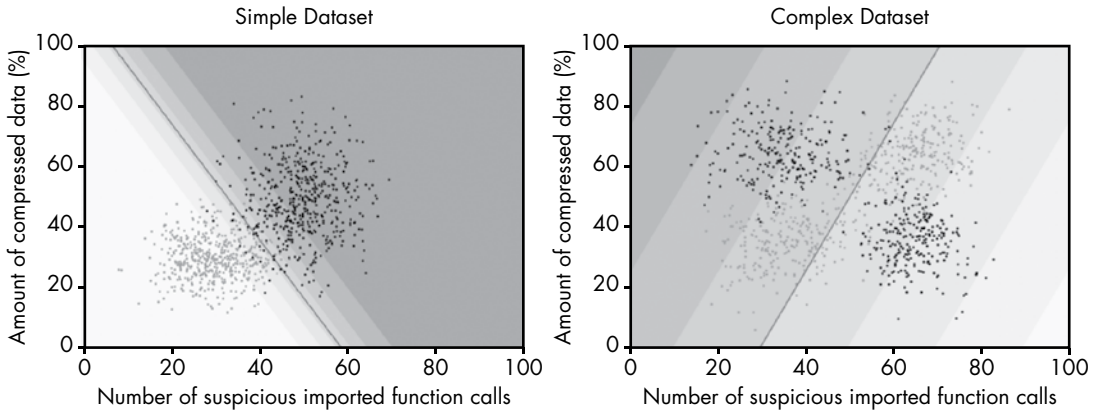


Figure 6-11: A decision boundary drawn through our sample datasets using logistic regression

The Math Behind Logistic Regression

Let's now look at the math behind how logistic regression detects malware samples. Listing 6-1 shows Pythonic pseudocode for computing the probability that a binary is malware using logistic regression.

```
def logistic_regression(compressed_data, suspicious_calls, learned_parameters): ❶
    compressed_data = compressed_data * learned_parameters["compressed_data_weight"] ❷
    suspicious_calls = suspicious_calls * learned_parameters["suspicious_calls_weight"]
    score = compressed_data + suspicious_calls + bias ❸
    return logistic_function(score)

def logistic_function(score): ❹
    return 1/(1.0+math.e**(-score))
```

Listing 6-1: Pseudocode using logistic regression to calculate probability

Let's step through the code to understand what this means. We first define the `logistic_regression` function ❶ and its parameters. Its parameters are the features of the binary (`compressed_data` and `suspicious_calls`) that represent the amount of compressed data and the number of suspicious calls it makes, respectively, and the parameter `learned_parameters` stands for the elements of the logistic regression function that were learned by training the logistic regression model on training data. I discuss how the parameters were learned later in this chapter; for now, just accept that they were derived from the training data.

Then, we take the `compressed_data` feature ❷ and multiply it by the `compressed_data_weight` parameter. This weight scales the feature up or down, depending on how indicative of malware the logistic regression function thinks this feature is. Note that the weight can also be negative, which indicates that the logistic regression model thinks that the feature is an indicator of a file being benign.

On the line below that, we perform the same step for the `suspicious_calls` parameter. Then, we add these two weighted features together ❸, plus add in a parameter called the bias parameter (also learned from training data). In sum, we take the `compressed_data` feature, scaled by how indicative of maliciousness we believe it to be, add the `suspicious_calls` feature, also scaled by how indicative of maliciousness we believe it to be, and add the bias parameter, which indicates how suspicious the logistic regression model thinks we should be of files in general. The result of these additions and multiplications is a score indicating how likely it is that a given file is malicious.

Finally, we use `logistic_function` ❹ to convert our suspiciousness score into a probability. Figure 6-12 visualizes how this function works.

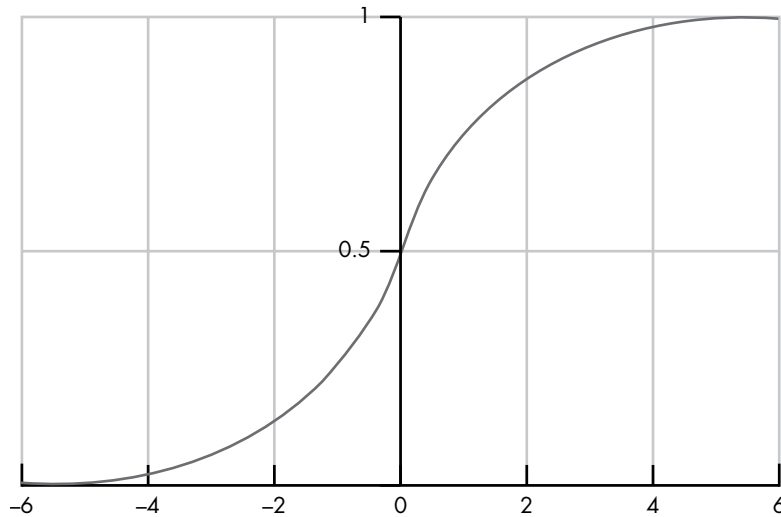


Figure 6-12: A plot of the logistic function used in logistic regression

Here, the logistic function takes a score (shown on the x-axis) and translates it into a value that's bounded between 0 and 1 (a probability).

How the Math Works

Let's return to the decision boundaries you saw in Figure 6-11 to see how this math works in practice. Recall how we compute our probability:

```
logistic_function(feature1_weight * feature1 + feature2_weight*feature2 + bias)
```

For example, if we were to plot the resulting probabilities at every point in the feature spaces shown in Figure 6-11 using the same feature weights and bias parameter, we'd wind up with the shaded regions shown in the same figure, which shows where the model “thinks” malicious and benign samples lie, and with how much confidence.

If we were then to set a threshold of 0.5 (recall that at a probability of greater than 50 percent, files are defined as malicious), the line in

Figure 6-11 would appear as our decision boundary. I encourage you to experiment with my sample code, plug in some feature weights and a bias term, and try it yourself.

NOTE

Logistic regression doesn't constrain us to using only two features. In reality, we usually use scores or hundreds or even thousands of features with logistic regression. But the math doesn't change: we just compute our probability as follows for any number of features:

```
logistic_function(feature1 * feature1_weight + feature2 * feature2_weight +  
feature3 * feature3_weight ... + bias)
```

So how exactly does logistic regression learn to place the decision boundary in the right place based on the training data? It uses an iterative, calculus-based approach called *gradient descent*. We won't get into the details of this approach in this book, but the basic idea is that the line, plane, or hyperplane (depending on the number of features you're using) is iteratively adjusted such that it maximizes the probability that the logistic regression model will get the answer right when asked if a data point in the training set is either a malware sample or a benignware sample.

You can train logistic regression models to bias the logistic regression learning algorithm toward coming up with simpler or more complex theories about what constitutes malware and benignware. These training methods are beyond the scope of this book, but if you're interested in learning about these helpful methods, I encourage you to Google “logistic regression and regularization” and read explanations of them online.

When to Use Logistic Regression

Logistic regression has distinct advantages and disadvantages relative to other machine learning algorithms. An advantage of logistic regression is that one can easily interpret what a logistic regression model thinks constitutes benignware and malware. For example, we can understand a given logistic regression model by looking at its feature weights. Features that have high weight are those the model interprets as malicious. Features with negative weight are those the model believes are benignware. Logistic regression is a fairly simple approach, and when the data you're working with contains clear indicators of maliciousness, it can work well. But when the data is more complex, logistic regression often fails.

Now let's explore another simple machine learning approach that can express much more complex decision boundaries: k-nearest neighbors.

K-Nearest Neighbors

K-nearest neighbors is a machine learning algorithm based on the idea that if a binary in the feature space is close to other binaries that are malicious, then it's malicious, and if its features place it close to binaries that are benign, it must be benign. More precisely, if the majority of the

k closest binaries to an unknown binary are malicious, the file is malicious. Note that k represents the number of nearby neighbors that we pick and define ourselves, depending on how many neighbors we think should be involved in determining whether a sample is benign or malicious.

In the real world, this makes intuitive sense. For example, if you have a dataset of weights and heights of both basketball players and table tennis players, chances are that the basketball players' weights and heights are likely closer to one another than they are to the measurements of table tennis players. Similarly, in a security setting, malware will often have similar features to other malware, and benignware will often have similar features to other benignware.

We can translate this idea into a k -nearest neighbors algorithm to compute whether a binary is malicious or benign using the following steps:

1. Extract the binary's features and find the k samples that are closest to it in the feature space.
2. Divide the number of malware samples that are close to the sample by k to get the percentage of nearest neighbors that are malicious.
3. If enough of the samples are malicious, define the sample as malicious.

Figure 6-13 shows how k -nearest neighbors algorithm works at a high level.

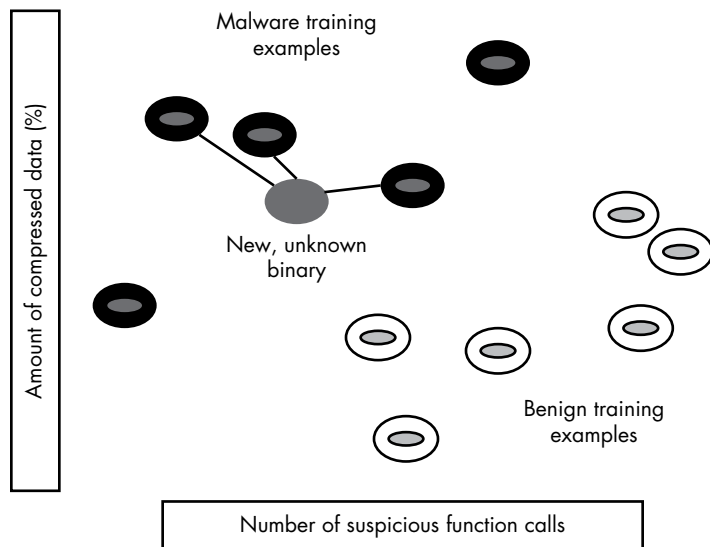


Figure 6-13: An illustration of the way k -nearest neighbors can be used to detect previously unseen malware

We see a set of malware training examples in the upper left and a set of benignware examples in the lower right. We also see a new, unknown binary that is connected to its three nearest neighbors. In this case, we've set k to 3,

meaning we're looking at the three nearest neighbors to unknown binaries. Because all three of the nearest neighbors are malicious, we'd classify this new binary as malicious.

The Math Behind K-Nearest Neighbors

Let's now discuss the math that allows us to compute the distance between new, unknown binaries' features and the samples in the training set. We use a *distance function* to do this, which tells us the distance between our new example and the examples in the training set. The most common distance function is *Euclidean distance*, which is the length of the shortest path between two points in our feature space. Listing 6-2 shows pseudocode for Euclidean distance in our sample two-dimensional feature space.

```
import math
def euclidean_distance(compression1,suspicious_calls1, compression2, suspicious_calls2): ❶
    comp_distance = (compression1-compression2)**2 ❷
    call_distance = (suspicious_calls1-suspicious_calls2)**2 ❸
    return math.sqrt(comp_distance + call_distance) ❹
```

Listing 6-2: Pseudocode for writing the *euclidean_distance* function

Let's walk through how the math in this code works. Listing 6-2 takes a pair of samples and computes the distance between them based on the differences between their features. First, the caller passes in the features of the binaries ❶, where *compression1* is the compression feature of the first example, *suspicious_calls1* is the suspicious_calls feature of the first example, *compression2* is the compression feature of the second example, and *suspicious_calls2* is the suspicious calls feature of the second example.

Then we compute the squared difference between the compression features of each sample ❷, and we compute the squared difference between the suspicious calls feature of each sample ❸. We won't cover the reason we use squared distance, but note that the resulting difference is always positive. Finally, we compute the square root of the two differences, which is the Euclidean distance between the two feature vectors, and return it to the caller ❹. Although there are other ways to compute distances between examples, Euclidean distance is the most commonly used with the k-nearest neighbors algorithm, and it works well for security data science problems.

Choosing the Number of Neighbors That Vote

Let's now look at the kinds of decision boundaries and probabilities that a k-nearest neighbors algorithm produces for the sample datasets we're using in this chapter. In Figure 6-14, I set *k* to 5, thus allowing five closest neighbors to "vote."

K-Nearest Neighbors, 5 Neighbors

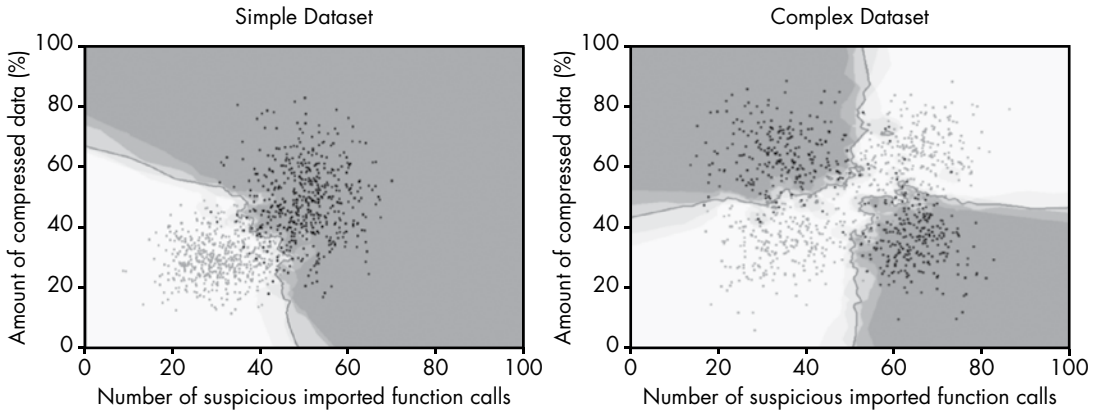


Figure 6-14: The decision boundaries created by k -nearest neighbors when k is set to 5

But in Figure 6-15, I set k to 50, allowing the 50 closest neighbors to “vote.”

K-Nearest Neighbors, 50 Neighbors

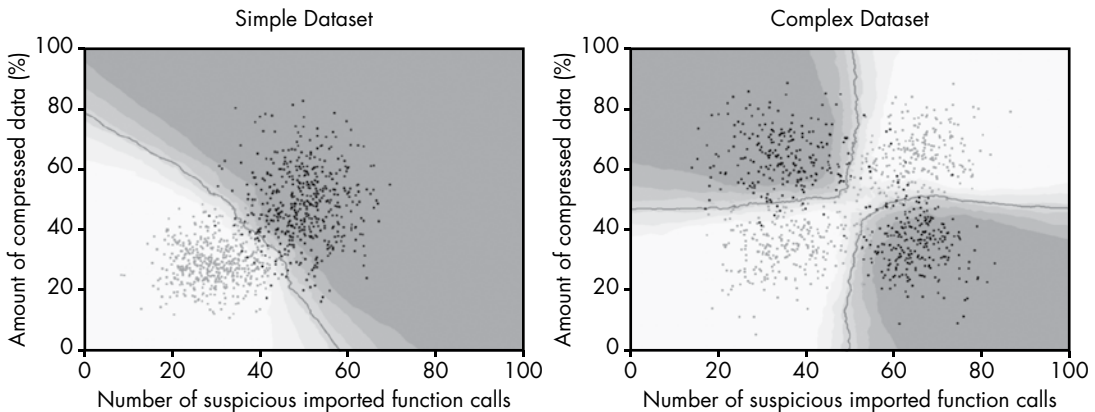


Figure 6-15: The decision boundaries created by k -nearest neighbors when k is set to 50

Note the dramatic difference between the models depending on the number of neighbors that vote. The model in Figure 6-14 shows a gnarly, complex decision boundary for both datasets, which is overfit in the sense that it draws local decision boundaries around outliers, but underfit because it fails to capture the simple, general trends. In contrast, the model in Figure 6-15 is well-fit to both datasets, because it doesn’t get distracted by outliers and cleanly identifies general trends.

As you can see, k -nearest neighbors can produce a much more complex decision boundary than logistic regression. We can control the complexity of this boundary to guard against both over- and underfitting by changing k , the number of neighbors that get to vote on whether a sample is

malicious or benign. Whereas the logistic regression model in Figure 6-11 got it completely wrong, k-nearest neighbors does well at separating the malware from the benignware, especially when we let 50 neighbors vote. Because k-nearest neighbors is not constrained by a linear structure and is simply looking at the nearest neighbors of each point to make a decision, it can create decision boundaries with arbitrary shapes, thus modeling complex datasets much more effectively.

When to Use K-Nearest Neighbors

K-nearest neighbors is a good algorithm to consider when you have data where features don't map cleanly onto the concept of suspiciousness, but closeness to malicious samples is a strong indicator of maliciousness. For example, if you're trying to classify malware into families that share code, k-nearest neighbors might be a good algorithm to try, because you want to classify a malware sample into a family if its features are similar to known members of a given family.

Another reason to use k-nearest neighbors is that it provides clear explanations of *why* it has made a given classification decision. In other words, it's easy to identify and compare similarities between samples and an unknown sample to figure out why the algorithm has classified it as malware or benignware.

Decision Trees

Decision trees are another frequently used machine learning method for solving detection problems. Decision trees automatically generate a series of questions through a training process to decide whether or not a given binary is malware, similar to the game Twenty Questions. Figure 6-16 shows a decision tree that I automatically generated by training it on the simple dataset we've been using in this chapter. Let's follow the flow of the logic in the tree.

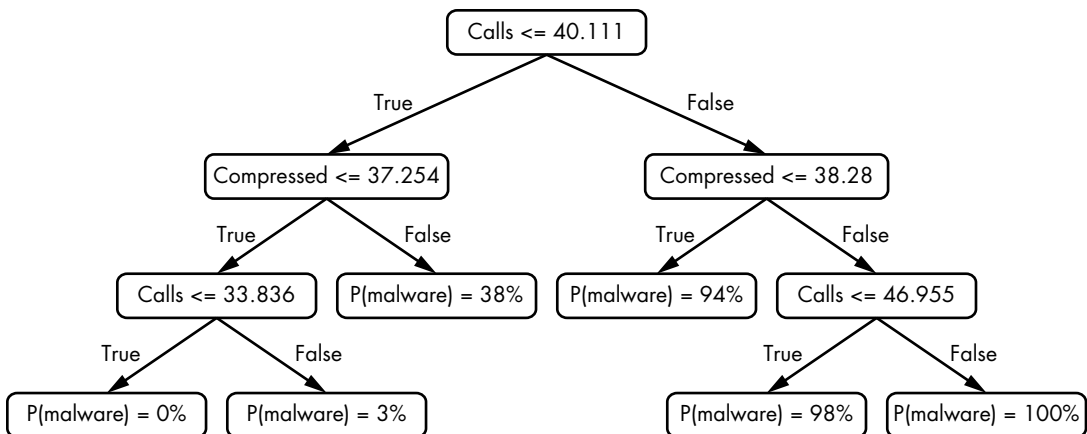


Figure 6-16: A decision tree learned for our simple dataset example

The decision tree flow starts when we input the features we've extracted from a new, previously unseen binary into the tree. Then the tree defines the series of questions to ask of this binary's features. The box at the top of the tree, which we call a *node*, asks the first question: is the number of suspicious calls in the tree less than or equal to 40.111? Note that the decision tree uses a floating point number here because we've normalized the number of suspicious calls in each binary to a range between 0 and 100. If the answer is "yes," we ask another question: is the percentage of compressed data in the file less than or equal to 37.254? If the answer is "yes," we proceed to the next question: is the number of suspicious calls in the binary less than or equal to 33.836? If the answer is "yes," we reach the end of the decision tree. At this point, the probability that the binary is malware is 0 percent.

Figure 6-17 shows a geometrical interpretation of this decision tree.

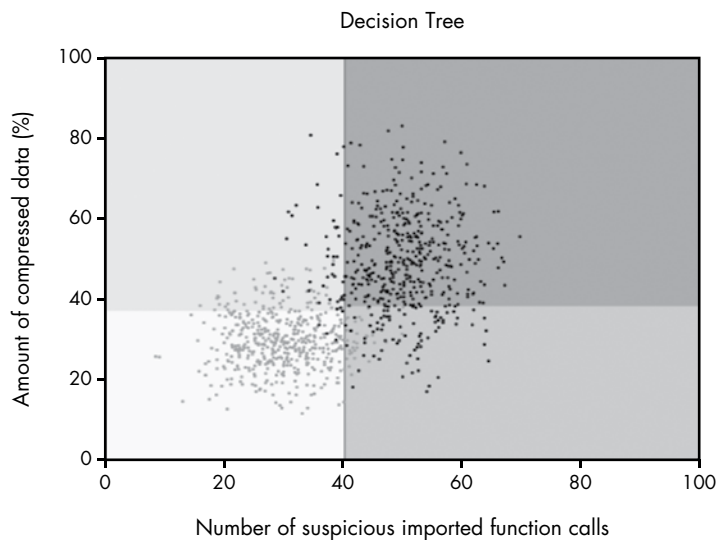


Figure 6-17: The decision boundary created by a decision tree for our simple dataset example

Here, the shaded regions indicate where the decision tree thinks samples are malicious. The lighter regions indicate where the decision tree thinks samples are benign. The probabilities assigned by the series of questions and answers in Figure 6-16 should correspond with those in the shaded regions in Figure 6-17.

Choosing a Good Root Node

So how do we use a machine learning algorithm to generate a decision tree like this from training data? The basic idea is that the decision tree starts with an initial question called a *root node*. The best root node is the one for which we get a "yes" answer for *most if not all* samples of one type, and a "no" answer for *most if not all* samples of the other type. For example,

in Figure 6-16, the root node question asks whether a previously unseen binary has 40.111 or fewer calls. (Note that the number of calls per binary here is normalized to a 0 to 100 scale, making floating point values valid.) As you can see from the vertical line in Figure 6-17, most of the benign data has less than this number, while most of the malware data has more than this number of suspicious calls, making this a good initial question to ask.

Picking Follow-Up Questions

After choosing a root node, pick the next questions using a method similar to the one we used to pick the root node. For example, the root node allowed us to split the samples into two groups: one group that has less than or equal to 40.111 suspicious calls (negative feature space) and another that has more than 40.111 suspicious calls (positive feature space). To choose the next question, we just need questions that will further distinguish the samples in each area of the feature space into malicious and benign training examples.

We can see this in the way the decision tree is structured in Figure 6-16 and 6-17. For example, Figure 6-16 shows that after we ask an initial “root” question about the number of suspicious calls binaries make, we ask questions about how much compressed data binaries have. Figure 6-17 shows why we do this based on the data: after we ask our first question about suspicious function calls, we have a crude decision boundary that separates most malware from most benignware in the plot. How can we refine the decision boundary further by asking follow-up questions? It’s clear visually that the next best question to ask, which will refine our decision boundary, will be about the amount of compressed data in the binaries.

When to Stop Asking Questions

At some point in our decision tree creation process, we need to decide when the decision tree should stop asking questions and simply determine whether a binary file is benign or malicious based on our certainty about our answer. One way is to simply limit the number of questions our decision tree can ask, or to limit its *depth* (the maximum number of questions we can ask of any binary). Another is to allow the decision tree to keep growing until we’re absolutely certain about whether or not every example in our training set is malware or benignware based on the structure of the tree.

The advantage of constraining the size of the tree is that if the tree is simpler, we have a greater chance of getting the answer right (think of Occam’s razor—the simpler the theory, the better). In other words, there’s less chance that the decision tree will overfit the training data if we keep it small.

Conversely, allowing the tree to grow to maximum size can be useful if we are *underfitting* the training data. For example, allowing the tree to grow further will increase the complexity of the decision boundary, which we’ll want to do if we’re underfitting. In general, machine learning practitioners

usually try multiple depths, or allow for maximum depth on previously unseen binaries, repeating this process until they get the most accurate results.

Using Pseudocode to Explore Decision Tree Generation Algorithms

Now let's examine an automated decision tree generation algorithm. You learned that the basic idea behind this algorithm is to create the root node in the tree by finding the question that best increases our certainty about whether the training examples are malicious or benign, and then to find subsequent questions that will further increase our certainty. The algorithm should stop asking questions and make a decision once its certainty about the training examples has surpassed some threshold we set in advance.

Programmatically, we can do this recursively. The Python-like pseudocode in Listing 6-3 shows the complete process for building a decision tree in simplified form.

```
tree = Tree()
def add_question(training_examples):
    ❶ question = pick_best_question(training_examples)
    ❷ uncertainty_yes, yes_samples = ask_question(question, training_examples, "yes")
    ❸ uncertainty_no, no_samples = ask_question(question, training_examples, "no")
    ❹ if not uncertainty_yes < MIN_UNCERTAINTY:
        add_question(yes_samples)
    ❺ if not uncertainty_no < MIN_UNCERTAINTY:
        add_question(no_samples)
❻ add_question(training_examples)
```

Listing 6-3: Pseudocode for building a decision tree algorithm

The pseudocode recursively adds questions to a decision tree, beginning with the root node and working its way down until the algorithm feels confident that the decision tree can provide a highly certain answer about whether a new file is benign or malicious.

When we start building the tree, we use `pick_best_question()` to pick our root node ❶ (for now, don't worry about how this function works). Then, we look at how much uncertainty we now have about the training samples for which the answer is "yes" to this initial question ❷. This will help us to decide if we need to keep asking questions about these samples or if we can stop, and predict whether the samples are malicious or benign. We do the same for the samples for which we answered "no" for the initial question ❸.

Next, we check if the uncertainty we have about the samples for which we answered "yes" (`uncertainty_yes`) is sufficiently low to decide whether they are malicious or benign ❹. If we can determine whether they're malicious or benign at this point, we don't ask any additional questions. But if we can't, we call `add_question()` again, using `yes_samples`, or the number of samples for which we answered "yes," as our input. This is a classic example of *recursion*, which is a function that calls itself. We're using recursion to

repeat the same process we performed for the root node with a subset of training examples. The next if statement does the same thing for our “no” examples ⑤. Finally, we call our decision tree building function on our training examples ⑥.

How exactly `pick_best_question()` works involves math that is beyond the scope of this book, but the idea is simple. To pick the best question at any point in the decision tree building process, we look at the training examples about which we’re still uncertain, enumerate all the questions we could ask about them, and then pick the one that best reduces our uncertainty about whether the examples are malware or benignware. We measure this reduction in uncertainty using a statistical measurement called *information gain*. This simple method for picking the best question works surprisingly well.

NOTE

This is a simplified example of how real-world, decision tree–generating, machine learning algorithms work. I’ve left out the math required to calculate how much a given question increases our certainty about whether or not a file is bad.

Let’s now look at the behavior of decision trees on the two sample datasets we’ve been using in this chapter. Figure 6-18 shows the decision boundary learned by a decision tree detector.

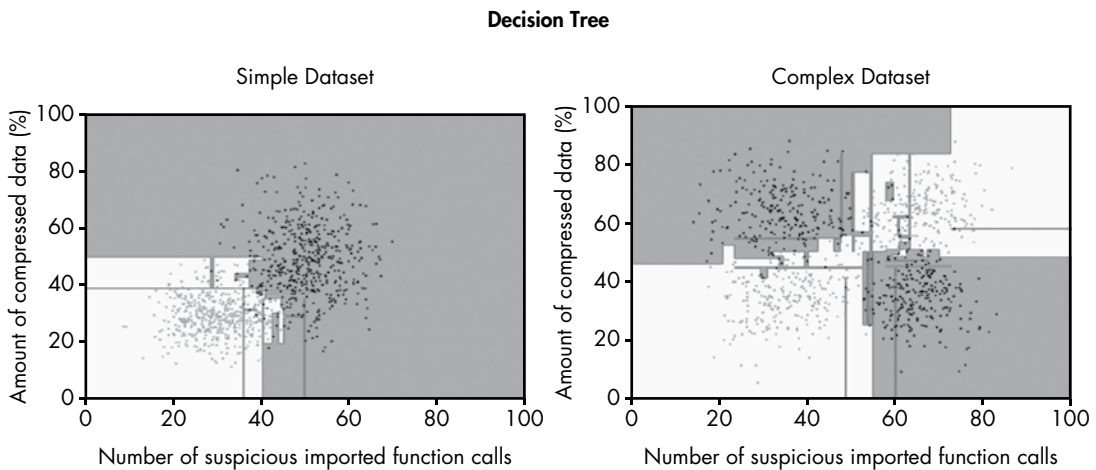


Figure 6-18: Decision boundaries for our sample datasets produced by a decision tree approach

In this case, instead of setting a maximum depth for the trees, we allow them to grow to the point where there are no false positives or false negatives relative to the training data so that every training sample is correctly classified.

Notice that decision trees can only draw horizontal and vertical lines in the feature space, even when it seems clear and obvious that a curved or diagonal line might be more appropriate. This is because decision trees

only allow us to express simple conditions on individual features (such as greater than or equal to and less than or equal to), which always leads to horizontal or vertical lines.

You can also see that although the decision trees in these examples succeed in separating the benignware from the malware, the decision boundaries look highly irregular and have strange artifacts. For example, the malware region extends into the benignware region in strange ways, and vice versa. On the positive side, the decision tree does far better than logistic regression at creating a decision boundary for the complex dataset.

Let's now compare the decision trees in Figure 6-18 to the decision tree models in Figure 6-19.

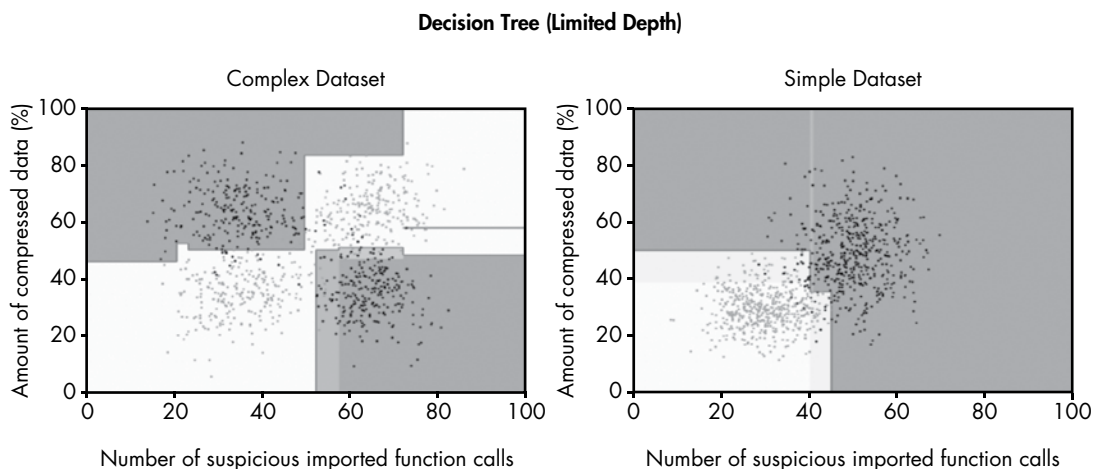


Figure 6-19: Decision boundaries for our sample datasets produced by a limited-depth decision tree

The decision trees in Figure 6-19 use the same decision tree generation algorithm used for Figure 6-18, except I limit the tree depth to five nodes. This means that for any given binary, I can ask a maximum of five questions of its features.

The result is dramatic. Whereas the decision tree models shown in Figure 6-18 are clearly overfit, focusing on outliers and drawing overly complex boundaries that fail to capture the general trend, the decision trees in Figure 6-19 fit the data much more elegantly, identifying a general pattern in both datasets without focusing on outliers (with one exception, the skinnier decision region in the upper-right area of the simple dataset). As you can see, picking a good maximum decision tree depth can have a big effect on your decision tree-based machine learning detector.

When to Use Decision Trees

Because decision trees are expressive and simple, they can learn both simple and highly irregular boundaries based on simple yes-or-no questions. We can also set the maximum depth to control how simple or complex their theories of what constitutes malware versus benignware should be.

Unfortunately, the downside to decision trees is that they often simply do not result in very accurate models. The reason for this is complex, but it's related to the fact that decision trees express jagged decision boundaries, which don't fit the training data in ways that generalize to previously unseen examples very well.

Similarly, decision trees don't usually learn accurate probabilities around their decision boundaries. We can see this by inspecting the shaded regions around the decision boundary in Figure 6-19. The decay is not natural or gradual and doesn't happen in the regions it should—areas where the malware and benignware examples overlap.

Next, I discuss the random forest approach, which combines multiple decision trees to yield far better results.

Random Forest

Although the security community relies heavily on decision trees for malware detection, they almost never use them individually. Instead, hundreds or thousands of decision trees are used in concert to make detections through an approach called *random forest*. Instead of training one decision tree, we train many, usually a hundred or more, but we train each decision tree differently so that it has a different perspective on the data. Finally, to decide whether a new binary is malicious or benign, we allow the decision trees to vote. The probability that a binary is malware is the number of positive votes divided by the total number of trees.

Of course, if all the decision trees are identical, they would all vote the same way, and the random forest would simply replicate the results of the individual decision trees. To address this, we want the decision trees to have different perspectives on what constitutes malware and benignware, and we use two methods, which I discuss next, to induce this diversity into our collection of decision trees. By inducing diversity, we generate a “wisdom of crowds” dynamic in our model, which typically results in a more accurate model.

We use the following steps to generate a random forest algorithm:

1. Training: for every tree out of the number we plan to generate (typically 100 or more)
 - Randomly sample some training examples from our training set.
 - Build a decision tree from the random sample.
 - For each tree that we build, each time we consider “asking a question,” consider asking a question of only a handful of features, and disregard the other features.
2. Detection on a previously unseen binary
 - Run detection for each individual tree on the binary.
 - Decide whether or not the binary is malware based on the number of trees that voted “yes.”

To understand this in more detail, let's examine the results generated by the random forest approach on our two sample datasets, as shown in Figure 6-20. These results were generated using 100 decision trees.

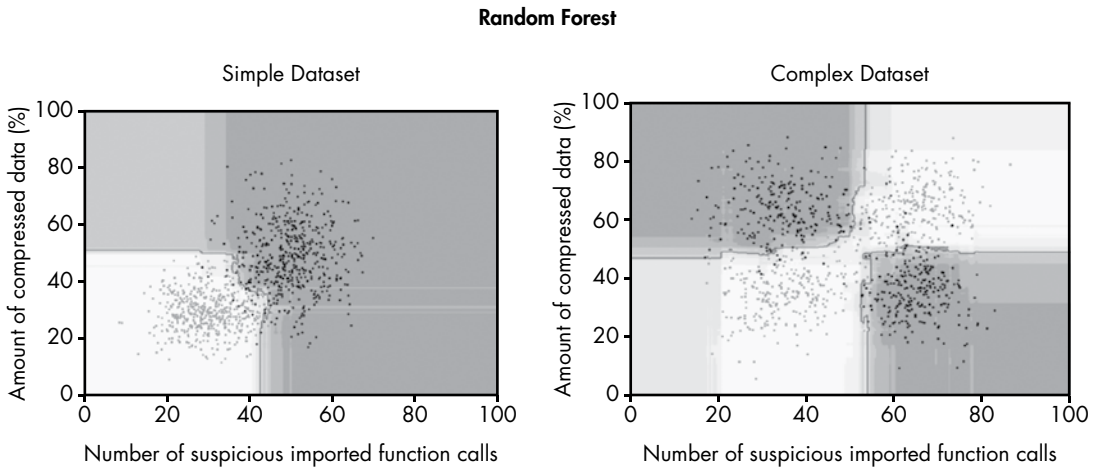


Figure 6-20: Decision boundaries created using the random forest approach

In contrast to the individual decision tree results shown in Figures 6-18 and 6-19, random forest can express much smoother and more intuitive decision boundaries for both simple and complex datasets than individual decision trees. Indeed, the random forest model fits the training dataset very cleanly, with no jagged edges; the model seems to have learned good theories about what constitutes “malicious versus benign” for both datasets.

Additionally, the shaded regions are intuitive. For example, the further you get from benign or malicious examples, the less certainty random forest has about whether examples are malicious or benign. This bodes well for random forest’s performance on previously unseen binaries. In fact, as you’ll see in the next chapter, random forest is the best performing model on previously unseen binaries of all the approaches discussed in this chapter.

To understand why random forest draws such clean decision boundaries compared to individual decision trees, let’s think about what the 100 decision trees are doing. Each tree sees only about two-thirds of the training data, and only gets to consider a randomly selected feature whenever it makes a decision about what question to ask. This means that behind the scenes, we have 100 diverse decision boundaries that get *averaged* to create the final decision boundaries in the examples (and the shaded regions). This “wisdom of crowds” dynamic creates an aggregate opinion that can identify the trends in the data in a much more sophisticated way than individual decision trees can.

Summary

In this chapter, you got a high-level introduction to machine learning–based malware detection as well as four major approaches to machine learning: logistic regression, k-nearest neighbors, decision trees, and random forests. Machine learning–based detection systems can automate the work of writing detection signatures, and they often perform better in practice than custom written signatures.

In the following chapters, I'll show you how these approaches perform on real-world malware detection problems. Specifically, you'll learn how to use open source, machine learning software to build machine learning detectors to accurately classify files as either malicious or benign, and how to use basic statistics to evaluate the performance of your detectors on previously unseen binaries.

7

EVALUATING MALWARE DETECTION SYSTEMS



In the previous chapter, you learned how machine learning can help you build malware detectors. In this chapter, you learn the basic concepts necessary to predict how malware detection systems will perform. The ideas you learn here will prove crucial in improving any malware detection system you build, because without a way to measure your system's performance, you will not know how to improve it. Please note that while this chapter is dedicated to introducing basic evaluation concepts, Chapter 8 continues this thread, introducing essential evaluation concepts like cross-validation.

First, I introduce the basic ideas behind detection accuracy evaluation, and then I introduce more advanced ideas concerning the environment in which you deploy your system when evaluating its performance. To do this, I walk you through an evaluation of a hypothetical malware detection system.

Four Possible Detection Outcomes

Suppose you run a malware detection system on a software binary and get the system's "opinion" about whether the binary is malicious or benign. As illustrated in Figure 7-1, four possible outcomes may occur.

	Example is malicious	Example is not malicious
Detector alarms	True positive	False positive
Detector does not alarm	False negative	True negative

Figure 7-1: The four possible detection outcomes

These outcomes can be defined as follows:

True positive The binary is malware and the system says it is malware.

False negative The binary is malware and the system says it's not malware.

False positive The binary is not malware and the system says it is malware.

True negative The binary is not malware and the system says it's not malware.

As you can see, there are two scenarios in which your malware detection system can produce inaccurate results: false negatives and false positives. In practice, true positive and true negative results are what we desire, but they are often difficult to obtain.

You'll see these terms used throughout this chapter. In fact, most of detection evaluation theory is built on this simple vocabulary.

True and False Positive Rates

Now suppose you want to test the detection system's accuracy using a set of benignware and malware. You can run the detector on each binary and keep count of which of the four possible outcomes the detector gives you over the entire test set. At this point, you need some summary statistics to give you an overall sense of the system's accuracy (that is, how likely it is that your system will generate false positives or false negatives).

One such summary statistic is the *true positive rate* of the detection system, which you can calculate by dividing the number of true positives on your test set by the total number of malware samples in your test set. Because this calculates the percentage of malware samples your system is able to detect, it measures your system's ability to recognize malware when it "sees" malware.

However, simply knowing that your detection system will raise alarms when it sees malware is insufficient to evaluate its accuracy. For example, if you only used the true positive rate as an evaluation criterion, a simple function that says "yes, this is malware" on all files would yield a perfect true positive rate. The real test of a detection system is whether or not it says "yes, this is malware" when it sees malware and "no, this is not malware" when it sees benignware.

To measure a system's ability to discern whether something is not malware, you also need to measure the system's *false positive rate*, which is the rate at which your system issues a malware alarm when it sees benignware. You can calculate your system's false positive rate by dividing the number of benign samples the system flags as malware by the total number of benign samples tested.

Relationship Between True and False Positive Rates

When designing a detection system, you want to keep the false positive rate as low as possible while keeping the true positive rate as high as possible. Unless you build a truly perfect malware detection system that is always right (which is really an impossibility given the evolving nature of malware), there will always be tension between the desire for a high true positive and the desire for a low false positive rate.

To see why this is the case, imagine a detection system that, before deciding whether or not a binary is malware, adds up all the evidence that the binary is malware to create a *suspiciousness score* for the binary. Let's call this hypothetical suspiciousness-score-generating system MalDetect. Figure 7-2 shows an example of the values that MalDetect might output for 12 sample binaries, where the circles represent individual software binaries. The further to the right a binary, the higher the suspiciousness score given by MalDetect.

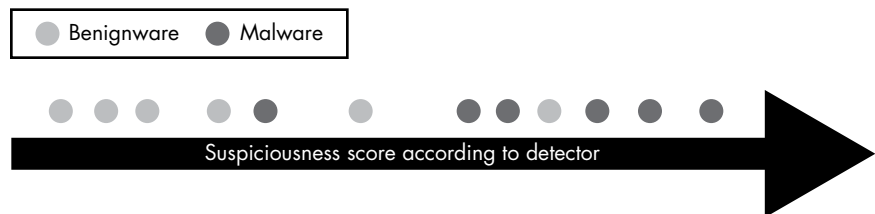


Figure 7-2: *Suspiciousness scores output by the hypothetical MalDetect system for individual software binaries*

Suspiciousness scores are informative, but in order to calculate MalDetect's true positive rate and false positive rate on our files, we

need to convert MalDetect’s suspiciousness scores to “yes” or “no” answers regarding whether or not a given software binary is malicious. To do this, we use a threshold rule. For example, we decide that if the suspiciousness score is greater or equal to some number, the binary in question raises a malware alarm. If the score is lower than the threshold, it doesn’t.

Such a threshold rule is the standard way to convert a suspiciousness score into a binary detection choice, but where should we set the threshold? The problem is that there is no right answer. Figure 7-3 shows the conundrum: the higher we set the threshold, the less likely we are to get false positives, but the more likely we are to get false negatives.

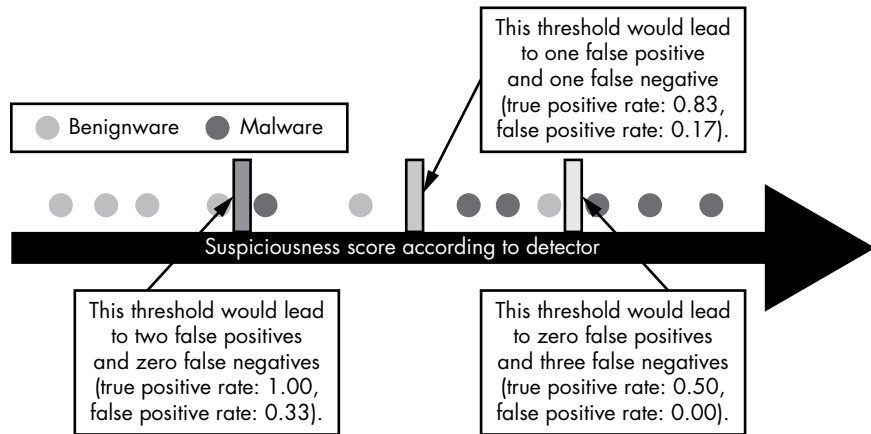


Figure 7-3: An illustration of the relationship between false positive rate and true positive rate when deciding on a threshold value

For example, let’s consider the leftmost threshold shown in Figure 7-3, where binaries to the left of the threshold are classified as benign and binaries to its right are classified as malware. Because this threshold is low, we get a great true positive rate (classifying 100 percent of the malware samples correctly) but a terrible false positive rate (falsely classifying 33 percent of the benign samples as malicious).

Our intuition might be to increase the threshold so that only samples with a higher suspiciousness score are deemed to be malware. Such a solution is given by the middle threshold in Figure 7-3. Here, the false positive rate drops to 0.17, but unfortunately the true positive rate drops as well, to 0.83. If we continue to move the threshold to the right, as shown by the rightmost threshold, we eliminate any false positives, but detect only 50 percent of the malware.

As you can see, there is no such thing as a perfect threshold. A detection threshold that yields a low false positive rate (good) will tend to miss more malware, yielding a low true positive rate (bad). Conversely, using a detection threshold that has a high true positive rate (good) will also increase the false positive rate (bad).

ROC Curves

The tradeoff between the true positive rate and false positive rate of detection systems is a universal problem for all detectors, not just malware detectors. Engineers and statisticians have thought long and hard about this phenomenon and come up with the *Receiver Operating Characteristic (ROC)* curve to describe and analyze it.

NOTE

If you're confused by the phrase Receiver Operating Characteristic, don't worry about it—this phrase is confusing and pertains to the context in which ROC curves were originally developed, which is radar-based detection of physical objects.

ROC curves characterize a detection system by plotting false positive rates against their associated true positive rates at various threshold settings. This helps us evaluate the tradeoff between lower false positive rates and higher true positive rates, and in doing so determine the “best” threshold for our situation.

For example, for our hypothetical MalDetect system from Figure 7-3, the system's true positive rate is 0.5 when its false positive rate is 0 (low threshold), and the system's true positive rate is 1.00 when the false positive rate is 0.33 (high threshold).

Figure 7-4 shows how this works in more detail.

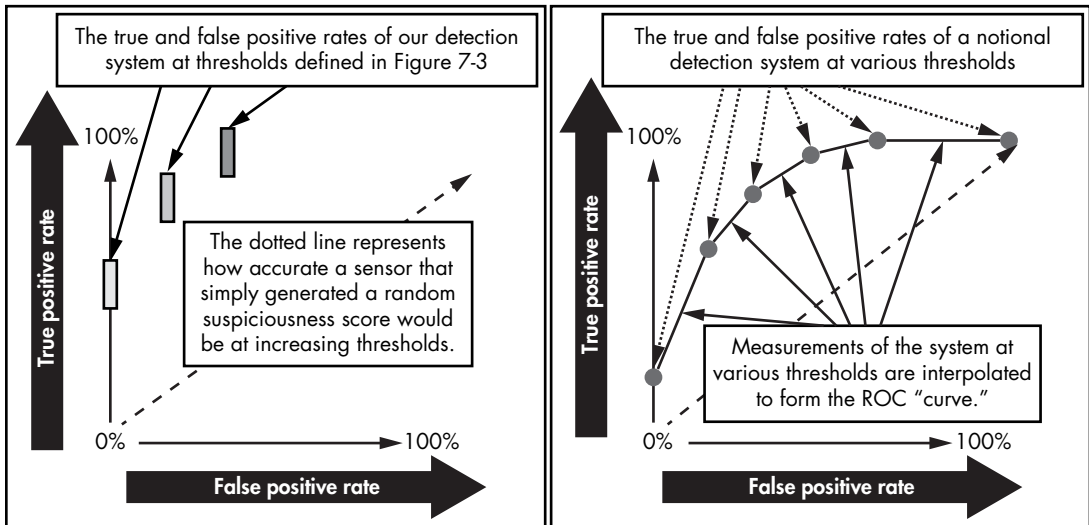


Figure 7-4: An illustration of what ROC curves mean and how they are constructed

To build the ROC curve, we start with the three thresholds used in Figure 7-3 and plot their resulting false and true positive rates, shown in the left half of Figure 7-3. The plot on the right of Figure 7-4 shows the same thing, but for all possible thresholds. As you can see, the higher the false positive rates, the higher the true positive rates. Similarly, the lower the false positive rates, the lower the true positive rates.

The “curve” of the ROC curve is a line within the two-dimensional ROC plot that represents how we think our detection system will do on its true positive rate over all possible false positive values, and how we think our detection system will do on its false positive rate over all possible true positive values. There are multiple ways of generating such a curve, but that goes beyond the scope of this book.

One simple method, however, is to try many threshold values, observe the corresponding false and true positive rates, plot them, and connect the dots using a line. This connected line, shown in the right plot of Figure 7-4, becomes our ROC curve.

Considering Base Rates in Your Evaluation

As you’ve seen, ROC curves can tell you how your system will perform in terms of the rate at which it calls malicious binaries malicious (true positive rate) and the rate at which it calls benign binaries malicious (false positive rate). However, ROC curves will not tell you the *percentage* of your system’s alarms that will be true positives, which we call the *precision* of the system. The precision of a system is related to the percentage of binaries the system encounters that are actually malware, which we call the *base rate*. Here’s a breakdown of each term:

Precision The percentage of system detection alarms that are true positives (meaning that they are detections of actual malware). In other words, *precision* is the detection system’s number of *true positives* / (*true positives* + *false positives*) when tested against some set of binaries.

Base rate The percentage of the data fed to the system that has the quality we are looking for. In our case, *base rate* refers to the percentage of binaries that are *actually malware*.

We discuss how these two metrics are related in the next section.

How Base Rate Affects Precision

Although a detection system’s true and false positive rates do not change when the base rate changes, the system’s precision is affected by changes in the malware base rate—often dramatically. To see why this is true, let’s consider the following two cases.

Suppose the false positive rate of MalDetect is 1 percent and the true positive rate is 100 percent. Now suppose we set MalDetect loose on a network that we know upfront has no malware on it (perhaps the network has just been created from scratch in a laboratory). Because we know in advance there is no malware on the network, every alarm the MalDetect throws will by definition be a false positive, because the only binaries that MalDetect encounters will be benignware. In other words, precision will be 0 percent.

In contrast, if we run MalDetect on a dataset composed of entirely malware, none of its alarms will ever be false positives: there simply will

never be an opportunity for MalDetect to generate a false positive since there is no benignware in the software dataset. Therefore, precision will be 100 percent.

In both of these extreme cases, the base rates have a huge impact on MalDetect's precision, or the probability that its alarm is a false positive.

Estimating Precision in a Deployment Environment

You now know that depending on the proportion of malware in a test dataset (base rate), your system will yield very different precision values. What if you want to estimate the precision your system will have based on an estimate of the base rate of the environment in which you deploy it? All you have to do is use your deployment environment's estimated base rate to estimate the variables in the precision formula: *true positives / (true positives + false positives)*. You'll need three numbers:

- **True positive rate (TPR)** of the system, or the percentage of malware samples the system will correctly detect
- **False positive rate (FPR)** of the system, or the percentage of benign samples the system will incorrectly alarm on
- **Base rate (BR)** of the binaries against which you will use the system (for example, the percentage of binaries downloaded from piracy sites you expect will be malware, if this is what you'll be using your system on)

The numerator of the precision equation—the number of true positives—can be estimated by *true positive rate × base rate*, giving you the percentage of malware your system will correctly detect. Similarly, the denominator of the equation—that is, *(true positives + false positives)*—can be estimated by *true positive rate × base rate + false positive rate × (1 – base rate)*, giving you the percentage of *all* binaries the system will alarm on by calculating the number of malware binaries that will be detected correctly plus the fraction of benignware binaries for which false positives will be issued.

In sum, you calculate the expected precision of your system as follows:

$$\text{precision} = \frac{\text{true positive rate} \times \text{base rate}}{\text{true positive rate} \times \text{base rate} + \text{false positive rate} \times (1 - \text{base rate})}$$

Let's consider another example to see how base rate can have a profound impact on the performance of a detection system. For example, suppose we have a detection system that has an 80 percent true positive rate and a 10 percent false positive rate, and 50 percent of the software binaries we run it against are expected to be malware. This would lead to an expected precision of 89 percent. But when the base rate is 10 percent, our precision drops to 47 percent.

What happens if our base rate is very low? For example, in a modern enterprise network, very few software binaries are actually malware. Using our precision equation, if we assume a base rate of 1 percent (1 in 100 binaries are malware), we get a precision of about 7.5 percent, which

means that 92.5 percent of our system's alarms would be false positives! And if we assume a base rate of 0.1 percent (1 in 1000 binaries are likely to be malware), we get a precision of 1 percent, meaning 99 percent of our system's alarms would be false positives! Finally, at a base rate of 0.01 percent (1 in 10,000 binaries are likely to be malware—probably the most realistic assumption on an enterprise network), our expected precision drops to 0.1 percent, meaning the overwhelming majority of our system's alerts will be false positives.

One takeaway from this analysis is that detection systems that have high false positive rates will almost never be useful in enterprise settings, because their precision will be far too low. Therefore, a key goal in building malware detection systems is to minimize the false positive rate such that the precision of the system is reasonable.

Another related takeaway is that when you do the ROC curve analysis introduced earlier in this chapter, you should effectively ignore false positive rates over, say, 1 percent, if you are developing your system to be deployed in an enterprise setting, because any higher false positive rate will likely result in a system that has such low precision that it is rendered useless.

Summary

In this chapter, you learned basic detection evaluation concepts, including true positive rate, false positive rate, ROC curves, base rates, and precision. You saw how maximizing the true positive rate and minimizing the false positive rate are both important in building a malware detection system. Because of the way base rate affects precision, reducing the false positive rate is particularly important if you want to deploy your detection system within an enterprise.

If you don't feel completely fluent in these concepts, don't worry. You'll get more practice with them in the next chapter, where you'll build and evaluate a malware detection system from the ground up. In the process, you'll learn additional machine learning-specific evaluation concepts that will help you improve your machine learning-based detectors.

8

BUILDING MACHINE LEARNING DETECTORS



Today, thanks to high-quality open source software that handles the heavy mathematical lifting of implementing machine learning systems, anyone who knows basic Python and understands the key concepts can use machine learning.

In this chapter, I show you how to build machine learning malware detection systems using *scikit-learn*, the most popular—and the best, in my opinion—open source machine learning package available. This chapter contains a lot of sample code. The major code blocks are accessible in the directory *malware_data_science/ch8/code*, and the corresponding sample data is accessible in the directory *malware_data_science/ch8/data* in the code and data (and on the virtual machine) accompanying this book.

By following along with the text, examining the sample code, and trying out the provided examples, you should be comfortable building and evaluating your own machine learning systems by the end of the chapter. You also learn to build a general malware detector and use the necessary tools to build malware detectors for specific malware families. The skills

you gain here will have a broad application, allowing you to apply machine learning to other security problems, such as detecting malicious emails or suspicious network streams.

First, you learn the terminology and concepts you need to know before using `scikit-learn`. Then, you use `scikit-learn` to implement a basic decision tree detector based on the decision tree concepts you learned in Chapter 6. Next, you learn how to integrate feature extraction code with `scikit-learn` to build a real-world malware detector that uses real-world feature extraction and a random forest approach to detect malware. Finally, you learn how to use `scikit-learn` to evaluate machine learning systems with the sample random forest detector.

Terminology and Concepts

Let's review some terminology first. The open source library `scikit-learn` (`sklearn` for short) has become popular in the machine learning community because it's both powerful and easy to use. Many data scientists use the library within the computer security community and in other fields, and many rely on it as their main toolbox for performing machine learning tasks. Although `sklearn` is constantly being updated with new machine learning approaches, it provides a consistent programming interface that makes using these machine learning approaches simple.

Like many machine learning frameworks, `sklearn` requires training data in *vector* form. Vectors are arrays of numbers where each index in the array corresponds to a single feature of the training example software binary. For example, if the two features of software binaries our machine learning detector uses are `is compressed` and `contains encrypted data`, then our feature vector for a training example binary could be `[0,1]`. Here, the first index in the vector would represent whether or not the binary is compressed, with the zero indicating "no," and the second index would represent whether or not the binary contains encrypted data, with the one indicating "yes."

Vectors can be awkward to work with because you have to remember what feature each index maps to. Fortunately, `sklearn` provides helper code that translates other data representations to vector form. For example, you can use `sklearn`'s `DictVectorizer` class to transform dictionary representations of your training data (for instance, `{"is compressed":1, "contains encrypted data":0}`) into the vector representation that `sklearn` operates on, like `[0,1]`. Later, you can use the `DictVectorizer` to recover the mapping between the vector's indices and the original feature names.

To train an `sklearn`-based detector, you need to pass in two separate objects to `sklearn`: feature vectors (as described previously) and a label vector. A *label vector* contains one number per training example, which corresponds, in our case, to whether or not the example is malware or benignware. For instance, if we pass three training examples to `sklearn`, and then pass the label vector `[0,1,0]`, we're telling `sklearn` that the first sample is benignware, the second sample is malware, and the third is benignware. By convention, machine learning engineers use a capital X

variable to represent the training data and a lowercase *y* variable to represent the labels. The difference in case reflects the convention in mathematics of capitalizing variables that represent matrices (which we can think of as arrays of vectors) and lowercasing variables that represent individual vectors. You'll see this convention used in machine learning sample code online, and I use this convention for the remainder of this book to get you comfortable with it.

The *sklearn* framework uses other terminology that you might find new as well. Instead of calling machine learning–based detectors “detectors,” *sklearn* calls them “classifiers.” In this context, the term *classifier* simply means a machine learning system that categorizes things into two or more categories. Therefore, a *detector* (the term I use throughout this book) is a special type of a classifier that places things into two categories, like malware and benignware. Also, instead of using the term *training*, *sklearn*'s documentation and API often use the term *fit*. For example, you'll see a sentence like “fit a machine learning classifier using training examples,” which is the equivalent to saying “train a machine learning detector using training examples.”

Finally, instead of using the term *detect* in the context of classifiers, *sklearn* uses the term *predict*. This term is used in *sklearn*'s framework, and in the machine learning community more generally, whenever a machine learning system is used to perform a task, whether to predict the value of a stock a week from now or detect whether an unknown binary is malware.

Building a Toy Decision Tree–Based Detector

Now that you're familiar with *sklearn*'s technical terminology, let's create a simple decision tree along the lines of what we discussed in Chapter 6, using the *sklearn* framework. Recall that decision trees play a “20 questions” type of game in which they ask a series of questions about input vectors to arrive at a decision concerning whether these vectors are malicious or benign. We walk through building a decision tree classifier, step by step, and then explore an example of a complete program. Listing 8-1 shows how to import the requisite modules from *sklearn*.

```
from sklearn import tree
from sklearn.feature_extraction import DictVectorizer
```

Listing 8-1: Importing sklearn modules

The first module we import, *tree*, is *sklearn*'s decision tree module. The second module, *feature_extraction*, is *sklearn*'s helper module from which we import the *DictVectorizer* class. The *DictVectorizer* class conveniently translates the training data provided in readable, dictionary form to the vector representation that *sklearn* requires to actually train machine learning detectors.

After we import the modules we need from *sklearn*, we instantiate the requisite *sklearn* classes, as shown in Listing 8-2.

```
classifier = ❶tree.DecisionTreeClassifier()
vectorizer = ❷DictVectorizer(sparse=❸False)
```

Listing 8-2: Initializing the decision tree classifier and vectorizer

The first class we instantiate, `DecisionTreeClassifier` ❶, represents our detector. Although `sklearn` provides a number of parameters that control exactly how our decision tree will work, here we don't select any parameters so that we're using `sklearn`'s default decision tree settings.

The next class we instantiate is `DictVectorizer` ❷. We set `sparse` to `False` ❸ in the constructor, telling `sklearn` that we do not want it to use sparse vectors, which save memory but are complicated to work with. Because `sklearn`'s decision tree module can't use sparse vectors, we turn this feature off.

Now that we have instantiated our classes, we can initialize some sample training data, as shown in Listing 8-3.

```
# declare toy training data
❶ training_examples = [
    {'packed':1,'contains_encrypted':0},
    {'packed':0,'contains_encrypted':0},
    {'packed':1,'contains_encrypted':1},
    {'packed':1,'contains_encrypted':0},
    {'packed':0,'contains_encrypted':1},
    {'packed':1,'contains_encrypted':0},
    {'packed':0,'contains_encrypted':0},
    {'packed':0,'contains_encrypted':0},
]
❷ ground_truth = [1,1,1,1,0,0,0,0]
```

Listing 8-3: Declaring training and label vectors

In this example, we initialize two structures—feature vectors and a label vector—that together comprise our training data. The feature vectors, assigned to the `training_examples` variable ❶, are given in dictionary form. As you can see, we're using two simple features. The first is `packed`, which represents whether a given file is packed, and the second is `contains_encrypted`, which represents whether the file contains encrypted data. The label vector, which is assigned to the `ground_truth` variable ❷, represents whether each training example is malicious or benign. In this book, and in general among security data scientists, 0 always stands for benign and 1 always stands for malicious. In this case, the label vector declares that the first four feature vectors are malicious and the second four are benign.

Training Your Decision Tree Classifier

Now that we've declared our training vectors and label vector, let's train our decision tree model by calling the decision tree class instance's `fit` method, as shown in Listing 8-4.

```
# initialize the vectorizer with the training data
❶ vectorizer.fit(training_examples)
```

```
# transform the training examples to vector form
❷ X = vectorizer.transform(training_examples)
y = ground_truth # call ground truth 'y', by convention
```

Listing 8-4: Initializing the vectorizer class with training data

The code in Listing 8-4 first initializes the vectorizer class that we initialized in Listing 8-2 by calling the fit method ❶. Here, the fit method tells sklearn to create a mapping between the packed feature and the contains_encrypted feature and vector array indices. Then we transform our dictionary-based feature vectors into numerical vector form by calling the vectorizer class's transform method ❷. Recall that we assign our feature vectors to a variable called X and our label vector to a variable called y, which is the naming convention in the machine learning community.

Now that we're all set up with our training data, we can train our decision tree detector by calling the fit method on the decision tree classifier instances, like this:

```
# train the classifier (a.k.a. 'fit' the classifier)
classifier.fit(X,y)
```

As you can see, training the sklearn detector is as simple as that. But behind the scenes, sklearn is going through the algorithmic process of identifying a good decision tree for accurately detecting whether new software is malicious or benign, along the lines of the algorithm we discussed in the previous chapter.

Now that we've trained the detector, let's use the code in Listing 8-5 to detect whether a binary is malicious or benign.

```
test_example = ❶{'packed':1,'contains_encrypted':0}
test_vector = ❷vectorizer.transform(test_example)
❸ print classifier.predict(test_vector) # prints [1]
```

Listing 8-5: Determining whether a binary is malicious

Here, we instantiate a dictionary-based feature vector for a hypothetical software binary ❶, translate it to numerical vector form using vectorizer ❷, which we declared earlier in our code, and then run the decision tree detector we built ❸ to determine whether or not the binary is malicious. You'll see when we run the code that the classifier “thinks” that the new binary is malicious (because it gives a “1” as its output), and you'll see why this is the case when we visualize our decision tree.

Visualizing the Decision Tree

We can visualize the decision tree that sklearn has automatically created based on our training data, as shown in Listing 8-6.

```
# visualize the decision tree
with open(❶"classifier.dot","w") as output_file:
```

```

❷ tree.export_graphviz(
    classifier,
    feature_names=vectorizer.get_feature_names(),
    out_file=output_file
)

import os
os.system("dot classifier.dot -Tpng -o classifier.png")

```

Listing 8-6: Creating an image file of the decision tree using GraphViz

Here, we open a file called *classifier.dot* ❶ to which we write a network representation of our decision tree using the `export_graphviz()` function that `sklearn`'s `tree` module provides. Then we call `tree.export_graphviz` ❷ to write a GraphViz *.dot* file to *classifier.dot*, which writes a network representation of the decision tree to disk. Finally, we use the GraphViz `dot` command line program to create an image file that visualizes the decision tree, in a form that corresponds to what you learned about decision trees in Chapter 6. When you run this, you should get an output image file called *classifier.png* that looks like Figure 8-1.

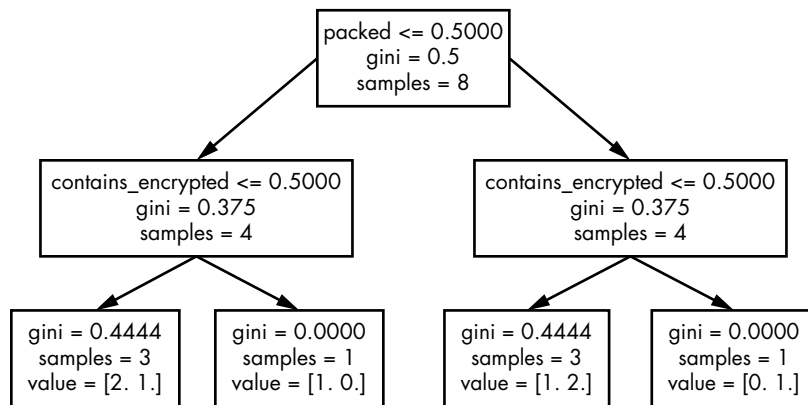


Figure 8-1: Decision tree visualization

Although this decision tree visualization should be familiar from Chapter 6, it contains some new vocabulary. The first line in each box contains the name of the feature about which the node asks a question (in machine learning parlance, we say that the node “splits on” this feature). For example, the first node splits on the feature “packed”: if a binary is not packed, we move along the left arrow; otherwise, we move along the right arrow.

The second line of text in each box refers to that node’s *gini index*, which measures how much inequality there is between the malware and benignware training examples that match that node. The higher the gini index, the more skewed the samples that match that node are toward either benignware or malware. This means that a high gini index in each node is good, because the more the training examples skew toward either malware

or benignware, the more sure we are about whether new test examples are malware or benignware. The third line in each box just gives the number of training examples that matched that node.

You'll notice that in the leaf nodes of the tree, the text in the box is different. These nodes don't "ask a question;" instead, they provide an answer to the question "is this binary malicious or benign?" For example, in the leftmost leaf node, we have "value = [2. 1.]," which means that two benign training examples matched this node (not packed and not encrypted) and one malware training example matched the node. That is, if we reach this node, we'd assign a probability of 33 percent to the binary being malware (1 malware sample / 3 total samples = 33 percent). The gini value in these boxes shows how much information is gained about whether the binary is malware or benignware when we split on the question directly leading up to these nodes. As you can see, it can be useful to inspect visualizations of decision trees generated by sklearn to understand how our decision trees are making detections.

Complete Sample Code

Listing 8-7 shows the complete code for the decision tree workflow I have described thus far. This code should be easily legible to you now that we have worked through the code, piece by piece.

```
#!/usr/bin/python

# import sklearn modules
from sklearn import tree
from sklearn.feature_extraction import DictVectorizer

# initialize the decision tree classifier and vectorizer
classifier = tree.DecisionTreeClassifier()
vectorizer = DictVectorizer(sparse=False)

# declare toy training data
training_examples = [
    {'packed':1,'contains_encrypted':0},
    {'packed':0,'contains_encrypted':0},
    {'packed':1,'contains_encrypted':1},
    {'packed':1,'contains_encrypted':0},
    {'packed':0,'contains_encrypted':1},
    {'packed':1,'contains_encrypted':0},
    {'packed':0,'contains_encrypted':0},
    {'packed':0,'contains_encrypted':0},
]
ground_truth = [1,1,1,1,0,0,0,0]

# initialize the vectorizer with the training data
vectorizer.fit(training_examples)

# transform the training examples to vector form
X = vectorizer.transform(training_examples)
y = ground_truth # call ground truth 'y', by convention
```

```
# train the classifier (a.k.a. 'fit' the classifier)
classifier.fit(X,y)

test_example = {'packed':1,'contains_encrypted':0}
test_vector = vectorizer.transform(test_example)
print `classifier.predict(test_vector)` # prints [1]

#visualize the decision tree
with open("classifier.dot","w") as output_file:
    tree.export_graphviz(
        classifier,
        feature_names=vectorizer.get_feature_names(),
        out_file=output_file
    )

import os
os.system("dot classifier.dot -Tpng -o classifier.png")
```

Listing 8-7: Complete decision tree workflow sample code

The sample machine learning malware detector we just explored demonstrates how to get started with sklearn’s functionality, but it’s missing some essential features required for a real-world malware detector. Let’s now explore what a real-world malware detector entails.

Building Real-World Machine Learning Detectors with sklearn

To build a real-world detector, you need to use industrial-strength features of software binaries as well as write code to extract these features from software binaries. Industrial-strength features are those that reflect the content of binaries in all their complexity, which means we need to use hundreds or thousands of features. By “extracting” features I mean that you have to write code that identifies the presence of these features within binaries. You also need to use thousands of training examples and train a machine learning model at scale. Finally, you need to use sklearn’s more advanced detection approaches because the simple decision tree approaches we just explored don’t provide sufficient detection accuracy.

Real-World Feature Extraction

The sample features I used previously, such as `is_packed` and `contains_encrypted` data, are simple toy examples, and these two features alone will never result in a working malware detector. As I mentioned previously, real-world malware detection systems use hundreds, thousands, or even millions of features. For example, a machine learning–based detector might use millions of character strings that occur in software binaries as features. Or it might use the values of software binary Portable Executable (PE) headers, the functions imported by a given binary, or some combination of all of these. Although we’ll work only with string features in this chapter, let’s take a moment to explore

common categories of features used in machine learning–based malware detection, starting with the string features.

String Features

The string features of a software binary are all the contiguous strings of printable characters in the file that are at least some minimum length (in this book, this minimum is set to five characters). For example, suppose a binary file contains the following sequences of printable characters:

```
["A", "The", "PE executable", "Malicious payload"]
```

In this case, the strings we can use as features would be "PE executable" and "Malicious payload" because these two strings have more than five characters in them.

To transform string features into a format that `sklearn` can understand, we need to put them into a Python dictionary. We do this by using the actual strings as dictionary keys and then setting their values to 1 to indicate that the binary in question contains that string. For example, the previous sample binary would get a feature vector of {"PE executable": 1, "Malicious payload": 1}. Of course, most software binaries have hundreds of printable strings in them, not just two, and these strings can contain rich information about what a program does.

In fact, string features work well with machine learning–based detection because they capture so much information about software binaries. If the binary is a packed malware sample, then it's likely to have few informative strings, which in itself can be a giveaway that the file is malicious. On the other hand, if parts of the file's resources section are not packed or obfuscated, then those strings reveal much about the file's behavior. For example, if the binary program in question makes HTTP requests, it's common to see strings such as "GET %s" in that file's set of strings.

String features have some limitations, however. For example, they don't capture anything about the actual logic of a binary program, because they don't include actual program code. So, although strings can be useful features even on packed binaries, they don't reveal what packed binaries actually do. As a result, detectors based on string features are not ideal for detecting packed malware.

Portable Executable (PE) Header Features

PE header features are extracted from the PE header metadata that resides in every Windows `.exe` and `.dll` file. For more information on the format of these headers, refer to Chapter 1. To extract PE features from static program binaries, you can use the code given in that chapter, and then encode file features in Python dictionary form, where the header field name is the dictionary key and the field value is the value corresponding to each key.

PE header features complement string features well. For example, whereas string features often do a good job of capturing the function calls and network transmissions made by a program, like the "GET %s" example,

PE header features capture information like a program binary's compile timestamp, the layout of its PE sections, and which of those sections are marked executable and how large they are on disk. They also capture the amount of memory a program allocates upon startup, and many other run-time characteristics of a program binary that string features don't capture.

Even when you're dealing with packed binaries, PE header features can still do a decent job of distinguishing packed malware from packed benignware. This is because although we cannot see packed binaries' code because of obfuscation, we can still see how much space the code takes up on disk and how the binary is laid out on disk or compressed over a series of file sections. These are telling details that can help a machine learning system distinguish malware from benignware. On the downside, PE header features don't capture the actual instructions a program executes when it is run, or the functions that it calls.

Import Address Table (IAT) Features

The Import Address Table (IAT), which you learned about in Chapter 1, is also an important source of machine learning features. The IAT contains a list of functions and libraries that a software binary imports from external DLL files. As such, the IAT contains important information about program behavior that you can use to complement the PE header features described in the previous section.

To use the IAT as a source of machine learning features, you need to represent each file as a dictionary of features, where the name of the imported library and function is the key, and the key maps to a 1, which indicates that the file in question contains that specific import (for example, the key "KERNEL32.DLL:LoadLibraryA", where KERNEL32.DLL is the DLL and LoadLibraryA is the function call). The feature dictionary resulting from computing IAT features in this way for a sample would look like { KERNEL32.DLL:LoadLibraryA: 1, ... }, where we'd assign a 1 to any keys observed in a binary.

In my experience building malware detectors, I have found that IAT features rarely work well on their own—although these features capture useful high-level information about program behavior, the malware often obfuscates the IAT to make itself look like benignware. Even when malware contains no obfuscation, it often imports the same DLL calls that benignware also imports, making it hard to distinguish between malware and benignware simply based on IAT information. Finally, when malware is packed (compressed or encrypted, such that the real malware code is only visible after the malware executes and uncompresses or unencrypts itself), the IAT only contains imports that the packer uses, not the imports that the malware uses. That said, when you use IAT features in conjunction with other features like PE header features and string features, they can boost system accuracy.

N-grams

So far you've learned about machine learning features that don't involve any concept of ordering. For example, we discussed string features to

check whether or not a binary has a particular string, but not whether a particular string comes before or after another string in the layout of the binary on disk.

But sometimes order matters. For example, we might find that an important malware family imports only commonly used functions, but imports them in a very specific order, such that when we observe the functions in that order, we know we're seeing that malware family and not benignware. To capture this kind of order information, you can use a machine learning concept called an N-gram.

N-grams sound more exotic than they are: they just involve laying out your features in the sequence in which they occur and then sliding a window of length n over the sequence, treating the sequence of features inside the window at each step as a single, aggregate feature. For example, if we had the sequence ["how", "now", "brown", "cow"] and we wanted to extract N-gram features of length 2 ($n = 2$) from this sequence, we would have [("how", "now"), ("now", "brown"), ("brown", "cow")] as our features.

In the context of malware detection, some kinds of data are most naturally represented as N-gram features. For example, when you disassemble a binary into its constituent instructions, like ["inc", "dec", "sub", "mov"], it makes sense to then use the N-gram approach to capture these sequences of instructions because representing a sequence of instructions can be useful in detecting particular malware implementations. Alternatively, when you're executing binaries to examine their dynamic behavior, you can use the N-gram approach to represent binaries' sequences of API calls or high-level behaviors.

I recommend experimenting with N-gram features in your machine learning-based malware detection systems whenever you're working with data that occurs in some type of sequence. It often takes some trial and error to determine what number you should set n to, which determines the length of your N-grams. This trial and error involves varying the n value to see which value yields the best accuracy on your test data. Once you find the right number, N-grams can be powerful features for capturing the actual sequential behaviors of program binaries, thereby boosting system accuracy.

Why You Can't Use All Possible Features

Now that you know the strengths and weaknesses of different categories of features, you may be wondering why you can't use all these features at once to build the best possible detector. There are a few reasons using all possible features is not a good idea.

First, extracting all the features we just explored takes a long time, which compromises how quickly your system can scan files. More importantly, if you use too many features on machine learning algorithms, you can run into memory issues and your system can take too long to train. This is why when building your systems, I recommend trying different features and honing in on those that work well on the kinds of malware you're trying to detect (and the kinds of benignware you're trying to avoid generating false positives on).

Unfortunately, even if you do home in on one category of features, like string features, you'll often have more features than most machine learning algorithms can handle. When using string features, you must have one feature for every unique string that occurs in your training data. For example, if training sample A contains the string "hello world", and training sample B contains the string "hello world!", then you'll need to treat "hello world" and "hello world!" as two separate features. This means that when you're dealing with thousands of training samples, you'll quickly encounter thousands of unique strings, and your system will end up using that many features.

Using the Hashing Trick to Compress Features

To get around the problem of having too many features, you can use a popular and straightforward solution called the *hashing trick*, also known as *feature hashing*. The idea is as follows: suppose you have a million unique string features in your training set, but the machine learning algorithm and hardware you're using can only deal with 4,000 unique features across the whole training set. You need some way of compressing a million features down to a feature vector that's 4,000 entries long.

The hashing trick makes these million features fit within a feature space of 4,000 by hashing each feature into one of 4,000 indices. Then you add the value of your original feature to the number at that index in your 4,000-dimensional feature vector. Of course, features often collide with this approach because their values are added together along the same dimension. This might affect system accuracy because the machine learning algorithm you're using can't "see" the value of individual features anymore. But in practice, this degradation in accuracy is often very small, and the benefit you get from the compressed representation of your features far outweighs this slight degradation that occurs because of the compression operation.

Implementing the Hashing Trick

To make these ideas clearer, I walk you through sample code that implements the hashing trick. Here I show this code to illustrate how the algorithm works; later, we'll use `sklearn`'s implementation of this function. Our sample code starts with a function declaration:

```
def apply_hashing_trick(feature_dict, vector_size=2000):
```

The `apply_hashing_trick()` function takes two parameters: the original feature dictionary and the size of the vector we store the smaller feature vector in after we apply the hashing trick.

Next, we create the new feature array using the following code:

```
new_features = [0 for x in range(vector_size)]
```

The `new_features` array stores the feature information after applying the hashing trick. Then we perform the key operations of the hashing trick inside a `for` loop, like in Listing 8-8.

```
for key in ❶feature_dict:
    array_index = ❷hash(key) % vector_size
    new_features[array_index] += ❸feature_dict[key]
```

Listing 8-8: Using a for loop to perform a hash operation

Here, we use a for loop to iterate over every feature in the feature dictionary ❶. To do this, first we hash the keys of the dictionary (in the case of string features, these would correspond to the software binaries' individual strings) modulo `vector_size` such that the hash values are bounded between zero and `vector_size - 1` ❷. We store the result of this operation in the `array_index` variable.

Still within the for loop, we increment the value of the `new_feature` array entry at index `array_index` by the value in our original feature array ❸. In the case of string features, where our feature values are set to 1 to indicate that the software binary has that particular string, we would increment this entry by one. In the case of PE header features, where features have a range of values (for example, corresponding to the amount of memory a PE section will take up), we would add the value of the feature to the entry.

Finally, outside of the for loop, we simply return the `new_features` dictionary, like this:

```
return new_features
```

At this point, sklearn can operate on `new_features` using just thousands instead of millions of unique features.

Complete Code for the Hashing Trick

Listing 8-9 shows the complete code for the hashing trick, which should now be familiar to you.

```
def apply_hashing_trick(feature_dict,vector_size=2000):
    # create an array of zeros of length 'vector_size'
    new_features = [0 for x in range(vector_size)]

    # iterate over every feature in the feature dictionary
    for key in feature_dict:

        # get the index into the new feature array
        array_index = hash(key) % vector_size

        # add the value of the feature to the new feature array
        # at the index we got using the hashing trick
        new_features[array_index] += feature_dict[key]

    return new_features
```

Listing 8-9: Complete code for implementing the hashing trick

As you have seen, the feature hashing trick is easy to implement on your own, and doing so ensures that you understand how it works. However, you can also just use sklearn's implementation, which is easy to use and more optimized.

Using sklearn's FeatureHasher

To use sklearn's built-in implementation instead of implementing your own hashing solution, you need to first import sklearn's FeatureHasher class, like this:

```
from sklearn.feature_extraction import FeatureHasher
```

Next, instantiate the FeatureHasher class:

```
hasher = FeatureHasher(n_features=2000)
```

To do this, you declare `n_features` to be the size of the new array that results from applying the hashing trick.

Then, to apply the hashing trick to some feature vectors, you simply run them through the FeatureHasher class's transform method:

```
features = [{'how': 1, 'now': 2, 'brown': 4}, {'cow': 2, '.': 5}]
hashed_features = hasher.transform(features)
```

The result is effectively the same as our custom implementation of the feature hashing trick shown in Listing 8-9. The difference is that here we're simply using sklearn's implementation, since it's easier to use a well-maintained machine learning library than our own code. The complete sample code is shown in Listing 8-10.

```
from sklearn.feature_extraction import FeatureHasher
hasher = FeatureHasher(n_features=10)
features = [{'how': 1, 'now': 2, 'brown': 4}, {'cow': 2, '.': 5}]
hashed_features = hasher.transform(features)
```

Listing 8-10: Implementing FeatureHasher

There are a few things to note about feature hashing before we move on. First, as you may have guessed, feature hashing obfuscates the feature information you pass into a machine learning algorithm because it adds feature values together simply based on the fact that they hash to the same bin. This means that, in general, the fewer bins you use (or the more features you hash into some fixed numbers of bins), the worse your algorithm will perform. Surprisingly, machine learning algorithms can still work well even when using the hashing trick, and because we simply can't deal with millions or billions of features on modern hardware, we usually have to use the feature hashing trick in security data science.

Another limitation of the feature hashing trick is that it makes it difficult or impossible to recover the original features you hashed when

analyzing the internals of your model. Take the example of decision trees: because we're hashing arbitrary features into each entry of our feature vectors, we don't know which of the features we added to a given entry is causing a decision tree algorithm to split on this entry, since any number of features could have caused the decision tree to think splitting on this entry was a good idea. Although this is a significant limitation, security data scientists live with it because of the huge benefits of the feature hashing trick in compressing millions of features down to a manageable number.

Now that we've gone over the building blocks required for building a real-world malware detector, let's explore how to build an end-to-end machine learning malware detector.

Building an Industrial-Strength Detector

From a software requirements perspective, our real-world detector will need to do three things: extract features from software binaries for use in training and detection, train itself to detect malware using training data, and actually perform detection on new software binaries. Let's walk through the code that does each of these things, which will show you how it all fits together.

You can access the code I use in this section at *malware_data_science/ch8/code/complete_detector.py* in the code accompanying this book, or at the same location in the virtual machine provided with this book. A one-line shell script, *malware_data_science/ch8/code/run_complete_detector.sh*, shows how to run the detector from the shell.

Extracting Features

To create our detector, the first thing we implement is code to extract features from training binaries (I skip over boilerplate code here and focus on the core functions of the program). Extracting features involves extracting the relevant data from training binaries, storing these features within a Python dictionary, and then, if we think our number of unique features will become prohibitively large, transforming them using sklearn's implementation of the hashing trick.

For simplicity's sake, we use only string features and choose to use the hashing trick. Listing 8-11 shows how to do both.

```
def get_string_features(❶path, ❷hasher):
    # extract strings from binary file using regular expressions
    chars = r" -~"
    min_length = 5
    string_regexp = '[%s]{%d,}' % (chars, min_length)
    file_object = open(path)
    data = file_object.read()
    pattern = re.compile(string_regexp)
    strings = pattern.findall(data)

    # store string features in dictionary form
    ❸ string_features = {}
```

```

    for string in strings:
        string_features[string] = 1

    # hash the features using the hashing trick
    ❹ hashed_features = hasher.transform([string_features])

    # do some data munging to get the feature array
    hashed_features = hashed_features.todense()
    hashed_features = numpy.asarray(hashed_features)
    hashed_features = hashed_features[0]

    # return hashed string features
    ❺ print "Extracted {0} strings from {1}".format(len(string_features),path)
    return hashed_features

```

Listing 8-11: Defining the `get_string_features` function

Here, we declare a single function called `get_string_features` that takes the path to the target binary ❶ and an instance of sklearn’s feature hashing class ❷ as its arguments. Then we extract the target file’s strings using a regular expression, which parses out all printable strings of minimum length 5. Then, we store the features in a Python dictionary ❸ for further processing by setting each string’s value to 1 in the dictionary, simply indicating that that feature is present in the binary.

Next, we hash the features using sklearn’s hashing trick implementation by calling `hasher`. Notice that we wrap the `string_features` dictionary in a Python list as we pass it into the `hasher` instance ❹ because sklearn requires that we pass in a list of dictionaries to be transformed, rather than a single dictionary.

Because we passed in our feature dictionary as a list of dictionaries, features are returned as a list of arrays. Additionally, they are returned in *sparse* format, a compressed representation that can be useful for processing large matrices, which we won’t discuss in this book. We need to get our data back into a normal `numpy` vector.

To get the data back into normal format, we call `.todense()` and `.asarray()`, and then select the first array in the list of `hasher` results to recover our final feature vector. The last step in the function is simply to return the feature vector `hashed_features` ❺ to the caller.

Training the Detector

Because sklearn does most of the hard work of training machine learning systems, training a detector requires only a small amount of code once we’ve extracted machine learning features from our target binaries.

To train a detector, we first need to extract features from our training examples, and then instantiate the feature hasher and the sklearn machine learning detector we wish to use (in this case, we use a random forest classifier). Then we need to call sklearn’s `fit` method on the detector to train it on the examples’ binaries. Finally, we save the detector and feature hasher to disk so we can use them when we want to scan files in the future.

Listing 8-12 shows the code for training the detector.

```
def ❶get_training_data(benign_path,malicious_path,hasher):
    def ❷get_training_paths(directory):
        targets = []
        for path in os.listdir(directory):
            targets.append(os.path.join(directory,path))
        return targets
    ❸malicious_paths = get_training_paths(malicious_path)
    ❹benign_paths = get_training_paths(benign_path)
    ❺X = [get_string_features(path,hasher)
        for path in malicious_paths + benign_paths]
    y = [1 for i in range(len(malicious_paths))]
        + [0 for i in range(len(benign_paths))]
    return X, y
def ❻train_detector(X,y,hasher):
    classifier = tree.RandomForestClassifier()
    ❼classifier.fit(X,y)
    ❽pickle.dump((classifier,hasher),open("saved_detector.pkl","w+"))
```

Listing 8-12: Programming sklearn to train the detector

Let's start by declaring the `get_training_data()` function ❶, which extracts features from training examples we provide. The function has three arguments: a path to a directory containing examples of benign binary programs (`benign_path`), a path to a directory containing examples of malicious binary programs (`malicious_path`), and an instance of the sklearn `FeatureHasher` class used to do feature hashing (`hasher`).

Next, we declare `get_training_paths()` ❷, a local helper function that gets us the list of absolute file paths for files occurring in a given directory. In the next two lines, we use `get_training_paths` to get us the lists of paths that occur in the malicious ❸ and benign ❹ training example directories.

Finally, we extract our features and create our label vector. We do this by calling the `get_string_features` function described in Listing 8-11 on every training example file path ❺. Notice that the label vector has a 1 for every malicious path and a 0 for every benign path, such that the numbers at the indices in the label vector correspond to the label of the feature vectors at those same indices in the `X` array. This is the form in which sklearn expects feature and label data, and it allows us to tell the library the label for each feature vector.

Now that we've finished extracting features and created our feature vector `X` and our label vector `y`, we're ready to tell sklearn to train our detector using the feature vectors and the label vector.

We do this using the `train_detector()` function ❻, which takes three arguments: the training example feature vectors (`X`), the label vector (`y`), and the instance of sklearn's feature hasher (`hasher`). In the function body we instantiate `tree.RandomForestClassifier`, the sklearn detector. Then we pass `X` and `y` into the detector's `fit` method to train it ❼, and then use the Python `pickle` module ❽ to save the detector and hasher for future use.

Running the Detector on New Binaries

Now let's go over how to use the saved detector we just trained to detect malware in new program binaries. Listing 8-13 shows how to write the `scan_file()` function to do this.

```
def scan_file(path):
    if not os.path.exists("saved_detector.pkl"):
        print "Train a detector before scanning files."
        sys.exit(1)
    ❶ with open("saved_detector.pkl") as saved_detector:
        classifier, hasher = pickle.load(saved_detector)
        features = ❷get_string_features(path,hasher)
        result_proba = ❸classifier.predict_proba(features)[1]
        # if the user specifies malware_paths and
        # benignware_paths, train a detector
    ❹ if result_proba > 0.5:
        print "It appears this file is malicious!",`result_proba`
    else:
        print "It appears this file is benign.",`result_proba`
```

Listing 8-13: Running the detector on new binaries

Here, we declare the `scan_file()` function to scan a file to determine whether it's malicious or benign. Its only argument is the path to the binary that we are going to scan. The function's first job is to load the saved detector and hasher from the pickle file to which they were saved ❶.

Next, we extract features from the target file using the function `get_string_features` ❷ we defined in Listing 8-11.

Finally, we call the detector's `predict` method to decide whether or not the file in question is malicious, given the features extracted. We do this using the `predict_proba` method ❸ of the classifier instance and selecting the second element of the array that it returns, which corresponds to the probability that the file is malicious. If this probability is above 0.5, or 50 percent ❹, we say the file is malicious; otherwise, we tell the user that it's benign. We can change this decision threshold to something much higher to minimize false positives.

What We've Implemented So Far

Listing 8-14 shows the code for this small-scale but realistic malware detector in its entirety. I hope that the code reads fluidly to you now that you've seen how each individual piece works.

```
#!/usr/bin/python
```

```
import os
import sys
import pickle
import argparse
import re
import numpy
```

```

from sklearn.ensemble import RandomForestClassifier
from sklearn.feature_extraction import FeatureHasher

def get_string_features(path, hasher):
    # extract strings from binary file using regular expressions
    chars = r" -~"
    min_length = 5
    string_regex = '[%s]{%d,}' % (chars, min_length)
    file_object = open(path)
    data = file_object.read()
    pattern = re.compile(string_regex)
    strings = pattern.findall(data)

    # store string features in dictionary form
    string_features = {}
    for string in strings:
        string_features[string] = 1

    # hash the features using the hashing trick
    hashed_features = hasher.transform([string_features])

    # do some data munging to get the feature array
    hashed_features = hashed_features.todense()
    hashed_features = numpy.asarray(hashed_features)
    hashed_features = hashed_features[0]

    # return hashed string features
    print "Extracted {0} strings from {1}".format(len(string_features), path)
    return hashed_features

def scan_file(path):
    # scan a file to determine if it is malicious or benign
    if not os.path.exists("saved_detector.pkl"):
        print "Train a detector before scanning files."
        sys.exit(1)
    with open("saved_detector.pkl") as saved_detector:
        classifier, hasher = pickle.load(saved_detector)
    features = get_string_features(path, hasher)
    result_proba = classifier.predict_proba([features])[0, 1]
    # if the user specifies malware_paths and
    # benignware_paths, train a detector
    if result_proba > 0.5:
        print "It appears this file is malicious!", `result_proba`
    else:
        print "It appears this file is benign.", `result_proba`

def train_detector(benign_path, malicious_path, hasher):
    # train the detector on the specified training data
    def get_training_paths(directory):
        targets = []
        for path in os.listdir(directory):
            targets.append(os.path.join(directory, path))
        return targets
    malicious_paths = get_training_paths(malicious_path)
    benign_paths = get_training_paths(benign_path)

```

```

X = [get_string_features(path,hasher) for path in malicious_paths + benign_paths]
y = [1 for i in range(len(malicious_paths))] + [0 for i in range(len(benign_paths))]
classifier = tree.RandomForestClassifier(64)
classifier.fit(X,y)
pickle.dump((classifier,hasher),open("saved_detector.pkl","w+"))

def get_training_data(benign_path,malicious_path,hasher):
    def get_training_paths(directory):
        targets = []
        for path in os.listdir(directory):
            targets.append(os.path.join(directory,path))
        return targets
    malicious_paths = get_training_paths(malicious_path)
    benign_paths = get_training_paths(benign_path)
    X = [get_string_features(path,hasher) for path in malicious_paths + benign_paths]
    y = [1 for i in range(len(malicious_paths))] + [0 for i in range(len(benign_paths))]
    return X, y

parser = argparse.ArgumentParser("get windows object vectors for files")
parser.add_argument("--malware_paths",default=None,help="Path to malware training files")
parser.add_argument("--benignware_paths",default=None,help="Path to benignware training files")
parser.add_argument("--scan_file_path",default=None,help="File to scan")
args = parser.parse_args()

hasher = FeatureHasher(20000)
if args.malware_paths and args.benignware_paths:
    train_detector(args.benignware_paths,args.malware_paths,hasher)
elif args.scan_file_path:
    scan_file(args.scan_file_path)
else:
    print "[*] You did not specify a path to scan," \
          " nor did you specify paths to malicious and benign training files" \
          " please specify one of these to use the detector.\n"
    parser.print_help()

```

Listing 8-14: Basic machine learning malware detector code

Writing a machine learning–based malware detector is great, but evaluating and improving its performance is necessary if you’re going to deploy the detector with any confidence in its efficacy. Next, you learn different ways to evaluate the performance of your detector.

Evaluating Your Detector’s Performance

Conveniently, sklearn contains code that makes it easy to evaluate detection systems using measurements like ROC curves, which you learned about in Chapter 7. The sklearn library also provides additional evaluation functionality specific to evaluating machine learning systems. For example, you can use sklearn’s functions for performing cross-validation, which is a powerful method for predicting how well your detector will work when you deploy it.

In this section, you learn how to use `sklearn` to plot ROC curves that show your detector's accuracy. You also learn about cross-validation and how to implement it with `sklearn`.

Using ROC Curves to Evaluate Detector Efficacy

Recall that Receiver Operating Characteristic (ROC) curves measure the changes in a detector's true positive rate (the percentage of malware that it successfully detects) and false positive rate (the percentage of benignware that it falsely flags as malware) as you adjust its sensitivity.

The higher the sensitivity, the more false positives you will get but the greater your detection rate. The lower the sensitivity, the fewer false positives but also the fewer detections you'll get. To compute a ROC curve you need a detector that can output a *threat score* such that the higher its value the more likely it is that a binary is malicious. Conveniently, `sklearn`'s implementations of decision trees, logistic regression, k-nearest neighbors, random forests, and other machine learning approaches covered in this book all provide the option of outputting a threat score that reflects whether a file is malware or benignware. Let's explore how we can use ROC curves to determine a detector's accuracy.

Computing ROC Curves

To compute a ROC curve for the machine learning detector we built in Listing 8-14, we need to do two things: first, define an experimental setup, and second, implement the experiment using `sklearn`'s `metrics` module. For our basic experimental setup, we'll split our training examples in half such that we use the first half for training and the second half for computing the ROC curve. This split simulates the problem of detecting zero-day malware. Basically, by splitting the data, we're telling the program, "show me one set of benignware and malware that I'll use to learn how to identify malware and benignware, and then show me the other set to test me on how well I learned the concept of malware and benignware." Because the detector has never seen the malware (or benignware) in the test set, this evaluation setup is a simple way to predict how well the detector will do against truly new malware.

Implementing this split with `sklearn` is straightforward. First, we add an option to the argument parser class of our detector program to signal that we want to evaluate the detector's accuracy, like this:

```
parser.add_argument("--evaluate", default=False,
                    action="store_true", help="Perform cross-validation")
```

Then, in the part of the program where we process command line arguments, shown in Listing 8-15, we add another `elif` clause that handles the case that the user has added `-evaluate` to the command line arguments.

```

elif args.malware_paths and args.benignware_paths and args.evaluate:
    ❶ hasher = FeatureHasher()
    X, y = ❷get_training_data(
        args.benignware_paths,args.malware_paths,hasher)
    evaluate(X,y,hasher)
def ❸evaluate(X,y,hasher):
    import random
    from sklearn import metrics
    from matplotlib import pyplot

```

Listing 8-15: Running the detector on new binaries

Let's walk through this code in detail. First, we instantiate an sklearn feature hasher ❶, get the training data we require for our evaluation experiment ❷, and then call a function named evaluate ❸, which takes the training data (X, y) and the feature hasher instance (hasher) as its parameters and then imports three modules we need to perform the evaluation. We use the random module to randomly select which training examples to use for training the detector and which to use for testing it. We use the metrics module from sklearn to compute the ROC curve, and we use the pyplot module from matplotlib (the de facto standard Python library for data visualization) to visualize the ROC curve.

Splitting Data into Training and Test Sets

Now that we've randomly sorted the X and y arrays corresponding to our training data, we can split these arrays into equally sized training and test sets, as shown in Listing 8-16, which continues defining the evaluate() function begun in Listing 8-15.

```

❶ X, y = numpy.array(X), numpy.array(y)
❷ indices = range(len(y))
❸ random.shuffle(indices)
❹ X, y = X[indices], y[indices]
    splitpoint = len(X) * 0.5
❺ splitpoint = int(splitpoint)
❻ training_X, test_X = X[:splitpoint], X[splitpoint:]
    training_y, test_y = y[:splitpoint], y[splitpoint:]

```

Listing 8-16: Splitting the data into training and test sets

First, we convert X and y into numpy arrays ❶, and then we create a list of indices corresponding to the number of elements in X and y ❷. Next, we randomly shuffle these indices ❸ and reorder X and y based on this new order ❹. This sets us up to randomly assign samples to either our training set or our test set, ensuring that we don't split the samples simply by the order in which they occur in our experimental data directory. To complete the random split, we divide the arrays in half by finding the array index that evenly splits the dataset in half, rounding this point to the nearest integer using the int() function ❺, and then actually splitting the X and y arrays into training and test sets ❻.

Now that we have our training and test sets, we can instantiate and train our decision tree detector using the training data using the following:

```
classifier = RandomForestClassifier()  
classifier.fit(training_X, training_y)
```

Then we use the trained classifier to get scores for our test examples corresponding to the likelihood that these test examples are malicious:

```
scores = classifier.predict_proba(test_X)[:,-1]
```

Here, we call the `predict_proba()` method on our classifier, which predicts the probability that our test examples are benignware or malware. Then, using `numpy` indexing magic, we pull out only the probabilities that the samples are malicious, as opposed to benign. Keep in mind that these probabilities are redundant (for example, if the probability an example is malicious is 0.99, then the probability it's benign is 0.01, since probabilities add up to 1.00), so all we need is the malware probability here.

Computing the ROC Curve

Now that we've computed malware probabilities (which we can also refer to as "scores") using our detector, it's time to compute our ROC curve. We do this by first calling the `roc_curve` function within `sklearn`'s `metrics` module, like this:

```
fpr, tpr, thresholds = metrics.roc_curve(test_y, scores)
```

The `roc_curve` function tests a variety of *decision thresholds*, or score thresholds above which we would deem a software binary to be malicious, and measures what the false positive rate and true positive rate of the detector would be if we were to use that detector.

You can see that the `roc_curve` function takes two arguments: the label vector for our test examples (`test_y`) and the scores array, which contains our detector's judgment about how malicious it thinks each training example is. The function returns three related arrays: `fpr`, `tpr`, and `thresholds`. These arrays are all of equal length, such that the false positive rate, true positive rate, and decision threshold at each index correspond to one another.

Now we can use `matplotlib` to visualize the ROC curve we just calculated. We do this by calling the `plot` method on `matplotlib`'s `pyplot` module, as shown here:

```
pyplot.plot(fpr, tpr, 'r-')  
pyplot.xlabel("Detector false positive rate")  
pyplot.ylabel("Detector true positive rate")  
pyplot.title("Detector ROC Curve")  
pyplot.show()
```

We call the `xlabel`, `ylabel`, and `title` methods to label the chart's axes and title, and then the `show` method to make the chart window pop up. The resulting ROC curve is shown in Figure 8-2.

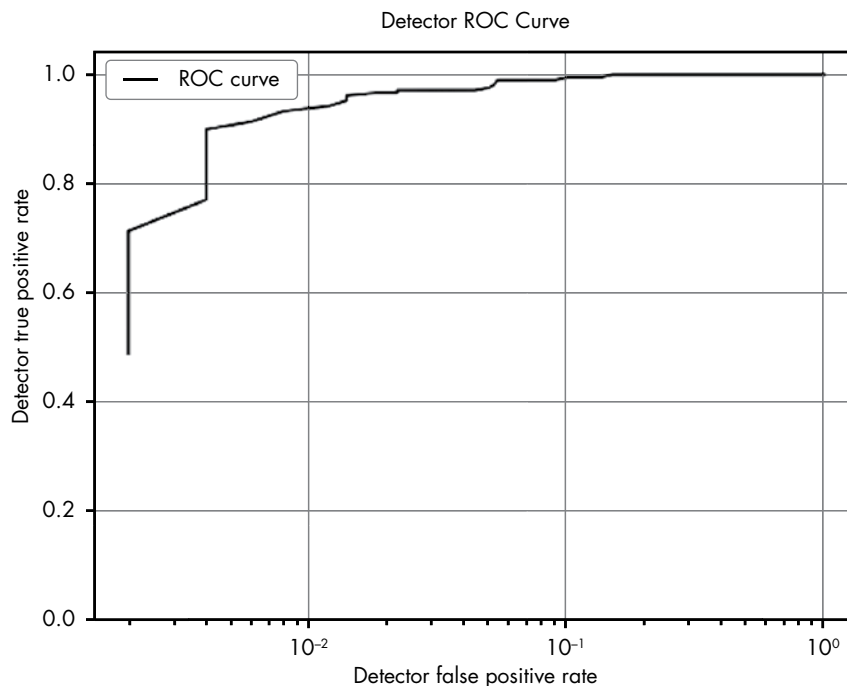


Figure 8-2: Visualizing the detector's ROC curve

You can see from the plot in Figure 8-2 that our detector performs well for such a basic example. At around a 1 percent false positive rate (10^{-2}), it can detect about 94 percent of the malware samples in the test set. We're only training it on a few hundred training examples here; to get better accuracy we'd need to train it on tens of thousands, hundreds of thousands, or even millions of examples (alas, scaling machine learning to this degree is beyond the scope of this book).

Cross-Validation

Although visualizing the ROC curve is useful, we can actually do better at predicting our detector's real-world accuracy by performing many experiments on our training data, not just one. Recall that to perform our test, we split our training examples in half, training the detector on the first half and testing it on the second half. This is really an insufficient test of our detector. In the real world, we won't be measured on our accuracy on this particular set of test examples but rather on our accuracy on new, previously unseen malware. To get a better sense of how we'll perform once we

deploy, we need to run more than just one experiment on one set of test data; we need to perform many experiments on many test sets and get a sense of the overall trend in accuracy.

We can use *cross-validation* to do this. The basic idea behind cross-validation is to split our training examples into a number of folds (here I use three folds, but you can use more). For example, if you had 300 examples and decided to split them into three folds, the first 100 samples would go in the first fold, the second 100 would go in the second fold, and the third 100 would go in the third fold.

Then we run three tests. In the first test, we train the system on folds 2 and 3 and test the system on fold 1. On the second test, we repeat this process but train the system on folds 1 and 3 and test the system on fold 2. On the third test, as you can probably predict by now, we train the system on folds 1 and 2 and test the system on fold 3. Figure 8-3 illustrates this cross-validation process.

	Fold 1	Fold 2	Fold 3
Experiment 1	Use for testing	Use for training	Use for training
Experiment 2	Use for training	Use for testing	Use for training
Experiment 3	Use for training	Use for training	Use for testing

Figure 8-3: A visualization of a sample cross-validation process

The sklearn library makes implementing cross-validation easy. To do this, let's rewrite our evaluate function from Listing 8-15 as `cv_evaluate`.

```
def cv_evaluate(X,y,hasher):
    import random
    from sklearn import metrics
    from matplotlib import pyplot
    from sklearn.cross_validation import KFold
```

We start the `cv_evaluate()` function the same way we started our initial evaluation function, except that here we also import the `KFold` class from sklearn's `cross_validation` module. *K-fold cross-validation*, or `KFold` for short, is synonymous with the type of cross-validation I just discussed and is the most common way to do cross-validation.

Next, we convert our training data to numpy arrays so that we can use numpy's enhanced array indexing on it:

```
X, y = numpy.array(X), numpy.array(y)
```

The following code actually starts the cross-validation process:

```
fold_counter = 0
for train, test in KFold(len(X),3,shuffle=True):
```

```
② training_X, training_y = X[train], y[train]
  test_X, test_y = X[test], y[test]
```

We first instantiate the `KFold` class, passing in the number of training examples we have as the first parameter and the number of folds we'd like to use as the second argument. The third argument, `shuffle=True` ❶, tells `sklearn` to randomly sort our training data before dividing it into three folds. The `KFold` instance is actually an iterator that gives a different training or test example split on each iteration. Within the `for` loop, we assign the training instances and test instances to the `training_X` and `training_y` arrays ❷ that contain the corresponding elements.

After preparing the training and test data, we're ready to instantiate and train the `RandomForestClassifier`, as you've learned to do previously in this chapter:

```
classifier = RandomForestClassifier()
classifier.fit(training_X, training_y)
```

Finally, we compute a ROC curve for this particular fold and then plot a line that represents this ROC curve:

```
scores = classifier.predict_proba(test_X)[:,-1]
fpr, tpr, thresholds = metrics.roc_curve(test_y, scores)
pyplot.semilogx(fpr, tpr, label="Fold number {}".format(fold_counter))
fold_counter += 1
```

Note that we don't call the `matplotlib` `show` method to display the chart just yet. We do this after all the folds have been evaluated and we're ready to show all three lines at once. As we did in the previous section, we label our axes and give the plot a title, like this:

```
pyplot.xlabel("Detector false positive rate")
pyplot.ylabel("Detector true positive rate")
pyplot.title("Detector Cross-Validation ROC Curves")
pyplot.legend()
pyplot.grid()
pyplot.show()
```

The resulting ROC curve is shown in Figure 8-4.

As you can see, our results were similar on every fold, but there is definitely some variation. Our detection rate (true positive rate) over the three runs averages about 90 percent at a 1 percent false positive rate. This estimate, which takes into account all three cross-validation experiments, is a more accurate estimate of our detector's performance than we'd get if we just ran one experiment on our data; in that case, which samples we happened to use for training and testing would lead to a somewhat random outcome. By running more experiments, we can get a more robust sense of our solution's efficacy.

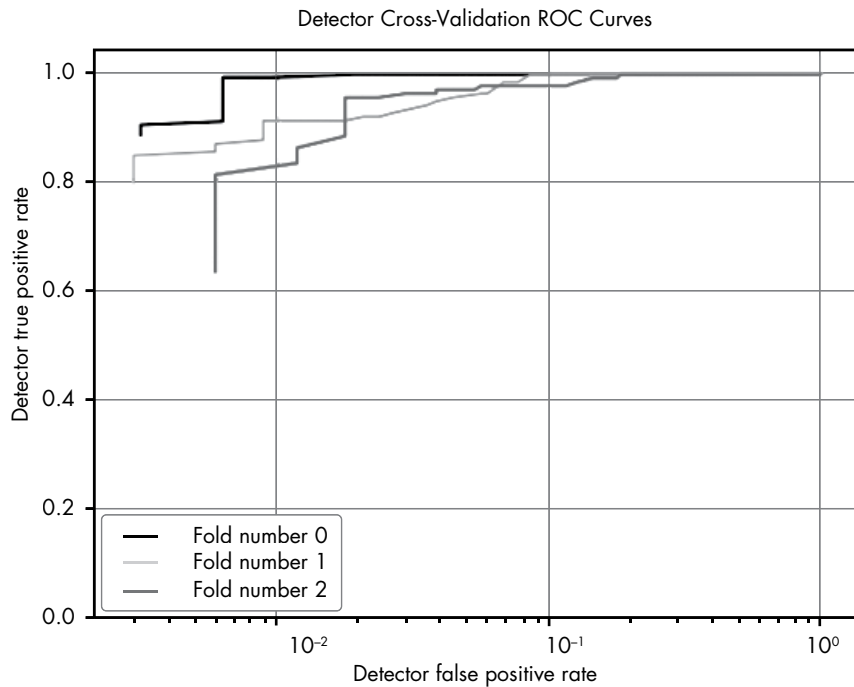


Figure 8-4: Plotting the detector’s ROC curve using cross-validation

Note that these results are not great because we’re training on a very small amount of data: a few hundred malware and benignware samples. At my day job, where we train large-scale machine learning malware detection systems, we usually train on hundreds of millions of samples. You don’t need hundreds of millions of samples to train your own malware detector, but you’ll want to assemble datasets of at least tens of thousands of samples to start getting really good performance (for example, a 90 percent detection rate at a 0.1 percent false positive rate).

Next Steps

So far, I covered how to use Python and `sklearn` to extract features from a training dataset of software binaries, and then train and evaluate a decision tree–based machine learning approach. To improve the system, you can use features other than or in addition to printable string features (for example, the PE header, instruction N-gram, or Import Address Table features discussed previously), or you could use a different machine learning algorithm.

To make the detector more accurate, I recommend going beyond `sklearn`’s `RandomForestClassifier` (`sklearn.ensemble.RandomForestClassifier`) to try other classifiers. Recall from the previous chapter that *random forest detectors* are also based on decision trees, but instead of just one decision tree, they build many decision trees, randomizing the way they are built. To

determine whether a new file is malware or benignware, each of these decision trees makes individual decisions, which we combine by summing them up and dividing them by the total number of trees to get the average result.

You can also use other algorithms that sklearn provides, such as logistic regression. Using any of these algorithms can be as simple as doing a search and replace in the sample code discussed in this chapter. For example, in this chapter we instantiate and train our decision tree as follows:

```
classifier = RandomForestClassifier()  
classifier.fit(training_X,training_y)
```

But you can simply replace that code with this:

```
from sklearn.linear_model import LogisticRegression  
classifier = LogisticRegression()  
classifier.fit(training_X,training_y)
```

This replacement yields a logistic regression detector instead of a decision tree–based detector. By computing a new cross validation–based evaluation of this Logistic Regression detector and comparing it to the results from Figure 8-4, you could determine which works better.

Summary

In this chapter, you learned the ins and outs of building machine learning–based malware detectors. Specifically, you learned how to extract features from software binaries for machine learning, how to compress these features using the hashing trick, and how to train machine learning–based malware detectors using these extracted features. You also learned how to plot ROC curves to examine the relationship between a detector’s detection threshold and its true and false positive rates. Finally, you learned about cross-validation, a more advanced evaluation concept, and other possible extensions to enhance the detector used in this chapter.

This concludes this book’s discussion of machine learning–based malware detection using sklearn. We’ll cover another set of machine learning methods, known as deep learning methods or artificial neural networks in Chapters 10 and 11. You now have the basic knowledge necessary to effectively use machine learning in the context of malware identification. I encourage you to read more about machine learning. Because computer security is in many ways a data analysis problem, machine learning is here to stay in the security industry and will continue to be useful not only in detecting malicious binaries but also in detecting malicious behavior in network traffic, system logs, and other contexts.

In the next chapter, we’ll take a deep dive into visualizing malware relationships, which can help us quickly understand the similarities and differences between large numbers of malware samples.

9

VISUALIZING MALWARE TRENDS



Sometimes the best way to analyze malware collections is to visualize them. Visualizing security data allows us to quickly recognize trends in malware and within the threat landscape at large. These visualizations are often far more intuitive than nonvisual statistics, and they can help communicate insights to diverse audiences. For example, in this chapter, you see how visualization can help us identify the types of malware prevalent in a dataset, the trends within malware datasets (the emergence of ransomware as a trend in 2016, for example), and the relative efficacy of commercial antivirus systems at detecting malware.

Working through these examples, you come away understanding how to create your own visualizations that can lead to valuable insights by using the Python data analysis package `pandas`, as well as the Python data visualization packages `seaborn` and `matplotlib`. The `pandas` package is used mostly for loading and manipulating data and doesn't have much to do with data visualization itself, but it's very useful for preparing data for visualization.

Why Visualizing Malware Data Is Important

To see how visualizing malware data can be helpful, let's go through two examples. The first visualization addresses the following question: is the antivirus industry's ability to detect ransomware improving? The second visualization asks which malware types have trended over the period of a year. Let's look at the first example shown in Figure 9-1.

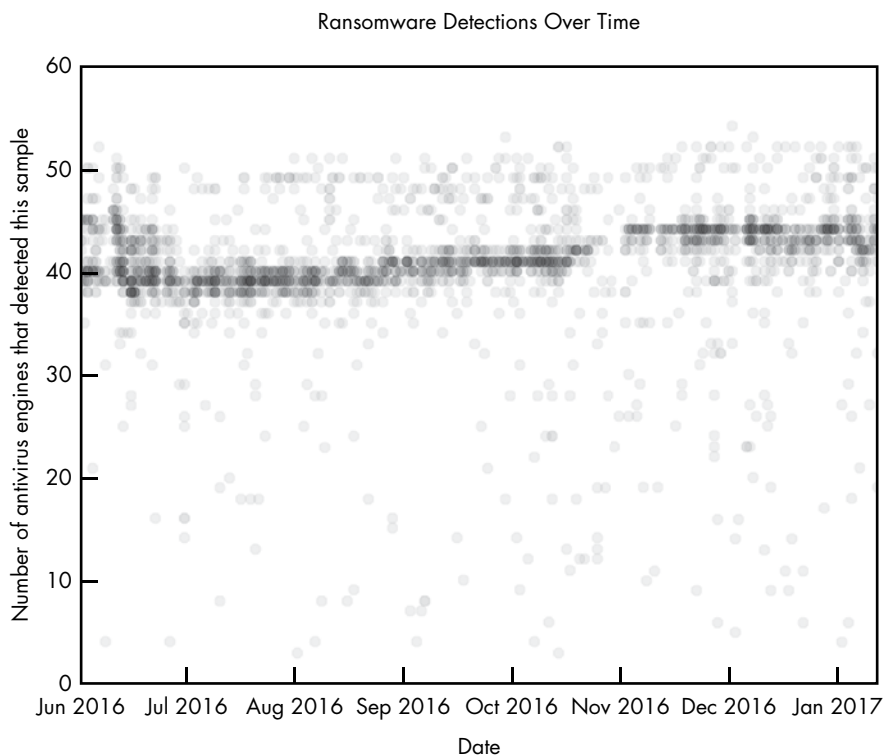


Figure 9-1: Visualization of ransomware detections over time

I created this ransomware visualization using data collected from thousands of ransomware malware samples. This data contains the results of running 57 separate antivirus engines against each file. Each circle represents a malware sample. The y-axis represents how many detections, or positives, each malware sample received from the antivirus engines when it was scanned. Keep in mind that while this y-axis stops at 60, the maximum count for a given scan is 57, the total number of scanners. The x-axis represents when each malware sample was first seen on the malware analysis site *VirusTotal.com* and scanned.

In this plot, we can see the antivirus community's ability to detect these malicious files started off relatively strong in June 2016, dipped around July 2016, and then steadily rose over the rest of the year. By the end of 2016,

ransomware files were still missed by an average of about 25 percent of anti-virus engines, so we can conclude that the security community remained somewhat weak at detecting these files during this time.

To extend this investigation, you could create a visualization that shows *which* antivirus engines are detecting ransomware and at what rate, and how they are improving over time. Or you could look at some other category of malware (for example, Trojan horses). Such plots are useful in deciding which antivirus engines to purchase, or deciding which kinds of malware you might want to design custom detection solutions for—perhaps to supplement a commercial antivirus detection system (for more on building custom detection systems, see Chapter 8).

Now let's look at Figure 9-2, which is another sample visualization, created using the same dataset used for Figure 9-1.

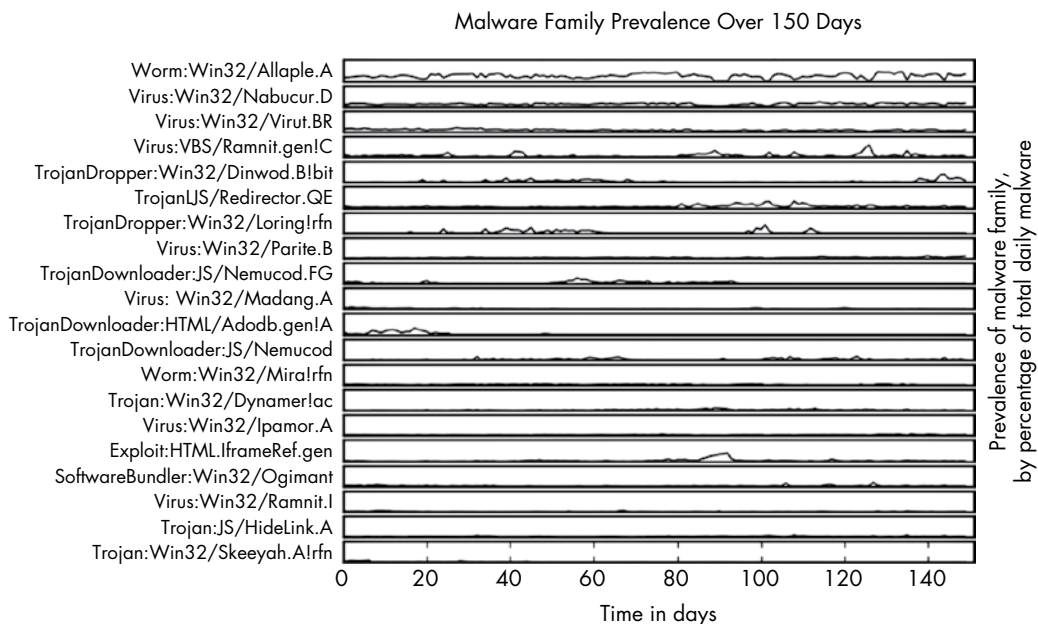


Figure 9-2: Visualization of per-family malware detections over time

Figure 9-2 shows the top 20 most common malware families and how frequently they occurred relative to one another over a 150-day period. The plot reveals some key insights: whereas the most popular malware family, Allaple.A, occurred consistently over the 150-day span, other malware families, like Nemucod.FG, were prevalent for shorter spans of time and then went silent. A plot like this, generated using malware detected on your own workplace's network, can reveal helpful trends showing what types of malware are involved in attacks against your organization over time. Without the creation of a comparison figure such as this one, understanding and comparing the relative peaks and volumes of these malware types over time would be difficult and time consuming.

These two examples show how useful malware visualization can be. The rest of this chapter shows how to create your own visualizations. We start by discussing the sample dataset used in this chapter and then we use the pandas package to analyze the data. Finally, we use the matplotlib and seaborn packages to visualize the data.

Understanding Our Malware Dataset

The dataset we use contains data describing 37,000 unique malware binaries collected by VirusTotal, a malware detection aggregation service. Each binary is labeled with four fields: the number of antivirus engines (out of 57) that flagged the binary as malicious (I call this the number of *positives* associated with each sample), the size of each binary, the binary's *type* (bitcoin miner, keylogger, ransomware, trojan, or worm), and the date on which the binary was first seen. We'll see that even with this fairly limited amount of metadata for each binary, we can analyze and visualize the data in ways that reveal important insights into the dataset.

Loading Data into pandas

The popular Python data analysis library pandas makes it easy to load data into analysis objects called DataFrames, and then provides methods to slice, transform, and analyze that repackaged data. We use pandas to load and analyze our data and prep it for easy visualization. Let's use Listing 9-1 to define and load some sample data into the Python interpreter.

```
In [135]: import pandas

In [136]: example_data = [❶{'column1': 1, 'column2': 2},
...: {'column1': 10, 'column2': 32},
...: {'column1': 3, 'column2': 58}]

In [137]: ❷pandas.DataFrame(example_data)
Out[137]:
```

	column1	column2
0	1	2
1	10	32
2	3	58

Listing 9-1: Loading data into pandas directly

Here we define some data, which we call `example_data`, as a list of Python dictionaries ❶. Once we have created this list of dicts, we pass it to the DataFrame constructor ❷ to get the corresponding pandas DataFrame. Each of these dicts becomes a row in the resulting DataFrame. The keys in the dicts (`column1` and `column2`) become columns. This is one way to load data into pandas directly.

You can also load data from external CSV files. Let's use the code in Listing 9-2 to load this chapter's dataset (available on the virtual machine or in the data and code archive that accompany this book).

```
import pandas
malware = pandas.read_csv("malware_data.csv")
```

Listing 9-2: Loading data into pandas from an external CSV file

When you import *malware_data.csv*, the resulting malware object should look something like this:

	positives	size	type	fs_bucket
0	45	251592	trojan	2017-01-05 00:00:00
1	32	227048	trojan	2016-06-30 00:00:00
2	53	682593	worm	2016-07-30 00:00:00
3	39	774568	trojan	2016-06-29 00:00:00
4	29	571904	trojan	2016-12-24 00:00:00
5	31	582352	trojan	2016-09-23 00:00:00
6	50	2031661	worm	2017-01-04 00:00:00

We now have a pandas DataFrame composed of our malware dataset. It has four columns: positives (the number of antivirus detections out of 57 anti-virus engines for that sample), size (the number of bytes that malware binary takes up on disk), type (the type of malware, such as Trojan horse, worm, and so on), and fs_bucket (the date on which this malware was first seen).

Working with a pandas DataFrame

Now that we have our data in a pandas DataFrame, let's look at how to access and manipulate it by calling the describe() method, as shown in Listing 9-3.

```
In [51]: malware.describe()
Out[51]:
```

	positives	size
count	37511.000000	3.751100e+04
mean	39.446536	1.300639e+06
std	15.039759	3.006031e+06
min	3.000000	3.370000e+02
25%	32.000000	1.653960e+05
50%	45.000000	4.828160e+05
75%	51.000000	1.290056e+06
max	57.000000	1.294244e+08

Listing 9-3: Calling the describe() method

As shown in Listing 9-3, calling the describe() method shows some useful statistics about our DataFrame. The first line, count, counts the total number of non-null positives rows, and the total number of non-null rows. The second line gives the mean, or average number of positives per sample, and the mean size of the malware samples. Next comes the standard deviation for both positives and size, and the minimum value of each column in all the samples in the dataset. Finally, we see percentile values for each of the columns and the maximum value for the columns.

Suppose we'd like to retrieve the data for one of the columns in the malware DataFrame, such as the positives column (to view the average number of detections each file has, for example, or plot a histogram showing the distribution of positives over the dataset). To do this, we simply write `malware['positives']`, which returns the positives column as a list of numbers, as shown in Listing 9-4.

```
In [3]: malware['positives']
Out[3]:
0      45
1      32
2      53
3      39
4      29
5      31
6      50
7      40
8      20
9      40
--snip--
```

Listing 9-4: Returning the positives column

After retrieving a column, we can compute statistics on it directly. For example, `malware['positives'].mean()` computes the mean of the column, `malware['positives'].max()` computes the maximum value, `malware['positives'].min()` computes the minimum value, and `malware['positives'].std()` computes the standard deviation. Listing 9-5 shows examples of each.

```
In [7]: malware['positives'].mean()
Out[7]: 39.446535682866362

In [8]: malware['positives'].max()
Out[8]: 57

In [9]: malware['positives'].min()
Out[9]: 3

In [10]: malware['positives'].std()
Out[10]: 15.039759380778822
```

Listing 9-5: Calculating the mean, maximum, and minimum values and the standard deviation

We can also slice and dice the data to do more detailed analysis. For example, Listing 9-6 computes the mean positives for the trojan, bitcoin, and worm types of malware.

```
In [67]: malware[malware['type'] == 'trojan']['positives'].mean()
Out[67]: 33.43822473365119

In [68]: malware[malware['type'] == 'bitcoin']['positives'].mean()
Out[68]: 35.857142857142854
```

```
In [69]: malware[malware['type'] == 'worm']['positives'].mean()
Out[69]: 49.90857904874796
```

Listing 9-6: Calculating the average detection rates of different malwares

We first select the rows of the DataFrame where type is set to trojan using the following notation: `malware[malware['type'] == 'trojan']`. To select the positives column of the resulting data and compute the mean, we extend this expression as follows: `malware[malware['type'] == 'trojan']['positives'].mean()`. Listing 9-6 yields an interesting result, which is that worms get detected more frequently than bitcoin mining and Trojan horse malware. Because $49.9 > 35.8$ and 33.4 , on average, malicious worm samples (49.9) are detected by more vendors than malicious bitcoin and trojan samples (35.8, 33.4).

Filtering Data Using Conditions

We can select a subset of the data using other conditions as well. For example, we can use “greater than” and “less than” style conditions on numerical data like malware file size to filter the data, and then compute statistics on the resulting subsets. This can be useful if we’re interested in finding out whether the effectiveness of the antivirus engines is related to file size. We can check this using the code in Listing 9-7.

```
In [84]: malware[malware['size'] > 1000000]['positives'].mean()
Out[84]: 33.507073192162373
```

```
In [85]: malware[malware['size'] > 2000000]['positives'].mean()
Out[85]: 32.761442050415432
```

```
In [86]: malware[malware['size'] > 3000000]['positives'].mean()
Out[86]: 27.20672682526661
```

```
In [87]: malware[malware['size'] > 4000000]['positives'].mean()
Out[87]: 25.652548725637182
```

```
In [88]: malware[malware['size'] > 5000000]['positives'].mean()
Out[88]: 24.411069317571197
```

Listing 9-7: Filtering the results by malware file size

Take the first line in the preceding code: first, we subset our DataFrame by only samples that have a size over one million (`malware[malware['size'] > 1000000]`). Then we grab the positives column and calculate the mean (`['positives'].mean()`), which is about 33.5. As we do this for higher and higher file sizes, we see that the average number of detections for each group goes down. This means we’ve discovered that there is indeed a relationship between malware file size and the average number of antivirus engines that detect those malware samples, which is interesting and merits further investigation. We explore this visually next by using `matplotlib` and `seaborn`.

Using matplotlib to Visualize Data

The go-to library for Python data visualization is matplotlib; in fact, most other Python visualization libraries are essentially convenience wrappers around matplotlib. It's easy to use matplotlib with pandas: we use pandas to get, slice, and dice the data we want to plot, and we use matplotlib to plot it. The most useful matplotlib function for our purposes is the plot function. Figure 9-3 shows what the plot function can do.

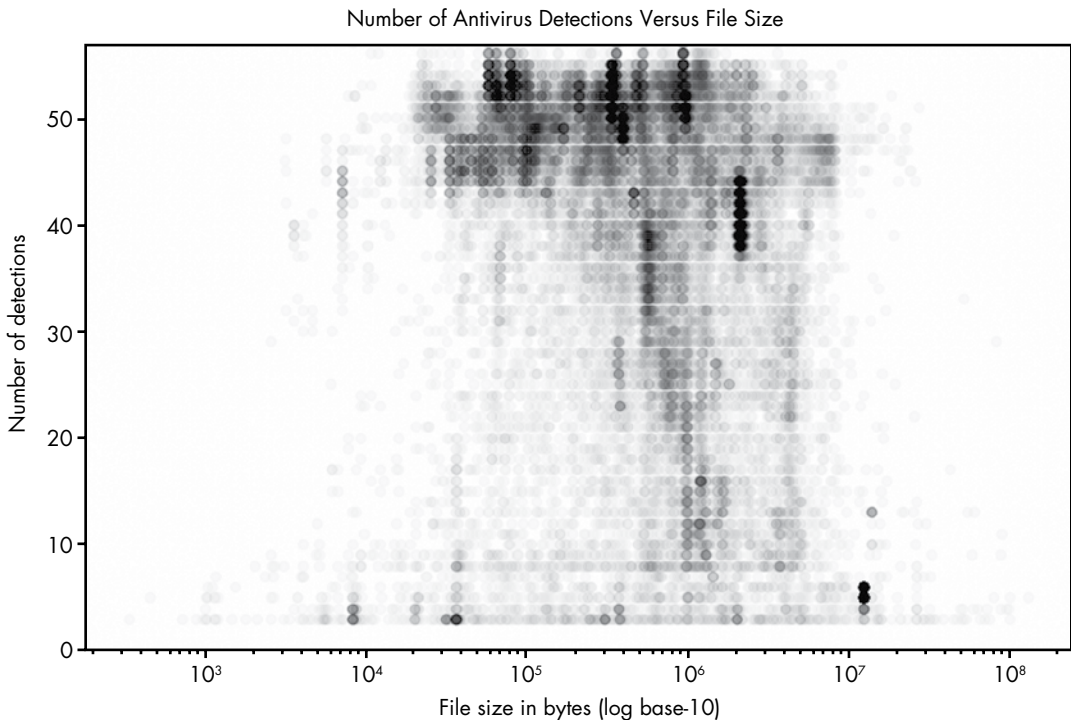


Figure 9-3: A plot of malware samples' sizes and the number of antivirus detections

Here, I plot the positives and size attributes of our malware dataset. An interesting result emerges, as foreshadowed by our discussion of pandas in the previous section. It shows that small files and very large files are rarely detected by most of the 57 antivirus engines that scanned these files. Files of middling size (around $10^{4.5}$ – 10^7) are detected by most engines, however. This may be because small files don't contain enough information to allow engines to determine they are malicious, and big files are too slow to scan, causing many antivirus systems to punt on scanning them at all.

Plotting the Relationship Between Malware Size and Vendor Detections

Let's walk through how to make the plot shown in Figure 9-3 by using the code in Listing 9-8.

```

❶ import pandas
from matplotlib import pyplot
malware = ❷ pandas.read_csv("malware_data.csv")
pyplot.plot(❸malware['size'], ❹malware['positives'],
            ❺'bo', ❻alpha=0.01)
pyplot.xscale(❼"log")
❸ pyplot.ylim([0,57])
pyplot.xlabel("File size in bytes (log base-10)")
pyplot.ylabel("Number of detections")
pyplot.title("Number of Antivirus Detections Versus File Size")
❹ pyplot.show()

```

Listing 9-8: Visualizing data using the `plot()` function

As you can see, it doesn't take much code to render this plot. Let's walk through what each line does. First, we import ❶ the necessary libraries, including pandas and the matplotlib library's pyplot module. Then we call the `read_csv` function ❷, which, as you learned earlier, loads our malware data-set into a pandas `DataFrame`.

Next we call the `plot()` function. The first argument to the function is the malware size data ❸, and the next argument is the malware positives data ❹, or the number of positive detections for each malware sample. These arguments define the data that matplotlib will plot, with the first argument representing the data to be shown on the x-axis and the second representing the data to be shown on the y-axis. The next argument, 'bo' ❺, tells matplotlib what color and shape to use to represent the data. Finally, we set `alpha`, or the transparency of the circles, to 0.1 ❻, so we can see how dense the data is within different regions of the plot, even when the circles completely overlap each other.

NOTE

The `b` in `bo` stands for blue, and the `o` stands for circle, meaning that we're telling matplotlib to plot blue circles to represent our data. Other colors you can try are green (`g`), red (`r`), cyan (`c`), magenta (`m`), yellow (`y`), black (`k`), and white (`w`). Other shapes you can try are a point (`.`), a single pixel per data point (`,`), a square (`s`), and a pentagon (`p`). For complete details, see the matplotlib documentation at <http://matplotlib.org>.

After we call the `plot()` function, we set the scale of the x-axis to be logarithmic ❼. This means that we'll be viewing the malware size data in terms of powers of 10, making it easier to see the relationships between very small and very large files.

Now that we've plotted our data, we label our axes and title our plot. The x-axis represents the size of the malware file ("File size in bytes (log base-10)"), and the y-axis represents the number of detections ("Number of detections"). Because there are 57 antivirus engines we're analyzing, we set the y-axis scale to the range 0 to 57 at ❸. Finally, we call the `show()` function ❹ to display the plot. We could replace this call with `pyplot.savefig("myplot.png")` if we wanted to save the plot as an image instead.

Now that we've gone through an initial example, let's do another.

Plotting Ransomware Detection Rates

This time, let's try reproducing Figure 9-1, the ransomware detection plot I showed at the beginning of this chapter. Listing 9-9 presents the entire code that plots our ransomware detections over time.

```
import dateutil
import pandas
from matplotlib import pyplot

malware = pandas.read_csv("malware_data.csv")
malware['fs_date'] = [dateutil.parser.parse(d) for d in malware['fs_bucket']]
ransomware = malware[malware['type'] == 'ransomware']
pyplot.plot(ransomware['fs_date'], ransomware['positives'], 'ro', alpha=0.05)
pyplot.title("Ransomware Detections Over Time")
pyplot.xlabel("Date")
pyplot.ylabel("Number of antivirus engine detections")
pyplot.show()
```

Listing 9-9: Plotting ransomware detection rates over time

Some of the code in Listing 9-9 should be familiar from what I've explained thus far, and some won't be. Let's walk through the code, line by line:

```
import dateutil
```

The helpful Python package `dateutil` enables you to easily parse dates from many different formats. We import `dateutil` because we'll be parsing dates so we can visualize them.

```
import pandas
from matplotlib import pyplot
```

We also import the `matplotlib` library's `pyplot` module as well as `pandas`.

```
malware = pandas.read_csv("malware_data.csv")
malware['fs_date'] = [dateutil.parser.parse(d) for d in malware['fs_bucket']]
ransomware = malware[malware['type'] == 'ransomware']
```

These lines read in our dataset and create a filtered dataset called `ransomware` that contains only ransomware samples, because that's the type of data we're interested in plotting here.

```
pyplot.plot(ransomware['fs_date'], ransomware['positives'], 'ro', alpha=0.05)
pyplot.title("Ransomware Detections Over Time")
pyplot.xlabel("Date")
```

```
pyplot.ylabel("Number of antivirus engine detections")
pyplot.show()
```

These five lines of code mirror the code in Listing 9-8: they plot the data, title the plot, label its x- and y-axes, and then render everything to the screen (see Figure 9-4). Again, if we wanted to save this plot to disk, we could replace the `pyplot.show()` call with `pyplot.savefig("myplot.png")`.

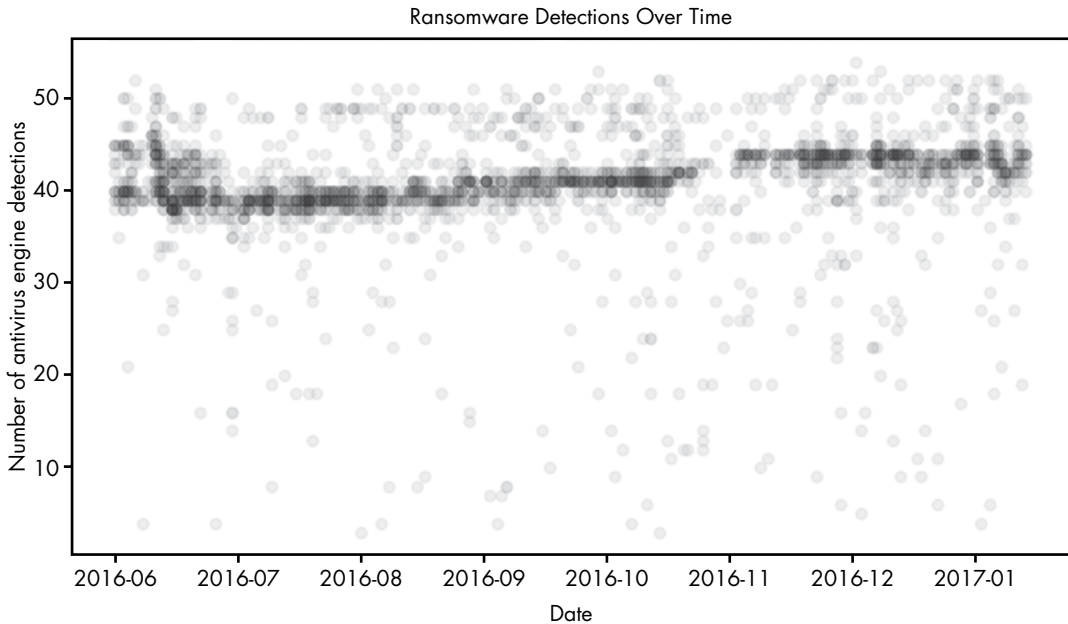


Figure 9-4: Visualization of ransomware detections over time

Let's try one more plot using the `plot()` function.

Plotting Ransomware and Worm Detection Rates

This time, instead of just plotting ransomware detections over time, let's also plot worm detections in the same graph. What becomes clear in Figure 9-5 is that the antivirus industry is better at detecting worms (an older malware trend) than ransomware (a newer malware trend).

In this plot, we see how many antivirus engines detected malware samples (y-axis) over time (x-axis). Each red dot represents a `type="ransomware"` malware sample, whereas each blue dot represents a `type="worm"` sample. We can see that on average, more engines detect worm samples than ransomware samples. However, the number of engines detecting both samples has been trending slowly up over time.

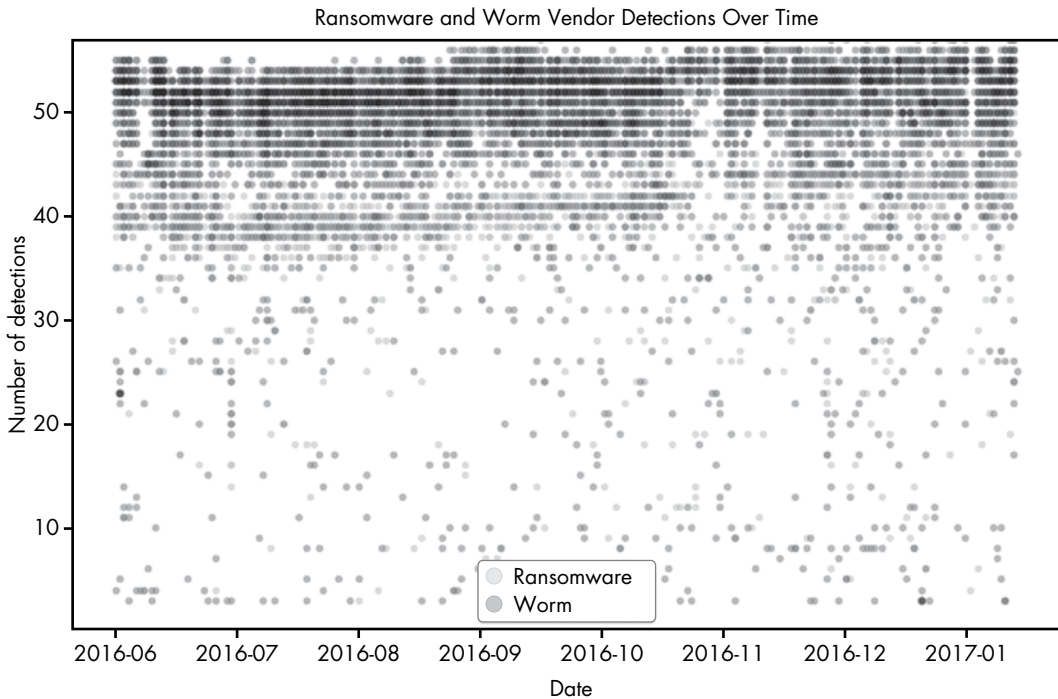


Figure 9-5: Visualization of ransomware and worm malware detections over time

Listing 9-10 shows the code for making this plot.

```
import dateutil
import pandas
from matplotlib import pyplot

malware = pandas.read_csv("malware_data.csv")
malware['fs_date'] = [dateutil.parser.parse(d) for d in malware['fs_bucket']]

ransomware = malware[malware['type'] == 'ransomware']
worms = malware[malware['type'] == 'worm']

pyplot.plot(ransomware['fs_date'], ransomware['positives'],
            'ro', label="Ransomware", markersize=3, alpha=0.05)
pyplot.plot(worms['fs_date'], worms['positives'],
            'bo', label="Worm", markersize=3, alpha=0.05)
pyplot.legend(framealpha=1, markerscale=3.0)
pyplot.xlabel("Date")
pyplot.ylabel("Number of detections")
pyplot.ylim([0, 57])
pyplot.title("Ransomware and Worm Vendor Detections Over Time")
pyplot.show()
```

Listing 9-10: Plotting ransomware and worm detection rates over time

Let's walk through the code by looking at the first part of Listing 9-10:

```
import dateutil
import pandas
from matplotlib import pyplot

malware = pandas.read_csv("malware_data.csv")
malware['fs_date'] = [dateutil.parser.parse(d) for d in malware['fs_bucket']]

ransomware = malware[malware['type'] == 'ransomware']
❶ worms = malware[malware['type'] == 'worm']
--snip--
```

The code is similar to the previous example. The difference thus far is that we create the worm filtered version of our data ❶ using the same method with which we create the ransomware filtered data. Now let's take a look at the rest of the code:

```
--snip--
❷ pyplot.plot(ransomware['fs_date'], ransomware['positives'],
              'ro', label="Ransomware", markersize=3, alpha=0.05)
❸ pyplot.plot(worms['fs_bucket'], worms['positives'],
              'bo', label="Worm", markersize=3, alpha=0.05)
❹ pyplot.legend(framealpha=1, markerscale=3.0)
pyplot.xlabel("Date")
pyplot.ylabel("Number of detections")
pyplot.ylim([0,57])
pyplot.title("Ransomware and Worm Vendor Detections Over Time")
pyplot.show()
pyplot.gcf().clf()
```

The main difference between this code and Listing 9-9 is that we call the `plot()` function twice: once for the ransomware data using the `ro` selector ❷ to create red circles, and once more on the worm data using the `bo` selector ❸ to create blue circles for the worm data. Note that if we wanted to plot a third dataset, we could do this too. Also, unlike Listing 9-9, here, at ❹, we create a legend for our figure showing that the blue marks stand for worm malware and the red marks stand for ransomware. The parameter `framealpha` determines how translucent the background of the legend is (by setting it to 1, we make it completely opaque), and the parameter `markerscale` scales the size of the markers in the legend (in this case, by a factor of three).

In this section, you've learned how to make some simple plots in `matplotlib`. However, let's be honest—they're not gorgeous. In the next section, we're going to use another plotting library that should allow us to give our plots a more professional look, and help us implement more complex visualizations quickly.

Using seaborn to Visualize Data

Now that we've discussed pandas and matplotlib, let's move on to seaborn, which is a visualization library actually built on top of matplotlib but wrapped up in a slicker container. It includes built-in themes to style our graphics as well as premade higher-level functions that save time in performing more complicated analyses. These features make it simple and easy to produce sophisticated, beautiful plots.

To explore seaborn, let's start by making a bar chart showing how many examples of each malware type we have in our dataset (see Figure 9-6).

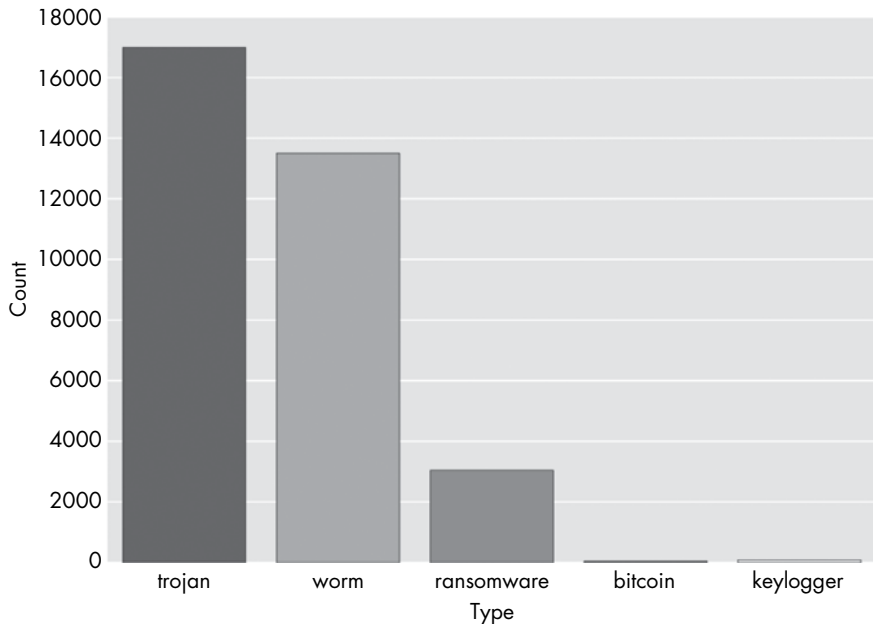


Figure 9-6: Bar chart plot of the different kinds of malware in this chapter's dataset

Listing 9-11 shows the code to make this plot.

```
import pandas
from matplotlib import pyplot
import seaborn

❶ malware = pandas.read_csv("malware_data.csv")
❷ seaborn.countplot(x='type', data=malware)
❸ pyplot.show()
```

Listing 9-11: Creating a bar chart of malware counts by type

In this code, we first read in our data via `pandas.read_csv` ❶ and then use seaborn's `countplot` function to create a barplot of the `type` column in our DataFrame ❷. Finally, we make the plot appear by calling `pyplot`'s `show()`

method at ❸. Recall that seaborn wraps matplotlib, which means we need to ask matplotlib to display our seaborn figures. Now let's move on to a more complex sample plot.

Plotting the Distribution of Antivirus Detections

The premise for the following plot is as follows: suppose we want to understand the distribution (frequency) of antivirus detections across malware samples in our dataset to understand what percentage of malware is missed by most antivirus engines, and what percentage is detected by most engines. This information gives us a view of the efficacy of the commercial antivirus industry. We can do this by plotting a bar chart (a histogram) showing, for each number of detections, the proportion of malware samples that had that number of detections, as shown in Figure 9-7.

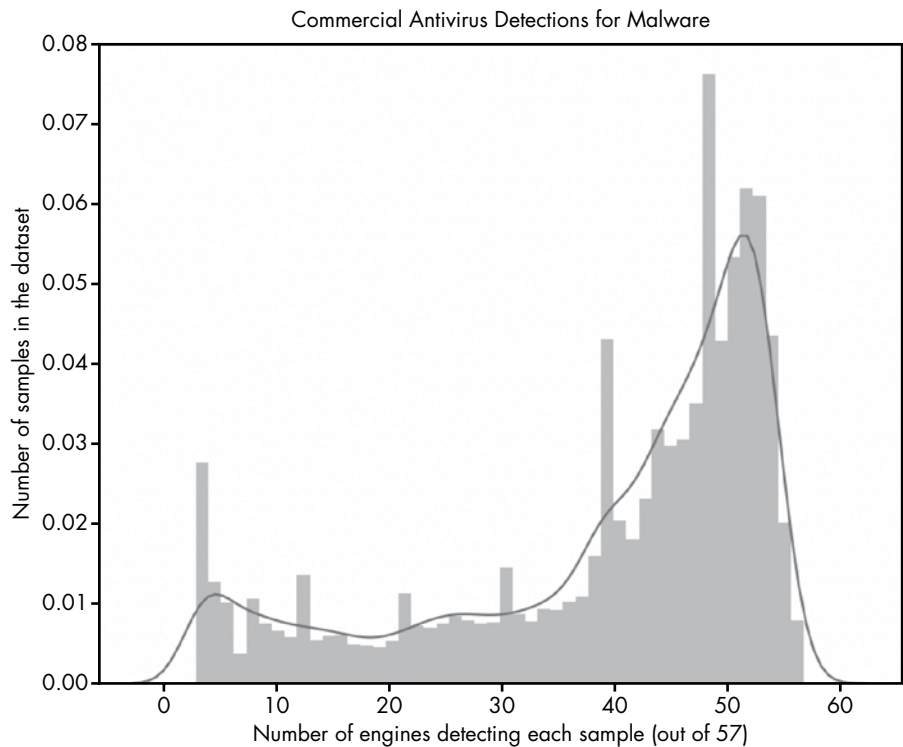


Figure 9-7: Visualization of distribution of antivirus detections (positives)

The x-axis of this figure represents categories of malware samples, sorted by how many out of 57 total antivirus engines detected them. If a sample was detected as malicious by 50 of 57 engines, it is placed at 50, if it was only detected by 10 engines out of 57, it goes in the 10 category. The height of each bar is proportional to how many total samples ended up in that category.

The plot makes it clear that many malware samples are detected by most of our 57 antivirus engines (shown by the big bump in frequencies in the upper-rightmost region of the plot) but also that a substantial minority of samples are detected by a small number of engines (shown in the leftmost region of the plot). We don't show samples that were detected by fewer than five engines because of the methodology I used to construct this dataset: I define malware as samples that five or more antivirus engines detect. This plotted result, with substantial numbers of samples receiving just 5–30 detections, indicates that there is still significant disagreement between engines in malware detection. A sample that was detected as malware by 10 out of 57 engines either indicates that 47 engines failed to detect it, or that 10 made a mistake and issued a false positive on a benign file. The latter possibility is very unlikely, because antivirus vendors' products have very low false-positive rates: it's much more likely that most engines missed these samples.

Making this plot requires just a few lines of plotting code, as shown in Listing 9-12.

```
import pandas
import seaborn
from matplotlib import pyplot
malware = pandas.read_csv("malware_data.csv")
❶ axis = seaborn.distplot(malware['positives'])
❷ axis.set(xlabel="Number of engines detecting each sample (out of 57)",
          ylabel="Amount of samples in the dataset",
          title="Commercial Antivirus Detections for Malware")
pyplot.show()
```

Listing 9-12: Plotting distribution of positives

The seaborn package has a built-in function to create distribution plots (histograms), and so all we've done is pass the `distplot` function the data we wanted to display, which is `malware['positives']` ❶. Then we use the axis object returned by seaborn to configure the plot title, x-axis label, and y-axis label to describe our plot ❷.

Now let's try a seaborn plot with two variables: the number of positive detections for malware (files with five or more detections) and their file sizes. We created this plot before with matplotlib in Figure 9-3, but we can achieve a more attractive and informative result using seaborn's `jointplot` function. The resulting plot, shown in Figure 9-8, is richly informative but takes a bit of effort to understand at first, so let's walk through it.

This plot is similar to the histogram we made in Figure 9-7, but instead of displaying the distribution of a single variable via bar heights, this plot shows the distributions of *two* variables (the size of a malware file, on the x-axis, and the number of detections, on the y-axis) via color intensity. The darker the region, the more data is in that region. For example, we can see that files most commonly have a size of about $10^{5.5}$ and a positives value of about 53. The subplots on the top and right of the main plots show a smoothed version of the frequencies of the size and detections data, which reveal the distribution of detections (as we saw in the previous plot) and file sizes.

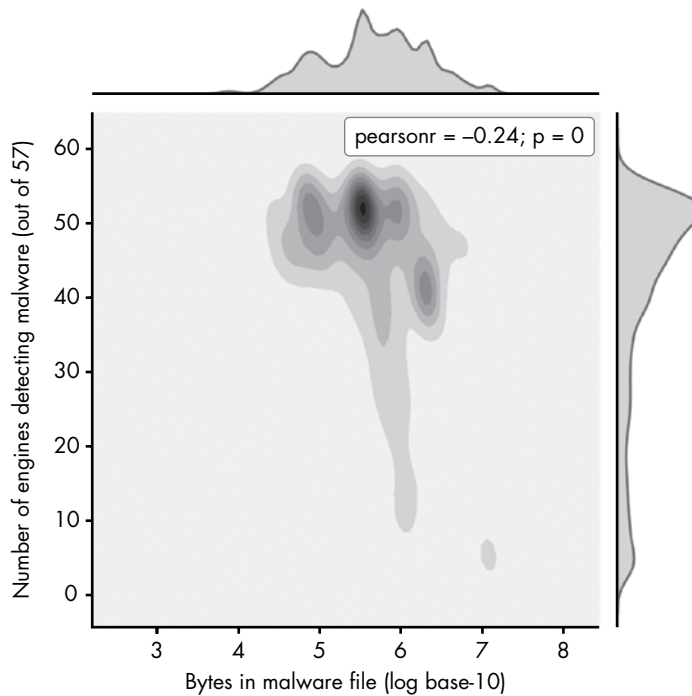


Figure 9-8: Visualization of the distribution of malware file sizes versus positive detections

The center plot is the most interesting, because it shows the relationship between size and positives. Instead of showing individual data points, like in Figure 9-3 with `matplotlib`, it shows the overall trend in a way that's much clearer. This shows that very large malware files (size 10^6 and greater) are less commonly detected by antivirus engines, which tells us we might want to custom-build a solution that specializes in detecting such malware.

Creating this plot just requires one plotting call to `seaborn`, as shown in Listing 9-13.

```
import pandas
import seaborn
import numpy
from matplotlib import pyplot

malware = pandas.read_csv("malware_data.csv")
❶ axis=seaborn.jointplot(x=numpy.log10(malware['size']),
                        y=malware['positives'],
                        kind="kde")
❷ axis.set_axis_labels("Bytes in malware file (log base-10)",
                      "Number of engines detecting malware (out of 57)")
pyplot.show()
```

Listing 9-13: Plotting the distribution of malware file sizes vs. positive detections

Here, we use seaborn's `jointplot` function to create a joint distribution plot of the positives and size columns in our DataFrame ❶. Also, somewhat confusingly, for seaborn's `jointplot` function, we have to call a different function than in Listing 9-11 to label our axes: the `set_axis_labels()` function ❷, whose first argument is the x-axis label and whose second argument is the y-axis label.

Creating a Violin Plot

The last plot type we explore in this chapter is the seaborn violin plot. This plot allows us to elegantly explore the distribution of a given variable across several malware types. For example, suppose we're interested in seeing the distribution of file sizes per malware type in our dataset. In this case, we can create a plot like Figure 9-9.

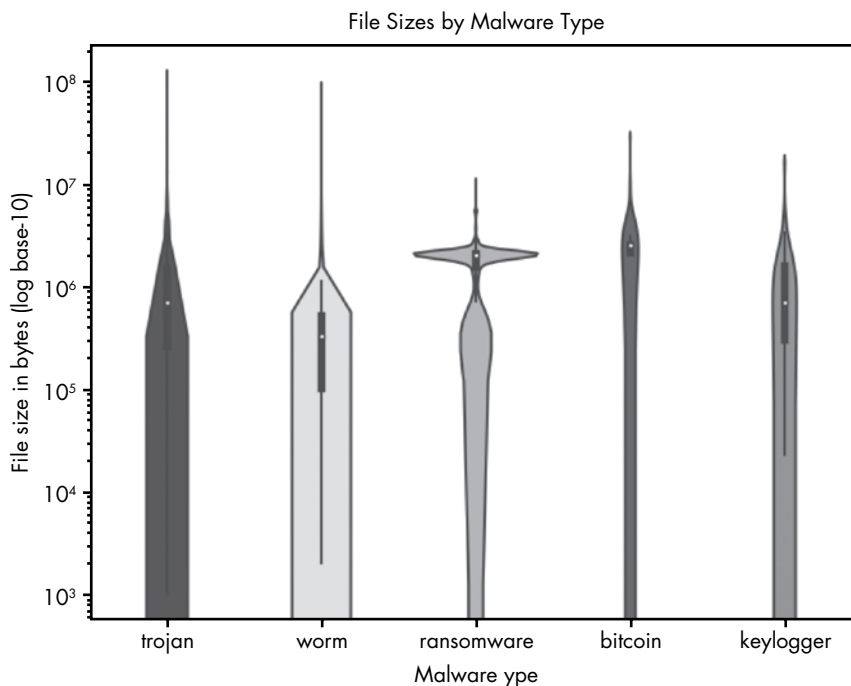


Figure 9-9: Visualization of file sizes by malware type

On the y-axis of this plot are file sizes, represented as powers of 10. On the x-axis we enumerate each malware type. As you can see, the thickness of the bars representing each file type varies at different size levels, which show how much of the data for that malware type is of that size. For example, you can see that there's a substantial number of very large ransomware files, and that worms tend to have smaller file sizes—probably because worms aim to spread rapidly across a network, and worm authors thus tend to minimize their file sizes. Knowing these patterns could potentially help us to classify unknown files better (a larger file being more likely to be ransomware and less likely to be a worm), or teach us what file sizes we should focus on in a defensive tool targeted at a specific type of malware.

Making the violin plot takes one plotting call, as shown in Listing 9-14.

```
import pandas
import seaborn
from matplotlib import pyplot

malware = pandas.read_csv("malware_data.csv")

❶ axis = seaborn.violinplot(x=malware['type'], y=malware['size'])
❷ axis.set(xlabel="Malware type", ylabel="File size in bytes (log base-10)",
           title="File Sizes by Malware Type", yscale="log")
❸ pyplot.show()
```

Listing 9-14: Creating a violin plot

In Listing 9-14, first we create the violin plot ❶. Next we tell seaborn to set the axis labels and title and to set the y-axis to log-scale ❷. Finally, we make the plot appear ❸. We can also make an analogous plot showing the number of positives for each malware type, as shown in Figure 9-10.

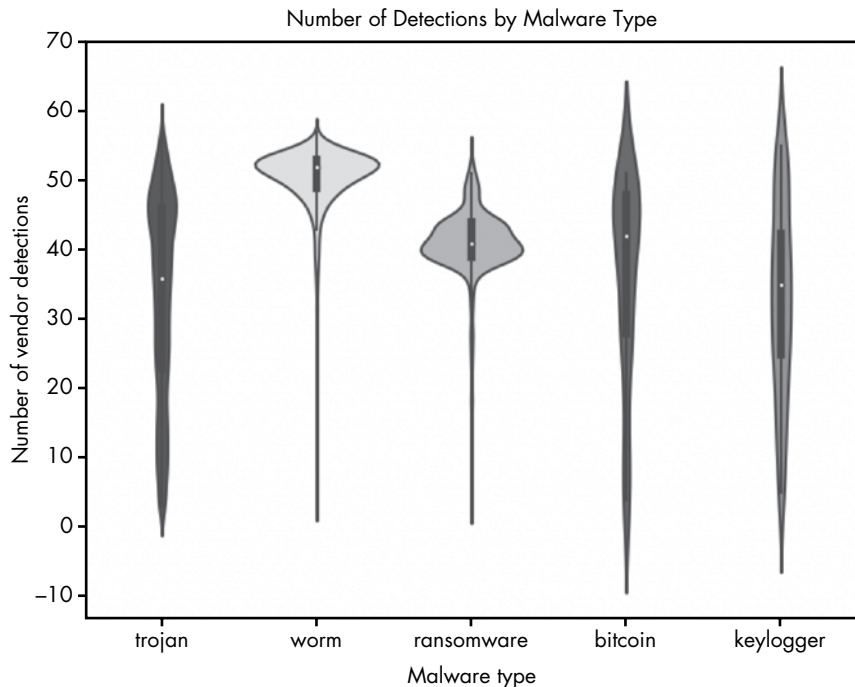


Figure 9-10: Visualization of the number of antivirus positives (detections) per malware type

The only difference between Figure 9-9 and Figure 9-10 is that instead of looking at file size on the y-axis, we're looking at the number of positives each file received. The results show some interesting trends. For example, ransomware is almost always detected by more than 30 scanners. The bitcoin, trojan, and keylogger malware types, in contrast, are detected by less than 30 scanners a substantial portion of the time, meaning more of these types

are slipping past the security industry's defenses (folks who don't have the scanners that detect these files installed are likely getting infected by these samples). Listing 9-15 shows how to create the plot shown in Figure 9-10.

```
import pandas
import seaborn
from matplotlib import pyplot

malware = pandas.read_csv("malware_data.csv")

axis = seaborn.violinplot(x=malware['type'], y=malware['positives'])
axis.set(xlabel="Malware type", ylabel="Number of vendor detections",
         title="Number of Detections by Malware Type")
pyplot.show()
```

Listing 9-15: Visualizing antivirus detections per malware type

The only differences in this code and the previous are that we pass the `violinplot` function different data (`malware['positives']` instead of `malware['size']`), we label the axes differently, we set the title differently, and we omit setting the y-axis scale to `log-10`.

Summary

In this chapter, you learned how visualization of malware data allows you to get macroscopic insights into trending threats and the efficacy of security tools. You used `pandas`, `matplotlib`, and `seaborn` to create your own visualizations and gain insight into sample datasets.

You also learned how to use methods like `describe()` in `pandas` to show useful statistics and how to extract subsets of your dataset. You then used these subsets of data to create your own visualizations to assess improvements in antivirus detections, analyze trending malware types, and answer other broader questions.

These are powerful tools that transform the security data you have into actionable intelligence that can inform the development of new tools and techniques. I hope you'll learn more about data visualizations and incorporate them into your malware and security analysis workflow.

10

DEEP LEARNING BASICS



Deep learning is a type of machine learning that has advanced rapidly in the past few years, due to improvements in processing power and deep learning techniques.

Usually, *deep learning* refers to deep, or many-layered, neural networks, which excel at performing very complex, often historically human-centric tasks, like image recognition and language translation.

For example, detecting whether a file contains an exact copy of some malicious code you've seen before is simple for a computer program and doesn't require advanced machine learning. But detecting whether a file contains malicious code that is somewhat similar to malicious code you've seen before is a far more complex task. Traditional signature-based detection schemes are rigid and perform poorly on never-before-seen or obfuscated malware, whereas deep learning models can see through superficial changes and identify core features that make a sample malicious. The same

goes for network activity, behavioral analysis, and other related fields. This ability to pick out useful characteristics within a mass of noise makes deep learning an extremely powerful tool for cybersecurity applications.

Deep learning is just a type of machine learning (we covered machine learning in general in Chapters 6 and 7). But it often leads to models that achieve better accuracy than approaches we discussed in these preceding chapters, which is why the entire field of machine learning has emphasized deep learning in the last five years or so. If you're interested in working at the cutting edge of security data science, it's essential to learn how to use deep learning. A note of caution, however: deep learning is harder to understand than the machine learning approaches we discussed early in this book, and it requires some commitment, and high-school level calculus, to fully understand. You'll find that the time you invest in understanding it will pay dividends in your security data science work in terms of your ability to build more accurate machine learning systems. So we urge you to read this chapter carefully and work at understanding it until you get it! Let's get started.

What Is Deep Learning?

Deep learning models learn to view their training data as a nested hierarchy of concepts, which allows them to represent incredibly complex patterns. In other words, these models not only take into consideration the original features you give them, but automatically combine these features to form new, optimized meta-features, which they then combine to form even more features, and so on.

“Deep” also refers to the architecture used to accomplish this, which usually consists of multiple layers of processing units, each using the previous layer's outputs as its inputs. Each of these processing units is called a *neuron*, and the model architecture as a whole is called a *neural network*, or a *deep neural network* when there are many layers.

To see how this architecture can be helpful, let's think about a program that attempts to classify images either as a bicycle or a unicycle. For a human, this is an easy task, but programming a computer to look at a grid of pixels and tell which object it represents is quite difficult. Certain pixels that indicate that a unicycle exists in one image will mean something else entirely in the next if the unicycle has moved slightly, been placed at a different angle, or has a different color.

Deep learning models get past this by breaking the problem down into more manageable pieces. For example, a deep neural network's first layer of neurons might first break down the image into parts and just identify low-level visual features, like edges and borders of shapes in the image. These created features are fed into the next layer of the network to find patterns among the features. These patterns are then fed into subsequent layers, until the network is identifying general shapes and, eventually, complete objects. In our unicycle example, the first layer might find lines, the second might see lines forming circles, and the third might identify that certain circles are actually wheels. In this way, instead of looking at a mass of pixels,

the model can see that each image has a certain number of “wheel” meta-features. It can then, for example, learn that two wheels likely indicate a bicycle, whereas one wheel means a unicycle.

In this chapter, we focus on how neural networks actually work, both mathematically and structurally. First, I use a very basic neural network as an example to explain exactly what a neuron is and how it connects to other neurons to create a neural network. Second, I describe the mathematical processes used to train these networks. Finally, I describe some popular types of neural networks, how they’re special, and what they’re good at. This will set you up nicely for Chapter 11, where you’ll actually create deep learning models in Python.

How Neural Networks Work

Machine learning models are simply big mathematical functions. For example, we take input data (such as an HTML file represented as a series of numbers), apply a machine learning function (such as a neural network), and we get an output that tells us how malicious the HTML file looks. Every machine learning model is just a function containing adjustable parameters that get optimized during the training process.

But how does a deep learning function actually work and what does it look like? Neural networks are, as the name implies, just networks of many neurons. So, before we can understand how neural networks work, we first need to know what a neuron is.

Anatomy of a Neuron

Neurons themselves are just a type of small, simple function. Figure 10-1 shows what a single neuron looks like.

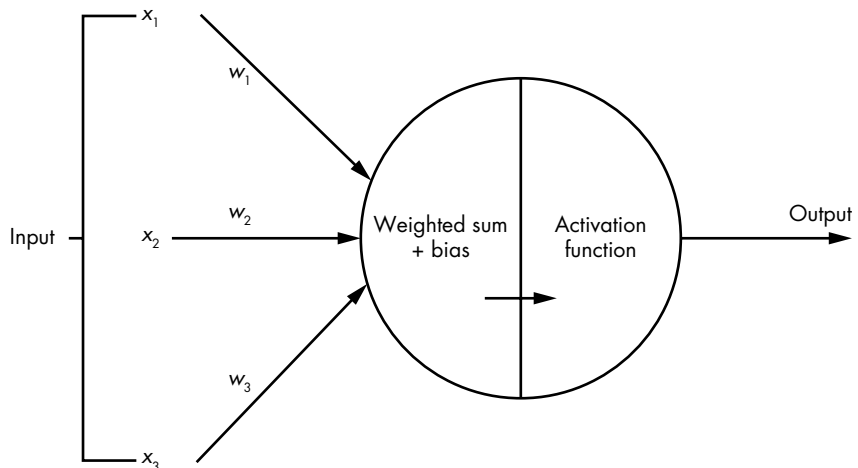


Figure 10-1: Visualization of a single neuron

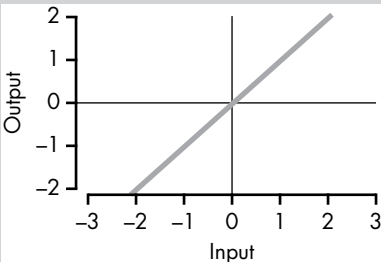
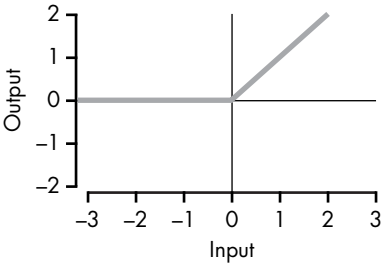
You can see that input data comes in from the left, and a single output number comes out on the right (though some types of neurons generate multiple outputs). The value of the output is a function of the neuron's input data and some parameters (which are optimized during training). Two steps occur inside every neuron to transform the input data into the output.

First, a weighted sum of the neuron's inputs is calculated. In Figure 10-1, each input number, x_i , travelling into the neuron gets multiplied by an associated *weight* value, w_i . The resulting values are added together (yielding a weighted sum) to which a *bias* term is added. The bias and weights are the parameters of the neuron that are modified during training to optimize the model.

Second, an *activation function* is applied to the weighted sum plus bias value. The purpose of an activation function is to apply a nonlinear transformation to the weighted sum, which is a *linear* transformation of the neuron's input data. There are many common types of activation functions, and they tend to be quite simple. The only requirement of an activation function is that it's differentiable, which enables us to use backpropagation to optimize parameters (we discuss this process shortly in "Training Neural Networks" on page 189).

Table 10-1 shows a variety of other common activation functions and explains which ones tend to be good for which purposes.

Table 10-1: Common Activation Functions

Name	Plot	Equation	Description
Identity		$f(x) = x$	Basically: no activation function!
ReLU		$f(x) = \begin{cases} 0 & \text{for } x < 0 \\ x & \text{for } x \geq 0 \end{cases}$	Just $\max(0, x)$. ReLUs enable fast learning and are more resilient to the vanishing gradient problem (explained later in this chapter) compared to other functions, like the sigmoid.

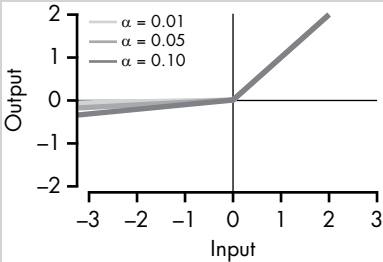
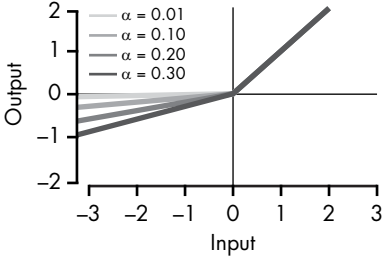
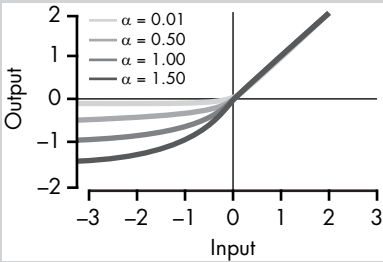
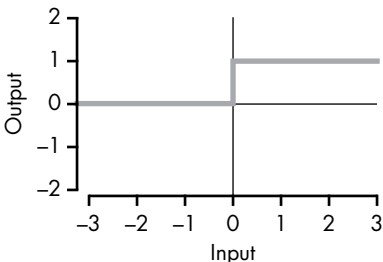
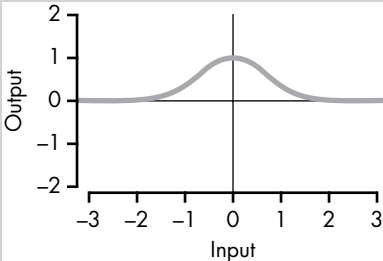
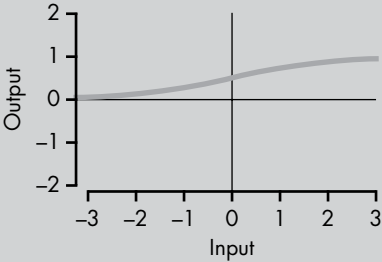
Name	Plot	Equation	Description
Leaky ReLU		$f(x) = \begin{cases} \alpha x & \text{for } x < 0 \\ x & \text{for } x \geq 0 \end{cases}$	Like normal ReLU, but instead of 0, a small constant fraction of x is returned. Generally you choose α to be very small, like 0.01. Also, α stays fixed during training.
PReLU		$f(x) = \begin{cases} \alpha x & \text{for } x < 0 \\ x & \text{for } x \geq 0 \end{cases}$	This is just like leaky ReLU, but in PReLU, α is a parameter whose value is optimized during the training process, along with the standard weight and bias parameters.
ELU		$f(x) = \begin{cases} \alpha (e^x - 1) & \text{for } x < 0 \\ x & \text{for } x \geq 0 \end{cases}$	Like PReLU in that α is a parameter, but instead of going down infinitely with a slope of α when $x < 0$, the curve is bounded by α , because e^x will always be between 0 and 1 when $x < 0$.
Step		$f(x) = \begin{cases} 0 & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}$	Just a step function: the function returns 0 unless $x \geq 0$, in which case the function returns 1.
Gaussian		$f(x) = e^{-x^2}$	A bell-shaped curve whose maximum value tops out at 1 when $x = 0$.

Table 10-1: Common Activation Functions, *continued*

Name	Plot	Equation	Description
Sigmoid		$f(x) = \frac{e^x}{e^x + 1}$	Because of the vanishing gradient problem (explained later in this chapter), sigmoid activation functions are often only used in the final layer of a neural network. Because the output is continuous and bounded between 0 and 1, sigmoid neurons are a good proxy for output probabilities.
Softmax	(multi-output)	$f(x) = \frac{e^{x_j}}{\sum_{k=1}^{K} e^{x_k}}$ for $j = 1, 2, \dots, K$	Outputs multiple values that sum to 1. Softmax activation functions are often used in the final layer of a network to represent classification probabilities, because Softmax forces all outputs from a neuron to sum to 1.

Rectified linear unit (ReLU) is by far the most common activation function used today, and it's simply $\max(0, s)$. For example, let's say your weighted sum plus bias value is called s . If s is above zero, then your neuron's output is s , and if s is equal to or below zero, then your neuron's output is 0. You can express the entire function of a ReLU neuron as simply $\max(0, \text{weighted-sum-of-inputs} + \text{bias})$, or more concretely, as the following for n inputs:

$$\max\left(0, \sum_{i=1}^n w_i * x_i + b\right)$$

Nonlinear activation functions are actually a key reason why networks of such neurons are able to approximate any continuous function, which is a big reason why they're so powerful. In the following sections, you learn how neurons are connected together to form a network, and later you'll gain an understanding of why nonlinear activation functions are so important.

A Network of Neurons

To create a neural network, you arrange neurons in a *directed graph* (a network) with a number of layers, connecting to form a much larger function. Figure 10-2 shows an example of a small neural network.

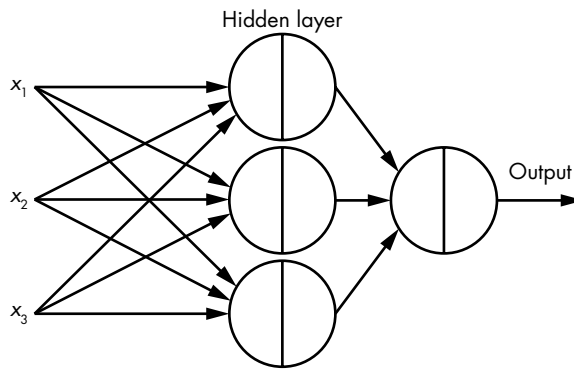


Figure 10-2: Example of a very small, four-neuron neural network, where data is passed from neuron to neuron via the connections.

In Figure 10-2, we have our original inputs: x_1 , x_2 , and x_3 on the left side. Copies of these x_i values are sent along the connections to each neuron in the *hidden layer* (a layer of neurons whose output is not the final output of the model), resulting in three output values, one from each neuron. Finally, each output of these three neurons is sent to a final neuron, which outputs the neural network's final result.

Every connection in a neural network is associated with a *weight* parameter, w , and every neuron also contains a *bias* parameter, b (added to the weighted sum), so the total number of optimizable parameters in a basic neural network is the number of edges connecting an input to a neuron, plus the number of neurons. For example, in the network shown in Figure 10-2, there are 4 total neurons, plus $9 + 3$ edges, yielding a total of 16 optimizable parameters. Because this is just an example, we're using a very small neural network—real neural networks often have thousands of neurons and millions of connections.

Universal Approximation Theorem

A striking aspect of neural networks is that they are *universal approximators*: given enough neurons, and the right weight and bias values, a neural network can emulate basically any type of behavior. The neural network shown in Figure 10-2 is *feed-forward*, which means the data is always flowing forward (from left to right in the image).

The *universal approximation theorem* describes the concept of universal-ity more formally. It states that a feed-forward network with a single hidden layer of neurons with nonlinear activation functions can approximate (with an arbitrarily small error) any continuous function on a compact subset of $\mathbf{R}^n$.¹ That's a bit of a mouthful, but it just means that with enough neurons, a neural network can *very* closely approximate any continuous, bounded function with a finite number of inputs and outputs.

1. $\mathbf{R}^n$ can be thought of as an n -dimensional Euclidian space, where all numbers are real numbers. For example, $\mathbf{R}^2$ represents all possible real-valued tuples of length 2, like $(3.5, -5)$.

In other words, the theorem states that regardless of the function we want to approximate, there's theoretically some neural network with the right parameters that can do the job. For example, if you draw a squiggly, continuous function, $f(x)$, like in Figure 10-3, there exists some neural network such that for every possible input of x , $f(x) \approx \text{network}(x)$, no matter how complicated the function $f(x)$. This is one reason neural networks can be so powerful.

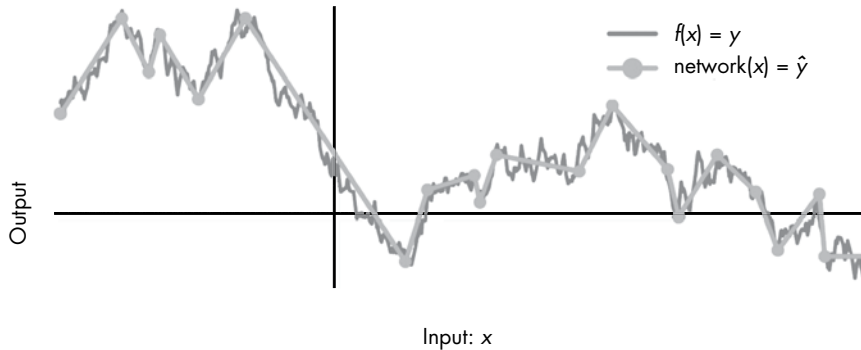


Figure 10-3: Example of how a small neural net could approximate a funky function. As the number of neurons grows, the difference between y and $\hat{y}$ will approach 0.

In the next sections, we build a simple neural network by hand to help you understand how and why we can model such different types of behavior, given the right parameters. Although we do this on a very small scale using just a single input and output, the same principle holds true when you're dealing with multiple inputs and outputs, and incredibly complex behaviors.

Building Your Own Neural Network

To see this universality in action, let's try building our own neural network. We start with two ReLU neurons, using a single input x , as shown in Figure 10-4. Then, we see how different weight and bias values (parameters) can be used to model different functions and outcomes.

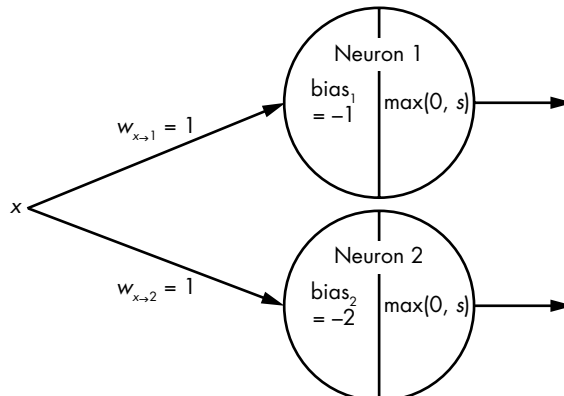


Figure 10-4: Visualization of two neurons being fed input data x

Here, both neurons have a weight of 1, and both use a ReLU activation function. The only difference between the two is that neuron₁ applies a bias value of -1, while neuron₂ applies a bias value of -2. Let's see what happens when we feed neuron₁ a few different values of x . Table 10-2 summarizes the results.

Table 10-2: Neuron₁

Input	Weighted sum	Weighted sum + bias	Output
x	$x * w_{x \rightarrow 1}$	$x * w_{x \rightarrow 1} + bias_1$	$\max(0, x * w_{x \rightarrow 1} + bias_1)$
0	$0 * 1 = 0$	$0 + -1 = -1$	$\max(0, -1) = 0$
1	$1 * 1 = 1$	$1 + -1 = 0$	$\max(0, 0) = 0$
2	$2 * 1 = 2$	$2 + -1 = 1$	$\max(0, 1) = 1$
3	$3 * 1 = 3$	$3 + -1 = 2$	$\max(0, 2) = 2$
4	$4 * 1 = 4$	$4 + -1 = 3$	$\max(0, 3) = 3$
5	$5 * 1 = 5$	$5 + -1 = 4$	$\max(0, 4) = 4$

The first column shows some sample inputs for x , and the second shows the resulting weighted sum. The third column adds the bias parameter, and the fourth column applies the ReLU activation function to yield the neuron's output for a given input of x . Figure 10-5 shows the graph of the neuron₁ function.

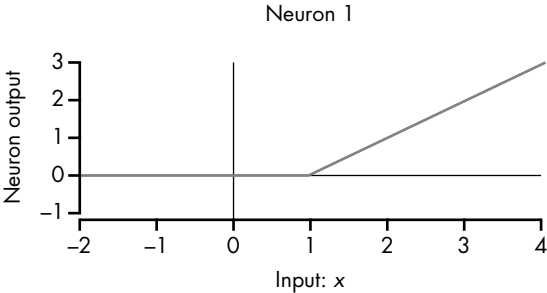


Figure 10-5: Visualization of neuron₁ as a function. The x -axis represents the neuron's single input value, and the y -axis represents the neuron's output.

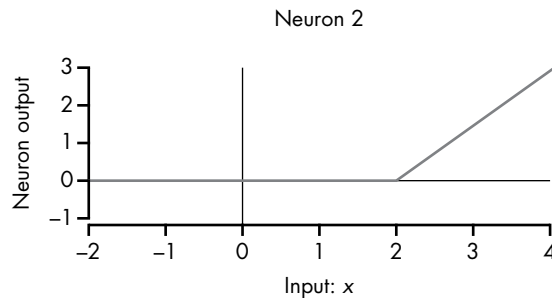
Because neuron₁ has a bias of -1, the output of neuron₁ stays at 0 until the weighted sum goes above 1, and then it goes up with a certain slope, as you can see in Figure 10-5. That slope of 1 is associated with the $w_{x \rightarrow 1}$ weight value of 1. Think about what would happen with a weight of 2: because the weighted sum value would double, the angle in Figure 10-5 would occur at $x = 0.5$ instead of $x = 1$, and the line would go up with a slope of 2 instead of 1.

Now let's look at neuron₂, which has a bias value of -2 (see Table 10-3).

Table 10-3: Neuron₂

Input	Weighted sum	Weighted sum + bias	Output
x	$x * w_{x \rightarrow 2}$	$(x * w_{x \rightarrow 2}) + \text{bias}_2$	$\max(0, (x * w_{x \rightarrow 2}) + \text{bias}_2)$
0	$0 * 1 = 0$	$0 + -2 = -2$	$\max(0, -2) = 0$
1	$1 * 1 = 1$	$1 + -2 = -1$	$\max(0, -1) = 0$
2	$2 * 1 = 2$	$2 + -2 = 0$	$\max(0, 0) = 0$
3	$3 * 1 = 3$	$3 + -2 = 1$	$\max(0, 1) = 1$
4	$4 * 1 = 4$	$4 + -2 = 2$	$\max(0, 2) = 2$
5	$5 * 1 = 5$	$5 + -2 = 3$	$\max(0, 3) = 3$

Because neuron₂'s bias is -2 , the angle in Figure 10-6 occurs at $x = 2$ instead of $x = 1$.

*Figure 10-6: Visualization of neuron₂ as a function*

So now we've built two very simple functions (neurons), both doing nothing over a set period, then going up infinitely with a slope of 1. Because we're using ReLU neurons, the slope of each neuron's function is affected by its weights, while its bias and weight terms both affect where the slope begins. When you use other activation functions, similar rules apply. By adjusting parameters, we could change the angle and slope of each neuron's function however we wanted.

In order to achieve universality, however, we need to combine neurons together, which will allow us to approximate more complex functions. Let's connect our two neurons up to a third neuron, as shown in Figure 10-7. This will create a small three-neuron network with a single hidden layer, composed of neuron₁ and neuron₂.

In Figure 10-7, input data x is sent to both neuron₁ and neuron₂. Then, neuron₁ and neuron₂'s outputs are sent as inputs to neuron₃, which yields the network's final output.

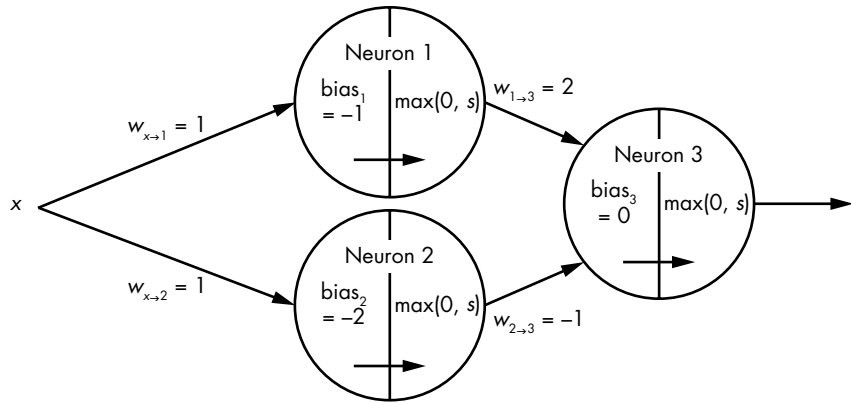


Figure 10-7: Visualization of a small three-neuron network

If you inspect the weights in Figure 10-7, you'll notice that the weight $w_{1 \rightarrow 3}$ is 2, doubling neuron₁'s contribution to neuron₃. Meanwhile, $w_{2 \rightarrow 3}$ is -1 , inverting neuron₂'s contribution. In essence, neuron₃ is simply applying its activation function to $\text{neuron}_1 * 2 - \text{neuron}_2$. Table 10-4 summarizes the inputs and corresponding outputs for the resulting network.

Table 10-4: A Three-Neuron Network

Original network input	Inputs to neuron ₃		Weighted sum	Weighted sum + bias	Final network output
x	neuron ₁	neuron ₂	$(\text{neuron}_1 * w_{1 \rightarrow 3}) + (\text{neuron}_2 * w_{2 \rightarrow 3})$	$(\text{neuron}_1 * w_{1 \rightarrow 3}) + (\text{neuron}_2 * w_{2 \rightarrow 3}) + \text{bias}_3$	$\max(0, (\text{neuron}_1 * w_{1 \rightarrow 3}) + (\text{neuron}_2 * w_{2 \rightarrow 3}) + \text{bias}_3)$
0	0	0	$(0 * 2) + (0 * -1) = 0$	$0 + 0 + 0 = 0$	$\max(0, 0) = 0$
1	0	0	$(0 * 2) + (0 * -1) = 0$	$0 + 0 + 0 = 0$	$\max(0, 0) = 0$
2	1	0	$(1 * 2) + (0 * -1) = 2$	$2 + 0 + 0 = 2$	$\max(0, 2) = 2$
3	2	1	$(2 * 2) + (1 * -1) = 3$	$4 + -1 + 0 = 3$	$\max(0, 3) = 3$
4	3	2	$(3 * 2) + (2 * -1) = 4$	$6 + -2 + 0 = 4$	$\max(0, 4) = 4$
5	4	3	$(4 * 2) + (3 * -1) = 5$	$8 + -3 + 0 = 5$	$\max(0, 5) = 5$

The first column shows original network input, x , followed by the resulting outputs of neuron₁ and neuron₂. The rest of the columns show how neuron₃ processes the outputs: the weighted sum is calculated, bias is added, and finally in the last column the ReLU activation function is applied to achieve the neuron and network outputs for each original input value for x . Figure 10-8 shows the network's function graph.

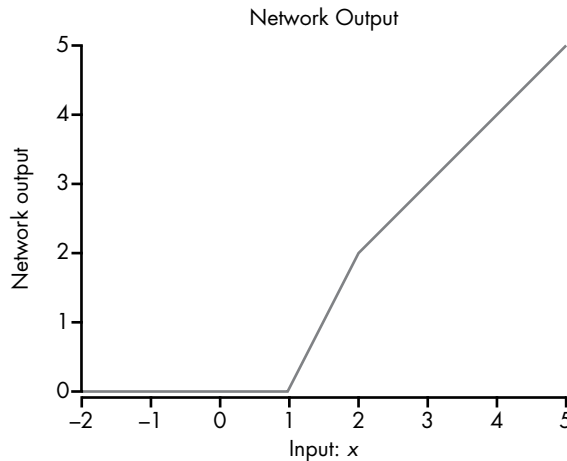


Figure 10-8: Visualization of our network's inputs and associated outputs

We can see that through the combination of these simple functions, we can create a graph that goes up for any period or slope desired over different points, as we did in Figure 10-8. In other words, we're much closer to being able to represent any finite function for our input x !

Adding Another Neuron to the Network

We've seen how to make our network's function's graph go up (with any slope) by adding neurons, but how would we make the graph go down? Let's add another neuron (neuron₄) to the mix, as shown in Figure 10-9.

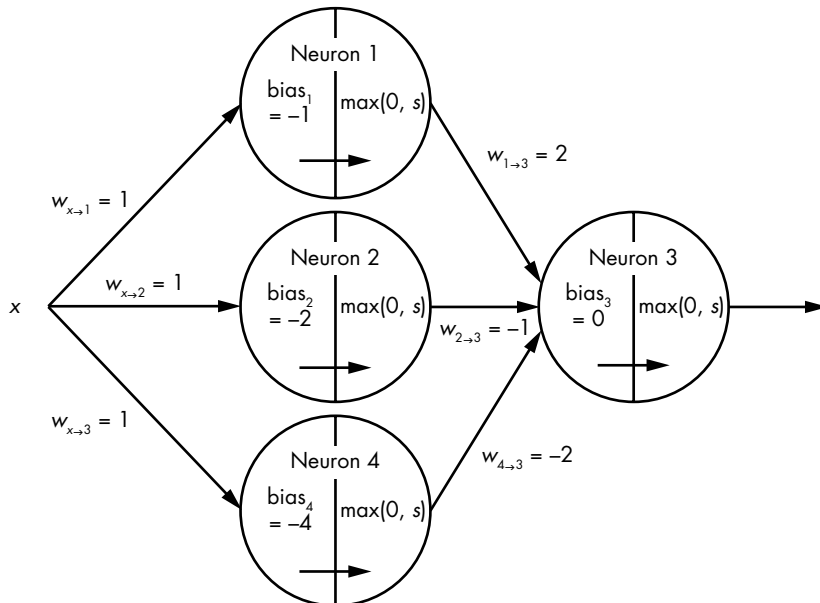


Figure 10-9: Visualization of a small four-neuron network with a single hidden layer

In Figure 10-9, input data x is sent to neuron₁, neuron₂, and neuron₄. Their outputs are then fed as inputs to neuron₃, which yields the network's final output. Neuron₄ is the same as neuron₁ and neuron₂, but with its bias set to -4 . Table 10-5 summarizes the output of neuron₄.

Table 10-5: Neuron₄

Input	Weighted sum	Weighted sum + bias	Output
x	$x * w_{x \rightarrow 4}$	$(x * w_{x \rightarrow 4}) + \text{bias}_4$	$\max(0, (x * w_{x \rightarrow 4}) + \text{bias}_4)$
0	$0 * 1 = 0$	$0 + -4 = -4$	$\max(0, -4) = 0$
1	$1 * 1 = 1$	$1 + -4 = -3$	$\max(0, -3) = 0$
2	$2 * 1 = 2$	$2 + -4 = -2$	$\max(0, -2) = 0$
3	$3 * 1 = 3$	$3 + -4 = -1$	$\max(0, -1) = 0$
4	$4 * 1 = 4$	$4 + -4 = 0$	$\max(0, 0) = 0$
5	$5 * 1 = 5$	$5 + -4 = 1$	$\max(0, 1) = 1$

To make our network graph descend, we subtract neuron₄'s function from that of neuron₁ and neuron₂ in neuron₃'s weighted sum by setting the weight connecting neuron₄ to neuron₃ to -2 . Table 10-6 shows the new output of the entire network.

Table 10-6: A Four-Neuron Network

Original network input	Inputs to neuron ₃			Weighted sum	Weighted sum + bias	Final network output
x	neuron ₁	neuron ₂	neuron ₄	$(\text{neuron}_1 * w_{1 \rightarrow 3}) + (\text{neuron}_2 * w_{2 \rightarrow 3}) + (\text{neuron}_4 * w_{4 \rightarrow 3})$	$(\text{neuron}_1 * w_{1 \rightarrow 3}) + (\text{neuron}_2 * w_{2 \rightarrow 3}) + (\text{neuron}_4 * w_{4 \rightarrow 3}) + \text{bias}_3$	$\max(0, (\text{neuron}_1 * w_{1 \rightarrow 3}) + (\text{neuron}_2 * w_{2 \rightarrow 3}) + (\text{neuron}_4 * w_{4 \rightarrow 3}) + \text{bias}_3)$
0	0	0	0	$(0 * 2) + (0 * -1) + (0 * -2) = 0$	$0 + 0 + 0 + 0 = 0$	$\max(0, 0) = 0$
1	0	0	0	$(0 * 2) + (0 * -1) + (0 * -2) = 0$	$0 + 0 + 0 + 0 = 0$	$\max(0, 0) = 1$
2	1	0	0	$(1 * 2) + (0 * -1) + (0 * -2) = 2$	$2 + 0 + 0 + 0 = 2$	$\max(0, 2) = 2$
3	2	1	0	$(2 * 2) + (1 * -1) + (0 * -2) = 3$	$4 + -1 + 0 + 0 = 3$	$\max(0, 3) = 3$
4	3	2	0	$(3 * 2) + (2 * -1) + (0 * -2) = 4$	$6 + -2 + 0 + 0 = 4$	$\max(0, 4) = 4$
5	4	3	1	$(4 * 2) + (3 * -1) + (1 * -2) = 5$	$8 + -3 + -2 + 0 = 3$	$\max(0, 3) = 3$

Figure 10-10 shows what this looks like.

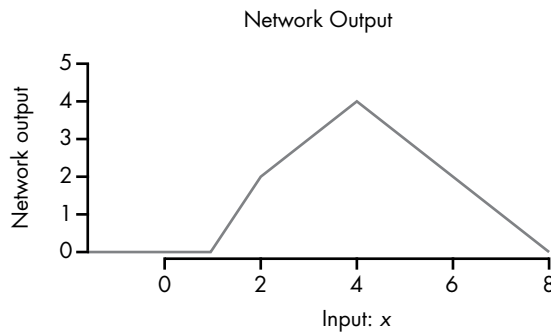


Figure 10-10: Visualization of our four-neuron network

Hopefully, now you can see how the neural network architecture allows us to move up and down at any rate over any points on the graph, just by combining a number of simple neurons (universality!). We could continue adding more neurons to create far more sophisticated functions.

Automatic Feature Generation

You’ve learned that a neural network with a single hidden layer can approximate any finite function with enough neurons. That’s a pretty powerful idea. But what happens when we have multiple hidden layers of neurons? In short, automatic feature generation happens, which is perhaps an even more powerful aspect of neural networks.

Historically, a big part of the process of building machine learning models was feature extraction. For an HTML file, a lot of time would be spent deciding what numeric aspects of an HTML file (number of section headers, number of unique words, and so on) might aid the model.

Neural networks with multiple layers and automatic feature generation allow us to offload a lot of that work. In general, if you give fairly raw features (such as characters or words in an HTML file) to a neural network, each layer of neurons can learn to represent those raw features in ways that work well as inputs to later layers. In other words, a neural network will learn to count the number of times the letter *a* shows up in an HTML document, if that’s particularly relevant to detecting malware, with no real input from a human saying that it is or isn’t.

In our image-processing bicycle example, nobody specifically told the network that edges or wheel meta-features were useful. The model learned that those features were useful as inputs to the next neuron layer during the training process. What’s especially useful is that these lower-level learned features can be used in different ways by later layers, which means that deep neural networks can estimate many incredibly complex patterns using far fewer neurons and parameters than a single-layered network could.

Not only do neural networks perform a lot of the feature extraction work that previously took a lot of time and effort, they do it in an optimized and space-efficient way, guided by the training process.

Training Neural Networks

So far, we've explored how, given a large number of neurons and the right weights and bias terms, a neural network can approximate complex functions. In all our examples so far, we set those weight and bias parameters manually. However, because real neural networks normally contain thousands of neurons and millions of parameters, we need an efficient way to optimize these values.

Normally, when training a model, we start with a training dataset and a network with a bunch of non-optimized (randomly initialized) parameters. Training requires optimizing parameters to minimize an objective function. In supervised learning, where we're trying to train our model to be able to predict a label, like 0 for "benign" and 1 for "malware," that *objective function* is going to be related to the network's prediction error during training. For some given input x (for example, a specific HTML file), this is the difference between the label y we know is correct (for example, 1.0 for "is malware") and the output $\hat{y}$ we get from the current network (for example, 0.7). You can think of the error as the difference between the predicted label $\hat{y}$ and the known, true label y , where $\text{network}(x) = \hat{y}$, and the network is trying to approximate some unknown function f , such that $f(x) = y$. In other words, $\text{network} = \hat{f}$.

The basic idea behind training networks is to feed a network an observation, x , from your training dataset, receive some output, $\hat{y}$, and then figure out how changing your parameters will shift $\hat{y}$ closer to your goal, y . Imagine you're in a spaceship with various knobs. You don't know what each knob does, but you know the direction you want to go in (y). To solve the problem, you step on the gas and note the direction you went ($\hat{y}$). Then, you turn a knob just a *tiny* bit and step on the gas again. The difference between your first and second directions tells you how much that knob affects your direction. In this way, you can eventually figure out how to fly the spaceship quite well.

Training a neural network is similar. First, you feed a network an observation, x , from your training dataset, and you receive some output, $\hat{y}$. This step is called *forward propagation* because you feed your input x forward through the network to get your final output $\hat{y}$. Next, you determine how each parameter affects your output $\hat{y}$. For example, if your network's output is 0.7, but you know the correct output should be closer to 1, you can try increasing a parameter, w , just a little bit, seeing whether $\hat{y}$ gets closer to or further away from y , and by how much.² This is called the partial derivative of $\hat{y}$ with respect to w , or $\partial\hat{y}/\partial w$.

Parameters all throughout the network are then nudged just a *tiny* bit in a direction that causes $\hat{y}$ to shift a little closer to y (and therefore network closer to f). If $\partial\hat{y}/\partial w$ is positive, then you know you should increase w by a

2. In practice, increasing the parameter slightly and then reevaluating the network's resulting output isn't necessary. This is because the entire network is a differentiable function, which means that we can just calculate $(\partial\hat{y}/\partial w)$ precisely and more rapidly using calculus. However, I find that thinking in terms of nudging and reevaluating tends to be more intuitive than using derivatives calculus.

small amount (specifically, proportional to $\partial(y - \hat{y})/\partial w$), so that your new $\hat{y}$ will move slightly away from 0.7 and toward 1 (y). In other words, you teach your network to approximate the *unknown* function f by correcting its mistakes on training data with *known* labels.

The process of iteratively calculating these partial derivatives, updating parameters, and then repeating is called *gradient descent*. However, with a network of thousands of neurons, millions of parameters, and often millions of training observations, all of that calculus requires a lot of computation. To get around this, we use a neat algorithm called *backpropagation* that makes these calculations computationally feasible. At its core, backpropagation allows us to efficiently calculate partial derivatives along computational graphs like a neural network!

Using Backpropagation to Optimize a Neural Network

In this section, we construct a simple neural network to showcase how backpropagation works. Let's assume that we have a training example whose value is $x = 2$ and an associated true label of $y = 10$. Usually, x would be an array of many values, but let's stick to a single value to keep things simple. Plugging in these values, we can see in Figure 10-11 that our network outputs a $\hat{y}$ value of 5 with an input x value of 2.

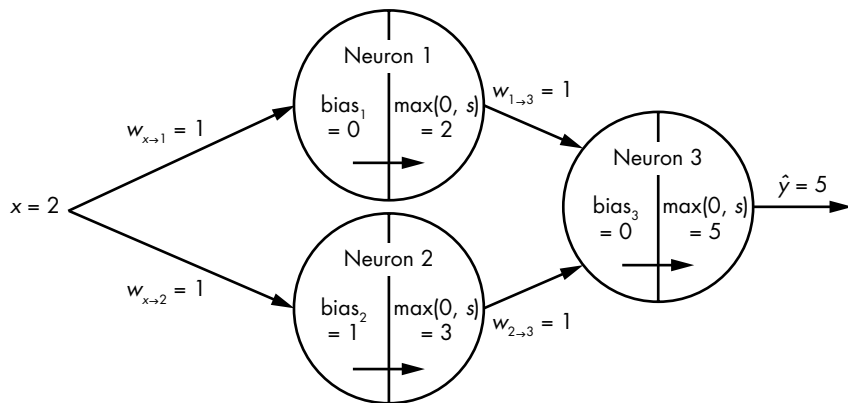


Figure 10-11: Visualization of our three-neuron network, with an input of $x = 2$

To nudge our parameters so that our network's output $\hat{y}$, given $x = 2$, moves closer to our known y value of 10, we need to calculate how $w_{1 \rightarrow 3}$ affects our final output $\hat{y}$. Let's see what happens when we increase $w_{1 \rightarrow 3}$ by just a bit (say, 0.01). The weighted sum in neuron₃ becomes $1.01 * 2 + (1 * 3)$, making the final output $\hat{y}$ change from 5 to 5.02, resulting in an increase of 0.02. In other words, the partial derivative of $\hat{y}$ with respect to $w_{1 \rightarrow 3}$ is 2, because changing $w_{1 \rightarrow 3}$ yields twice that change in $\hat{y}$.

Because y is 10 and our current output $\hat{y}$ (given our current parameter values and $x = 2$) is 5, we now know that we should increase $w_{1 \rightarrow 3}$ by a small amount to move y closer to 10.

That's fairly simple. But we need to be able to know which direction to push *all* parameters in our network, not just ones in a neuron in the final layer. For example, what about $w_{x \rightarrow 1}$? Calculating $\partial \hat{y} / \partial w_{x \rightarrow 1}$ is more complicated because it only *indirectly* affects $\hat{y}$. First, we ask neuron₃'s function how $\hat{y}$ is affected by neuron₁'s output. If we change the output of neuron₁ from 2 to 2.01, the final output of the neuron₃ changes from 5 to 5.01, so $\partial \hat{y} / \partial \text{neuron}_1 = 1$. To know how much $w_{x \rightarrow 1}$ affects $\hat{y}$, we just have to multiply $\partial \hat{y} / \partial \text{neuron}_1$ by how much $w_{x \rightarrow 1}$ affects the output of neuron₁. If we change $w_{x \rightarrow 1}$ from 1 to 1.01, the output of neuron₁ changes from 2 to 2.02, so $\partial \text{neuron}_1 / \partial w_{x \rightarrow 1}$ is 2. Therefore:

$$\frac{\partial \hat{y}}{\partial w_{x \rightarrow 1}} = \frac{\partial \hat{y}}{\partial \text{neuron}_1} * \frac{\partial \text{neuron}_1}{\partial w_{x \rightarrow 1}}$$

Or:

$$\frac{\partial \hat{y}}{\partial w_{x \rightarrow 1}} = 1 * 2 = 2$$

You may have noticed that we just used the chain rule.³

In other words, to figure out how a parameter like $w_{x \rightarrow 1}$ deep inside a network affects our final output $\hat{y}$, we multiply the partial derivatives at each point along the path between our parameter $w_{x \rightarrow 1}$ and $\hat{y}$. This means that if $w_{x \rightarrow 1}$ is fed into a neuron whose outputs are fed into ten other neurons, calculating $w_{x \rightarrow 1}$'s effect on $\hat{y}$ would involve summing over all the paths that led from $w_{x \rightarrow 1}$ to $\hat{y}$, instead of just one. Figure 10-12 visualizes the paths affected by the sample weight parameter $w_{x \rightarrow 2}$.

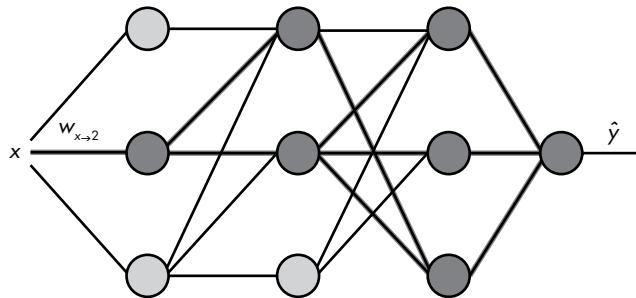


Figure 10-12: Visualization of the paths affected by $w_{x \rightarrow 2}$ (shown in dark gray): the weight associated with the connection between input data x and the middle neuron in the first (leftmost) layer

3. The chain rule is a formula for calculating the derivative of composite functions. For example, if f and g are both functions, and h is the composite function $h(x) = f(g(x))$, then the chain rule states that $h'(x) = f'(g(x)) * g'(x)$, where $f'(x)$ indicates the partial derivative of a function, f , with respect to x .

Note that the hidden layers in this network are not fully connected layers, which helps explain why the second hidden layer's bottom neuron isn't highlighted.

Path Explosion

But what happens when our network gets even larger? The number of paths we need to add to calculate the partial derivative of a low-level parameter increases exponentially. Consider a neuron whose output is fed into a layer of 1,000 neurons, whose outputs are fed into 1,000 more neurons, whose outputs are then fed into a final output neuron.

That results in one million paths! Luckily, going over every single path and then summing them to get the $\partial\hat{y}/(\partial\text{parameter})$ is not necessary. This is where backpropagation comes in handy. Instead of walking along every single path that leads to our final output(s), $\hat{y}$, partial derivatives are calculated layer by layer, starting from the top down, or backward.

Using the chain rule logic from the last section, we can calculate any partial derivative $\partial\hat{y}/\partial w$, where w is a parameter connecting an output from layer $_{i-1}$ to a neuron $_i$ in layer $_i$, by summing over the following for all neuron $_{i+1}$, where each neuron $_{i+1}$ is a neuron in layer $_{i+1}$ to which neuron $_i$ (w 's neuron) is connected:

$$\frac{\partial\hat{y}}{\partial\text{neuron}_{i+1}} * \frac{\partial\text{neuron}_{i+1}}{\partial\text{neuron}_i} * \frac{\partial\text{neuron}_i}{\partial w}$$

By doing this layer by layer from the top down, we limit path explosion by consolidating derivatives at each layer. In other words, derivatives calculated in a top-level layer $_{i+1}$ (like $\partial\hat{y}/\partial\text{neuron}_{i+1}$) are recorded to help calculate derivatives in layer $_i$. Then to calculate derivatives in layer $_{i-1}$, we use the saved derivatives from layer $_i$ (like $\partial\hat{y}/\partial\text{neuron}_i$). Then, layer $_{i-2}$ uses derivatives from layer $_{i-1}$, and so on and so forth. This trick greatly reduces the amount of calculations we have to repeat and helps us to train neural networks quickly.

Vanishing Gradient

One issue that very deep neural networks face is the *vanishing gradient* problem. Consider a weight parameter in the first layer of a neural network that has ten layers. The signal it gets from backpropagation is the summation of all paths' signals from this weight's neuron to the final output.

The problem is that each path's signal is likely to be incredibly tiny, because we calculate that signal by multiplying partial derivatives at each point along the ten-neuron-deep path, all of which tend to be numbers smaller than 1. This means that a low-level neuron's parameters are updated based on the summation of a massive number of very tiny numbers, many of which end up canceling one another out. As a result, it can be difficult for a network to coordinate sending a strong signal down

to parameters in lower layers. This problem gets exponentially worse as you add more layers. As you learn in the following section, certain network designs try to get around this pervasive problem.

Types of Neural Networks

For simplicity's sake, every example I've shown you so far uses a type of network called a feed-forward neural network. In reality, there are many other useful network structures you can use for different classes of problems. Let's discuss some of the most common classes of neural networks and how they could be applied in a cybersecurity context.

Feed-Forward Neural Network

The simplest (and first) kind of neural network, a feed-forward neural network, is kind of like a Barbie doll with no accessories: other types of neural networks are usually just variations on this "default" structure. The feed-forward architecture should sound familiar: it consists of stacks of layers of neurons. Each layer of neurons is connected to some or all neurons in the next layer, but connections never go backward or form cycles, hence the name "feed forward."

In feed-forward neural networks, every connection that exists is connecting a neuron (or original input) in layer i to a neuron in layer $j > i$. Each neuron in layer i doesn't necessarily have to connect to every neuron in layer $i + 1$, but all connections must be feeding forward, connecting previous layers to later layers.

Feed-forward networks are generally the kind of network you throw at a problem first, unless you already know of another architecture that works particularly well on the problem at hand (such as convolutional neural networks for image recognition).

Convolutional Neural Network

A *convolutional neural network (CNN)* contains convolutional layers, where the input that feeds into each neuron is defined by a window that slides over the input space. Imagine a small square window sliding over a larger picture where only the pixels visible through the window will be connected to a specific neuron in the next layer. Then, the window slides, and the new set of pixels are connected to a new neuron. Figure 10-13 illustrates this.

The structure of these networks encourages localized feature learning. For example, it's more useful for a network's lower layers to focus on the relationship between nearby pixels in an image (which form edges, shapes, and so on) than to focus on the relationship between pixels randomly scattered across an image (which are unlikely to mean much). The sliding windows explicitly force this focus, which improves and speeds up learning in areas where local feature extraction is especially important.

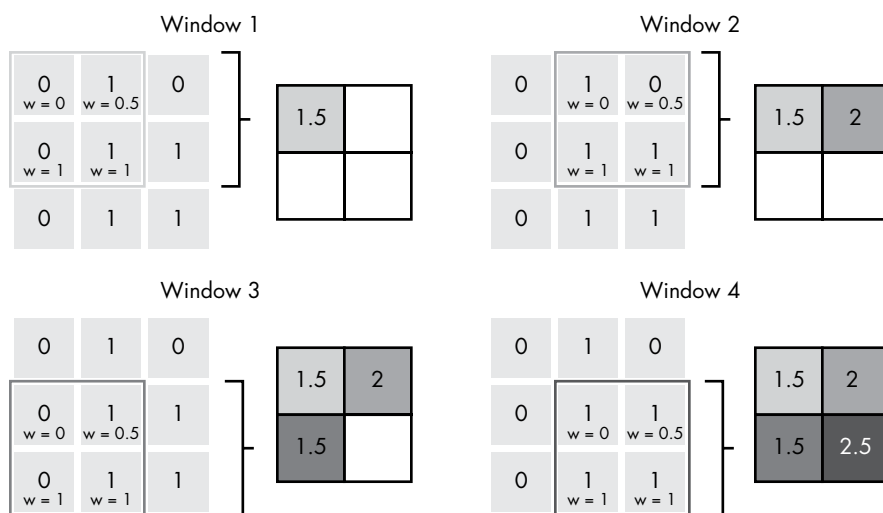


Figure 10-13: Visualization of a 2×2 convolutional window sliding over a 3×3 input space with a stride (step size) of 1, to yield a 2×2 output

Because of their ability to focus on localized sections of the input data, convolutional neural networks are extremely effective at image recognition and classification. They've also been shown to be effective for certain types of natural language processing, which has implications for cybersecurity.

After each convolutional window's values are fed to specific neurons in a convolutional layer, a sliding window is again slid over *these* neurons' outputs, but instead of them being fed to standard neurons (for example, ReLUs) with weights associated with each input, they're fed to neurons that have no weights (that is, fixed at 1) and a max (or similar) activation function. In other words, a small window is slid over the convolutional layer's outputs, and the maximum value of each window is taken and passed to the next layer. This is called a *pooling layer*. The purpose of pooling layers is to "zoom out" on the data (usually, an image), thereby reducing the size of the features for faster computation, while retaining the most important information.

Convolutional neural networks can have one or multiple sets of convolutional and pooling layers. A standard architecture might include a convolutional layer, a pooling layer, followed by another set of convolutional and pooling layers, and finally a few fully connected layers, like in feed-forward networks. The goal of this architecture is that these final fully connected layers receive fairly high-level features as inputs (think wheels on a unicycle), and as a result are able to accurately classify complex data (such as images).

Autoencoder Neural Network

An *autoencoder* is a type of neural network that tries to compress and then decompress an input with minimal difference between the original training input and the decompressed output. The goal of an autoencoder is to learn

an efficient representation for a set of data. In other words, autoencoders act like optimized lossy compression programs, where they compress input data into a smaller representation, then decompress it back to its original input size.

Instead of the neural network optimizing parameters by minimizing the difference between known labels (y) and predicted labels ($\hat{y}$) for a given input x , the network tries to minimize the difference between the original input x and the reconstructed output $\hat{x}$.

Structurally, autoencoders are usually very similar to standard feed-forward neural networks, except that middle layers contain fewer neurons than early and later stage layers, as shown in Figure 10-14.

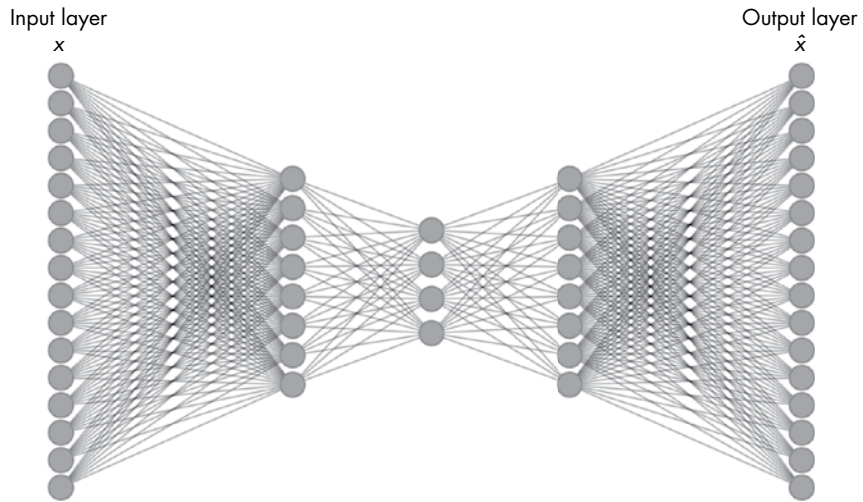


Figure 10-14: Visualization of an autoencoder network

As you can see, the middle layer is much smaller than the leftmost (input) and rightmost (output) layers, which each have the same size. The last layer should always contain the same number of outputs as the original inputs, so each training input x_i can be compared to its compressed and reconstructed cousin $\hat{x}_i$.

After an autoencoder network has been trained, it can be used for different purposes. Autoencoder networks can simply be used as efficient compress/decompress programs. For example, autoencoders trained to compress image files can create images that look far clearer than the same image compressed via JPEG to the same size.

Generative Adversarial Network

A *generative adversarial network* (GAN) is a system of *two* neural networks competing with each other to improve themselves at their respective tasks. Typically, the *generative* network tries to create fake samples (for example, some sort of image) from random noise. Then a second *discriminator* network

attempts to tell the difference between real samples and the fake, generated samples (for example, distinguishing between real images of a bedroom and generated images).

Both neural networks in a GAN are optimized with backpropagation. The generator network optimizes its parameters based on how well it fooled the discriminator network in a given round, while the discriminator network optimizes its parameters based on how accurately it could discriminate between generated and real samples. In other words, their loss functions are direct opposites of one another.

GANs can be used to generate real-looking data or enhance low-quality or corrupted data.

Recurrent Neural Network

Recurrent networks (RNNs) are a relatively broad class of neural networks in which connections between neurons form directed cycles whose activation functions are dependent on time-steps. This allows the network to develop a memory, which helps it learn patterns in sequences of data. In RNNs, the inputs, the outputs, or both the inputs and outputs are some sort of time series.

RNNs are great for tasks where data order matters, like connected handwriting recognition, speech recognition, language translation, and time series analysis. In the context of cybersecurity, they're relevant to problems like network traffic analysis, behavioral detection, and static file analysis. Because program code is similar to natural language in that order matters, it can be treated as a time series.

One issue with RNNs is that due to the vanishing gradient problem, each time-step introduced in an RNN is similar to an entire extra layer in a feed-forward neural network. During backpropagation, the vanishing gradient problem causes signals in lower-level layers (or in this case, earlier time-steps) to become incredibly faint.

A *long short-term memory (LSTM) network* is a special type of RNN designed to address this problem. LSTMs contain *memory cells* and special neurons that try to decide what information to remember and what information to forget. Tossing out most information greatly limits the vanishing gradient problem because it reduces path explosion.

ResNet

A *ResNet* (short for *residual network*) is a type of neural network that creates *skip connections* between neurons in early/shallow layers of the network to deeper layers by skipping one or more intermediate layers. Here the word *residual* refers to the fact that these networks learn to pass numerical information directly between layers, without that numerical information having to pass through the kinds of activation functions we illustrated in Table 10-1.

This structure helps greatly reduce the vanishing gradient problem, which enables ResNets to be incredibly deep—sometimes more than 100 layers.

Very deep neural networks excel at modeling extremely complex, odd relationships in input data. Because ResNets are able to have so many layers, they are especially suited to complex problems. Like feed-forward neural networks, ResNets are important more because of their general effectiveness at solving complex problems rather than their expertise in very specific problem areas.

Summary

In this chapter, you learned about the structure of neurons and how they are connected together to form neural networks. You also explored how these networks are trained via backpropagation, and you discovered some benefits and issues that neural networks have, such as universality, automatic feature generation, and the vanishing gradient problem. Finally, you learned the structures and benefits of a few common types of neural networks.

In the next chapter, you'll actually build neural networks to detect malware, using Python's Keras package.

11

BUILDING A NEURAL NETWORK MALWARE DETECTOR WITH KERAS



A decade ago, building a functioning, scalable, and fast neural network was time consuming and required quite a lot of code. In the past few years, however, this process has become far less painful, as more and more high-level interfaces to neural network design have been developed. The Python package `Keras` is one of these interfaces.

In this chapter, I walk you through how to build a sample neural network using the `Keras` package. First, I explain how to define a model's architecture in `Keras`. Second, we train this model to differentiate between benign and malicious HTML files, and you learn how to save and load such models. Third, using the Python package `sklearn`, you learn how to evaluate the model's accuracy on validation data. Finally, we use what you've learned to integrate validation accuracy reporting into the model training process.

I encourage you to read this chapter while reading and editing the associated code in the data accompanying this book. You can find all the code

discussed in this chapter there (organized into parameterized functions to make things easier to run and adjust), as well as a few extra examples. By the end of this chapter, you'll feel ready to start building some networks of your own!

To run code listings in this chapter, you not only need to install the packages listed in this chapter's *ch11/requirements.txt* file (`pip install -r requirements.txt`), but also follow the directions to install one of Keras's backend engines on your system (TensorFlow, Theano, or CNTK). Install TensorFlow by following the directions here: <https://www.tensorflow.org/install/>.

Defining a Model's Architecture

To build a neural network, you need to define its architecture: which neurons go where, how they connect to subsequent neurons, and how data flows through the whole thing. Luckily, Keras provides a simple, flexible interface to define all this. Keras actually supports two similar syntaxes for model definition, but we're going to use the Functional API syntax, as it's more flexible and powerful than the other ("sequential") syntax.

When designing a model, you need three things: input, stuff in the middle that processes the input, and output. Sometimes your models will have multiple inputs, multiple outputs, and very complex stuff in the middle, but the basic idea is that when defining a model's architecture, you're just defining how the input—your data, such as features relating to an HTML file—flows through various neurons (stuff in the middle), until finally the last neurons end up yielding some output.

To define this architecture, Keras uses layers. A *layer* is a group of neurons that all use the same type of activation function, all receive data from a previous layer, and all send their outputs to a subsequent layer of neurons. In a neural network, input data is generally fed to an initial layer of neurons, which sends its outputs to a subsequent layer, which sends its outputs to another layer, and so on and so forth, until the last layer of neurons generates the network's final output.

Listing 11-1 is an example of a simple model defined using Keras's functional API syntax. I encourage you to open a new Python file to write and run the code yourself as we walk through the code, line by line. Alternatively, you can try running the associated code in the data accompanying this book, either by copying and pasting parts of the *ch11/model_architecture.py* file into an ipython session or by running `python ch11/model_architecture.py` in a terminal window.

```
❶ from keras import layers
❷ from keras.models import Model

input = layers.Input(❸shape=(1024,), ❹dtype='float32')
❺ middle = layers.Dense(units=512, activation='relu')(input)
❻ output = layers.Dense(units=1, activation='sigmoid')(middle)
```

```

⑦ model = Model(inputs=input, outputs=output)
model.compile(⑧optimizer='adam',
              ⑨loss='binary_crossentropy',
              ⑩metrics=['accuracy'])

```

Listing 11-1: Defining a simple model using functional API syntax

First, we import the Keras package's layers submodule ❶ as well as the Model class from Keras's models submodule ❷.

Next, we specify what kind of data this model will accept for one observation by passing a shape value (a tuple of integers) ❸ and a data type (string) ❹ to the layers.Input() function. Here, we declared that the input data to our model will be an array of 1,024 floats. If our input was, for example, a matrix of integers instead, the first line would look more like `input = Input(shape=(100, 100), dtype='int32')`.

NOTE

If the model takes in variable-sized inputs on one dimension, you can use None instead of a number—for example, (100, None,).

Next, we specify the layer of neurons that this input data will be sent to. To do this, we again use the layers submodule we imported, specifically the Dense function ❺, to specify that this layer will be a densely connected (also called fully connected) layer, which means that every output from the previous layer is sent to every neuron in this layer. Dense is the most common type of layer you'll likely use when developing Keras models. Others allow you to do things like change the shape of the data (Reshape) and implement your own custom layer (Lambda).

We pass the Dense function two arguments: `units=512`, to specify that we want 512 neurons in this layer, and `activation='relu'`, to specify that we want these neurons to be rectified linear unit (ReLU) neurons. (Recall from Chapter 10 that ReLU neurons use a simple type of activation function that outputs whichever is larger: either 0, or the weighted sum of the neuron's inputs.) We use `layers.Dense(units=512, activation='relu')` to define the layer, and then the last part of the line—(input)—declares the input to this layer (namely, our input object). It's important to understand that this passing of input to our layer is how data flow is defined in the model, as opposed to the ordering of the lines of the code.

In the next line, we define our model's output layer, which again uses the Dense function. But this time, we designate only a single neuron to the layer and use a 'sigmoid' activation function ❻, which is great for combining a lot of data into a single score between 0 and 1. The output layer takes the (middle) object as input, declaring that the outputs from our 512 neurons in our middle layer should all be sent to this neuron.

Now that we've defined our layers, we use the Model class from the models submodule to wrap up all these layers together as a model ❼. Note that you *only* have to specify your input layer(s) and output layer(s). Because each layer after the first is given the preceding layer as input, the final output

layer contains all the information the model needs about the previous layers. We could have 10 more middle layers declared between our input and output layers, but the line of code at ⑦ would remain the same.

Compiling the Model

Finally, we need to compile our model. We've defined the model's architecture and flow of data, but we haven't yet specified how we want the model to perform its training. To do this, we use our model's own `compile` method and pass it three parameters:

- The first parameter, optimizer ⑧, specifies the type of backpropagation algorithm to use. You can specify the name of the algorithm you wish to use via a character string like we did here, or you can import an algorithm directly from `keras.optimizers` to pass in specific parameters to the algorithm or even design your own.
- The loss parameter ⑨ specifies the thing that is minimized during the training process (backpropagation). Specifically, this specifies the formula you wish to use to represent the difference between your true training labels and your model's predicted labels (output). Again, you can specify the name of a loss function, or pass in an actual function, like `keras.losses.mean_squared_error`.
- Lastly, for the metrics parameter ⑩, you can pass a list of metrics that you want Keras to report when analyzing model performance during and after training. Again, you can pass strings or actual metric functions, like `['categorical_accuracy', keras.metrics.top_k_categorical_accuracy]`.

After running the code in Listing 11-1, run `model.summary()` to see the model structure printed to your screen. Your output should look something like Figure 11-1.

```
In [2]: model.summary()
```

Layer (type)	Output Shape	Param #
input_1 (InputLayer)	(None, 1024)	0
dense_1 (Dense)	(None, 512)	524800
dense_2 (Dense)	(None, 1)	513
Total params: 525,313		
Trainable params: 525,313		
Non-trainable params: 0		

Figure 11-1: Output of `model.summary()`

Figure 11-1 shows the output of `model.summary()`. Each layer's description is printed to the screen, along with the number of parameters associated with that layer. For example, the `dense_1` layer has 524,800 parameters because each of its 512 neurons gets a copy of each of the 1,024 input values from the input layer, meaning that there are $1,024 \times 512$ weights. Add 512 bias parameters, and you get $1,024 \times 512 + 512 = 524,800$.

Although we haven't yet trained our model or tested it on validation data, this is a compiled Keras model that is ready to train!

NOTE

Check out the sample code in `ch11/model_architecture.py` for an example of a slightly more complex model!

Training the Model

To train our model, we need training data. The virtual machine that comes with this book includes a set of about half a million benign and malicious HTML files. This consists of two folders of benign (`ch11/data/html/benign_files/`) and malicious (`ch11/data/html/malicious_files/`) HTML files. (Remember not to open these files in a browser!) In this section, we use these to train our neural network to predict whether an HTML file is benign (0) or malicious (1).

Extracting Features

To do this, we first need to decide how to represent our data. In other words, what features do we want to extract from each HTML file to use as input to our model? For example, we could simply pass the first 1,000 characters in each HTML file to the model, we could pass in the frequency counts of all letters in the alphabet, or we could use an HTML parser to develop some more complex features. To make things easier, we'll transform each variable-length, potentially very large HTML file into a uniformly sized, compressed representation that allows our model to quickly process and learn important patterns.

In this example, we transform each HTML file into a 1,024-length vector of category counts, where each category count represents the number of tokens in the HTML file whose hash resolved to the given category. Listing 11-2 shows the feature extraction code.

```
import numpy as np
import murmur
import re
import os

def read_file(sha, dir):
    with open(os.path.join(dir, sha), 'r') as fp:
        file = fp.read()
    return file
```

```
def extract_features(sha, path_to_files_dir,
                    hash_dim=1024, ❶split_regex=r"\s+"):
    ❷ file = read_file(sha=sha, dir=path_to_files_dir)
    ❸ tokens = re.split(pattern=split_regex, string=file)
    # now take the modulo(hash of each token) so that each token is replaced
    # by bucket (category) from 1:hash_dim.
    token_hash_buckets = [
        ❹ (murmur.string_hash(w) % (hash_dim - 1) + 1) for w in tokens
    ]
    # Finally, we'll count how many hits each bucket got, so that our features
    # always have length hash_dim, regardless of the size of the HTML file:
    token_bucket_counts = np.zeros(hash_dim)
    # this returns the frequency counts for each unique value in
    # token_hash_buckets:
    buckets, counts = np.unique(token_hash_buckets, return_counts=True)
    # and now we insert these counts into our token_bucket_counts object:
    for bucket, count in zip(buckets, counts):
        ❺ token_bucket_counts[bucket] = count
    return np.array(token_bucket_counts)
```

Listing 11-2: Feature extraction code

You don't have to understand all the details of this code to understand how Keras works, but I encourage you to read through the comments in the code to better understand what's going on.

The `extract_features` function starts by reading in an HTML file as a big string ❷ and then splits up this string into a set of tokens based on a regular expression ❸. Next, the numeric hash of each token is taken, and these hashes are divided into categories by taking the modulo of each hash ❹. The final set of features is the number of hashes in each category ❺, like a histogram bin count. If you want, you can try altering the regular expression `split_regex` ❶ that splits up the HTML file into chunks to see how it affects the resulting tokens and features.

If you skipped or didn't understand all that, that's okay: just know that our `extract_features` function takes the path to an HTML file as input and then transforms it into a feature array of length 1,024, or whatever `hash_dim` is.

Creating a Data Generator

Now we need to make our Keras model actually train on these features. When working with small amounts of data already loaded into memory, you can use a simple line of code like Listing 11-3 to train your model in Keras.

```
# first you would load in my_data and my_labels via some means, and then:
model.fit(my_data, my_labels, epochs=10, batch_size=32)
```

Listing 11-3: Training your model when data is already loaded into memory

However, this isn't really useful when you start working with large amounts of data, because you can't fit all your training data into your computer's memory at once. To get around this, we use the slightly more

complex but more scalable `model.fit_generator` function. Instead of passing in all the training data at once to this function, you pass a generator that yields training data in batches so that your computer's RAM won't choke.

Python generators work just like Python functions, except they have a `yield` statement. Instead of returning a single result, generators return an object that can be called again and again to yield many, or infinite, sets of results. Listing 11-4 shows how we can create our own data generator using our feature extraction function.

```
def my_generator(benign_files, malicious_files,
                path_to_benign_files, path_to_malicious_files,
                batch_size, features_length=1024):
    n_samples_per_class = batch_size / 2
    ❶ assert len(benign_files) >= n_samples_per_class
    assert len(malicious_files) >= n_samples_per_class
    ❷ while True:
        ben_features = [
            extract_features(sha, path_to_files_dir=path_to_benign_files,
                            hash_dim=features_length)
            for sha in np.random.choice(benign_files, n_samples_per_class,
                                       replace=False)
        ]
        mal_features = [
            ❸ extract_features(sha, path_to_files_dir=path_to_malicious_files,
                              hash_dim=features_length)
            ❹ for sha in np.random.choice(malicious_files, n_samples_per_class,
                                       replace=False)
        ]
        ❺ all_features = ben_features + mal_features
        labels = [0 for i in range(n_samples_per_class)] + [1 for i in range(
            n_samples_per_class)]

        idx = np.random.choice(range(batch_size), batch_size)
        ❻ all_features = np.array([np.array(all_features[i]) for i in idx])
        labels = np.array([labels[i] for i in idx])
        ❼ yield all_features, labels
```

Listing 11-4: Writing a data generator

First, the code makes two `assert` statements to check that enough data is there ❶. Then inside a `while` ❷ loop (so it'll just iterate forever), both benign and malicious features are grabbed by choosing a random sample ❹ of file keys and then extracting features for those files using our `extract_features` function ❸. Next, the benign and malicious features and associated labels (0 and 1) are concatenated ❺ and shuffled ❻. Finally, these features and labels are returned ❼.

Once instantiated, this generator should yield `batch_size` features and labels for the model to train on (50 percent malicious, 50 percent benign) each time the generator's `next()` method is called.

Listing 11-5 shows how to create a training data generator using the data that comes with this book, and how to train our model by passing the generator to our model's `fit_generator` method.

```

import os

batch_size = 128
features_length = 1024
path_to_training_benign_files = 'data/html/benign_files/training/'
path_to_training_malicious_files = 'data/html/malicious_files/training/'
steps_per_epoch = 1000 # artificially small for example-code speed!

❶ train_benign_files = os.listdir(path_to_training_benign_files)
❷ train_malicious_files = os.listdir(path_to_training_malicious_files)

# make our training data generator!
❸ training_generator = my_generator(
    benign_files=train_benign_files,
    malicious_files=train_malicious_files,
    path_to_benign_files=path_to_training_benign_files,
    path_to_malicious_files=path_to_training_malicious_files,
    batch_size=batch_size,
    features_length=features_length
)

❹ model.fit_generator(
    ❺ generator=training_generator,
    ❻ steps_per_epoch=steps_per_epoch,
    ❼ epochs=10
)

```

Listing 11-5: Creating the training generator and using it to train the model

Try reading through this code to understand what’s happening. After importing a necessary package and creating some parameter variables, we read the filenames for our benign ❶ and malicious training data ❷ into memory (but not the files themselves). We pass these values to our new `my_generator` function ❸ to get our training data generator. Finally, using our model from Listing 11-1, we use the model’s built-in `fit_generator` method ❹ to start training.

The `fit_generator` method takes three parameters. The generator parameter ❺ specifies the data generator that produces training data for each *batch*. During training, parameters are updated once per batch by averaging all the training observations’ signals for that batch. The `steps_per_epoch` parameter ❻ sets the number of batches we want the model to process each *epoch*. As a result, the total number of observations the model sees per epoch is `batch_size*steps_per_epoch`. By convention, the number of observations a model sees per epoch should be equal to the dataset size, but in this chapter and in the virtual machine sample code, I reduce `steps_per_epoch` to make our code run faster. The `epochs` parameter ❼ sets the number of epochs we want to run.

Try running this code in the `ch11/` directory that accompanies this book. Depending on the power of your computer, each training epoch will take a certain amount of time to run. If you’re using an interactive session, feel free to cancel the process (CTRL-C) after a few epochs if it’s taking a while. This

will stop the training without losing progress. After you cancel the process (or the code completes), you'll have a trained model! The readout on your virtual machine screen should look something like Figure 11-2.

```
Using TensorFlow backend.
I tensorflow/stream_executor/dso_loader.cc:135] successfully opened CUDA library libcublas.so.7.5 locally
I tensorflow/stream_executor/dso_loader.cc:135] successfully opened CUDA library libcudnn.so.5 locally
I tensorflow/stream_executor/dso_loader.cc:135] successfully opened CUDA library libcufft.so.7.5 locally
I tensorflow/stream_executor/dso_loader.cc:135] successfully opened CUDA library libcuda.so.1 locally
I tensorflow/stream_executor/dso_loader.cc:135] successfully opened CUDA library libcurand.so.7.5 locally
Epoch 1/10
W tensorflow/core/platform/cpu_feature_guard.cc:45] The TensorFlow library wasn't compiled to use SSE3 ins
W tensorflow/core/platform/cpu_feature_guard.cc:45] The TensorFlow library wasn't compiled to use SSE4.1 i
W tensorflow/core/platform/cpu_feature_guard.cc:45] The TensorFlow library wasn't compiled to use SSE4.2 i
W tensorflow/core/platform/cpu_feature_guard.cc:45] The TensorFlow library wasn't compiled to use AVX inst
W tensorflow/core/platform/cpu_feature_guard.cc:45] The TensorFlow library wasn't compiled to use AVX2 ins
W tensorflow/core/platform/cpu_feature_guard.cc:45] The TensorFlow library wasn't compiled to use FMA inst
NVIDIA: no NVIDIA devices found
E tensorflow/stream_executor/cuda/cuda_driver.cc:509] failed call to cuInit: CUDA_ERROR_UNKNOWN
I tensorflow/stream_executor/cuda/cuda_diagnostics.cc:145] kernel driver does not appear to be running on
39/39 [=====] - 7s 171ms/step - loss: 0.3463 - acc: 0.8476
Epoch 2/10
39/39 [=====] - 7s 168ms/step - loss: 0.2181 - acc: 0.9139
Epoch 3/10
39/39 [=====] - 7s 168ms/step - loss: 0.1864 - acc: 0.9253
Epoch 4/10
18/39 [=====>.....] - ETA: 3s - loss: 0.1871 - acc: 0.9262
```

Figure 11-2: Console output from training a Keras model

The top few lines note that TensorFlow, which is the default backend to Keras, has been loaded. You'll also see some warnings like in Figure 11-2; these just mean that the training will be done on CPUs instead of GPUs (GPUs are often around 2–20 times faster for training neural networks, but for the purposes of this book, CPU-based training is fine). Finally, you'll see a progress bar for each epoch indicating how much longer the given epoch will take, as well as the epoch's loss and accuracy metrics.

Incorporating Validation Data

In the previous section, you learned how to train a Keras model on HTML files using the scalable `fit_generator` method. As you saw, the model prints statements during training, indicating each epoch's current loss and accuracy statistics. However, what you really care about is how your trained model does on *validation data*, or data that it has never seen before. This better represents the kind of data your model will face in a real-life production environment.

When trying to design better models and figure out how long to train your model for, you should try to maximize *validation accuracy* rather than *training accuracy*, the latter of which was shown in Figure 11-2. Even better would be using validation files originating from dates after the training data to better simulate a production environment.

Listing 11-6 shows how to load our validation features into memory using our `my_generator` function from Listing 11-4.

```
import os
path_to_validation_benign_files = 'data/html/benign_files/validation/'
```



```

path_to_validation_malicious_files = 'data/html/malicious_files/validation/'
# get the validation keys:
val_benign_file_keys = os.listdir(path_to_validation_benign_files)
val_malicious_file_keys = os.listdir(path_to_validation_malicious_files)
# grab the validation data and extract the features:
❶ validation_data = my_generator(
    benign_files=val_benign_files,
    malicious_files=val_malicious_files,
    path_to_benign_files=path_to_validation_benign_files,
    path_to_malicious_files=path_to_validation_malicious_files,
    ❷ batch_size=10000,
    features_length=features_length
❸ ).next()

```

Listing 11-6: Reading validation features and labels into memory by using the `my_generator` function

This code is very similar to how we created our training data generator, except that the file paths have changed and now we want to load all the validation data into memory. So instead of just creating the generator, we create a validation data generator ❶ with a large `batch_size` ❷ equal to the number of files we want to validate on, and we immediately call its `.next()` ❸ method just once.

Now that we have some validation data loaded into memory, Keras allows us to simply pass `fit_generator()` our validation data during training, as shown in Listing 11-7.

```

model.fit_generator(
    ❶ validation_data=validation_data,
    generator=training_generator,
    steps_per_epoch=steps_per_epoch,
    epochs=10
)

```

Listing 11-7: Using validation data for automatic monitoring during training

Listing 11-7 is almost identical to the end of Listing 11-5, except that `validation_data` is now passed to `fit_generator` ❶. This helps enhance model monitoring by ensuring that validation loss and accuracy are calculated alongside training loss and accuracy.

Now, training statements should look something like Figure 11-3.

```

Epoch 1/10
39/39 [=====] - 8s 192ms/step - loss: 0.1146 - acc: 0.9571 - val_loss: 0.5067 - val_acc: 0.7690
Epoch 2/10
39/39 [=====] - 7s 184ms/step - loss: 0.1392 - acc: 0.9463 - val_loss: 0.2621 - val_acc: 0.8970
Epoch 3/10
39/39 [=====] - 7s 189ms/step - loss: 0.1234 - acc: 0.9527 - val_loss: 0.3382 - val_acc: 0.8790
Epoch 4/10
39/39 [=====] - 7s 189ms/step - loss: 0.0981 - acc: 0.9611 - val_loss: 0.2770 - val_acc: 0.8970
Epoch 5/10
39/39 [=====] - 7s 189ms/step - loss: 0.1232 - acc: 0.9541 - val_loss: 0.3053 - val_acc: 0.8790
Epoch 6/10
37/39 [=====>...] - ETA: 0s - loss: 0.1068 - acc: 0.9552

```

Figure 11-3: Console output from training a Keras model with validation data

Figure 11-3 is similar to Figure 11-2, except that instead of just showing training loss and acc metrics for each epoch, now Keras also calculates and shows `val_loss` (validation loss) and `val_acc` (validation accuracy) for each epoch. In general, if validation accuracy is going down instead of up, that's an indication your model is overfitting to your training data, and it would be best to halt training. If validation accuracy is going up, as is the case here, it means your model is still getting better and you should continue training.

Saving and Loading the Model

Now that you know how to build and train a neural network, let's go over how to save it so you can share it with others.

Listing 11-8 shows how to save our trained model to an `.h5` file ❶ and reload ❷ it (at a potentially later date).

```
from keras.models import load_model
# save the model
❶ model.save('my_model.h5')
# load the model back into memory from the file:
❷ same_model = load_model('my_model.h5')
```

Listing 11-8: Saving and loading Keras models

Evaluating the Model

In the model training section, we observed some default model evaluation metrics like training loss and accuracy as well as validation loss and accuracy. Let's now review some more complex metrics to better evaluate our models.

One useful metric for evaluating the accuracy of a binary predictor is called *area under the curve (AUC)*. The curve refers to a Receiver Operating Characteristic (ROC) curve (see Chapter 8), which plots false-positive rates (x-axis) against true-positive rates (y-axis) for all possible score thresholds.

For example, our model tries to predict whether a file is malicious by using a score between 0 (benign) and 1 (malicious). If we choose a relatively high score threshold to classify a file as malicious we'll get fewer false-positives (good) but also fewer true-positives (bad). On the other hand, if we choose a low score threshold, we'll likely have a high false-positive rate (bad) but a very high detection rate (good).

These two sample possibilities would be represented as two points on our model's ROC curve, where the first would be located toward the left side of the curve and the second near the right side. AUC represents all these possibilities by simply taking the area under this ROC curve, as shown in Figure 11-4.

In simple terms, an AUC of 0.5 represents the predictive capability of a coin flip, while an AUC of 1 is perfect.

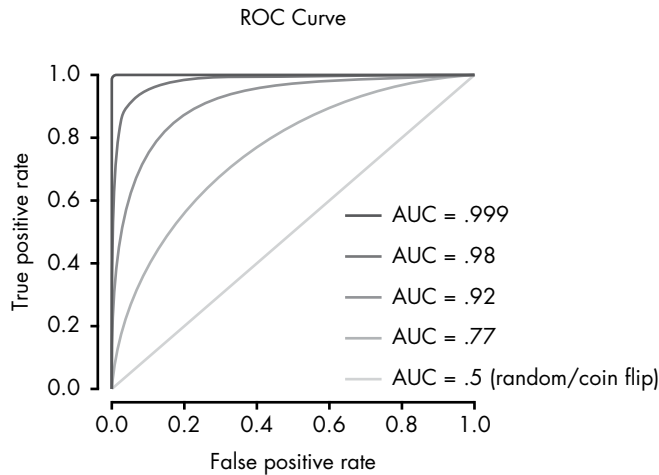


Figure 11-4: Various sample ROC curves. Each ROC curve (line) corresponds to a different AUC value.

Let's use our validation data to calculate validation AUC using the code in Listing 11-9.

```
from sklearn import metrics

❶ validation_labels = validation_data[1]
❷ validation_scores = [el[0] for el in model.predict(validation_data[0])]
❸ fpr, tpr, thres = metrics.roc_curve(y_true=validation_labels,
                                     y_score=validation_scores)
❹ auc = metrics.auc(fpr, tpr)
print('Validation AUC = {}'.format(auc))
```

Listing 11-9: Calculating validation AUC using *sklearn*'s metric submodule

Here, we split our `validation_data` tuple into two objects: the validation labels represented by `validation_labels` ❶, and flattened validation model predictions represented by `validation_scores` ❷. Then, we use the `metrics.roc_curve` function from *sklearn* to calculate false-positive rates, true-positive rates, and associated threshold values for the model predictions ❸. Using these, we calculate our AUC metric, again using an *sklearn* function ❹.

Although I won't go over the function code here, you can also use the `roc_plot()` function included in the `ch11/model_evaluation.py` file in the data accompanying this book to plot the actual ROC curve, as shown in Listing 11-10.

```
from ch11.model_evaluation import roc_plot
roc_plot(fpr=fpr, tpr=tpr, path_to_file='roc_curve.png')
```

Listing 11-10: Creating a ROC curve plot using the `roc_plot` function from this book's accompanying data, in `ch11/model_evaluation.py`

Running the code in Listing 11-10 should generate a plot (saved to `roc_curve.png`) that looks like Figure 11-5.

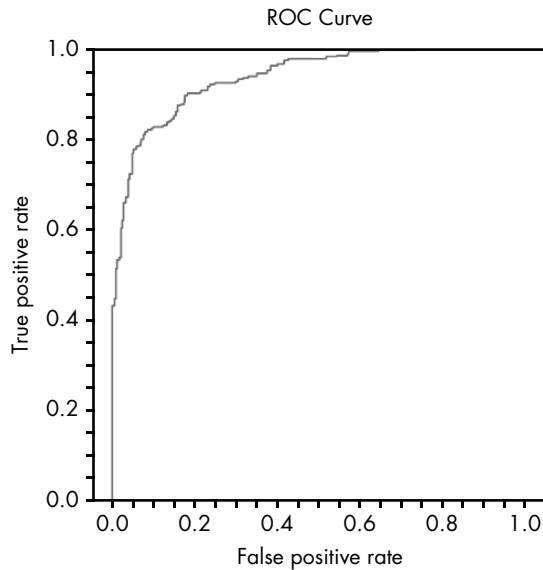


Figure 11-5: A ROC curve!

Each point in the ROC curve in Figure 11-5 represents a specific false-positive rate (x-axis) and true-positive rate (y-axis) associated with various model prediction thresholds ranging from 0 to 1. As false-positive rates increase, true-positive rates increase, and vice versa. In production environments, you generally have to pick a single threshold (a single point on this curve, assuming validation data mimics production data) with which to make your decision, based on your willingness to tolerate false positives, versus your willingness to risk allowing a malicious file to slip through the cracks.

Enhancing the Model Training Process with Callbacks

So far, you've learned how to design, train, save, load, and evaluate Keras models. Although this is really all you need to get a fairly good start, I also want to introduce Keras callbacks, which can make our model training process even better.

A Keras callback represents a set of functions that Keras applies during certain stages of the training process. For example, you can use a Keras callback to make sure that an `.h5` file is saved at the end of each epoch, or that validation AUC is printed to the screen at the end of each epoch. This can help record and inform you more precisely of how your model is doing during the training process.

We begin by using a built-in callback, and then we try writing our own custom callback.

Using a Built-in Callback

To use a built-in callback, simply pass your model's `fit_generator()` method a callback instance during training. We'll use the `callbacks.ModelCheckpoint` callback, which evaluates validation loss after each training epoch, and saves the current model to a file *if* the validation loss is smaller than any previous epoch's validation losses. To do this, the callback needs access to our validation data, so we'll pass that in to the `fit_generator()` method, as shown in Listing 11-11.

```
from keras import callbacks

model.fit_generator(
    generator=training_generator,
    # lowering steps_per_epoch so the example code runs fast:
    steps_per_epoch=50,
    epochs=5,
    validation_data=validation_data,
    callbacks=[
        callbacks.ModelCheckpoint(save_best_only=True,❶
                                ❷ filepath='results/best_model.h5',
                                ❸ monitor='val_loss')
    ],
)
```

Listing 11-11: Adding a `ModelCheckpoint` callback to the training process

This code ensures that the model is overwritten ^❶ to a single file, 'results/best_model.h5' ^❷, whenever 'val_loss' ^❸ (validation loss) reaches a new low. This ensures that the current saved model ('results/best_model.h5') always represents the best model across all completed epochs with regard to validation loss.

Alternatively, we can use the code in Listing 11-12 to save the model after every epoch to a *separate* file regardless of validation loss.

```
callbacks.ModelCheckpoint(save_best_only=False,❹
                          ❺ filepath='results/model_epoch_{epoch}.h5',
                          monitor='val_loss')
```

Listing 11-12: Adding a `ModelCheckpoint` callback to the training process that saves the model to a different file after each epoch

To do this, we use the same code in Listing 11-11 and the same function `ModelCheckpoint`, but with `save_best_only=False` ^❹ and a filepath that asks Keras to fill in the epoch number ^❺. Instead of only saving the single “best” version of our model, Listing 11-12's callback saves each epoch's version of our model, in *results/model_epoch_0.h5*, *results/model_epoch_1.h5*, *results/model_epoch_2.h5*, and so on.

Using a Custom Callback

Although Keras doesn't support AUC, we can design our own custom callback to, for example, allow us to print AUC to the screen after each epoch.

To create a custom Keras callback, we need to create a class that inherits from `keras.callbacks.Callback`, the abstract base class used to build new callbacks. We can add one or more of a selection of methods, which will be run automatically during training, at times that their names specify: `on_epoch_begin`, `on_epoch_end`, `on_batch_begin`, `on_batch_end`, `on_train_begin`, and `on_train_end`.

Listing 11-13 shows how to create a callback that calculates and prints validation AUC to the screen at the end of each epoch.

```
import numpy as np
from keras import callbacks
from sklearn import metrics

❶ class MyCallback(callbacks.Callback):

    ❷ def on_epoch_end(self, epoch, logs={}):
        ❸ validation_labels = self.validation_data[1]
        validation_scores = self.model.predict(self.validation_data[0])
        # flatten the scores:
        validation_scores = [el[0] for el in validation_scores]
        fpr, tpr, thres = metrics.roc_curve(y_true=validation_labels,
                                           y_score=validation_scores)
        ❹ auc = metrics.auc(fpr, tpr)
        print('\n\tEpoch {}, Validation AUC = {}'.format(epoch,
                                                         np.round(auc, 6)))

model.fit_generator(
    generator=training_generator,
    # lowering steps_per_epoch so the example code runs fast:
    steps_per_epoch=50,
    epochs=5,
    ❺ validation_data=validation_data,
    ❻ callbacks=[
        callbacks.ModelCheckpoint('results/model_epoch_{epoch}.h5',
                                monitor='val_loss',
                                save_best_only=False,
                                save_weights_only=False)
    ]
)
```

Listing 11-13: Creating and using a custom callback to print AUC to the screen after each training epoch

In this example, we first create our `MyCallback` class ❶, which inherits from `callbacks.Callbacks`. Keeping things simple, we overwrite a single method, `on_epoch_end` ❷, and give it two arguments expected by Keras: `epoch` and `logs` (a dictionary of log information), both of which Keras will supply when it calls the function during training.

Then, we grab the `validation_data` ❸, which is already stored in the `self` object thanks to `callbacks.Callback` inheritance, and we calculate and print out AUC ❹ like we did in “Evaluating the Model” on page 209. Note that for this code to work, the validation data needs to be passed to `fit_generator()` so that the callback has access to `self.validation_data` during training ❺. Finally, we tell the model to train and specify our new callback ❻. The result should look something like Figure 11-6.

```
Epoch 1/5
39/39 [=====] - 7s 186ms/step - loss: 0.1148 - acc: 0.9515 - val_loss: 0.3693 - val_acc: 0.8630
Epoch 0, Validation AUC = 0.922248
Epoch 2/5
39/39 [=====] - 7s 175ms/step - loss: 0.1308 - acc: 0.9507 - val_loss: 0.2938 - val_acc: 0.8640
Epoch 1, Validation AUC = 0.947984
Epoch 3/5
39/39 [=====] - 7s 179ms/step - loss: 0.1120 - acc: 0.9599 - val_loss: 0.3064 - val_acc: 0.8730
Epoch 2, Validation AUC = 0.949036
Epoch 4/5
39/39 [=====] - 7s 179ms/step - loss: 0.1134 - acc: 0.9625 - val_loss: 0.3167 - val_acc: 0.8520
Epoch 3, Validation AUC = 0.958548
Epoch 5/5
22/39 [====>.....] - ETA: 2s - loss: 0.1336 - acc: 0.9474
```

Figure 11-6: Console output from training a Keras model with a custom AUC callback

If what you really care about is minimizing validation AUC, this callback makes it easy to see how your model is doing during training, thus helping you assess whether you should stop the training process (for example, if validation accuracy is going consistently down over time).

Summary

In this chapter, you learned how to build your own neural network using Keras. You also learned to train, evaluate, save, and load it. You then learned how to enhance the model training process by adding built-in and custom callbacks. I encourage you to play around with the code accompanying this book to see what changes model architecture and feature extraction can have on model accuracy.

This chapter is meant to get your feet wet, but is not meant as a reference guide. Visit <https://keras.io> for the most up-to-date official documentation. I strongly encourage you to spend time researching aspects of Keras that interest you. Hopefully, this chapter has served as a good jumping-off point for all your security deep learning adventures!

12

BECOMING A DATA SCIENTIST



To conclude this book, let's take a step back and discuss how you can make a life and career as a malware data scientist or a security data scientist in general. Although this is a nontechnical chapter, it's just as important as the technical chapters in this book, if not more important. This is because becoming a successful security data scientist involves much more than simply understanding the subject matter.

In this chapter, we the authors share our own career paths to becoming professional security data scientists. You'll get a glimpse of what day-to-day life looks like as a security data scientist and what it takes to become an effective data scientist. We also share tips on how to approach data science problems and how to stay resilient in the face of inevitable challenges.

Paths to Becoming a Security Data Scientist

Because security data science is a new field, there are many paths to becoming a security data scientist. Whereas many data scientists receive formal training through graduate school, many others are self-taught. For example, I grew up in the 1990s computer hacking scene, where I learned to program in C and assembly and to write black-hat hacking tools. Later, I got a bachelor's degree and then a master's degree in the humanities before re-entering the tech world as a security software developer. Along the way, I taught myself data visualization and machine learning at night, finally moving into a formal security data science role at Sophos, a security research and development company. Hillary Sanders, my co-author on this book, studied statistics and economics in college, worked as a data scientist for a time, and later found work at a security company as a data scientist, picking up her security knowledge on the job.

Our team at Sophos is just as diverse. Our colleagues hold a number of degrees in a wide range of disciplines: psychology, data science, mathematics, biochemistry, statistics, and computer science. Although security data science is biased toward those with formal training in quantitative methods in science, it includes folks with varied backgrounds in these fields. And although scientific and quantitative training is helpful for learning security data science, my own experience suggests that it's also possible to enter and excel in our field with a nontraditional background, as long as you're willing to teach yourself.

Excelling in security data science hinges on one's willingness to constantly learn new things. This is because practical knowledge is just as important as theoretical knowledge in our field, and you pick up practical knowledge through *doing*, not through school work.

Being willing to learn new things is also important because machine learning, network analysis, and data visualization are constantly changing, so what you learn in school quickly goes out of date. For example, deep learning has only taken off as a trend in the years since around 2012, and has developed rapidly since, so almost everyone in data science who graduated before then has had to teach themselves these powerful ideas. This is good news for those seeking to enter security data science professionally. Since those already in the field have to constantly teach themselves new skills, you can get a foot in the door by already knowing those skills.

A Day in the Life of a Security Data Scientist

A security data scientist's job is to apply the type of skills taught in this book to hard security problems. But application of these skills tends to be embedded within a larger workflow that involves other skills as well. Figure 12-1 illustrates a typical workflow of a security data scientist, based on our experience and that of our colleagues at other companies and organizations.

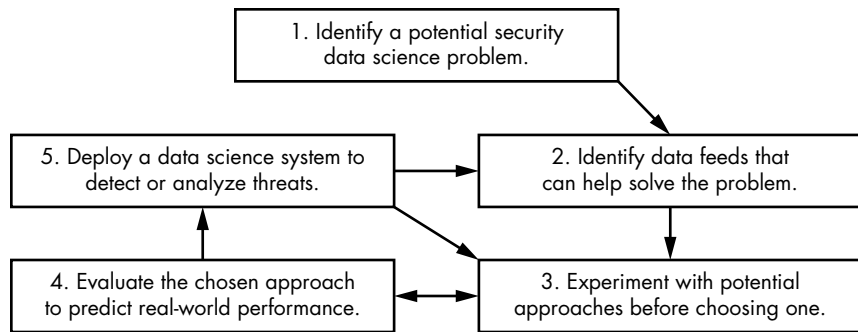


Figure 12-1: A model of the security data science workflow

As Figure 12-1 shows, the security data science workflow involves an interplay between five areas of work. The first area, *problem identification*, involves identifying security problems where data science can help. For example, we may hypothesize that identifying spear-phishing emails can be solved using data science methods, or that identifying the particular method used to obfuscate known malware is a problem worth investigating.

At this stage, any assumption that a given problem may be solvable with data science is just a hypothesis. When you have a hammer (data science), every problem can look like a nail (a machine learning, data visualization, or network analysis problem). We have to reflect on whether these problems are *truly* best addressed using data science methods, keeping in mind that it will take building a prototype data science solution and then testing this solution to better understand if data science actually provides the *best* solution.

When you're working within an organization, identifying a good problem almost always involves interacting with stakeholders who are not themselves data scientists. For example, within our company, we often interface with product managers, executives, software developers, and salespeople who think that data science is like a magic wand that can solve any problem, or that data science is akin to “artificial intelligence” and therefore has some magical ability to achieve unrealistic results.

The key thing to remember when dealing with such stakeholders is to be honest about the capabilities and limitations of data science-based approaches, and to maintain a shrewd, measured attitude so that you don't go chasing the wrong problems. You should discard problems for which there is no data to drive data science algorithms or no way to evaluate whether your data science approaches are actually working, as well as problems you can clearly solve better through more manual methods.

For example, here are some problems we declined after others asked us to solve them:

- **Automatically identifying rogue employees who may be leaking data to competitors.** There's not enough data to drive a machine learning algorithm, but this could be pursued using data visualization or network analysis.

- **Decrypting network traffic.** The mathematics of machine learning show that machine learning is simply not capable of decrypting weapons-grade encrypted data!
- **Automatically identifying phishing emails handcrafted to target specific employees based on detailed background knowledge of their lifestyle.** Again, there's not enough data to drive a machine learning algorithm, but this might be possible through the visualization of a time series or email data.

Once you *do* successfully identify a potential security data science problem, your next task is to identify data feeds you can use to help solve it using the data science techniques explained in this book. This is shown in step 2 of Figure 12-1. At the end of the day, if you don't have data feeds that you can use to train machine learning models, feed visualizations, or drive network analysis that solves your chosen security problem, data science is probably not going to help you.

After you've selected a problem and identified data feeds that will allow you to build a data science-based solution to the problem, it's time to begin building your solution. This actually happens in an iterative loop between steps 3 and 4 of Figure 12-1: you build something, evaluate it, improve it, reevaluate it, and so on.

Finally, once your system is ready, you deploy it, as shown in step 5 of Figure 12-1. As long as your system stays deployed, you'll need to go back and integrate new data feeds as they become available, try out new data science methods, and redeploy new versions of your system.

Traits of an Effective Security Data Scientist

Success in security data science depends a lot on your attitude. In this section, we list some mental attributes we've found are important to success in security data science work.

Open-Mindedness

Data is full of surprises, and this disrupts what we thought we knew about a problem. It's important to keep your mind open to your data proving your preconceived notions wrong. If you don't, you'll end up missing important learnings from your data, and even reading too much into random noise to convince yourself of a false theory. Fortunately, the more security data science you do, the more open-minded you'll be about "learning" from your data, and the more okay you'll be with how little you know and how much you have to learn from each new problem. In time, you'll come to both enjoy and expect surprises from your data.

Boundless Curiosity

Data science projects are very different from software engineering and IT projects in that they require exploring data to find patterns, outliers,

and trends, which we then leverage to build our systems. Identifying these dynamics is not easy: it often requires running hundreds of experiments or analyses to get a sense of the overall shape of your data and the stories hidden inside. Some people have a natural drive to run shrewdly designed experiments and to dig deeper into their data, almost addictively, whereas others don't. The former is the type of person who tends to succeed at data science. Curiosity is therefore a *requirement* in this field because it's what differentiates our ability to arrive at a deep understanding of our data versus a shallow one. The more you can cultivate an attitude of curiosity when building models and visualizations of your data, the more useful your systems will be.

Obsession with Results

Once you've defined a good security data science problem and have begun iteratively trying solutions and evaluating them, an obsession with results may take hold of you, particularly on machine learning projects. This is a good sign. For example, when I'm heavily involved in a machine learning project, I have multiple experiments running 24 hours a day, 7 days a week. This means that I might wake up multiple times a night to check on the status of the experiments, and often need to fix bugs and restart experiments at 3:00 in the morning. I tend to check in on my experiments before going to bed every night and multiple times throughout the weekend.

This kind of round-the-clock workflow is often necessary to build top-of-the-line security data science systems. Without it, it's easy to settle for mediocre results, failing to break out of ruts or overcome blockages built out of misplaced assumptions about the data.

Skepticism of Results

It's easy to fool yourself into thinking you're succeeding on a security data science project. For example, perhaps you set up your evaluation incorrectly, such that it appears your system's accuracy is much better than it actually is. Evaluating your system on data that's too similar to your training data or too dissimilar from real-world data is a common pitfall. You also might have inadvertently cherry-picked examples from your network visualization that *you* thought were useful but most users don't find much value in. Or perhaps you worked so hard on your approach that you convinced yourself that the evaluation statistics are good, when in fact they're not good enough to make your system useful in your real world. It's important to maintain a healthy level of skepticism of your results, lest you find yourself in an embarrassing situation someday.

Where to Go from Here

We've covered a lot in this book, but we've also barely scratched the surface. If this book has convinced you to pursue security data science in a serious way, we have two recommendations for you: first, begin applying the

tools you’ve learned in this book to problems you care about immediately. Second, read more books on data science and security data science. Here are some examples of problems you might consider applying your new-found skills to:

- Detecting malicious domain names
- Detecting malicious URLs
- Detecting malicious email attachments
- Visualizing network traffic to spot anomalies
- Visualizing email sender/recipient patterns to detect phishing emails

To expand your knowledge of data science methods, we recommend starting simple, with Wikipedia articles on the data science algorithms you want to learn more about. Wikipedia is a surprisingly accessible and authoritative resource when it comes to data science, and it’s free. For those who want to go deeper, especially in machine learning, we recommend picking up books on linear algebra, probability theory, statistics, graph analytics, and multivariable calculus, or taking free online courses. Learning these fundamentals will pay dividends for the rest of your data science career, because they are the foundation on which our field rests. Beyond focusing on these fundamentals, we also recommend taking courses on or reading more “applied” books about Python, `numpy`, `sklearn`, `matplotlib`, `seaborn`, `Keras`, and any other tools covered in this book that are used heavily in the data science community.

AN OVERVIEW OF DATASETS AND TOOLS



All data and code for this book
are available for download at *[http://](http://www.malwaredatascience.com/)*

www.malwaredatascience.com/. Be warned:

there is Windows malware in the data. If you unzip
the data on a machine with an antivirus engine run-
ning on it, many of the malware examples will likely
get deleted or quarantined.

NOTE

*We have modified a few bytes in each malware executable so as to disable it from
executing. That being said, you can't be too careful about where you store it. We
recommend storing it on a non-Windows machine that's isolated from your home
or business network.*

Ideally, you should only experiment with the code and data within an isolated virtual machine. For convenience, we've provided a VirtualBox Ubuntu instance at <http://www.malwaredatascience.com/> that has the data and code pre-loaded onto it, along with all the necessary open source libraries.

Overview of Datasets

Now let's walk through the datasets that accompany each chapter of this book.

Chapter 1: Basic Static Malware Analysis

Recall that in Chapter 1 we walk through basic static analysis of a malware binary called *ircbot.exe*. This malware is an *implant*, meaning it hides on users' systems and waits for commands from an attacker, allowing the attacker to collect private data from a victim's computer or achieve malicious ends like erasing the victim's hard drive. This binary is available in the data accompanying this book at *ch1/ircbot.exe*.

We also use an example of *fakepdfmalware.exe* in this chapter (located at *ch1/fakepdfmalware.exe*). This is a malware program that has an Adobe Acrobat/PDF desktop icon to trick users into thinking they're opening a PDF document when they're actually running the malicious program and infecting their systems.

Chapter 2: Beyond Basic Static Analysis: x86 Disassembly

In this chapter we explore a deeper topic in malware reverse engineering: analyzing x86 disassembly. We reuse the *ircbot.exe* example from Chapter 1 in this chapter.

Chapter 3: A Brief Introduction to Dynamic Analysis

For our discussion of dynamic malware analysis in Chapter 3, we experiment with a ransomware example stored in the path *ch3/d676d9dfab6a4242258362b8ff579cfe6e5e6db3f0cdd3e0069ace50f80af1c5* in the data accompanying this book. The filename corresponds to the file's SHA256 cryptographic hash. There's nothing particularly special about this ransomware, which we got by searching *VirusTotal.com*'s malware database for examples of ransomware.

Chapter 4: Identifying Attack Campaigns Using Malware Networks

Chapter 4 introduces the application of network analysis and visualization to malware. To demonstrate these techniques, we use a set of high-quality malware samples used in high-profile attacks, focusing our analysis on a set of malware samples likely produced by a group within the Chinese military known to the security community as *Advanced Persistent Threat 1* (or *APT1* for short).

These samples and the APT1 group that generated them were discovered and made public by cybersecurity firm Mandiant. In its report (excerpted here) titled “APT1: Exposing One of China’s Cyber Espionage Units” (<https://www.fireeye.com/content/dam/fireeye-www/services/pdfs/mandiant-apt1-report.pdf>), Mandiant found the following:

- Since 2006, Mandiant has observed APT1 compromise 141 companies spanning 20 major industries.
- APT1 has a well-defined attack methodology, honed over years and designed to steal large volumes of valuable intellectual property.
- Once APT1 has established access, they periodically revisit the victim’s network over several months or years and steal broad categories of intellectual property, including technology blueprints, proprietary manufacturing processes, test results, business plans, pricing documents, partnership agreements, and emails and contact lists from victim organizations’ leadership.
- APT1 uses some tools and techniques that we have not yet observed being used by other groups including two utilities designed to steal email: GETMAIL and MAPIGET.
- APT1 maintained access to victim networks for an average of 356 days.
- The longest time period APT1 maintained access to a victim’s network was 1,764 days, or four years and ten months.
- Among other large-scale thefts of intellectual property, we have observed APT1 stealing 6.5TB of compressed data from a single organization over a ten-month time period.
- In the first month of 2011, APT1 successfully compromised at least 17 new victims operating in 10 different industries.

As this excerpt of the report shows, the APT1 samples were used for high-stakes, nation state–level espionage. These samples are available in the data accompanying this book at *ch4/data/APT1_MALWARE_FAMILIES*.

Chapter 5: Shared Code Analysis

Chapter 5 reuses the APT1 samples used in Chapter 4. For convenience, these samples are also located in the Chapter 5 directory, at *ch5/data/APT1_MALWARE_FAMILIES*.

Chapter 6: Understanding Machine Learning–Based Malware Detectors and Chapter 7: Evaluating Malware Detection Systems

These conceptual chapters don’t require any sample data.

Chapter 8: Building Machine Learning Detectors

Chapter 8 explores building machine learning–based malware detectors and uses 1,419 sample binaries as a sample dataset for training your own machine learning detection system. These binaries are located at *ch8/data/benignware* for the benign samples and *ch8/data/malware* for the malware samples.

The dataset contains 991 benignware samples and 428 malware samples, and we got this data from *VirusTotal.com*. These samples are representative, in the malware case, of the kind of malware observed on the internet in 2017 and, in the benignware case, of the kind of binaries users uploaded to *VirusTotal.com* in 2017.

Chapter 9: Visualizing Malware Trends

Chapter 9 explores data visualization and uses the sample data in the file *ch9/code/malware_data.csv*. Of the 37,511 data rows in the file, each row shows a record of an individual malware file, when it was first seen, how many antivirus products detected it, and what kind of malware it is (for example, Trojan horse, ransomware, and so on). This data was collected from *VirusTotal.com*.

Chapter 10: Deep Learning Basics

This chapter introduces deep neural networks and doesn't use any sample data.

Chapter 11: Building a Neural Network Malware Detector with Keras

This chapter walks through building a neural network malware detector for detecting malicious and benign HTML files. Benign HTML files are from legitimate web pages, and the malicious web pages are from websites that attempt to infect victims via their web browsers. We got both of these datasets from *VirusTotal.com* using a paid subscription that allows access to millions of sample malicious and benign HTML pages.

All the data is stored at the root directory *ch11/data/html*. The benignware is stored at *ch11/data/html/benign_files*, and the malware is stored at *ch11/data/html/malicious_files*. Additionally, within each of these directories are the subdirectories *training* and *validation*. The *training* directories contain the files we train the neural network on in the chapter, and the *validation* directories contain the files we test the neural network on to assess its accuracy.

Chapter 12: Becoming a Data Scientist

Chapter 12 discusses how to become a data scientist and doesn't use any sample data.

Tool Implementation Guide

Although all the code in this book is *sample code*, intended to demonstrate the ideas in the book and not be taken whole cloth and used in the real world, some of the code we provide can be used as a tool in your own malware analysis work, particularly if you're willing to extend it for your own purposes.

NOTE

Intended as examples and starting places for full-fledged malware data science tools, these tools are not robustly implemented. They have been tested on Ubuntu 17 and are expected to work on this platform, but with a bit of work around installing the right requirements, you should be able to get the tools to work on other platforms like macOS and other flavors of Linux fairly easily.

In this section, we walk through the nascent tools provided in this book in the order in which they appear.

Shared Hostname Network Visualization

A shared hostname network visualization tool is given in Chapter 4 and is located at `ch4/code/listing-4-8.py`. This tool extracts hostnames from target malware files and then shows connections between the files based on common hostnames contained in them.

The tool takes a directory of malware as its input and then outputs three GraphViz files that you can then visualize. To install the requirements for this tool, run the command `run bash install_requirements.sh` in the `ch4/code` directory. Listing A-1 shows the “help” output from the tool, after which we discuss what the parameters mean.

```
usage: Visualize shared hostnames between a directory of malware samples
       [-h] target_path output_file malware_projection hostname_projection
```

positional arguments:

- | | |
|-----------------------|--------------------------------|
| ❶ target_path | directory with malware samples |
| ❷ output_file | file to write DOT file to |
| ❸ malware_projection | file to write DOT file to |
| ❹ hostname_projection | file to write DOT file to |

optional arguments:

- | | |
|------------|---------------------------------|
| -h, --help | show this help message and exit |
|------------|---------------------------------|
-

Listing A-1: Help output from the shared hostname network visualization tool given in Chapter 4

As shown in Listing A-1, the shared hostname visualization tool requires four command line arguments: `target_path` ❶, `output_file` ❷, `malware_projection` ❸, and `hostname_projection` ❹. The parameter `target_path` is the path to the directory of malware samples you'd like to analyze. The `output_file` parameter is a path to the file where the program will write a GraphViz `.dot` file representing the network that links malware samples to the hostnames they contain.

The `malware_projection` and `hostname_projection` parameters are also file paths and specify the locations where the program will write `.dot` files that represent these derived networks (for more on network projections, see Chapter 4). Once you’ve run the program, you can use the GraphViz suite discussed in Chapters 4 and 5 to visualize the networks. For example, you could use the command `fdp malware_projection.dot -Tpng -o malware_projection.png` to generate a file like the `.png` file rendered in Figure A-1 on your own malware datasets.

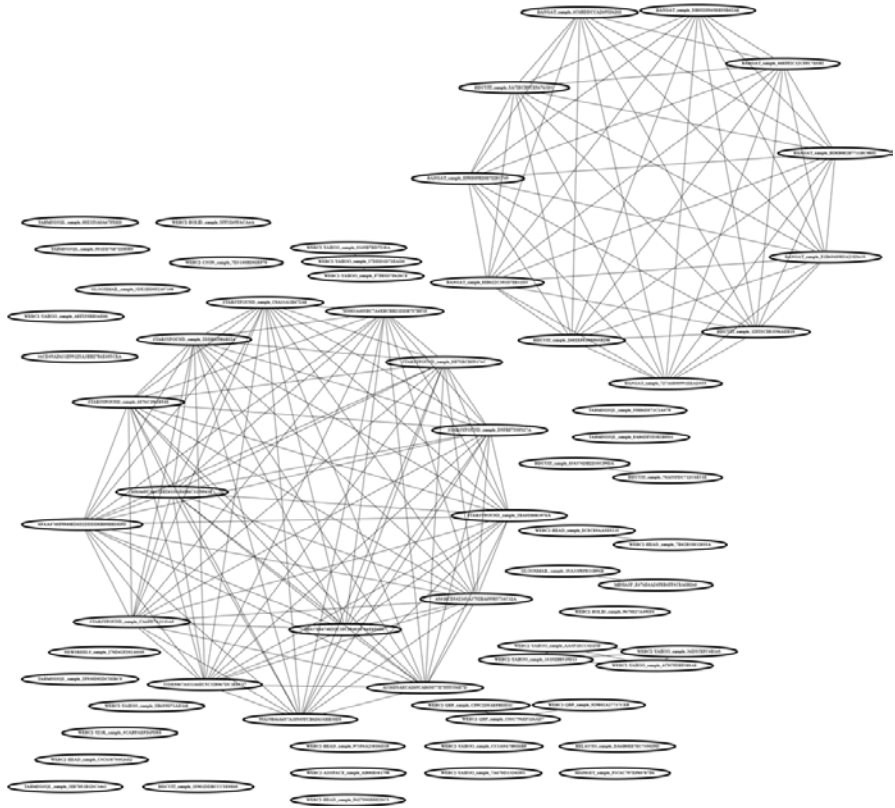


Figure A-1: Sample output from the shared hostname visualization tool given in Chapter 4

Shared Image Network Visualization

We present a shared image network visualization tool in Chapter 4, which is located at `ch4/code/listing-4-12.py`. This program shows network relationships between malware samples based on embedded images they share.

The tool takes a directory of malware as its input and then outputs three GraphViz files that you can then visualize. To install the requirements for this tool, run the command `run bash install_requirements.sh` in the `ch4/code` directory. Let’s discuss the parameters in the “help” output from the tool (see Listing A-2).

```
usage: Visualize shared image relationships between a directory of malware
samples
```

```
[-h] target_path output_file malware_projection resource_projection
```

```
positional arguments:
```

- ❶ target_path directory with malware samples
- ❷ output_file file to write DOT file to
- ❸ malware_projection file to write DOT file to
- ❹ resource_projection file to write DOT file to

```
optional arguments:
```

- h, --help show this help message and exit

Listing A-2: Help output from the shared resource network visualization tool given in Chapter 4

As shown in Listing A-2, the shared image relationships visualization tool requires four command line arguments: `target_path` ❶, `output_file` ❷, `malware_projection` ❸, and `resource_projection` ❹. Much like in the `shared_hostname` program, here `target_path` is the path to the directory of malware samples you'd like to analyze, and `output_file` is a path to the file where the program will write a GraphViz `.dot` file representing the bipartite graph that links malware samples to the images they contain (bipartite graphs are discussed in Chapter 4). The `malware_projection` and `resource_projection` parameters are also file paths and specify the locations where the program will write `.dot` files that represent these networks.

As with the `shared_hostname` program, once you've run the program, you can use the GraphViz suite to visualize the networks. For example, you could use the command `fdp resource_projection.dot -Tpng -o resource_projection.png` on your own malware datasets to generate a file like the `.png` file rendered in Figure 4-12 on page 55.

Malware Similarity Visualization

In Chapter 5, we discuss malware similarity and shared code analysis and visualization. The first sample tool we provide is given in `ch5/code/listing_5_1.py`. This tool takes a directory containing malware as its input and then visualizes shared code relationships between the malware samples in the directory. To install the requirements for this tool, run the command `run bash install_requirements.sh` in the `ch5/code` directory. Listing A-3 shows the help output for the tool.

```
usage: listing_5_1.py [-h] [--jaccard_index_threshold THRESHOLD]
target_directory output_dot_file
```

```
Identify similarities between malware samples and build similarity graph
```

```
positional arguments:
```

- ❶ target_directory Directory containing malware
- ❷ output_dot_file Where to save the output graph DOT file

optional arguments:

- h, --help show this help message and exit
- ❶ --jaccard_index_threshold THRESHOLD, -j THRESHOLD Threshold above which to create an 'edge' between samples

Listing A-3: Help output from the malware similarity visualization tool given in Chapter 5

When you run this shared code analysis tool from the command line, you need to pass in two command line arguments: `target_directory` ❶ and `output_dot_file` ❷. You can use the optional argument, `jaccard_index_threshold` ❸, to set the threshold the program uses with the Jaccard index similarity between two samples to determine whether or not to create an edge between those samples. The Jaccard index is discussed in detail in Chapter 5.

Figure A-2 shows sample output from this tool once you've rendered the `output_dot_file` with the command `fdp output_dot_file.dot -Tpng -o similarity_network.png`. This is the shared code network inferred by the tool for the APT1 malware samples we just described.

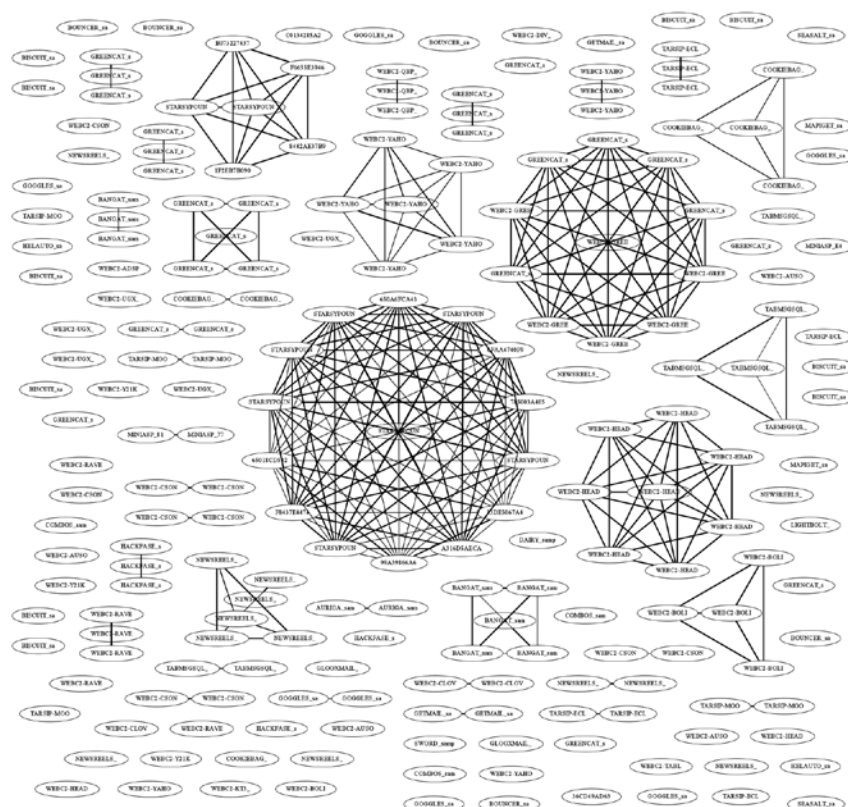


Figure A-2: Sample output from the malware similarity analysis tool given in Chapter 5

Malware Similarity Search System

The second code-sharing estimation tool we provide in Chapter 5 is given in *ch5/code/listing_5_2.py*. This tool allows you to index thousands of samples in a database and then perform a similarity search on them with a query malware sample, which lets you find malware samples that likely share code with that sample. To install the requirements for this tool, run the command `run bash install_requirements.sh` in the *ch5/code* directory. Listing A-4 shows the help output for the tool.

```
usage: listing_5_2.py [-h] [-l LOAD] [-s SEARCH] [-c COMMENT] [-w]
```

Simple code-sharing search system which allows you to build up a database of malware samples (indexed by file paths) and then search for similar samples given some new sample

optional arguments:

- h, --help show this help message and exit
 - ❶ -l LOAD, --load LOAD Path to directory containing malware, or individual malware file, to store in database
 - ❷ -s SEARCH, --search SEARCH Individual malware file to perform similarity search on
 - ❸ -c COMMENT, --comment COMMENT Comment on a malware sample path
 - ❹ -w, --wipe Wipe sample database
-

Listing A-4: Help output from the malware similarity search system given in Chapter 5

This tool has four modes in which it can be run. The first mode, **LOAD** ❶, loads malware into the similarity search database and takes a path as its parameter, which should point to a directory with malware in it. You can run **LOAD** multiple times and add new malware to the database each time.

The second mode, **SEARCH** ❷, takes the path to an individual malware file as its parameter and then searches for similar samples in the database. The third mode, **COMMENT** ❸, takes a malware sample path as its argument and then prompts you to enter a short textual comment about that sample. The advantage of using the **COMMENT** feature is that when you search for samples similar to a query malware sample, you see the comments corresponding to the similar sample, thus enriching your knowledge of the query sample.

The fourth mode, **wipe** ❹, deletes all the data in the similarity search database, in case you want to start over and index a different malware dataset. Listing A-5 shows some sample output from a **SEARCH** query, giving you a flavor for what the output from this tool looks like. Here we've indexed the APT1 samples described previously using the **LOAD** command and have subsequently searched the database for samples similar to one of the APT1 samples.

```

Showing samples similar to WEBC2-GREENCAT_sample_E54CE5F0112C9FDFE86DB17E85A5E2C5
Sample name                                     Shared code
[*] WEBC2-GREENCAT_sample_55FB1409170C91740359D1D96364F17B    0.9921875
[*] GREENCAT_sample_55FB1409170C91740359D1D96364F17B          0.9921875
[*] WEBC2-GREENCAT_sample_E83F60FB0E0396EA309FAF0AED64E53F    0.984375
[comment] This sample was determined to definitely have come from the advanced persistent
          threat group observed last July on our West Coast network
[*] GREENCAT_sample_E83F60FB0E0396EA309FAF0AED64E53F          0.984375

```

Listing A-5: Sample output for the malware similarity search system given in Chapter 5

Machine Learning Malware Detection System

The final tool you can use in your own malware analysis work is the machine learning malware detector used in Chapter 8, which can be found at *ch8/code/complete_detector.py*. This tool allows you to train a malware detection system on malware and benignware and then use this system to detect whether a new sample is malicious or benign. You can install the requirements for this tool by running the command `bash install.sh` in the *ch8/code* directory. Listing A-6 shows the help output for this tool.

```

usage: Machine learning malware detection system [-h]
                                           [--malware_paths MALWARE_PATHS]
                                           [--benignware_paths BENIGNWARE_PATHS]
                                           [--scan_file_path SCAN_FILE_PATH]
                                           [--evaluate]

optional arguments:
  -h, --help            show this help message and exit
  ❶ --malware_paths MALWARE_PATHS
                        Path to malware training files
  ❷ --benignware_paths BENIGNWARE_PATHS
                        Path to benignware training files
  ❸ --scan_file_path SCAN_FILE_PATH
                        File to scan
  ❹ --evaluate           Perform cross-validation

```

Listing A-6: Help output for the machine learning malware detection tool given in Chapter 8

This tool has three modes in which it can be run. The evaluate mode ❹, tests the accuracy of the system on the data you select for training and evaluating the system. You can invoke this mode by running `python complete_detector.py -malware_paths <path to directory with malware in it> --benignware_paths <path to directory with benignware in it> --evaluate`. This command will invoke a matplotlib window showing your detector's ROC curve (ROC curves are discussed in Chapter 7). Figure A-3 shows some sample output from evaluate mode.

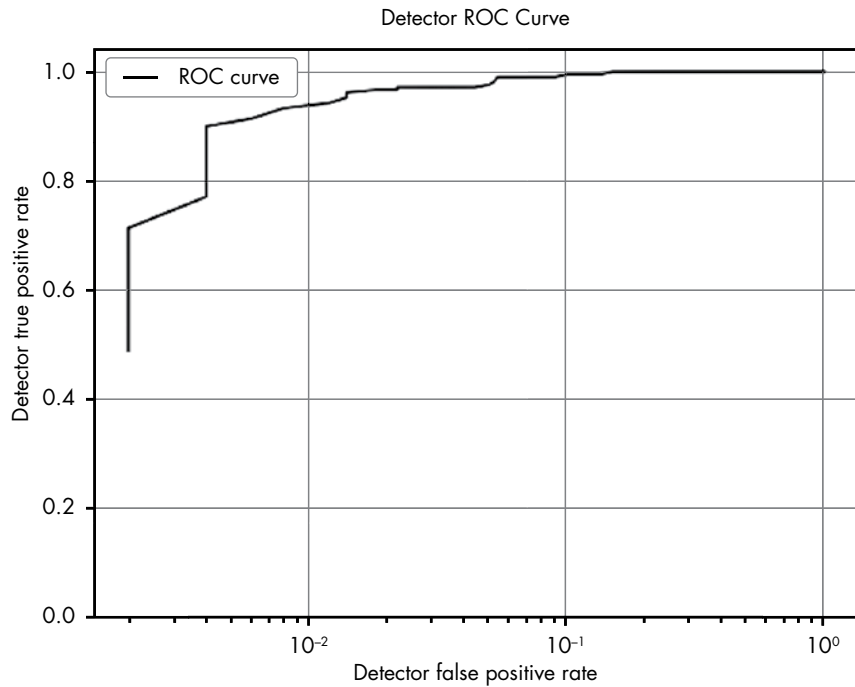


Figure A-3: Sample output from the malware detection tool provided in Chapter 8, run in evaluate mode

Training mode trains a malware detection model and saves it to disk. You can invoke this mode by running `python complete_detector.py -malware_paths ❶ <path to directory with malware in it> --benignware_paths ❷ <path to directory with benignware in it>`. Note that the only difference between this command invocation and the invocation of evaluate mode is that we've left off the `--evaluate` flag. The result of this command is that it generates a model that it saves to a file called `saved_detector.pkl`, which is saved in your current working directory.

The third mode, scan ❸, loads `saved_detector.pkl` and then scans a target file, predicting whether it's malicious or not. Make sure you have run training mode before running a scan. You can run a scan by running `python complete_detector.py -scan_file_path <PE EXE file>` in the directory where you trained the system. The output will be a probability that the target file is malicious.

INDEX

Note: Page numbers referring to figures and tables are followed by an italicized *f* or *t* respectively.

A

- activation functions
 - common, 178*t*–180*t*
 - defined, 178
- add_edge function, 41
- add_node function, 49–50
- add_question function, 112
- add arithmetic instruction, 15
- ADS (Alternate Data Streams), 29
- Advanced Persistent Threat 1 attacker group. *See* APT1 attacker group
- advanced persistent threats (APTs), 60
- Allaple.A malware family, 157, 157*f*
- Alternate Data Streams (ADS), 29
- anti-disassembly techniques, 22
- API calls, 32–33, 33*f*
- apply_hashing_trick function, 138
- APT1 (Advanced Persistent Threat 1)
 - attacker group, 37–39, 38*f*, 45–47, 45*f*–47*f*, 61, 61*f*, 76, 76*f*, 86, 222–223
- APTs (advanced persistent threats), 60
- ArchSMS family of Trojans, 55
- area under the curve (AUC), 209–210, 210*f*, 213
- arithmetic instructions, 15, 15*t*
- .asarray method, 142
- assembly language, defined, 12. *See also* x86 assembly language
- AT&T, 43
- AT&T syntax, 13
- attributes, 37
 - adding to nodes and edges, 42
 - and edges, 48–51
- AUC (area under the curve), 209–210, 210*f*, 213

- autoencoder neural networks, 194–195, 195*f*
- automatic feature generation, 188

B

- backpropagation, 190–192, 190*f*–191*f*
- bag of features model, 62–64, 63*f*
 - features, defined, 62
 - Jaccard index and, 65
 - N-grams, 63–64, 64*f*
 - order information and, 63–64
 - overview, 62–63
- bar charts (histograms), 168–170, 168*f*–169*f*
- base virtual memory address, 6
- basic blocks, 19–20
- bias parameter, 104
- bias term, 178, 181
- bipartite networks, 37–39, 38*f*
- bitcoin mining, 158, 160–161, 168*f*, 172*f*–173*f*, 173

C

- callbacks
 - built-in (Keras package), 212
 - creating shared callback
 - relationship network, 51–54
 - custom, 213–214, 214*f*
- call instruction, 17–18
- capstone module, 20
- Carerra, Ero, 5
- chain rule, 191–192
- cmp instruction, 18
- CNNs (convolutional neural networks), 193–194, 194*f*
- coarsenings, 46
- color attribute, 49
- comment_sample function, 82–84
- COMMENT mode, 229
- compile method, 202

- compressed_data_weight parameter, 103
- compressed_data parameter, 103–104
- conditional branches, defined, 15
- control flow, 17
 - graphs, 19–20, 19*f*
 - instructions, 17–18
 - registers, 14–15
- convolutional neural networks (CNNs), 193–194, 194*f*
- CPU registers, 13–15, 14*f*
 - general-purpose registers, 13–14
 - stack and control flow registers, 14–15
- cross_validation module, 151
- cross-validation, 150–153, 151*f*, 153*f*
- CuckooBox software platform, 27, 33–34, 59
- “curse of dimensionality,” 92
- cv_evaluate function, 151

D

- dapato malware family, 62, 67*f*–68*f*, 70*f*–72*f*
- DataFrame objects, 158–161
- data movement instructions, 15–20, 16*t*
 - basic blocks, 19–20, 19*f*
 - control flow graphs, 19–20, 19*f*
 - control flow instructions, 17–18
 - stack instructions, 16–17
- data science, iii, iv
 - applying to malware, v
 - importance of, iv–v
- .data section (in PE file format), 4
- dateutil package, 164
- dec arithmetic instruction, 15
- decision boundaries, 93–98, 95*f*–98*f*
 - identifying with k-nearest neighbors, 97–98, 97*f*–98*f*
 - identifying with logistic regression, 96–97, 96*f*–97*f*
 - overfit machine-learning model, 100, 101*f*
 - underfit machine-learning model, 99, 99*f*
 - well-fit machine-learning model, 100, 100*f*
- decision thresholds, 149
- DecisionTreeClassifier class, 130

- decision trees, 109–115, 109*f*–110*f*, 113*f*–114*f*
 - decision tree–based detectors, 129
 - importing modules, 129
 - initializing sample training data, 130
 - instantiating classes, 130
 - sample code, 133–134
 - training, 130–131
 - visualizing, 131–133, 132*f*
- follow-up questions, 111
- limiting depth or number of questions, 111–112
- pseudocode for, 112–113
- root node, 110–111
- when to use, 114–115
- deep learning, 175–197, 216. *See also* neural networks
 - automatic feature generation, 188
 - building neural networks, 182–188
 - neurons, 176
 - anatomy of, 177–180
 - networks of, 180–181
 - overview, 176–177
 - training neural networks, 189–193
 - types of neural networks, 193–197
 - universal approximation theorem, 181–182
- deep neural networks. *See* neural networks
- Dense function, 200–201
- describe method, 159
- detection accuracy evaluation, 119–126, 146–153
 - base rates and precision, 124–126
 - effect of base rate on precision, 124–125
 - estimating precision in deployment environment, 125–126
 - with cross-validation, 150–153, 151*f*, 153*f*
 - neural networks, 209–211, 210*f*–211*f*
 - possible detection outcomes, 120, 120*f*
 - with ROC curves, 123–124, 123*f*, 147–150, 150*f*
 - true and false positive rates, 120–124
 - relationship between, 121–122, 121*f*–122*f*
 - ROC curves, 123–124, 123*f*

- DictVectorizer class, 128–130
- directed graphs, 180
- distance functions, 107
- DLLs (dynamic-link libraries), 13
- DOS header (in PE file format), 3
- .dot format, 42
- dynamically downloaded data, 22–23
- dynamic analysis, 25–34
 - bag of features model, 63
 - dataset for, 222
 - for disassembly, 26
 - limitations of, 33–34
 - for malware data science, 26
 - typical malware behaviors, 27
 - using *malwr.com*, 26–33
 - analyzing results, 28–33
 - limitations, 33
 - loading files, 27–28
- dynamic API call–based similarity, 72, 72*f*
- dynamic-link libraries (DLLs), 13

E

- EAX register, 14
- EBP register, 14
- EBX register, 14
- ECX register, 14
- edges, 37
 - adding attributes, 42
 - adding to shared relationship networks, 41
 - adding visual attributes to, 48–51
 - color, 49, 49*f*
 - text labels, 50–51
 - width, 48–49, 48*f*
- EDX register, 14
- EFLAGS register, 15
- EIP register, 14–15
- ELU activation function, 179*t*
- entry point, 3, 19
- epochs parameter, 206
- ESP register, 14
- euclidean_distance function, 107
- Euclidean distance, 107
- evaluate function, 148
- evaluate mode, 231–232
- evaluating malware detection systems.
 - See* detection accuracy evaluation
- export_graphviz function, 132
- extract_features function, 204–205
- ExtractImages helper class, 56–57

F

- fakepdfmalware.exe*, 7
- false negatives, defined, 120, 120*f*
- false positives, 120, 120*f*
 - base rates and precision, 124–126
 - false positive rate, 121
 - relationship between true and false positive rates, 121–122, 121*f*–122*f*
 - ROC curves, 123–124, 123*f*
- fdp tool, 43–45, 45*f*, 76
- feature_extraction module, 129
- feature extraction, 134–138
 - Import Address Table features, 136
 - machine learning–based malware detectors, 90–92, 141–142
 - N-grams, 136–137
 - Portable Executable header features, 135–136
 - shared code analysis, 73, 75
 - string features, 135
 - training neural networks with Keras package, 203–204
 - why all possible features can't be used at once, 137–138
- FeatureHasher class, 140–141
- feature hashing. *See* hashing trick
- feature spaces, 93–98, 94*f*–98*f*
- feed-forward neural networks, 181, 181*f*, 193
- fit_generator function, 204–206, 208, 212, 214
- fit method, 130–131, 142
- flags, defined, 15
- format strings, 70
- forward propagation, 189–190

G

- Gaussian activation function, 179*t*
- generative adversarial networks (GANs), 195–196
- generator parameter, 206
- get_database function, 80–82
- get_string_features function, 141–142, 144
- get_strings function, 82
- get_training_data function, 143
- get_training_paths function, 143
- GETMAIL utility, 223
- getstrings function, 73–74
- G flag, 44

- gini index, 132, 132*f*
- gradient descent, 105, 190
- Graph constructor, 41, 52–53
- graphical image analysis, 7–8
 - converting extracted *.ico* files to *.png* graphics, 8
 - creating directory to hold extracted images, 7–8
 - extracting image resources using *wrestool*, 8
- GraphViz, 76
 - decision tree–based detectors, 131–133, 132*f*
 - malware network analysis, 43–51
 - adding visual attributes to nodes and edges, 48–51
 - fdp* tool, 44–45, 45*f*
 - neato* tool, 47–48, 47*f*
 - parameters, 44
 - sfdp* tool, 46–47, 46*f*
 - similarity graphs, 76
- ground_truth* variable, 130

H

- hashing trick (feature hashing), 138–141
 - complete code for, 139–140
 - FeatureHasher* class, 140–141
 - implementing, 138–139
- hidden layer, 181
- histograms (bar charts), 168–170, 168*f*–169*f*
- hostname_projection* argument, 225
- hyperplanes, 96, 97*f*

I

- IAT. *See* Import Address Table
- icoutils* toolkit, 5
- IDA Pro, 12
- .idata* section (imports) (in PE file format), 4
- Identity activation function, 178*t*
- Import Address Table (IAT), 4
 - dumping using *pefile*, 6–7
 - extracting features, 136
 - similarity analysis based on, 71, 71*f*
- imports analysis, 6–7
- inc* arithmetic instruction, 15
- information gain, 113
- Input* function, 200–201

- instruction sequence–based similarity, 68*f*
 - limitations of, 68–70
 - overview, 67–68
- Intel syntax, 13
- Internet Relay Chat (IRC), 2
- int* function, 148
- inverted indexing, 82
- ircbot.exe* bot, 2
 - disassembling, 20–21
 - dissecting, 5–7
 - dumping IAT, 6–7
 - strings analysis, 9–10

J

- jaccard_index_threshold* argument, 227–228
- jaccard* function, 73
- Jaccard index, 61, 65, 65*f*
 - building similarity graphs, 73–75
 - dynamic API call–based similarity, 72
 - instruction sequence–based similarity, 68
 - minhash* method, 77–79
 - scaling similarity comparisons, 77
 - strings–based similarity, 70
- jge* instruction, 18
- jmp* instructions, 18
- jointplot* function, 171–172

K

- Kaspersky, 62
- Keras* package, building neural networks with, 199–214
 - compiling model, 202–203, 202*f*
 - defining architecture of model, 200–202
 - evaluating model, 209–211, 210*f*–211*f*
 - layers, 200
 - saving and loading model, 209
 - syntaxes, 200
 - training model, 203–209, 211–214
 - built-in callbacks, 212
 - custom callbacks, 213–214, 214*f*
 - data generators, 204–207, 207*f*
 - feature extraction, 203–204
 - validation data, 207–209, 208*f*
- keyloggers, 158, 168*f*, 172*f*–173*f*, 173

- KFold class, 151–152
- K-fold cross-validation, 151
- k-nearest neighbors, 105–109, 106*f*, 108*f*
 - identifying decision boundaries
 - with, 97–98, 97*f*–98*f*
 - logistic regression vs., 108–109
 - math behind, 107
 - pseudocode for, 107
 - when to use, 109

L

- label attribute, 50–51
- layers submodule, 200–201
- lea instruction, 16
- Leaky ReLU activation function, 179*t*
- learned_parameters parameter, 103
- linear disassembly, 12
 - limitation of, 12
 - shared code analysis, 67–68
- LOAD mode, 229
- logistic_function function,
 - 103–104, 104*f*
- logistic_regression function, 103
- logistic regression, 102–105,
 - 103*f*–104*f*, 154
 - gradient descent, 105
 - identifying decision boundaries
 - with, 96–97, 96*f*–97*f*
 - k-nearest neighbors vs., 108–109
 - limitation of, 102
 - math behind, 103–104
 - plot of logistic function, 104*f*
 - pseudocode for, 103
 - when to use, 105
- long short-term memory (LSTM)
 - networks, 196
- Los Alamos National Laboratory, 41
- loss parameter, 201–202

M

- machine learning–based malware
 - detectors, 89–117, 127–154
 - building basic detectors, 129
 - sample code, 133–134
 - training, 130–131
 - visualizing, 131–133, 132*f*
 - building overview, 90–93
 - collecting training examples,
 - 90–91
 - designing good features, 92
 - extracting features, 90–92
 - reasons for, 89–90
 - testing system, 90, 93
 - training system, 90, 92–93
 - building real-world detectors,
 - 141–146
 - complete code for, 144–146
 - feature extraction, 141–142
 - running detector on new
 - binaries, 144
 - training, 142–143
- dataset for, 224
- decision boundaries, 93–98,
 - 95*f*–98*f*
- evaluating detector
 - performance, 146
 - cross-validation, 150–153,
 - 151*f*, 153*f*
 - ROC curves, 147–150, 150*f*
 - splitting data into training and
 - test sets, 148–149
- feature extraction, 134–138
 - Import Address Table
 - features, 136
 - N-grams, 136–137
 - Portable Executable header
 - features, 135–136
 - string features, 135
 - why all possible features can't
 - be used at once, 137–138
- feature spaces, 93–98, 94*f*–98*f*
- hashing trick, 138–141
 - complete code for, 139–140
 - FeatureHasher class, 140–141
 - implementing, 138–139
- overfitting and underfitting, 98–99,
 - 99*f*–101*f*
- supervised vs. unsupervised
 - algorithms, 93
- terminology and concepts, 128–129
- tool for, 230–232, 231*f*
- traditional algorithms vs., 90
- types of algorithms, 101, 102*f*
 - decision trees, 109–115,
 - 109*f*–110*f*, 113*f*–114*f*
 - k-nearest neighbors, 97–98,
 - 97*f*–98*f*, 105–109, 106*f*, 108*f*
 - logistic regression, 96–97, 96*f*–
 - 97*f*, 102–105, 103*f*–104*f*
 - random forest, 115–116, 116*f*

- malware_projection argument, 52,
- 225–227

- malware detection evaluation.
 - See* detection accuracy evaluation
 - malware network analysis, 35–58, 36*f*
 - attributes, defined, 37
 - bipartite networks, 37–39, 38*f*
 - creating shared callback
 - relationship network, 51–54, 225–226, 226*f*
 - code for, 52–54
 - importing modules, 51–52
 - parsing command line arguments, 52
 - saving networks to disk, 54
 - creating shared image relationship networks, 54–58, 55*f*, 226–227
 - extracting graphical assets, 57
 - parsing initial argument and file-loading code, 55–57
 - saving networks to disk, 58
 - dataset for, 222–223
 - edges, defined, 37
 - GraphViz, creating visualizations with, 43–51
 - fdp tool, 44–45, 45*f*
 - neato tool, 47–48, 47*f*
 - parameters, 44
 - sfdp tool, 46–47, 46*f*
 - visual attributes, 48–51
 - NetworkX library, creating networks with, 40–43
 - adding attributes, 42
 - adding nodes and edges, 41
 - saving networks to disk, 42–43
 - nodes, defined, 37
 - projections, 38
 - shared code analysis and, 60–61
 - visualization challenges, 39–40
 - distortion problem, 39–40, 40*f*
 - force-directed algorithms, 40
 - network layout, 39–40
 - malware samples, 61–62, 222–224
 - malwr.com*, 26–33, 28*f*
 - analyzing results on, 28–33
 - API calls, 32–33, 33*f*
 - modified system objects, 30–32
 - Screenshots panel, 30, 30*f*
 - Signatures panel, 29–30, 29*f*
 - Summary panel, 30–32, 31*f*–32*f*
 - limitations of, 33
 - loading files on, 27–28
 - Mandiant, 61, 76, 223
 - MAPIGET utility, 223
 - Mastercard, iii
 - matplotlib library, 148–150, 162–167, 162*f*
 - plotting ransomware and worm detection rates, 165–167, 166*f*
 - plotting ransomware detection rates, 164–165, 165*f*
 - plotting relationship between malware size and detection, 162–163
 - max function, 160
 - mean function, 160–161
 - memory cells, 196
 - metrics module, 147–148
 - metrics parameter, 201–202
 - min function, 81, 160
 - minhash approach
 - combined with sketching, 79
 - math behind, 78–79, 78*f*
 - overview, 77–78
 - minhash function, 82
 - ModelCheckpoint callback, 212
 - Model class, 201
 - models submodule, 201–202
 - mov instruction, 15–16
 - murmur module, 80, 82
 - mutexes, defined, 32
 - my_generator* function, 205, 207–208
 - MyCallback class, 213–214
- ## N
- neato tool, 47–48, 47*f*
 - Nemucod.FG malware family, 157, 157*f*
 - NetworkX library, 40–43
 - creating shared relationship networks, 41–42
 - overview, 41
 - saving networks to disk, 42–43
 - neural networks, 176, 177–188
 - automatic feature generation, 188
 - building
 - with four neurons, 186–188, 186*f*–187*f*, 187*t*
 - with three neurons, 184–186, 185*f*–186*f*, 185*t*
 - with two neurons, 182–184, 182*f*–184*f*, 183*t*–184*t*

- building with Keras package, 199–214
 - compiling model, 202–203, 202*f*
 - defining architecture of model, 200–202
 - evaluating model, 209–211, 210*f*–211*f*
 - saving and loading model, 209
 - training model, 203–209, 211–214
- dataset for, 224
- neurons, 176
 - anatomy of, 177–180, 177*f*, 178*t*–180*t*
 - networks of, 180–181, 181*f*
- training, 189–193
 - using backpropagation, 190–192, 190*f*–191*f*
 - using forward propagation, 189–190
 - vanishing gradient problem, 192–193
- types of, 193–197
 - autoencoder, 194–195, 195*f*
 - convolutional, 193–194, 194*f*
 - feed-forward, 193
 - generative adversarial, 195–196
 - recurrent, 196
 - ResNet, 196–197
- universal approximation theorem, 181–182, 182*f*
- neurons, 176
 - anatomy of, 177–180, 177*f*, 178*t*–180*t*
 - networks of, 180–181, 181*f*
- next method, 205, 208
- N-grams, 63–64, 64*f*
 - dynamic API call-based similarity, 72
 - extracting features, 136–137
 - instruction sequence-based similarity, 67–68
- nodes, 37
 - adding attributes, 42
 - adding to shared relationship networks, 41
 - adding visual attributes to, 48–51
 - color, 49, 49*f*
 - shape, 49–50, 50*f*
 - text labels, 50–51
 - width, 48–49
 - in decision trees, 110–111
- NUM_MINHASHES constant, 80–81

O

- objective function, 189
- optimizer parameter, 201–202
- optional header (in PE file format), 3–4
- output_dot_file argument, 227–228
- output_file argument, 52, 225, 227
- overfit machine-learning models, 98–99, 101*f*
- overlap parameter, 44

P

- packing, 21
 - difficulty of disassembling packed malware, 26
 - legitimate uses of, 22
- pandas package, 158–161
 - filtering data using conditions, 161
 - loading data, 158–159
 - manipulating DataFrame, 159–161
- Parkour, Mila, 61
- pasta malware family, 62, 67*f*–68*f*, 70*f*–72*f*
- PE. *See* Portable Executable file format
- PE (Portable Executable) header, 3, 135–136
- pecheck function, 73–74
- pefile module, 5–7
 - disassembly using, 20
 - dumping IAT, 6–7
 - installing, 5, 20
 - opening and parsing files, 5–6
 - pulling information from PE fields, 6
- pefile PE parsing module, 51–52
- penwidth attribute, 48–49
- persistent malware similarity search systems, 79–87
 - building
 - allowing users to search for and comment on samples, 82–84
 - implementing database functionality, 80–81
 - importing packages, 80
 - indexing samples into system's database, 82
 - loading samples, 85
 - obtaining minhashes and sketches, 81–82
 - parsing user command line arguments, 84–85

- persistent malware similarity search systems, *continued*
 - commenting on samples, 86
 - sample output, 86–87
 - searching for similar samples, 86
 - wiping database, 86
- `pick_best_question` function, 112–113
- `pickle` module, 143–144
- `plot` function, 162–163, 167
- `.png` format, 43
- pooling layer, 194
- `pop` instruction, 16–17
- Portable Executable (PE) file
 - format, 2–5
 - dissecting files using `pefile`, 5–7
 - entry point, 3
 - file structure, 2–5, 3*f*
 - DOS header, 3
 - optional header, 3–4
 - PE header, 3
 - section headers, 4–5
 - sections, defined, 4
- Portable Executable (PE) header, 3, 135–136
- position independence, 5
- precision, 124–126
 - effect of base rate on, 124–125
 - estimating in deployment environment, 125–126
- `predict_proba` method, 144, 149
- PRELU activation function, 179*t*
- program stack, defined, 14
- `projected_graph` function, 54
- projections, 38
- `push` instruction, 16–17
- `pyplot` module, 148–149, 163

R

- random forest
 - overview, 115–116, 116*f*
 - random forest–based detectors, 141–146
 - complete code for, 144–146
 - running detector on new binaries, 144
 - training, 142–143
- `RandomForestClassifier` class, 143, 152
- ransomware, 30–31, 31*f*, 155–158, 156*f*, 158, 164–168, 165*f*–166*f*, 168*f*, 172–173, 172*f*–173*f*
- `.rdata` section (in PE file format), 4

- Receiver Operating Characteristic curves. *See* ROC curves
- rectified linear unit (ReLU) activation function, 177*f*, 178*t*, 180, 182*f*, 183–185, 201
- recurrent neural networks (RNNs), 196
- registry keys, 32
- `.reloc` section (in PE file format), 5
- ReLU (rectified linear unit) activation function, 177*f*, 178*t*, 180, 182*f*, 183–185, 201
- ResNets (residual networks), 196–197
- `resource_projection` argument, 52, 227
- resource obfuscation, 22
- `ret` instruction, 17–18
- reverse engineering, 12
 - anti-disassembly techniques, 22
 - dynamic analysis for, 26
 - methods for, 12
 - shared code analysis, 60
 - using `pefile` and `capstone`, 20–21
- RNNs (recurrent neural networks), 196
- ROC (Receiver Operating Characteristic) curves, 123–124, 123*f*, 126, 147–150, 230–231, 231*f*
 - computing, 147–150
 - cross-validation, 151–152, 153*f*
 - neural networks, 209–210, 210*f*–211*f*
 - visualizing, 149, 150*f*
- `roc_curve` function, 149, 210
- `.rsrc` section (resources) (in PE file format), 4–5

S

- sandbox, 26
- Sanders, Hillary, 216
- `savefig` function, 165
- `scan_file` function, 144
- scan mode, 230–231
- scikit-learn (sklearn) machine learning package, 127–128
 - building basic decision tree–based detectors, 129–134
 - building random forest–based detectors, 141–146
 - evaluating detector performance, 146–153
 - feature extraction, 134–135
 - hashing trick, 140–141

- terminology and concepts, 128–129
 - classifiers, 129
 - fit, 129
 - label vectors, 128–129
 - prediction, 129
 - vectors, 128
- seaborn package, 168–174, 168*f*
 - creating violin plots, 172–174, 172*f*–173*f*
 - plotting distribution of antivirus detections, 169–172, 169*f*, 171*f*
- search_sample function, 82–84
- SEARCH mode, 229
- section headers (in PE file format), 4–5
 - .data section, 4
 - .idata section (imports), 4
 - .rdata section, 4
 - .reloc section, 5
 - .rsrc section (resources), 4–5
 - .text section, 4
- security data scientists, 215–220
 - expanding knowledge of methods, 219–220
 - paths to becoming, 216
 - traits of effective, 218–219
 - curiosity, 218–219
 - obsession with results, 219
 - open-mindedness, 218
 - skepticism of results, 219
 - willingness to learn, 216
 - workflow of, 216–218, 217*f*
 - data feed identification, 218
 - dealing with stakeholders, 217
 - deployment, 218
 - problem identification, 217–218
 - solution building and evaluation, 218
- self-modifying code, 12
- set_axis_labels function, 172
- sfdp tool, 46–47, 46*f*
- shape attribute, 49–50
- shared attribute analysis. *See* malware network analysis
- shared code analysis (similarity analysis), 59–87, 60, 61*f*
 - bag of features model, 62–64, 63*f*
 - features, defined, 62
 - N-grams, 63–64, 64*f*
 - order information and, 63–64
 - overview, 62–63
 - dataset for, 223
- Jaccard index, 64–65, 65*f*
- persistent malware similarity search systems, 79–87
 - allowing users to search for and comment on samples, 82–84
 - commenting on samples, 86
 - implementing database functionality, 80–81
 - importing packages, 80
 - indexing samples into system database, 82
 - loading samples, 85
 - obtaining minhashes and sketches, 81–82
 - parsing user command line arguments, 84–85
 - sample output, 86–87
 - searching for similar samples, 86
 - wiping database, 86
- scaling similarity comparisons, 77–79
 - difficulties with, 77
 - minhash method, 77–79, 78*f*
- similarity graphs, 73–76, 76*f*
 - declaring utility functions, 73–74
 - extracting features, 73, 75
 - importing libraries, 73
 - iterating through pairs, 75
 - Jaccard index threshold, 73
 - parsing user's command line arguments, 74
 - visualizing graphs, 76
- similarity matrices, 66–72, 66*f*–67*f*
 - concept of, 66
 - dynamic API call-based similarity, 72, 72*f*
 - Import Address Table-based similarity, 71, 71*f*
 - instruction sequence-based similarity, 67–70, 68*f*
 - strings-based similarity, 70–71, 70*f*
 - tools for, 227–230, 228*f*
- shared image relationship networks, 54–58, 55*f*, 226–227
 - extracting graphical assets, 57
 - parsing initial argument and file-loading code, 55–57
 - saving networks to disk, 58
- shelve module, 80
- show function, 152, 163, 165, 168

- Sigmoid activation function, 180*t*, 201
- `sim_graph` module, 80, 82
- similarity analysis. *See* shared code analysis
- similarity functions, 64–65
- similarity graphs, 73–76, 76*f*
 - declaring utility functions, 73–74
 - extracting features, 73, 75
 - importing libraries, 73
 - iterating through pairs, 75
 - Jaccard index threshold, 73
 - parsing user’s command line arguments, 74
 - visualizing graphs, 76
- similarity matrices, 66–72, 66*f*–67*f*
 - dynamic API call–based similarity, 72, 72*f*
 - Import Address Table–based similarity, 71, 71*f*
 - instruction sequence–based similarity, 67–70, 68*f*
 - strings-based similarity, 70–71, 70*f*
- SKETCH_RATIO constant, 80, 82
- sklearn. *See* scikit-learn machine learning package
- skor malware family, 62, 67*f*–68*f*, 70*f*–72*f*
- Softmax activation function, 180*t*
- Sophos, 216
- splines parameter, 44
- `split_regex` expression, 203–204
- stack, defined, 16
- stack instructions, 16–17
- stack management registers, 14–15
- static malware analysis, 1–23
 - dataset for, 222
 - disassembly and reverse engineering, 12
 - methods for, 12
 - using `pefile` and `capstone`, 20–21
 - graphical image analysis, 7–8
 - imports analysis, 6–7
 - limitations of, 21–23
 - anti-disassembly techniques, 22
 - dynamically downloaded data, 22–23
 - packing, 21–22
 - resource obfuscation, 22
 - `pefile` module, 5–7
 - Portable Executable file format, 2–5
 - strings analysis, 8–10

- `std` function, 160
- Step activation function, 179*t*
- `steps_per_epoch` parameter, 206
- `string_hash` function, 81–82
- strings
 - defined, 8
 - feature extraction, 135, 141–142
- strings analysis, 8–10
 - analyzing printable strings, 8–10
 - information revealed through, 8
 - printing all strings in a file to terminal, 8–9
- strings-based similarity, 70–71, 70*f*
- strings tool, 8–10
- sub arithmetic instruction, 15
- summary function, 202–203, 202*f*
- supernodes, 46
- `suspicious_calls` parameter, 103–104
- suspiciousness scores, 121–122, 121*f*–122*f*

T

- Target, iii
- `target_directory` argument, 227–228
- `target_path` argument, 52, 225, 227
- TensorFlow, 200, 207
- .text section (in PE file format), 4
- threat scores, 147
- .todense method, 142
- `train_detector` function, 143
- `training_examples` variable, 130
- transform method, 131, 140
- tree module, 129
- Trojans, 54–55, 55*f*, 158–161, 168*f*, 172*f*–173*f*, 173
- true negatives, defined, 120, 120*f*
- true positives, 120, 120*f*
 - base rates and precision, 124–126
 - relationship between true and false positive rates, 121–122, 121*f*–122*f*
 - ROC curves, 123–124, 123*f*
 - true positive rate, 121

U

- underfit machine-learning models, 98–99, 99*f*
- universal approximation theorem, 181–182, 182*f*
- UPX packer, 29

V

- validation_labels object, 210–211
- validation_scores object, 210
- vanishing gradient problem, 192–193
- vbna malware family, 62, 67*f*–68*f*, 70*f*–72*f*
- vectors, 128
- violin plots, 172–174, 172*f*–173*f*
- VirtualBox, vii–viii, 222
- virtual size, 6
- VirusTotal.com*, 29, 59
- visualization, 155–174
 - basic machine learning–based malware detectors, 131–133, 132*f*
 - dataset for, 224
 - importance of, 156–158, 157*f*
 - malware network analysis
 - challenges to, 40*f*
 - creating with GraphViz, 45*f*–47*f*
 - network analysis
 - challenges to, 39–40
 - creating with GraphViz, 43–51
 - ROC curves, 149, 150*f*, 152–153, 153*f*
 - shared code analysis, 76
 - using matplotlib, 162–167, 162*f*
 - plotting ransomware and worm detection rates, 165–167, 166*f*
 - plotting ransomware detection rates, 164–165, 165*f*
 - plotting relationship between malware size and detection, 162–163
 - using pandas, 158–161
 - filtering data using conditions, 161
 - loading data, 158–159
 - manipulating DataFrame, 159–161
 - using seaborn, 168–174, 168*f*
 - creating violin plots, 172–174, 172*f*–173*f*
 - plotting distribution of antivirus detections, 169–172, 169*f*, 171*f*

W

- webprefix malware family, 62, 67*f*–68*f*, 70*f*–72*f*
- weight attribute, 37
- weight parameter, 178, 181
- Wells Fargo, iii
- Wikipedia, 220
- wipe_database function, 80–81
- wipe mode, 229
- work method, 57
- worms, 158–161, 165–167, 166*f*, 168*f*, 172, 172*f*–173*f*
- wrestool tool, 55
 - downloading, 8
 - extracting image resources, 7–8
- write_dot function, 42–43

X

- x86 assembly language, 12–20
 - arithmetic instructions, 15, 15*t*
 - CPU registers, 13–15, 14*f*
 - general-purpose registers, 13–14
 - stack and control flow registers, 14–15
 - data movement instructions, 15–20, 16*t*
 - basic blocks and control flow graphs, 19–20, 19*f*
 - control flow instructions, 17–18
 - stack instructions, 16–17
 - dialects of, 13
 - shared code analysis, 67
- xtoober malware family, 62, 67*f*–68*f*, 70*f*–72*f*

Y

- yield statement, 205

Z

- zango malware family, 62, 67*f*–68*f*, 70*f*–72*f*

Malware Data Science is set in New Baskerville, Futura, Dogma, and TheSans-MonoCondensed.

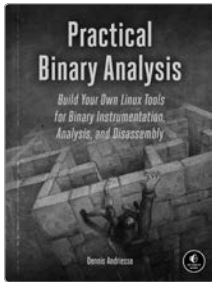
UPDATES

Visit <https://www.nostarch.com/malwaredatascience/> for updates, errata, and other information.

More no-nonsense books from



NO STARCH PRESS

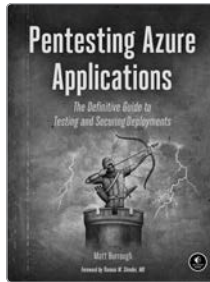


PRACTICAL BINARY ANALYSIS **Build Your Own Linux Tools for Binary Instrumentation, Analysis, and Disassembly**

by DENNIS ANDRIESSE

FALL 2018, 440 PP., \$49.95

ISBN 978-1-59327-912-7

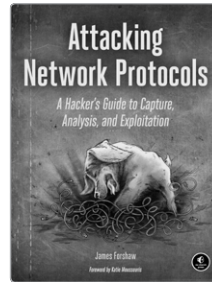


PENTESTING AZURE APPLICATIONS **The Definitive Guide to Testing and Securing Deployments**

by MATT BURROUGH

JULY 2018, 216 PP., \$39.95

ISBN 978-1-59327-863-2

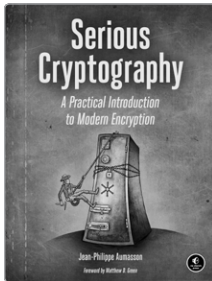


ATTACKING NETWORK PROTOCOLS **A Hacker's Guide to Capture, Analysis, and Exploitation**

by JAMES FORSHAW

DECEMBER 2017, 336 PP., \$49.95

ISBN 978-1-59327-750-5

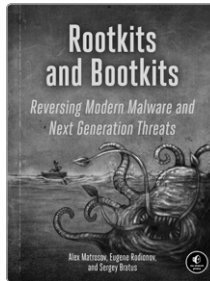


SERIOUS CRYPTOGRAPHY **A Practical Introduction to Modern Encryption**

by JEAN-PHILIPPE AUMASSON

NOVEMBER 2017, 312 PP., \$49.95

ISBN 978-1-59327-826-7

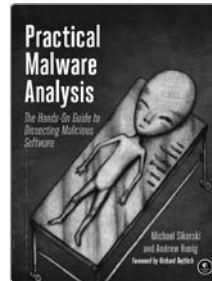


ROOTKITS AND BOOTKITS **Reversing Modern Malware and Next Generation Threats**

by ALEX MATROSOV, EUGENE RODIONOV, and SERGEY BRATUS

SPRING 2019, 504 PP., \$49.95

ISBN 978-1-59327-716-1



PRACTICAL MALWARE ANALYSIS **The Hands-On Guide to Dissecting Malicious Software**

by MICHAEL SIKORSKI and ANDREW HONIG

FEBRUARY 2012, 800 PP., \$59.95

ISBN 978-1-59327-290-6

PHONE:

1.800.420.7240 OR

1.415.863.9900

EMAIL:

SALES@NOSTARCH.COM

WEB:

WWW.NOSTARCH.COM

"Stay ahead of the changes in technology and the adversaries you're charged with defeating."

— Anup Ghosh, PhD, founder of Invincea, Inc

With millions of malware files created each year and a flood of security-related data generated every day, security has become a "big data" problem. So, when defending against malware, why not think like a data scientist?

In *Malware Data Science*, security data scientists Joshua Saxe and Hillary Sanders show you how to apply machine learning, statistics, and data visualization as you build your own detection and intelligence systems. Following an overview of basic reverse engineering concepts like static and dynamic analysis, you'll learn to measure code similarities in malware samples and use machine learning frameworks like scikit-learn and Keras to build and train your own detectors.

Learn how to:

- ✔ Identify new malware written by the same adversary groups through shared code analysis
- ✔ Catch zero-day malware by building your own machine learning detection system
- ✔ Use ROC curves to measure the accuracy of your malware detector to help you select the best approach to a security problem

- ✔ Use data visualization to identify and explore malware campaigns, trends, and relationships
- ✔ Use Python to implement deep neural network-based detection systems

Whether you're a malware analyst looking to add skills to your existing arsenal or a data scientist interested in attack detection and threat intelligence, *Malware Data Science* will help you stay ahead of the curve.

About the Authors

JOSHUA SAXE is chief data scientist at Sophos, a major security software vendor, where he helps invent data science technologies for detecting Android-, Windows-, and web-based malicious programs. Before joining Sophos, Saxe spent five years leading DARPA-funded security data research projects for the US government.

HILLARY SANDERS is a senior software engineer and data scientist at Sophos, where she has played a key role in inventing and productizing neural network, machine learning, and malware similarity analysis security technologies. She is a regular speaker at security conferences like Black Hat USA and BSides Las Vegas.



THE FINEST IN GEEK ENTERTAINMENT™
www.nostarch.com

Part of the proceeds from this book will be
donated to the Environmental Defense Fund.

Price: \$49.95 (\$65.95 CDN)

Shelve In: COMPUTERS/SECURITY

ISBN: 978-1-59327-859-5



9 781593 278595